Today 10:00 AM

daplanstan-main.zip File	casestudy(1).tex File	DP_Csestudy_instr... PDF
DP_Csestudy(1).pdf PDF	Pasted text(9).txt Document	Part1_DaPlanStan_... File

I am handling Part 2 — Design Pattern Discovery and Verification for our 23CSE455 case study on DaPlanStan.
The uploaded material includes the original source archive, faculty instructions, report template and the Part 1 handoff artifacts.
Repository: **https://github.com/GeertClaes/daplanstan**
Stated baseline commit:
da54d9ab8bfc118e99375fdd6c506efb20b66e5b
The source ZIP does not contain Git metadata, so do not treat the ZIP-to-commit match as verified.
Also note that the Part 1 human-verification tracker is not completed. Use the Part 1 architecture and evidence notes only as navigation aids. Any design-pattern claim must be independently checked against the source.
My scope is only pattern discovery in the original source. I am not doing evolution, refactoring or cross-language implementation here.
For this first step:

1. Read the faculty requirements and tell me exactly what I need to produce for pattern discovery, pattern counts, participant mapping, evidence and verification.
2. Inspect the repository structure and decide which source files should be examined first. Use the Part 1 handoff as a starting point but verify the files yourself.
3. Give me an inspection order. For each source area, explain what kind of collaboration is worth looking at, without assigning a pattern name yet.
4. Keep application code, Rails-provided behavior and dependency-provided behavior separate throughout the analysis.

5. Tell me what would count as enough evidence to confirm a pattern and what would only be a weak similarity.
6. List any information that is still uncertain before discovery starts.

Do not produce the final pattern inventory yet. I want to establish the method and inspection scope first.

Worked for 2m 54s

DaPlanStan — Design Pattern Discovery and Verification

23CSE455 | Preliminary requirements review and source-inspection plan

1. Important clarifications before discovery

After reviewing the faculty instructions, report template and Part 1 handoff, there are three important distinctions.

First, your team's Part 2 and the faculty's Part 2 are different.

Your team calls your responsibility **Part 2 — Design Pattern Discovery and Verification**. However, the official faculty report places design-pattern discovery under **Section 2: Part 1 Pattern discovery**. Faculty Part 2 concerns evolution and refactoring.

Therefore, your contribution belongs in the **baseline pattern-discovery section of the final report**, even though your team internally calls it Part 2. DP_Csestudy.pdf

Second, the baseline commit is not fully verified.

The stated commit `da54d9ab8bfc118e99375fdd6c506efb20b66e5b` exists in the public GitHub repository. GitHub However, the uploaded source ZIP contains no `.git` metadata, and its correspondence to that commit has not been independently established.

For now, the **uploaded ZIP is the actual source snapshot being inspected**, while the commit remains a stated rather than verified snapshot identity.

Third, Part 1's human verification is incomplete.

The handoff contains an architecture diagram, source-navigation notes and a verification CSV, but the CSV has no completed verification outcomes, reviewers or dates. This means the handoff is useful for selecting files, not for accepting design-pattern evidence.

No design-pattern instances are confirmed at this stage.

2. Exactly what the faculty requires you to produce

The official instructions and report template require a source-supported inventory, quantitative counts, participant mappings, diagrams, and documented verification. There is **no minimum pattern quota**. A small defensible inventory is preferable to a large speculative one. DP_Csestudy.pdf

A. Required deliverables

Deliverable	What you must prepare
Discovery methodology	Explain how source areas were selected, candidates identified and evidence checked
`patterns.csv`	One record per confirmed collaboration in the original code
`pattern_counts.csv`	Per-pattern instance counts, distinct pattern types and total instances
Participant mappings	Map every applicable pattern role to actual code symbols with file paths and line ranges
Pattern evidence	Code excerpts, relationships, invocation paths, behavior and problem solved
Diagrams	Editable UML/component/interaction diagrams and rendered images for important confirmed instances
Candidate review	Separate rejected and uncertain candidates with reasons
Human verification	Reviewer, decision, justification, source references, alternatives and available test results
LLM audit records	Exact prompts, inputs, raw outputs, displayed model/date and human evaluation
Report contribution	Baseline discovery methodology, totals, inventory, detailed instance records and limitations

These are based on faculty instructions, the report template and the team's handoff.
DP_Csestudy_instructions.pdf DP_Csestudy.pdf

B. `patterns.csv` — mandatory schema

EXACT COLUMN ORDER

CSV header Copy

```
source_version,pattern_name,pattern_type,instance_name,participant_roles,source_locations,intent_and_evidence
```

For each confirmed record:

1. `source_version`: `Original code`, with the precise archive identity and unverified commit relationship documented separately.
2. `pattern_name`: Exact recognized design-pattern name, once established.

3. `pattern_type` : Its recognized category, such as creational, structural, behavioral, GRASP or architectural.
4. `instance_name` : A meaningful name describing the specific collaboration.
5. `participant_roles` : Every applicable role mapped to its real class, module, interface or method.
6. `source_locations` : Source path and line range for every participant.
7. `intent_and_evidence` : The problem addressed and the actual interactions showing the proposed implementation.

Do not invent abstract classes or interfaces simply because a textbook diagram contains them. A single code symbol may perform multiple roles.

C. `pattern_counts.csv` — mandatory schema

EXACT COLUMN ORDER

CSV header Copy

```
source_version,summary,pattern_name,count
```

The faculty defines three kinds of summary rows:

`summary` value	Meaning
`per_pattern`	Number of independent confirmed collaborations for that pattern name
`total_pattern_types`	Number of distinct confirmed pattern names
`total_pattern_instances`	Total number of independently confirmed collaborations

A repeated call does not automatically create another instance. Likewise, two interchangeable concrete classes taking part in one collaboration do not create two instances.

The report additionally requires separate totals for **concrete participant classes**, **unique participant symbols**, **rejected candidates** and **uncertain candidates**. Concrete participant classes are classes implementing variable roles; unique participant symbols count distinct mapped symbols only once. Document the deduplication rules explicitly. DP_Csestudy.pdf

D. Verification and supporting records

The required CSVs do not have fields for reviewer, verification status or candidate rejection. Therefore, maintain separate supporting evidence records rather than changing the mandatory CSV headers.

A practical verification record should capture the candidate ID, proposed classification, source snapshot, origin, all participating symbols, exact code references, alternative explanations, reviewer's name, decision, date, and evidence/test paths.

For `llm.csv`, the faculty requires:

```
part,model_and_date,prompt_file,input_file,output_file,evaluation,verification_file
```

Record the real interaction and actual human correction. Never present an unperformed test or an unreviewed AI suggestion as verified. DP_Csestudy_instructions.pdf

Submission detail: The instructions specify a single team submission, not independent submissions for each teammate. The primary CSV specification uses `pattern_counts.csv`, while an illustrative folder hierarchy uses a different spelling. Use the explicitly defined CSV schema and agree on one canonical filename with the team before integration.

3. What I verified in the uploaded source

I inspected the archive's directory structure and read the application code behind the main collaboration flows identified by Part 1.

Source snapshot Static inspection completed

16 Ruby controller files	**14** Model and concern files	**5** Background job files
2 Mailbox files	**3** Mailer files	**18** Stimulus controller files

Counts are from the uploaded archive, including the three admin controllers and the single model concern. They are source-file counts, not design-pattern counts.

The archive includes 38 migrations. It has no `test/` or `spec/` directory. Its CI configuration contains static security, dependency and lint checks, but not an application test suite. I have not run the Rails application or its integrations.

The source specifies Ruby 3.4.2 and resolves Rails 8.1.3 in `Gemfile.lock`. These are declared versions, not confirmation of an installed runtime.

For reproducibility, the uploaded archive's SHA-256 is:

DAPLANSTAN-MAIN.ZIP

```
69af480a1a3bfecc7580b3b6b7b8f44f4df2414315d5434c8e40c67eb63475a9
```

This identifies the archive bytes, not the upstream Git commit.

4. Recommended inspection order

I recommend beginning with smaller collaborations whose participating methods are relatively easy to trace, then moving to workflows spanning more components.

The following is an **inspection sequence, not a pattern classification or pattern ranking**.

01 **Source boundaries and request entry points**

`config/routes.rb` · `app/controllers/application_controller.rb` · `Gemfile.lock` · `db/structure.sql`

Establish where requests enter the application, which common methods controllers inherit, how permissions are checked and which relationships belong to the current database schema.

This establishes context before investigating individual collaborations.

02 **Geocoding and reusable model behavior — first focused investigation**

`app/models/concerns/geocodable.rb` · `app/models/trip_item.rb` · `app/jobs/geocode_trip_items_job.rb` · `app/controllers/trips_controller.rb` · `config/initializers/geocoder.rb`

Examine how shared functionality is included, how the model adapts that behavior, how geocoding is triggered, and where external lookup begins.

The source explicitly contains `include Geocodable`, a model method that invokes `super`, a conditional model callback and a background job calling the model method. Investigate the exact responsibilities and behavior across these boundaries.

03 **Inbound email processing and confirmation**

`app/mailboxes/application_mailbox.rb` · `app/mailboxes/trip_mailbox.rb` · `app/jobs/parse_inbox_item_job.rb` · `app/models/inbox_item.rb` · `app/controllers/inbox_items_controller.rb` · `app/controllers/approved_senders_controller.rb`

Follow how an incoming email is associated with a trip, how sender approval affects processing, how extracted information is stored, and which action creates linked itinerary or expense records.

An important distinction is already visible: `ParseInboxItemJob#perform` stores parsed data and updates the parsing status. `InboxItemsController#update` separately calls the confirmation or relinking methods. These must not be collapsed into one automatic operation.

‹/›

04 **Trip creation, membership and permissions**

`app/controllers/trips_controller.rb` · `app/models/trip.rb` · `app/models/trip_member.rb` · `app/models/traveler.rb` · `app/controllers/trip_members_controller.rb` · `app/models/account.rb` · `app/models/user.rb`

Trace how trip creation coordinates owner membership, how travelers relate to trips and how different controller actions enforce access conditions.

Inspect real association paths, decision points and method calls. Do not infer interchangeable behavioral objects merely because the membership model has multiple role values.

05 **Itinerary import, record linking and expense handling**

`app/controllers/trip_items_controller.rb` · `app/models/inbox_item.rb` · `app/models/trip_item.rb` · `app/models/expense.rb` · `app/controllers/expenses_controller.rb` · `app/jobs/parse_receipt_job.rb`

Compare the application-owned creation paths for manually entered items, imported items and confirmed email data.

Investigate how existing records are located, how missing information is filled, how duplication is avoided, and how receipt data updates an expense. Check whether any behavior is genuinely delegated or whether it is ordinary conditional application logic.

06 **Other external integrations and account identity**

`app/jobs/fetch_trip_cover_image_job.rb` · `app/controllers/sessions_controller.rb` · `app/models/user_identity.rb` · `config/initializers/omniauth.rb` · `app/mailers/trip_mailer.rb` · `app/mailers/invitation_mailer.rb`

Examine what the application itself coordinates when talking to external services, which behavior is delegated to installed libraries, and how results update domain records.

Treat authentication provider configuration, outbound mail delivery and cover-image retrieval as separate workflows unless shared participant relationships are actually demonstrated.

07 **Browser-side component collaboration**

`app/javascript/controllers/timeline_controller.js` · `map_controller.js` · `trip_nav_controller.js` · `location_picker_controller.js` · relevant `app/views/trips/` templates

Trace event producers, listeners, data attributes and component state updates.

Source inspection found application-defined events such as `map:locate`, `map:reset` and `map:resize`, with handlers registered in the map controller. Determine the complete event path through the views and JavaScript before considering any classification.

08 **Remaining application source and coverage check**

Remaining controllers · helpers · frontend components · model and mailer files · application initializers · schema and migrations

Search for collaborations not represented in the Part 1 handoff. Also check callers and declarations outside the initially selected files.

The existing schema should establish current persistent entities. Older migrations provide historical context, not proof that deleted entities remain active participants.

Why this order?

It starts with easily traceable relationships before exploring longer workflows. It also includes a later source-wide sweep so that the Part 1 handoff does not artificially determine which patterns can be discovered.

The objective is to identify complete collaborations, not isolated classes with interesting names.

5. Keeping application, framework and dependency behavior separate

For every candidate, maintain an origin classification throughout the evidence record.

Origin	What belongs here	How to treat it
Application code	DaPlanStan's Ruby and JavaScript methods, custom concerns, controllers, jobs, mailboxes and domain rules	Primary evidence for application-owned collaborations
Rails-provided behavior	Active Record association machinery, callbacks, Rails routing, Active Job scheduling, Action Mailbox dispatch, Turbo and Stimulus lifecycle support	Explain its role, but do not claim Rails internals are implemented by DaPlanStan
Dependency/provider behavior	Geocoder, Nominatim, Anthropic, OmniAuth, Postmark, Pexels and Leaflet	Distinguish the application's integration code from the behavior implemented externally

For example, `Geocodable#geocode_with_bias` is application-written Ruby code, while `Geocoder.search` comes from a library. Both are relevant to understanding the workflow, but their implementation origins differ.

Similarly, an application job's `perform` method belongs to DaPlanStan, whereas background scheduling and execution infrastructure are supplied by Rails and its job backend.

A candidate that exists primarily in framework or library internals should not be counted as an application-implemented instance. If the faculty accepts a framework idiom, identify its origin explicitly and keep the counting rule transparent.

6. What counts as sufficient evidence?

The standard should be stricter than finding code that resembles a textbook example.

Evidence strength

Strong evidence — suitable for confirmation after review

A recognizable design problem is supported by the code's structure and behavior. All necessary participants are identifiable, their interactions can be traced through actual methods or declarations, and the explanation matches the intended pattern mechanism rather than merely its appearance.

Exact source lines, meaningful alternatives and an independent human decision support the claim. Tests or execution logs strengthen behavioral claims where available.

Uncertain evidence — keep under investigation

Some required relationships are present, but an essential participant, behavioral mechanism or actual invocation remains unclear.

This also covers cases where the implementation might be ordinary application coordination rather than a particular design pattern.

Weak similarity — do not confirm

A familiar class name, controller inheritance, a framework macro, a simple method call, an enum, an API request or a diagram resemblance without the required collaboration.

These observations may lead to investigation, but they are not independently sufficient evidence.

Verification procedure for each future candidate

1. State the problem the proposed collaboration appears to solve.
2. Locate every participating symbol, not just the most visible class.
3. Follow the actual declarations, method calls, delegation and relevant conditional paths.
4. Compare the observed mechanism with the recognized pattern's essential characteristics and consider simpler explanations.
5. Record file paths, exact line ranges, source excerpts and the behavior supported by those excerpts.
6. Separate static source confirmation from runtime confirmation. Record actual tests only when performed.
7. Obtain an independent human review and classify the candidate as confirmed, rejected or uncertain, with reasons.

Only **confirmed collaborations** should enter `patterns.csv`. Derive the instance counts from those records afterward, instead of estimating counts during discovery.

This avoids counting the same collaboration multiple times merely because it has multiple participants, call sites or concrete implementations.

7. Information that remains uncertain

The following issues should remain visible in the discovery notes rather than being silently resolved.

Open issue	Current position
Source ZIP versus stated commit	Commit exists, but exact archive-to-commit correspondence is unverified
Part 1 human verification	Tracker is unfilled; previous architecture claims lack recorded independent sign-off
Application startup	Not executed or verified during this review

Automated tests	No application `test/` or `spec/` suite found in the supplied snapshot
External integrations	Actual credentials, API responses and successful runtime behavior are unverified
Full candidate coverage	Initial file discovery and source tracing completed, but exhaustive candidate evaluation has not started
Pattern acceptance boundaries	Exact treatment of framework idioms and dependency-origin patterns must be explicit
Concrete-class count rule	Cross-instance deduplication must be standardized and documented
Faculty approval and submission naming	Approval evidence was not established; folder and filename examples contain inconsistencies
Human verification for new claims	Student reviewer, review dates and signed/recorded decisions are still pending

8. Completion criteria for this preliminary step

Current preparation checklist 7 of 9

- [x] Faculty deliverables and required CSV schemas identified
- [x] Team-versus-faculty part numbering clarified
- [x] Uploaded source archive and major source directories inspected
- [x] High-priority source relationships independently traced
- [x] Application/framework/dependency boundaries established
- [x] Evidence acceptance and rejection criteria defined
- [x] Remaining uncertainties recorded
- [ ] Baseline pattern claims verified by a human reviewer
- [ ] Final pattern inventory and counts produced

The appropriate next step is the first focused investigation: tracing the geocoding collaboration from its initiating code through the model, concern, job and external library boundary.

At that stage, each suspected pattern can be evaluated independently, with rejected alternatives preserved as evidence rather than removed from the record.

Current conclusion: The methodology and initial inspection scope are established. No pattern names, pattern instances, participant counts or final inventory have been assigned. That is the correct

boundary for this first step.

Let us start with the first source area you recommended: geocoding.
For now, do not try to force a pattern name onto it. I want to understand the collaboration first.
Inspect these files and any directly related callers you need:

1. `app/models/concerns/geocodable.rb`
2. `app/models/trip_item.rb`
3. `app/jobs/geocode_trip_items_job.rb`
4. `app/controllers/trips_controller.rb`
5. `config/initializers/geocoder.rb`

Trace the geocoding flow from the point where the application triggers it up to the external geocoding boundary.
For this investigation:

1. Identify the actual classes, modules and methods involved.
2. Show how control moves between them.
3. Separate code written by DaPlanStan from behavior supplied by Rails and the Geocoder dependency.
4. Record the exact source paths and reliable line ranges for the important interactions.
5. Explain the design problem this collaboration is solving.
6. Only after tracing the collaboration, list any design pattern that could reasonably describe it.
7. For each possible pattern, explain both why it might fit and why it might be a false positive.
8. Do not confirm anything yet. Mark each possible classification as a candidate only.
9. If the collaboration is simply shared behavior, framework usage or ordinary delegation rather than a recognized pattern, say so.

At the end give me:

1. The complete geocoding collaboration trace.
2. The relevant source evidence.
3. Candidate pattern classifications, if any.
4. The strongest alternative explanation.

5. What still needs to be checked before any candidate can be confirmed.

Worked for 1m 40s

Geocoding Collaboration — Source Investigation 01

Candidates only DaPlanStan original source ZIP · Static source analysis

1. Main finding

The geocoding implementation has **three distinct triggering paths**, rather than a single controller-to-job workflow.

1. **Manual request:** A user selects *Fix missing pins* on the trip page. `TripsController` identifies records without coordinates and schedules `GeocodeTripItemsJob`.
2. **Bulk import:** `TripItemsController` imports itinerary items, temporarily disables the model's geocoding callback, then schedules the same background job for newly created records needing coordinates.
3. **Ordinary record saving:** The `Geocodable` concern registers an Active Record `after_save` callback. This can invoke geocoding during a normal `TripItem` save without passing through `GeocodeTripItemsJob`.

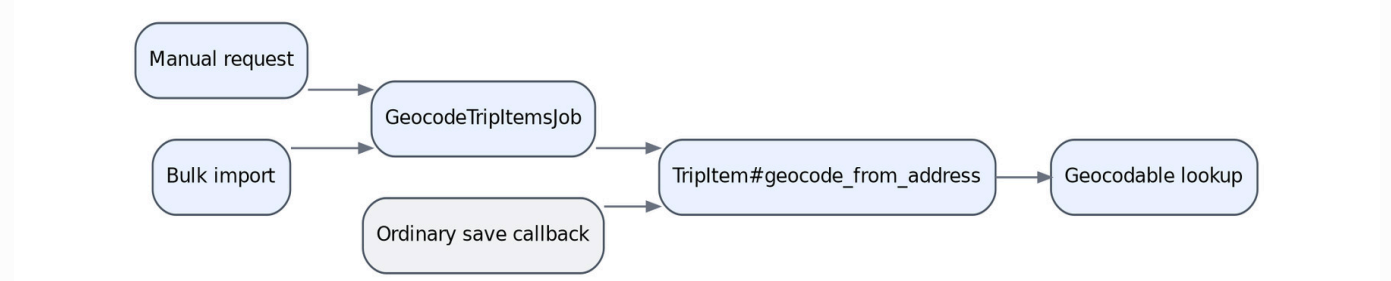
These paths ultimately reach the same application-level operation: `TripItem#geocode_from_address`, which delegates to the implementation supplied by the `Geocodable` module. ‹/› ‹/›

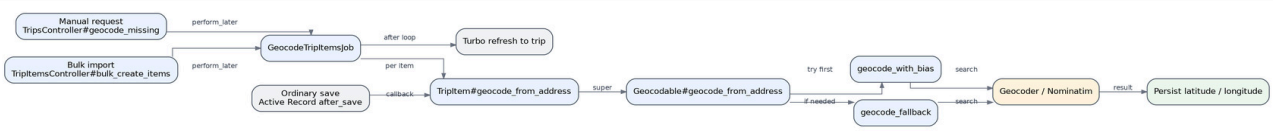
This distinction matters because **not every geocoding operation is asynchronous**. The manual and bulk-import paths enqueue a job, whereas the model callback is part of the normal record-save lifecycle.

The strongest initial explanation is shared model behavior combined with Rails callbacks, background-job orchestration and a Geocoder library integration. A recognized GoF pattern has not yet been demonstrated.

2. Complete geocoding collaboration trace

Control-flow overview





The two job-driven entry points converge at the same model method used by the normal save callback. The diagram shows logical calls, not a verified runtime execution.

Path A — Manual geocoding

The route is defined inside the `resources :trips` member block, under the application's `whats.` host constraint.

The user-facing button in `app/views/trips/show.html.erb:53–68` submits a POST request when missing-coordinate items exist. The controller's `show` method calculates that count using `!i.geocoded?` at lines 11–28.

The request then proceeds as follows:

1. Rails routes the request to `TripsController#geocode_missing`.
2. Controller authentication and `set_editable_trip` run first. The lookup restricts the trip to a user with an owner or planner membership.
3. The action selects IDs of trip items that fail `geocoded?`.
4. If any exist, the controller calls `GeocodeTripItemsJob.perform_later(missing_ids, @trip.id)`.
5. Rails Active Job supplies the queueing mechanism.
6. When the job executes, its `perform` method loads the trip and individually finds each item.
7. The job invokes `item.geocode_from_address`.

The application does not call the external geocoding service directly from the controller. ‹/› ‹/›

Path B — Bulk import

`TripItemsController#import` calls `bulk_create_items` after confirming the imported JSON.

The import method temporarily disables the `TripItem` save callback, creates the records, collects IDs without complete coordinates, and enqueues those IDs for the same background job. Its `ensure` block calls `TripItem.set_callback` afterward.

This is an important second entry path. It also introduces a question for later verification: the callback is restored without explicitly repeating its original conditional predicate. The actual post-import callback configuration should be inspected before making a behavioral claim about it. ‹/›

Path C — Automatic geocoding during saving

The `Geocodable` module registers:

‹/› Ruby

```ruby
after_save :geocode_from_address,
  if: -> {
    (saved_change_to_address? || saved_change_to_name?) &&
      !geocoded?
  }
```

This condition is true when the saved address or name changed and both latitude and longitude are not already present.

The callback can be reached through ordinary `TripItemsController#create` and `#update` operations. The source also contains `InboxItem` methods that create trip items, so those records can pass through the same model lifecycle. However, `InboxItem#fill_missing_links!` updates existing records using `update_columns`, which bypasses the normal callback. ‹/› ‹/›

Rails provides callback execution. The application supplies the callback condition and method.

3. How the model and concern collaborate

`TripItem` includes the `Geocodable` concern and defines its own method:

‹/› Ruby

```ruby
def geocode_from_address
  return if TRANSPORT_KINDS.include?(kind.to_s) && address.blank?
  super
end
```

Here, `TRANSPORT_KINDS` contains `flight`, `car`, `train` and `ferry`.

The model applies a domain-specific guard to avoid geocoding transport-company or transport-item names when no real address is supplied. Otherwise, `super` invokes the implementation found in the included concern.

This is ordinary, valid Ruby method lookup and module reuse. It is not, by itself, proof of any GoF pattern.

The concern then performs the actual lookup algorithm.

1. **Try the address-based lookup**

   `Geocodable#geocode_with_bias`, lines 27–45, extracts a location hint from the address, looks it up, then requests a full-address result with latitude/longitude parameters when a reference was found.

   If a reference and result are available, the method uses `Geocoder::Calculations.distance_between` and accepts the result only if the distance is under 50 km. When the hint lookup fails, it tries the full address without that distance check.

   The code attempts geographic biasing. Whether the configured Nominatim service meaningfully applies the supplied `lat` / `lon` parameters has not been verified.

2. **Try fallback queries**

   `Geocodable#geocode_fallback`, lines 46–67, builds alternative search strings from address components, the item's name combined with its trip title, and the item's name alone.

   For each query, it takes the first result. If another geocoded stop establishes a trip center, a fallback result more than 2,000 km from that center is rejected.

   `trip_geocoded_center`, lines 69–75, calculates an arithmetic mean of other trip items' saved latitude and longitude values.

3. **Persist coordinates or log failure**

   `Geocodable#geocode_from_address`, lines 13–24, runs the first method and, if it returns no result, the fallback method.

   If a result is available, it stores latitude and longitude using `update_columns`. Otherwise it logs a warning. Exceptions are rescued and logged as warnings.

   Rails documents that `update_columns` bypasses validations and callbacks. This prevents the coordinate write from starting another normal save-callback cycle. api.rubyonrails.org

These three stages are defined in `app/models/concerns/geocodable.rb:13-75`.

The external service boundary

The application calls `Geocoder.search` and `Geocoder::Calculations.distance_between`. The installed Geocoder dependency supplies those APIs; DaPlanStan does not implement the underlying

geocoding HTTP client in these files. The dependency documentation describes search queries, results and the pass-through `params` option. RubyDoc

The initializer specifies:

```ruby
Geocoder.configure(
  lookup:      :nominatim,
  use_https:   true,
  units:       :km,
  timeout:     10,
  http_headers: {
    "User-Agent" =>
      "Daplanstan/1.0 (#{ENV.fetch("POSTMARK_FROM_EMAIL", "noreply@daplanstan.com")})"
  }
)
```

Source: `config/initializers/geocoder.rb:1-7` . The lockfile declares Geocoder 1.8.6.

This establishes the configured service boundary, not evidence that any live request succeeded.

4. Complete participant and source-evidence register

All paths below are relative to the root of the uploaded source archive. Line numbers refer to that exact extracted snapshot, not a verified Git commit.

Actual symbol or interaction	Source location	Responsibility
Trip page button	`app/views/trips/show.html.erb:53-68`	User trigger for missing pins
Route declaration	`config/routes.rb:21-29`	Maps POST request to controller
`TripsController#geocode_missing`	`app/controllers/trips_controller.rb:94-103`	Selects missing items and enqueues job
`TripsController#set_editable_trip`	`app/controllers/trips_controller.rb:111-118`	Restricts trip access
`TripItemsController#bulk_create_items`	`app/controllers/trip_items_controller.rb:128-174`	Batch import, callback control and job queueing
`GeocodeTripItemsJob#perform`	`app/jobs/geocode_trip_items_job.rb:4-22`	Executes per-item geocoding

`TripItem` concern inclusion	`app/models/trip_item.rb:1–7`	Makes shared methods available
`TripItem#geocode_from_address`	`app/models/trip_item.rb:32–37`	Transport-item guard and `super`
`Geocodable` callback declaration	`app/models/concerns/geocodable.rb:1–7`	Registers automatic invocation
`Geocodable#geocoded?`	`app/models/concerns/geocodable.rb:9–11`	Determines coordinate completeness
`Geocodable#geocode_from_address`	`app/models/concerns/geocodable.rb:13–24`	Coordinates lookup and persistence
`Geocodable#geocode_with_bias`	`app/models/concerns/geocodable.rb:27–45`	Primary lookup and distance check
`Geocodable#geocode_fallback`	`app/models/concerns/geocodable.rb:46–67`	Alternative query sequence
`Geocodable#trip_geocoded_center`	`app/models/concerns/geocodable.rb:69–75`	Computes reference location
Geocoder configuration	`config/initializers/geocoder.rb:1–7`	Selects lookup provider and options
`Turbo::StreamsChannel.broadcast_refresh_to`	`app/jobs/geocode_trip_items_job.rb:20–21`	Requests a trip-page refresh

The major source declarations and calls have been cross-checked against the extracted files and direct application references. ‹/›

5. Application code versus supplied behavior

Layer	What is owned by DaPlanStan	What is supplied externally
Request	Controller actions and authorization queries	Rails routing, callbacks and controller infrastructure
Asynchronous processing	Job's `perform` method, loop and delays	Active Job queueing and execution machinery
Model	Transport guard, query construction and persistence decisions	Active Record persistence, associations and callbacks
Reuse	`Geocodable` methods and their application logic	Ruby module inclusion and ActiveSupport concern functionality
Geocoding	Query choice, 50-km and 2,000-km rules	Geocoder searches, distance calculation and Nominatim integration

Refresh	Request to broadcast a trip refresh	Turbo stream distribution and client refresh machinery

The central source of application-defined geocoding logic is `Geocodable`. The framework supplies infrastructure around it; the Geocoder dependency supplies lookup functionality beyond it.

6. The design problem being solved

DaPlanStan stores itinerary stops that may have full addresses, partial addresses, place names or no usable coordinates.

The collaboration is designed to convert that incomplete location information into coordinates that can be persisted and displayed on the trip map.

It attempts to solve several practical problems: avoiding obviously unsuitable transport-name lookups, using context to improve ambiguous searches, trying alternative search strings, avoiding excessive distance from known trip stops, and processing batches away from the initiating controller request.

After a job completes its loop, it requests a Turbo refresh. The trip page provides the coordinate data to its map controller, which creates Leaflet markers only for items with latitude and longitude values. ‹/›

The source establishes these design intentions. It does not establish the accuracy or operational reliability of the resulting coordinates.

7. Possible design-pattern classifications

Only now, after tracing the collaboration, are pattern names considered.

Candidate A — Template Method

Unconfirmed

Why it might fit: `TripItem#geocode_from_address` performs a preliminary check and invokes `super`, allowing the included concern's method to perform the main operation. That resembles behavior specialization around a shared implementation.

Why it might be a false positive: There is no clearly established invariant template algorithm with designated subclass-overridden primitive steps or hooks. The model simply guards an inherited module method.

Assessment: Weak resemblance; more naturally explained as method overriding and module reuse.

Candidate B — Strategy

Unconfirmed

Why it might fit: The implementation uses two lookup approaches, `geocode_with_bias` and `geocode_fallback`, depending on whether the earlier approach succeeds.

Why it might be a false positive: There are no interchangeable strategy objects, shared behavioral interfaces or runtime selection of independently encapsulated algorithms. Both methods are hardcoded into a single concern.

Assessment: Weak resemblance; fixed sequential fallback is a stronger description.

Candidate C — Adapter Unconfirmed

Why it might fit: The application uses a domain-oriented method, `geocode_from_address`, while the concern handles calls to the third-party `Geocoder.search` API and translates returned coordinates into model updates.

Why it might be a false positive: The concern does more than interface conversion, and there is no explicit incompatible target interface or separate adapter/adaptee object collaboration. It is simply an integration boundary offering convenient methods.

Assessment: Plausible superficial resemblance; insufficient evidence for a GoF Adapter.

Candidate D — Observer Unconfirmed

Why it might fit: The application reacts to model saves using callbacks and requests a Turbo refresh after background processing.

Why it might be a false positive: Rails already supplies callback dispatch, and Turbo supplies refresh broadcasting. The inspected application code does not show a separate application-defined subject/observer registration and notification collaboration.

Assessment: Framework event mechanisms are visible, but an application-implemented Observer has not been established.

No candidate is approved for the baseline pattern inventory. These are hypotheses to evaluate or reject, not four identified pattern instances.

8. Strongest alternative explanation

The geocoding collaboration is a shared model concern combined with procedural fallback logic, Rails lifecycle callbacks and optional background-job execution.

This explanation accounts for the observed code without inventing additional abstractions.

Specifically, `Geocodable` centralizes the reusable lookup and persistence behavior; `TripItem` provides a domain-specific guard; `TripsController` and `TripItemsController` initiate batch work; and the Geocoder gem provides the external lookup implementation.

A concern and a reusable method do not automatically constitute a recognized GoF design pattern.

9. Important unresolved behavior and verification work

There are several source-supported questions worth investigating before confirming any classification or claiming runtime correctness.

Issue	Why it matters
Callback restoration	The importer re-registers the callback without its original condition; inspect the resulting callback chain
Global callback modification	`skip_callback` changes class-level behavior; investigate possible interference between concurrent requests
Rate limiting	The job sleeps 1.1 seconds per item, but an item may cause several `Geocoder.search` calls, so per-request spacing is not proven
Job success logging	The job logs after invoking geocoding, even when the concern returns without coordinates or rescues an exception
External query parameters	Verify how Nominatim handles the `lat` and `lon` values sent through Geocoder's `params` option
Actual results	No geocoding responses or successful execution logs were examined
Pattern mechanism	No independently established interchangeable strategy, adapter contract, observer registration or template-hook protocol
Reviewer decision	No human review outcome has been recorded for any of these candidates

The callback-restoration and rate-limit issues are especially useful verification observations. They should be recorded as possible behavioral risks, not asserted as observed defects without tests.

Suggested verification cases

For subsequent human verification, the most informative cases would be a normal item with an address, a transport item with no address, an ambiguous address requiring fallback, an item with manual coordinates, a failed external lookup, and a bulk import followed by a normal save.

These would help confirm the flow's behavior, although passing functional tests alone would still not prove a particular design-pattern classification.

Investigation result

Geocoding investigation status Pending verification

Source trace	Established statically
Participant locations	Recorded
Pattern hypotheses	4 tentative classifications
Confirmed instances	None
Runtime verification	Not performed

Recommendation for the next investigation: Examine the callback and `super` method-resolution behavior first, then the actual structure of the two lookup approaches. That will help decide whether any recognized pattern is supported or whether this should be documented simply as framework-supported shared behavior.

For now, nothing from this geocoding investigation should be added to `patterns.csv` or its confirmed pattern counts.

Before moving to another part of the repository, I want to close the geocoding investigation properly.
You identified four possible classifications:

1. Template Method
2. Strategy
3. Adapter
4. Observer

None is currently confirmed.
Re-examine these four classifications using the source evidence already traced.
For each one:

1. State the essential mechanism that must exist for that pattern to be defensible.
2. Compare that requirement directly with the DaPlanStan implementation.
3. Identify the actual code evidence that supports the classification.
4. Identify the code evidence that works against it.
5. Decide whether the observed structure is better explained by ordinary Ruby/Rails behavior or third-party integration.
6. Give a final status of either REJECTED or UNCERTAIN. Do not mark anything CONFIRMED unless the source clearly

establishes the defining pattern mechanism.

7. If the result is UNCERTAIN, state exactly what missing source or runtime evidence would be needed to settle it.

At the end, give me one table:
Candidate | Required mechanism | What the source actually does | Final decision | Reason
Then give me the final geocoding conclusion that should be preserved in the candidate-review record.
Do not search for new patterns in this prompt and do not move to another source area yet.

Worked for 1m 36s

Geocoding Investigation — Final Candidate Review

DaPlanStan | Original source snapshot | 23CSE455

Re-examining the four classifications against their defining mechanisms produces the following preliminary decisions:

Template Method	REJECTED
Strategy	REJECTED
Adapter	REJECTED
Observer	UNCERTAIN

These are source-review assessments, not completed human verification decisions. Observer remains uncertain only as a framework-mediated Turbo Streams collaboration, not as a custom observer implementation.

One important detail deserves attention: the source contains both `turbo_stream_from @trip` in the trip view and `Turbo::StreamsChannel.broadcast_refresh_to(trip)` in the geocoding job. Unlike a callback alone, that pair provides concrete evidence of an intended publish-subscribe relationship. It merits further checking before dismissing Observer entirely. ‹/› ‹/›

1. Template Method — REJECTED

Essential mechanism: A superclass or inherited implementation defines the fixed structure of an algorithm and delegates selected steps to overridable operations, allowing subclasses or specialized implementations to vary those steps without replacing the algorithm's overall structure.

Supporting source evidence:

`TripItem` includes `Geocodable` at `app/models/trip_item.rb:2` and defines its own `geocode_from_address` method at lines 34–37.

```ruby
def geocode_from_address
  return if TRANSPORT_KINDS.include?(kind.to_s) && address.blank?
  super
end
```

The use of `super` superficially resembles overriding within an inherited algorithm.

Also, `Geocodable#geocode_from_address` at lines 13–23 defines a fixed sequence that invokes `geocode_with_bias` and `geocode_fallback`.

Evidence against the classification:

- `TripItem` overrides the entry method itself, not a designated algorithm step invoked by the concern's main method.
- Neither `geocode_with_bias` nor `geocode_fallback` is overridden by `TripItem`.
- The inspected source contains no other class including `Geocodable` or alternate implementation of those lookup steps.
- No explicit specialization protocol or customizable algorithm skeleton is demonstrated.

Better explanation: Ruby module inclusion, method overriding and a guard clause around reused functionality.

Final decision: REJECTED. The source demonstrates inherited method reuse, not the defining Template Method collaboration.

2. Strategy — REJECTED

Essential mechanism: A family of alternative algorithms is encapsulated behind a common behavioral contract. A context delegates to a selected implementation, allowing the algorithm to vary independently from the context.

Supporting source evidence:

At `app/models/concerns/geocodable.rb:13-14`:

```ruby
def geocode_from_address
  result = geocode_with_bias || geocode_fallback
```

Two different lookup approaches exist:

- `geocode_with_bias` — lines 27–44.
- `geocode_fallback` — lines 46–66.

The first attempts an address lookup with geographic context; the second tries alternative search queries.

Evidence against the classification:

- Both algorithms are methods of the same `Geocodable` module.
- There are no separate strategy objects or classes.
- No common interchangeable strategy contract is defined.
- The application does not select an algorithm implementation through configuration or composition.
- The `||` operator creates a fixed sequence: attempt the first method, then the second if needed.

Better explanation: Ordinary conditional fallback logic implemented through private methods.

Final decision: REJECTED. Having two different approaches is not sufficient to establish Strategy. The defining interchangeable algorithm collaboration is absent.

3. Adapter — REJECTED

Essential mechanism: An adapter translates an existing component's interface into a different interface expected by its client, allowing otherwise incompatible interfaces to collaborate.

Supporting source evidence:

At `app/models/concerns/geocodable.rb:32-35`, the application calls:

```ruby
ref = Geocoder.search(city_hint).first
return Geocoder.search(address).first unless ref

result = Geocoder.search(
  address,
  params: { lat: ref.latitude, lon: ref.longitude }
).first
```

The application exposes a domain-oriented method, `geocode_from_address`, and internally calls the Geocoder API. It subsequently stores returned latitude and longitude values using Active Record.

The initializer at `config/initializers/geocoder.rb:1-7` configures Geocoder to use Nominatim. ‹/› ‹/›

Evidence against the classification:

- No incompatible client/adaptee interfaces are shown.
- No explicit target interface is defined that the wrapper is required to satisfy.
- `Geocodable` does not primarily translate between interface contracts.
- Its main responsibility is implementing application-specific lookup rules and persistence decisions.
- It directly uses the Geocoder gem's existing API.

The Geocoder gem may itself abstract different lookup providers, but that is dependency-supplied behavior, not evidence that DaPlanStan implements an Adapter.

Better explanation: Third-party API integration encapsulated in a reusable model concern.

Final decision: REJECTED. Wrapping an API call does not automatically create an Adapter pattern.

4. Observer — UNCERTAIN

Essential mechanism: A subject maintains or uses a notification relationship with one or more observers. When a relevant event occurs, subscribed observers are notified and can update their state or behavior.

This can be implemented directly through application objects or indirectly through a framework's subscription and event-delivery infrastructure.

Supporting source evidence

There are two mechanisms that initially resemble Observer.

A. Active Record callback

At `app/models/concerns/geocodable.rb:4-7` :

‹/› Ruby

```ruby
included do
  after_save :geocode_from_address,
    if: -> {
      (saved_change_to_address? || saved_change_to_name?) &&
        !geocoded?
```

```
    }
end
```

A save event triggers application-defined behavior when the callback condition is satisfied.

However, this is a Rails lifecycle callback. The source does not demonstrate an application-owned subject managing observers or notifying registered observer objects.

B. Turbo Streams subscription and refresh

At `app/views/trips/show.html.erb:2`:

‹/› erb

```
<%= turbo_stream_from @trip %>
```

At `app/jobs/geocode_trip_items_job.rb:20–21`:

‹/› Ruby

```
# Broadcast a page refresh so map pins appear without the user doing anything
Turbo::StreamsChannel.broadcast_refresh_to(trip)
```

The job retrieves the `Trip` using its ID at lines 4–6, so its broadcast refers to the same logical trip resource used by the trip view.

This is stronger evidence than a generic callback: the application declares a stream subscription and requests a broadcast for the corresponding resource. ‹/› ‹/›

Evidence against confirmation

- Rails supplies the callback mechanism.
- Turbo supplies stream subscriptions, broadcasting and client-side stream handling.
- No independently implemented application-level observer interface, observer registry or notification dispatcher has been identified.
- The source alone does not verify that an active browser subscription actually receives and processes the broadcast.

- The application's use of a framework-supported publish-subscribe mechanism does not necessarily mean it should be counted as an application-implemented GoF Observer instance.

Better explanation

A framework-mediated reactive update mechanism using Rails callbacks and Turbo Streams.

This could be described as an Observer-like framework idiom, but that classification requires a more explicit decision about origin and evidence.

Exactly what remains to be checked

1. **Framework implementation:** Inspect the relevant installed Turbo Rails version's `turbo_stream_from` subscription handling and `broadcast_refresh_to` delivery path to establish how subscribers are registered and notified.
2. **Runtime behavior:** With an active trip page, execute the geocoding job and verify that the client receives and processes a refresh for the matching trip stream. Preserve the result as runtime evidence rather than assume success from method names.
3. **Origin and classification:** Determine whether the faculty's permitted framework-idiom category includes application-configured subscriptions whose implementation is supplied by Turbo, while keeping them separate from application-owned pattern instances.

Final decision: UNCERTAIN.

The Active Record callback alone is insufficient for Observer. The explicit Turbo subscription/broadcast pair supports further examination of a framework-mediated Observer collaboration, but not its confirmation as an application-implemented GoF pattern.

---

5. Final comparison table

Candidate	Required mechanism	What the source actually does	Final decision	Reason
Template Method	Fixed algorithm structure with overridable algorithm steps	`TripItem` adds a guard and calls `super` into the shared concern	**REJECTED**	No demonstrated overriding of designated algorithm steps
Strategy	Interchangeable algorithm implementations behind a shared contract	Two private lookup methods are called in a fixed fallback sequence	**REJECTED**	No independently selectable strategies or strategy contract
Adapter	Translation between incompatible interfaces	Application methods invoke Geocoder and	**REJECTED**	Ordinary third-party integration without

		persist returned results		demonstrated interface incompatibility
Observer	Registered or subscribed observers receive notifications when a subject changes	Active Record callback plus Turbo trip subscription and refresh broadcast	**UNCERTAIN**	Framework-mediated subscription is visible, but its implementation, runtime delivery and classification boundary remain unverified

6. Final geocoding conclusion for the candidate-review record

Writing

Geocoding Collaboration — Candidate Review

Source: DaPlanStan original source archive
Scope: Geocodable concern, TripItem model, GeocodeTripItemsJob, TripsController, related import controller, Turbo trip view and Geocoder configuration.

Review status: Static source analysis completed; independent human verification pending.

Observed collaboration:

The geocoding functionality is implemented through a shared `Geocodable` concern included in the `TripItem` model. The model adds a transport-specific guard before delegating to the concern's implementation. The concern performs a primary location lookup followed by fallback queries when necessary and persists the resulting coordinates.

Geocoding can be triggered through an Active Record save callback or through an application-defined background job initiated by controller actions. The application calls the Geocoder dependency for location searches and uses Turbo Streams to request a page refresh following job processing.

Candidate decisions:

1. Template Method — REJECTED. The implementation demonstrates Ruby method overriding and module reuse but not a fixed template algorithm with specialized algorithm steps.
2. Strategy — REJECTED. The lookup methods form a fixed conditional fallback sequence rather than interchangeable strategy implementations.
3. Adapter — REJECTED. The concern integrates an external API without demonstrating the incompatible-interface translation required by Adapter.

4. Observer — UNCERTAIN. Rails callbacks alone do not establish an application-owned Observer. However, the trip view's Turbo stream subscription and the job's matching refresh broadcast provide evidence of a possible framework-mediated Observer idiom. Framework behavior, runtime notification and classification boundaries require further verification.

Strongest alternative explanation:

The implementation is best understood as reusable Ruby model behavior combined with Rails lifecycle callbacks, background-job orchestration, sequential geocoding fallback logic, Geocoder dependency integration and Turbo-based UI refreshing.

Conclusion:

No application-implemented GoF pattern has been established in the geocoding collaboration. Three proposed classifications are rejected on structural grounds. One framework-mediated Observer classification remains uncertain pending targeted verification.

No geocoding instance should be entered in the confirmed `patterns.csv` inventory or included in confirmed pattern counts at this stage. Preserve these decisions in the rejected/uncertain candidate-review evidence and record any subsequent human verification separately.

Limitations: Findings are based on static inspection of the supplied source archive. Archive-to-commit correspondence and runtime behavior remain unverified.

The geocoding investigation is now sufficiently assessed to **close the three rejected hypotheses and retain only the specific Turbo-mediated Observer question as unresolved**. There is no need to reopen the rejected candidates unless new source evidence materially changes their structural assessment.

Now move to the second source area: inbound travel-email processing.
Do not start by assigning a pattern name. First trace what the application actually does.
Inspect these files and any directly related callers you need:

1. `app/mailboxes/application_mailbox.rb`
2. `app/mailboxes/trip_mailbox.rb`
3. `app/models/inbox_item.rb`
4. `app/jobs/parse_inbox_item_job.rb`
5. `app/controllers/inbox_items_controller.rb`

6. `app/controllers/approved_senders_controller.rb`
7. `app/models/approved_sender.rb`
8. Any directly related `TripItem`, `Expense`, `Trip` or routing code needed to complete the flow.

I want the investigation to answer these questions:

1. How does an incoming email enter the application and get associated with a trip?
2. What does `TripMailbox` itself decide or create?
3. What information is stored in `InboxItem` before parsing starts?
4. Exactly what does `ParseInboxItemJob` do?
5. What responsibility belongs to Anthropic and what responsibility remains in DaPlanStan code?
6. Does parsing itself create `TripItem` or `Expense` records, or does that happen later through confirmation/update logic?
7. Trace the exact conditions and call sites for methods such as `fill_missing_links!` or any equivalent record-linking methods.
8. How does approved-sender logic affect the workflow?
9. Which parts are application-written, which are Rails/Action Mailbox/Active Job behavior, and which are dependency/provider behavior?
10. Record the important source paths and reliable line ranges.

Once the collaboration is fully traced, identify any pattern classifications that are genuinely worth investigating.
For each possible pattern:

1. State the candidate pattern.
2. Explain the specific code behavior that suggests it.
3. Explain the strongest reason it may be a false positive.
4. State whether the apparent behavior is mainly application-owned, Rails-provided or dependency-provided.
5. Leave the candidate UNCONFIRMED in this step.

Do not force patterns such as Observer, Command, Chain of Responsibility, Adapter, Strategy or Factory merely because the workflow contains jobs, messages or multiple steps.
At the end give me:

1. Complete inbound-email collaboration trace.
2. Participant/source-evidence table.
3. Clear separation of application, Rails and Anthropic responsibilities.
4. Candidate pattern classifications, if any.
5. Strongest non-pattern explanation of the workflow.
6. Anything that still needs to be traced before candidate verification.

Worked for 2m 27s

Inbound Travel-Email Processing — Source Investigation 02

Pattern classification pending DaPlanStan · Uploaded original source ZIP · Static analysis

1. Main finding: Parsing and creating trip records are separate operations

The source establishes three important boundaries:

1. **Email intake:** `TripMailbox` identifies the trip, checks the sender and creates an `InboxItem`.
2. **AI extraction:** `ParseInboxItemJob` sends email information to Anthropic and saves the resulting structured data in the existing `InboxItem`.
3. **Record creation:** `InboxItemsController#update` separately invokes application methods that create or link `TripItem` and `Expense` records when the user selects confirmation or relinking.

Parsing an email does not itself create itinerary items or expenses. This distinction is directly supported by `ParseInboxItemJob#perform` at lines 99–112 and `InboxItemsController#update` at lines 16–35.

An approved sender allows an email to proceed to parsing, but approval is not the same as confirmation of its extracted travel information.

2. How incoming email enters and is assigned to a trip

The application's mailbox entry point is:

`app/mailboxes/application_mailbox.rb:1–3`

Ruby

```ruby
class ApplicationMailbox < ActionMailbox::Base
  routing(all: :trip)
end
```

The `routing(all: :trip)` declaration tells Rails Action Mailbox to route incoming messages to `TripMailbox`. It does not itself identify a particular trip.

At `app/mailboxes/trip_mailbox.rb:2–9`, `TripMailbox#process` extracts the first recipient address, normalizes it to lowercase and removes surrounding whitespace:

```ruby
to_address = mail.to&.first.to_s.downcase.strip
trip = Trip.find_by(inbound_email: to_address)
```

When no matching trip exists, the method logs a warning and returns. It does not create an `InboxItem` or enqueue parsing in that branch.

Where the recipient address comes from

`app/models/trip.rb:14–19` validates the inbound address and registers a callback to generate one when creating a trip.

`Trip#build_inbound_email`, lines 35–37, constructs an address from the parameterized trip title, a random hexadecimal suffix and the configured email domain.

The mailbox uses the address stored on `Trip`, not the trip title or message subject, to identify the destination.

Important limitation: The code checks the first address in `mail.to`. It does not iterate over every recipient to locate a matching trip.

These relationships are application-written behavior operating on Rails and Action Mailbox objects.

3. What `TripMailbox` decides and stores

After locating the trip, `TripMailbox#process` checks whether the sender is recognized.

At `app/mailboxes/trip_mailbox.rb:11–14`, the sender is approved if **any one** of these conditions is true:

- Their normalized email is recorded in the trip's `approved_senders` association.
- Their email matches a `Traveler` associated with the trip through `TripMember`.
- Their email matches the `User` account associated with a trip member's traveler record.

These are three application-defined database checks, not an Anthropic decision.

At lines 16–24, the mailbox creates an `InboxItem` using `trip.inbox_items.create!`.

Stored field	Source of value
`trip_id`	The matched `Trip` association
`from_email`	First sender address, normalized
`from_name`	Sender's display name, when available
`subject`	Email subject, or `"(no subject)"`
`raw_body`	Decoded text extracted from the email
`raw_html`	Decoded HTML content, when available
`received_at`	Email date, otherwise `Time.current`
`sender_status`	`approved` or `pending_approval`

For multipart email, `extract_text_body` prefers the decoded text part, then falls back to `mail.decoded`. `extract_html_body` attempts to extract the decoded HTML part. Both methods have exception fallbacks.

The source references are `app/mailboxes/trip_mailbox.rb:16-59`.

Other initial state

The database schema supplies these defaults:

Field	Initial state
`parse_status`	`unparsed`
`review_status`	`pending_review`
`parsed_data_json`	`NULL`
`reviewed_by_id`	`NULL`
`reviewed_at`	`NULL`

These defaults appear in `db/migrate/20260328112022_create_inbox_items.rb:3-18` and the current `db/structure.sql:36`.

The schema also includes `attachments_json`, and the model declares `has_many_attached :email_attachments`. However, the inspected `TripMailbox#process` does not populate either attachment representation. Attachment ingestion should not be claimed as implemented by this traced method.

What happens after the inbox record is created?

Recognized sender

`ParseInboxItemJob.perform_later(inbox_item.id)` is called at `app/mailboxes/trip_mailbox.rb:26-27`.

Unrecognized sender

No initial parsing job is enqueued. `notify_owners_of_pending_sender` sends a notification to each owner through `TripMailer.pending_sender_notification(...).deliver_later`, at `app/mailboxes/trip_mailbox.rb:55-59`.

The mailbox therefore coordinates intake, trip lookup, approval checking, inbox persistence and the initial next action. It does not itself extract structured travel details or create destination trip records.

4. Exactly what `ParseInboxItemJob` does

Primary source: `app/jobs/parse_inbox_item_job.rb:1-140`

The job inherits from `ApplicationJob` and declares `queue_as :default`. It also declares a Rails `retry_on StandardError` policy, although its own broad rescue handler is relevant to whether that policy would actually run.

Its implemented sequence is:

1. **Find the inbox record**

   `InboxItem.find_by(id: inbox_item_id)` returns the stored email. If no record exists, `perform` returns.

2. **Construct the prompt**

   `build_prompt` uses the trip title, sender name/email, subject and `raw_body` truncated to 4,000 characters. It does not use `raw_html` or attachments.

3. **Call Anthropic**

   `parse_with_anthropic` creates an `Anthropic::Client` and requests a message from `claude-haiku-4-5-20251001`, with a system prompt, a tool schema and the constructed email prompt.

4. **Extract the structured response**

   The application searches the response content for the first block whose type is `:tool_use`. It returns the block's input as a string-keyed hash, or `{}` when no such block exists.

5. **Save extraction results**

> `inbox_item.update!(parsed_data: data, parse_status: :parsed)` saves the extracted structure through `InboxItem#parsed_data=`.

6 **Handle errors**

On a rescued exception, it logs the failure and attempts to set `parse_status: :failed`.

The system prompt instructs the model to extract clearly stated information. The application-supplied tool schema defines four main output categories:

Structured key	Expected content
`travel_legs`	Flights, trains, ferries, buses and cars
`accommodations`	Stays, check-in/out dates and amounts
`bookings`	Tours, restaurants, tickets and other bookings
`shortlist_items`	Suggested places and activities

These are data returned to the application. The `record_travel_data` tool declaration does not itself call `TripItem.create!` or `Expense.create!`.

One important distinction: the code sets `parse_status: :parsed` even if `parse_with_anthropic` returns an empty hash. It has no visible semantic validation requiring the result to contain a usable travel record.

Anthropic versus DaPlanStan responsibility

Anthropic/provider	DaPlanStan application
Processes the submitted prompt and model instructions	Selects the model and constructs the prompt
Produces a structured tool-use response when successful	Defines the expected tool input schema
Supplies generated travel-information values	Extracts the tool input and saves it to `InboxItem`
Does not directly operate on the Rails database through this call	Later interprets the saved values and creates or updates domain records

Rails Active Job handles background execution, while DaPlanStan supplies the `perform` implementation and record updates.

The precise job logic is at `app/jobs/parse_inbox_item_job.rb:99–139`; the tool schema is at lines 20–97. ‹/›

5. Confirmation and record-linking flow

This is the most important application-owned part of the workflow.

Exact call sites

A source-wide search of the uploaded repository found these executable calls to `fill_missing_links!` and `confirm_and_create_items!`:

Caller	Location	Method invoked
User chooses *Add to trip*	`app/controllers/inbox_items_controller.rb:18–20`	`@item.confirm_and_create_items!(current_user)`
`InboxItem#confirm_and_create_items!`	`app/models/inbox_item.rb:34–35`	`fill_missing_links!(added_by)`
User chooses *Fix missing links*	`app/controllers/inbox_items_controller.rb:25–26`	`@item.fill_missing_links!(current_user)`

No call from `ParseInboxItemJob` to either method was found.

A. What happens when the user selects Add to trip?

The action is initiated from `app/views/inbox_items/show.html.erb:53–62` and handled by `InboxItemsController#update`.

Under the `confirm` branch, the controller first calls `confirm_and_create_items!` and then records `review_status: :confirmed`, `reviewed_by` and `reviewed_at`.

Within `InboxItem#confirm_and_create_items!`, lines 34–58:

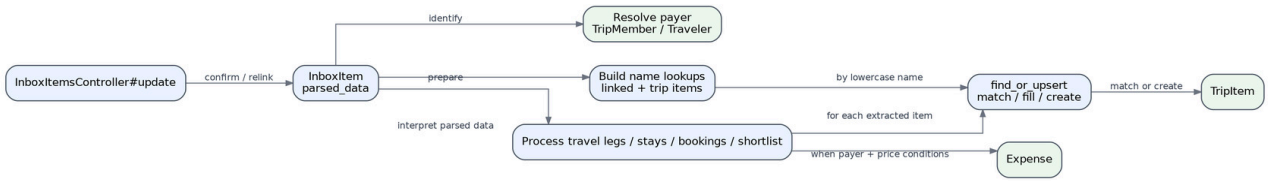
1. It invokes `fill_missing_links!`.
2. It reads the saved `parsed_data`.
3. If that data is not a Hash, it returns.
4. It processes `shortlist_items` using a category mapping.
5. It skips entries with blank names or names already found among the trip's itinerary items.
6. It creates new `TripItem` records with `status: :idea` for accepted shortlist entries.

There are separate rescue handlers for individual shortlist creation errors.

B. What `fill_missing_links!` actually does

`InboxItem#fill_missing_links!` occupies `app/models/inbox_item.rb:67–187`.

Record-linking algorithm



The method begins by identifying a possible payer. It first looks for a trip member whose traveler email matches the message sender; if that fails, it looks for a trip member's traveler associated with the user performing the action.

It then builds name-indexed lookups for itinerary items already linked to the inbox record and for all items belonging to the trip.

The local `find_or_upsert` lambda at lines 82–99:

- Prefers an item linked to the current inbox email, otherwise an item matching the lowercased name in the trip.
- For existing records, fills missing `inbox_item_id`, address and confirmation reference using `update_columns`.
- For unmatched names, creates a new confirmed `TripItem` associated with the inbox item and the reviewing user.

The term *upsert* here is the application's local method-variable name. The logic is implemented with Ruby lookups and Active Record operations, not a single database-native upsert statement.

C. How different extracted categories are handled

Category	Processing	Expense creation conditions
Travel legs	Converts transport modes to itinerary kinds; constructs names and destination addresses; detects a particular two-leg return-trip case	If a payer exists, selects the first leg with a price and creates an expense only when that selected item has no expense

Accommodations	Creates or links stays with dates, address, amount and confirmation number	Payer exists, price is present and the item has no expense
Other bookings	Maps booking type to an itinerary kind and creates or links the booking	Payer exists, price is present and the item has no expense
Shortlist ideas	Handled separately by `confirm_and_create_items!`	No corresponding expense creation in that method

The code for travel legs is at `app/models/inbox_item.rb:101–138`, accommodations at lines 140–158 and bookings at lines 160–187.

The shared `build_expense` method at lines 191–207 uses `trip.expenses.create!`, linking the `Expense` to the `TripItem`, `InboxItem`, reviewer and identified payer. It stores an amount, currency, description, category, date, booking source and confirmation timestamp.

Important condition: An extracted amount alone is insufficient to create an expense. The application must identify a `paid_by_traveler`, and the relevant expense must not already exist according to its lookup.

The source comments describe the linking routine as idempotent. Its name matching and expense-existence checks support that intention, but concurrency and duplicate behavior have not been independently tested.

D. What happens with other review actions?

`app/controllers/inbox_items_controller.rb:16–35` also handles:

`action_type`	**Actual application behavior**
`confirm`	Create/link itinerary data, then mark the inbox record confirmed
`discard`	Mark the inbox record discarded; no linking method is called
`relink`	Call `fill_missing_links!` without processing shortlist ideas
`retry`	Set parsing status to unparsed and enqueue a new parsing job

The `retry` branch does not itself check sender approval or require a previously failed parsing status.

All of these actions execute through Rails controller routing, but the branching and model method calls are application-written.

6. How approved-sender management affects processing

`ApprovedSendersController` provides separate actions for adding and removing sender approvals.

Approval: `app/controllers/approved_senders_controller.rb:5–20`

The controller optionally looks up an existing `InboxItem` using `params[:inbox_item_id]`, then creates an `ApprovedSender` for the trip containing the submitted email, approving user and approval timestamp.

When the request identifies an inbox item, the controller also changes that particular item's `sender_status` to `approved` and enqueues `ParseInboxItemJob`.

If no inbox item ID is supplied, it adds the sender to the approved list but does not automatically enqueue jobs for every previously received email from that address.

Removal: `app/controllers/approved_senders_controller.rb:22–26`

The controller destroys the selected approved-sender entry. It does not retrospectively change the status of existing inbox records.

Validation: `app/models/approved_sender.rb:1–6`

The model requires an email, requires `approved_at` and enforces trip-scoped email uniqueness.

The route for these controller actions is declared at `config/routes.rb:45`. The controller uses `current_user.trips.find(params[:trip_id])` at lines 30–32; there is no separate owner-role check in the inspected action.

A source-level inconsistency worth flagging

In `app/views/inbox_items/show.html.erb:53–75`, the conditional order is:

```erb
<% if @item.pending_review? %>
  ...
<% elsif @item.pending_approval? %>
  ...
```

New inbox records have `review_status: pending_review` by default, including those from unrecognized senders. Therefore, the first branch appears to take precedence over the approval branch for those new records.

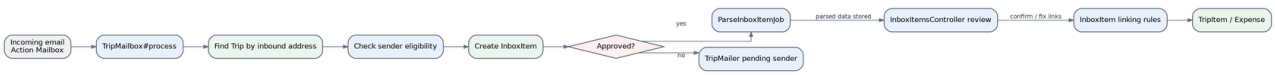
Meanwhile, `InboxItemsController#update` does not check `sender_status` or `parse_status` before executing the `confirm` or `relink` actions.

This creates a **possible approval/review gating inconsistency**: the mailbox delays automatic parsing for unknown senders, but the review controller does not independently enforce that gate. A real user-interaction test is required before recording the full impact as a verified defect.

It is relevant to explaining control flow, but it is not by itself evidence of a design pattern.

7. Complete collaboration trace

Inbound email to trip data



The diagram represents intended application paths established from static source. It does not assert that webhook delivery, queued execution or user review has been tested.

The configured external ingress is Postmark (`config/application.rb:52–53`). The setup documentation describes an Action Mailbox webhook under `/rails/action_mailbox/postmark/inbound_emails` ; actual Postmark delivery was not tested. The application's ordinary `config/routes.rb` declares the trip inbox and approval actions, while Action Mailbox provides its own ingress infrastructure.

8. Participant and source-evidence register

All paths are relative to the root of `daplanstan-main.zip` . The ranges below are from the uploaded source snapshot; they are not independently tied to the stated upstream commit.

Participant / method	Exact source location	Observed responsibility
Action Mailbox ingress configuration	`config/application.rb:52–53`	Configure Postmark ingress

`ApplicationMailbox`	`app/mailboxes/application_mailbox.rb:1–3`	Route incoming emails to `TripMailbox`
`Trip#build_inbound_email`	`app/models/trip.rb:31–37`	Generate trip-specific email address
`TripMailbox#process`	`app/mailboxes/trip_mailbox.rb:2–31`	Locate trip, check sender, create inbox record and choose next action
`TripMailbox#extract_text_body`	`app/mailboxes/trip_mailbox.rb:35–43`	Extract text content
`TripMailbox#extract_html_body`	`app/mailboxes/trip_mailbox.rb:45–53`	Extract HTML content
`TripMailbox#notify_owners_of_pending_sender`	`app/mailboxes/trip_mailbox.rb:55–60`	Request notification emails
`TripMailer#pending_sender_notification`	`app/mailers/trip_mailer.rb:2–10`	Construct owner notification
`ApprovedSendersController#create`	`app/controllers/approved_senders_controller.rb:5–20`	Register approval and optionally enqueue parsing
`ApprovedSender`	`app/models/approved_sender.rb:1–6`	Persist approved-sender association and validation
`ParseInboxItemJob#perform`	`app/jobs/parse_inbox_item_job.rb:99–112`	Coordinate AI extraction and persist its result
`ParseInboxItemJob#build_prompt`	`app/jobs/parse_inbox_item_job.rb:116–125`	Prepare input for extraction
`ParseInboxItemJob#parse_with_anthropic`	`app/jobs/parse_inbox_item_job.rb:127–139`	Invoke Anthropic and extract tool input
`InboxItem#parsed_data` and setter	`app/models/inbox_item.rb:19–25`	Serialize and deserialize structured data
`InboxItemsController#update`	`app/controllers/inbox_items_controller.rb:16–35`	Handle confirmation, discard, relink and retry
`InboxItem#confirm_and_create_items!`	`app/models/inbox_item.rb:34–58`	Perform linking, then add shortlist ideas
`InboxItem#fill_missing_links!`	`app/models/inbox_item.rb:67–187`	Find, update or create trip records
`InboxItem#build_expense`	`app/models/inbox_item.rb:191–207`	Construct an associated expense
`TripItem`	`app/models/trip_item.rb:1–28`	Persist linked itinerary data, types and statuses

`Expense`	`app/models/expense.rb:1-28`	Persist linked expense data
Inbox review interface	`app/views/inbox_items/show.html.erb:53-83, 130-137`	Submit review and linking actions
Trip inbox routes	`config/routes.rb:44-45`	Route inbox and sender-management requests

9. Ownership of the collaboration

Area	DaPlanStan application code	Rails / dependency / provider
Email reception	Configure ingress and mailbox routing	Postmark transport and Action Mailbox ingestion
Trip matching	Recipient normalization and `Trip.find_by`	Rails Active Record query machinery
Sender checks	Approval conditions and notifications	Active Record associations and Action Mailer delivery
Parsing	Prompt, schema, model selection and result persistence	Anthropic SDK and remote model generation
Background execution	Job's `perform` logic	Active Job / configured Solid Queue
Confirmation	Review actions and record-linking rules	Rails controller infrastructure and persistence
UI updates	`broadcasts_refreshes_to :trip` declaration	Turbo broadcasting and stream subscriptions

The substantive business logic—sender eligibility, extracted-data interpretation, existing-item matching, conditional expense creation and review decisions—is written in DaPlanStan.

The frameworks and providers supply transport, lifecycle management, queue execution, database APIs, model inference and UI-update infrastructure.

10. Pattern candidates worth investigating

No GoF classification is established by this source trace. There are three narrower hypotheses worth retaining for skeptical review.

Candidate A — GRASP Information Expert

UNCONFIRMED

Why it might fit: `InboxItem` owns the parsed-data interface, belongs to the relevant trip, and contains methods that use this information to create and link related records (`app/models/inbox_item.rb:19-25, 34-207`).

False-positive risk: These are responsibilities placed on an Active Record model. That could simply represent model-centric application code, not a deliberate responsibility allocation justified by GRASP Information Expert. The method also reaches across `Trip`, `TripItem`, `Expense`, `Traveler` and `User` data.

Origin: Mainly application-owned business logic, supported by Active Record.

Verification focus: Determine whether `InboxItem` genuinely holds the information needed to justify ownership of these operations, or whether it is only coordinating responsibilities belonging to several other entities.

Candidate B — GRASP Controller

UNCONFIRMED

Why it might fit: `TripMailbox#process` acts as a handler for an inbound system event. It coordinates lookup, authorization-related checks, inbox-record creation and delegation to background processing or notification.

False-positive risk: Receiving an event in a Rails mailbox is a framework convention. A method that coordinates several calls is not automatically a meaningful GRASP Controller assignment.

Origin: Application-owned event handling within Rails-provided mailbox infrastructure.

Verification focus: Evaluate the division of responsibility between `ApplicationMailbox`, `TripMailbox`, `InboxItem` and the parsing job against the GRASP Controller definition, not merely their names or inheritance.

Candidate C — Observer-like Turbo refresh

UNCONFIRMED

Why it might fit: `InboxItem` declares `broadcasts_refreshes_to :trip` at `app/models/inbox_item.rb:5`, and `app/views/inbox_items/index.html.erb:1` declares `turbo_stream_from @trip`.

Together, these are evidence of a framework-configured model-change notification and stream subscription relationship.

False-positive risk: The actual observer/subscription machinery is implemented by Turbo and Rails. The application does not visibly define its own observer interface, registry or notification implementation.

Origin: Predominantly framework-provided, with application-defined declarations.

Verification focus: Trace actual subscription and broadcast behavior, then determine whether it qualifies under the faculty's framework-idiom category. This may overlap with the same trip-stream

infrastructure observed during the geocoding investigation; it must not automatically become a separate counted pattern instance.

The use of `ParseInboxItemJob`, Anthropic tool input, lookup methods and `find_or_upsert` does **not** currently justify proposing Command, Chain of Responsibility, Strategy, Adapter or Factory as additional candidates. Those labels would require defining mechanisms that this inspection has not demonstrated.

11. Strongest non-pattern explanation

The complete workflow is best described as:

A Rails Action Mailbox intake pipeline with a sender-approval gate, asynchronous AI-based data extraction, persisted intermediate results and user-triggered domain-record linking.

The application uses straightforward procedural decisions, Active Record associations, background jobs and methods on its `InboxItem` model. Much of the framework behavior is declarative.

This explanation accounts for the collaboration without inventing interfaces, interchangeable implementations or design-pattern participants.

12. Remaining verification questions

The static trace is complete enough to support candidate review, but the following points remain unresolved:

1. **Inbound transport:** Has Postmark successfully delivered a real email through the configured Action Mailbox ingress into `TripMailbox`?
2. **Approval enforcement:** Does the inbox view's conditional order cause an unknown sender's email to display confirmation controls before approval, and are there any runtime protections not visible in the inspected application methods?
3. **Parsing execution:** Does the job return a usable Anthropic tool result for representative emails? Its rescue handler also needs checking against Rails' configured retry behavior.
4. **Record linking:** Do repeated or concurrent confirmation requests remain duplicate-safe? The source's name matching and expense-existence checks demonstrate an intention, not a tested guarantee.
5. **Partial failures:** What happens when individual item or expense creation fails but the controller subsequently marks the inbox record confirmed? No transaction covering the entire operation is visible.
6. **Attachments:** Are inbound attachments preserved through another framework mechanism? The inspected mailbox method does not copy them to `InboxItem`.
7. **Turbo updates:** Are subscribed browser clients actually refreshed following `InboxItem` changes, and should that framework behavior qualify for the inventory?

8. **Human verification:** A reviewer still needs to assess the GRASP responsibility assignments and decide whether they are meaningful enough to be counted.

Investigation outcome

Inbound-email investigation

Verification pending

Control-flow trace	Established from source
Parsing versus record creation	Confirmed as separate code paths
Source locations	Recorded
Pattern hypotheses	3 unconfirmed
Confirmed pattern instances	0
Runtime testing	Not performed

Conclusion: The inbound-email source demonstrates an application-controlled review and linking workflow, not automatic record creation by Anthropic. GRASP Information Expert, GRASP Controller and a Turbo-mediated Observer idiom are possible classifications for later verification, but none has yet met the standard for confirmation.

The next appropriate activity within this source area is to evaluate these specific candidates against their essential mechanisms and either reject them or preserve narrowly defined uncertainties, just as was done for geocoding.

Before moving to another part of the repository, verify the three candidates found in the inbound-email investigation:

1. GRASP Information Expert
2. GRASP Controller
3. Turbo-mediated Observer

Check them against the actual source rather than the wording of the previous answer.
For each candidate:

1. State the essential idea or responsibility rule that must exist for the pattern to be defensible.
2. Identify the exact DaPlanStan classes, modules or methods that would play the relevant roles.

3. Give the source paths and reliable line ranges.
4. Explain the actual collaboration or responsibility allocation that supports the classification.
5. Explain the strongest evidence against the classification.
6. Check whether the same behavior is better explained as normal Rails/Active Record/Action Mailbox/Turbo usage.
7. State whether the behavior is mainly application-owned, framework-provided or a mix of both.
8. Give a final decision:
   CONFIRMED
   REJECTED
   UNCERTAIN
9. If CONFIRMED, give a stable instance name that could later be used in `patterns.csv`.
10. If UNCERTAIN, state exactly what evidence is still missing.

For the Observer candidate, also check whether this is actually the same trip-stream notification mechanism already seen in the geocoding investigation. If it is the same collaboration, do not count it twice just because it appears in another workflow.
At the end give one table:
Candidate | Actual participants | Source evidence | Strongest objection | Origin | Final decision
Then give a short final conclusion for the inbound-email candidate-review record.
Do not search for new patterns in this prompt.

Worked for 1m 25s

Inbound Travel-Email Processing — Final Pattern Verification

DaPlanStan | Original uploaded source ZIP | Candidate Review 02

I re-examined the actual source declarations, method bodies and call sites rather than accepting the previous pattern suggestions.

GRASP Information Expert	CONFIRMED
GRASP Controller	CONFIRMED
Turbo-mediated Observer	UNCERTAIN

Confirmations are based on static source evidence. Independent human sign-off and runtime verification have not been performed.

1. GRASP Information Expert — CONFIRMED

Essential responsibility rule

GRASP Information Expert recommends assigning a responsibility to the class that possesses, or has direct access to, the information necessary to perform it.

It does not require a special interface, inheritance hierarchy or named pattern class.

For this investigation, the specific responsibility is **interpreting stored email-extraction data and converting it into linked itinerary and expense records**.

Actual participants and roles

Information Expert: `InboxItem`

Relevant methods:

- `InboxItem#parsed_data` — `app/models/inbox_item.rb:19–21`
- `InboxItem#parsed_data=` — lines 23–25
- `InboxItem#confirm_and_create_items!` — lines 34–58
- `InboxItem#fill_missing_links!` — lines 67–187
- `InboxItem#build_expense` — lines 191–207

Requesting client: `InboxItemsController#update`, `app/controllers/inbox_items_controller.rb:16–35`.

Collaborating domain objects: `Trip`, `TripItem`, `Expense` and `Traveler`.

Supporting evidence

`InboxItem` stores the structured AI result in `parsed_data_json` and exposes it through `parsed_data`.

The controller delegates record creation to the model:

```ruby
when "confirm"
  @item.confirm_and_create_items!(current_user)
```

The model then accesses the information it owns:

```ruby
```

```
def fill_missing_links!(added_by)
  data = parsed_data
  return unless data.is_a?(Hash)
```

Using that information, it identifies existing itinerary items, creates missing ones and conditionally creates expenses.

The most important supporting locations are `app/controllers/inbox_items_controller.rb:16-26`, `app/models/inbox_item.rb:67-99` and `app/models/inbox_item.rb:101-207`. ‹/›

This is more than merely using Active Record. DaPlanStan has explicitly assigned the application-specific interpretation and linking responsibility to the object holding the extracted email data.

Strongest objection

`InboxItem` does not possess all the required information independently.

It must query the associated trip, existing itinerary items, members and expenses. Its implementation also mixes extraction interpretation, deduplication, record creation and expense-related rules.

This raises a legitimate responsibility-design concern: the model may be coordinating more behavior than is ideal.

However, GRASP Information Expert allows collaboration with other objects. It does not require that all necessary data physically reside inside one class.

The essential observation remains that `InboxItem` owns the extracted data and has been given the responsibility for interpreting it.

Application versus framework

Mainly application-owned.

Rails supplies model associations, database querying and persistence methods. It does not supply the application's mapping of travel legs, accommodations, bookings and shortlist entries into DaPlanStan records.

Final decision

CONFIRMED

The source establishes a defensible application-specific GRASP Information Expert responsibility.

Stable instance name: `Parsed Email to Trip Record Linking`

Scope of confirmation: The assignment of extracted-email interpretation and linking responsibility to `InboxItem`, not a claim that every responsibility in this model is ideally allocated.

---

2. GRASP Controller — CONFIRMED

Essential responsibility rule

GRASP Controller recommends assigning the handling of a system event to an object representing the system, a subsystem or the relevant use case.

Such an object receives the event, coordinates the required actions and delegates responsibilities to appropriate collaborators.

It does not need to inherit from `ActionController`.

For this investigation, the system event is **the arrival of an inbound travel email**.

Actual participants and roles

System-event controller: `TripMailbox#process`, `app/mailboxes/trip_mailbox.rb:2–31`.

Event routing entry point: `ApplicationMailbox`, `app/mailboxes/application_mailbox.rb:1–3`.

Delegated collaborators: `Trip`, `InboxItem`, `ParseInboxItemJob` and `TripMailer`.

Supporting evidence

`ApplicationMailbox` specifies:

```ruby
class ApplicationMailbox < ActionMailbox::Base
  routing(all: :trip)
end
```

This routes email events into `TripMailbox`.

Inside `TripMailbox#process`, the application implements the event coordination.

It identifies the destination trip at lines 3–9, evaluates sender eligibility at lines 11–14 and creates an `InboxItem` at lines 16–24.

It then determines what happens next:

```ruby
if approved
  ParseInboxItemJob.perform_later(inbox_item.id)
else
  notify_owners_of_pending_sender(trip, inbox_item)
end
```

The mailbox does not attempt to implement AI extraction itself. It delegates that responsibility to a separate job. It also delegates notification construction and delivery to a mailer.

This creates a clear application-defined separation between the event handler and subsequent work.

Strongest objection

`TripMailbox` inherits from `ActionMailbox::Base`. Receiving inbound email through its `process` method is a normal Rails convention.

Consequently, simply discovering a `TripMailbox` class or a `process` method would not be enough to establish GRASP Controller.

The decisive evidence is the application-written coordination inside that method: trip identification, sender checks, inbox persistence and conditional delegation.

The framework determines *how the event reaches the handler*. DaPlanStan determines *what that handler does with the event*.

Application versus framework

Mixed, with the relevant responsibility application-owned.

Action Mailbox supplies inbound-event dispatch and mailbox lifecycle behavior.

DaPlanStan supplies the event-handling decisions and orchestration.

Final decision

CONFIRMED

The source establishes an application-defined system-event controller for incoming travel email.

Stable instance name: `Inbound Travel Email Intake Coordination`

Scope of confirmation: The `TripMailbox#process` event-handling responsibility. This does not automatically classify every Rails controller or mailbox in the repository as a separate GRASP

Controller instance.

---

3. Turbo-mediated Observer — UNCERTAIN

Essential mechanism

Observer requires a notification relationship between a subject or event source and one or more interested observers.

For a framework-mediated implementation, the source should establish a subscription, a notification publisher and a delivery mechanism connecting them.

Unlike the GRASP candidates, this is a collaboration whose defining behavior largely depends on external framework machinery.

Actual participants

Application-defined refresh source:

`InboxItem`, `app/models/inbox_item.rb:5`

```ruby
broadcasts_refreshes_to :trip
```

Trip-stream subscriber declarations:

`app/views/inbox_items/index.html.erb:1`

```erb
<%= turbo_stream_from @trip %>
```

`app/views/trips/show.html.erb:2`

```erb
<%= turbo_stream_from @trip %>
```

Additional application-defined publisher from geocoding:

`GeocodeTripItemsJob#perform`, `app/jobs/geocode_trip_items_job.rb:20-21`

Ruby

```ruby
Turbo::StreamsChannel.broadcast_refresh_to(trip)
```

Supporting evidence

The `InboxItem` model declares automatic refresh broadcasting to its associated trip.

Both the inbox index and the trip show page subscribe using the trip object.

The geocoding job also explicitly requests a broadcast for a trip.

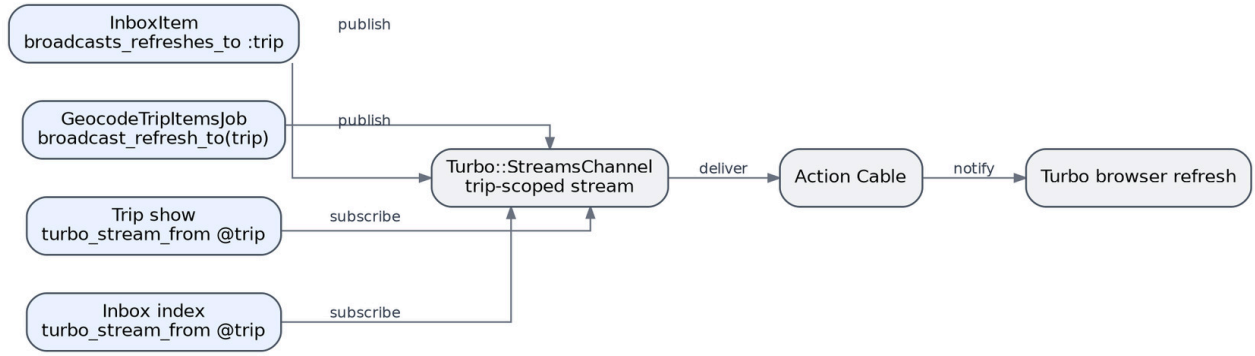
A repository-wide search identified these application declarations and found no additional custom observer-registration implementation in the inspected application source.

Together, these statements establish an intended trip-scoped notification mechanism.

Is this the same mechanism found during geocoding?

Yes, at the stream/subscription level.

Shared trip-stream notification mechanism



Logical framework-mediated publication and subscription, subject to verification of Turbo's actual delivery behavior.

The two features have different broadcast triggers:

- Inbox processing uses a model-level broadcasting declaration.
- Geocoding explicitly invokes a refresh broadcast inside the job.

However, both target the same kind of trip-scoped stream, and the subscribers use the same `@trip` subscription mechanism.

Do not create two confirmed Observer records simply because two workflows use this infrastructure.

If subsequently verified and accepted as a framework idiom, the sensible provisional identity would be `Trip-Scoped Turbo Refresh Notifications`, with both triggering paths documented as participants or usages. Final instance boundaries should follow the faculty's counting rule.

Strongest objection

The actual subscription lifecycle, event dispatch and notification handling are supplied by Turbo Rails and related Rails infrastructure.

DaPlanStan declares that behavior, but the source archive does not include the dependency implementation itself.

It is therefore unclear whether the correct inventory classification should be a framework-implemented Observer idiom or simply application usage of Turbo's existing broadcasting feature.

What is still missing?

The unresolved candidate requires three specific checks.

Framework source verification: Inspect the installed `turbo-rails` version 2.0.23, identified in `Gemfile.lock:424`, and trace how `broadcasts_refreshes_to`, `turbo_stream_from` and `broadcast_refresh_to` establish and use subscriptions.

Runtime verification: Open a trip page with an active stream subscription, trigger an `InboxItem` update and separately trigger the geocoding broadcast. Verify that the intended refresh reaches the subscriber.

Classification and counting review: Establish whether framework-mediated subscriptions are eligible as framework-idiom instances, and document the shared trip-stream collaboration once rather than multiplying it by triggers or pages.

Application versus framework

Mainly framework-provided, with application-defined configuration and broadcast calls.

Final decision

UNCERTAIN

The subscription and publishing declarations are visible, but the complete mechanism and its eligibility as a counted pattern instance have not been independently verified.

---

4. Final comparison table

Candidate	Actual participants	Source evidence	Strongest objection	Origin	Final decision
GRASP Information Expert	`InboxItem`, `InboxItemsController`, `Trip`, `TripItem`, `Expense`	`inbox_item.rb:19–25, 34–58, 67–207`; `inbox_items_controller.rb:16–26`	`InboxItem` coordinates data held in several domain objects and may have excessive responsibilities	Mainly application-owned	**CONFIRMED**
GRASP Controller	`ApplicationMailbox`, `TripMailbox`, `Trip`, `InboxItem`, `ParseInboxItemJob`, `TripMailer`	`application_mailbox.rb:1–3`; `trip_mailbox.rb:2–31, 55–60`	Mailbox lifecycle is provided by Action Mailbox; class naming alone proves nothing	Mixed; event decisions application-owned	**CONFIRMED**
Turbo-mediated Observer	`InboxItem`, Turbo stream declarations, `GeocodeTripItemsJob`, trip views	`inbox_item.rb:5`; `inbox_items/index.html.erb:1`; `trips/show.html.erb:2`; `geocode_trip_items_job.rb:20–21`	Subscriber registration and notification machinery belong to Turbo; framework implementation and delivery remain unverified	Mainly framework-provided	**UNCERTAIN**

5. Final candidate-review record

Writing

Inbound Travel-Email Processing — Final Candidate Review

Source: DaPlanStan original source ZIP
Review method: Static inspection of actual application methods, declarations and call sites.
Human verification: Pending.

GRASP Information Expert — CONFIRMED

Instance name: Parsed Email to Trip Record Linking.

The `InboxItem` model owns the parsed email data and implements its interpretation, matching and conversion into related trip records. The responsibility is supported by `InboxItem#confirm_and_create_items!`, `#fill_missing_links!` and `#build_expense`. The application-specific responsibility allocation is defensible as GRASP Information Expert.

GRASP Controller — CONFIRMED

Instance name: Inbound Travel Email Intake Coordination.

`TripMailbox#process` handles the incoming-email system event. It identifies the trip, checks sender eligibility, creates the inbox record and delegates parsing or notification. Although Action Mailbox supplies the event infrastructure, these coordination decisions belong to DaPlanStan and support GRASP Controller.

Turbo-mediated Observer — UNCERTAIN

The application declares model-level refresh broadcasting and trip-scoped Turbo subscriptions. The same trip-stream mechanism is also used by the geocoding workflow. Framework implementation, runtime notification and counting eligibility require verification. It must not be counted separately for each workflow or broadcast trigger.

Overall conclusion

Two application-owned GRASP responsibility assignments are supported by static source evidence. One framework-mediated notification candidate remains uncertain. The two GRASP instances may be proposed for the baseline inventory using the stable names above, subject to independent human review. The Observer candidate must remain outside confirmed pattern counts until verification is complete.

No additional pattern classifications were introduced during this review.

Result: The inbound-email candidate review supports two source-confirmed GRASP instances and one unresolved Turbo framework candidate. The Observer question remains shared with geocoding rather than becoming a new independent count.

Now move to the next source area: trip creation, membership and permission handling.
As before, do not begin by assigning pattern names. Trace the actual collaboration first.
Inspect these files and any directly related callers needed to complete the flow:

1. `app/controllers/trips_controller.rb`
2. `app/models/trip.rb`
3. `app/models/trip_member.rb`
4. `app/models/traveler.rb`
5. `app/controllers/trip_members_controller.rb`
6. `app/models/account.rb`
7. `app/models/user.rb`
8. Relevant routes, views or supporting models where needed.

I want to understand three things separately:

1. Trip creation

Trace what happens from the request to create a trip through persistence, owner membership creation and any follow-up work such as cover-image processing.

2. Membership management

Trace how travelers become members of a trip, how membership roles are represented and how members are added, updated or removed.

3. Permission handling

Trace how the application decides whether a user can view or modify a trip.

Do not assume that role values such as owner, planner or viewer constitute Strategy, State or any other pattern.
For the investigation:

1. Identify the actual classes and methods participating in each flow.
2. Give the important source paths and reliable line ranges.
3. Explain where responsibility for creating the trip and its initial membership is placed.
4. Explain how `Trip`, `TripMember`, `Traveler`, `Account` and `User` relate during these operations.
5. Trace the actual permission checks rather than describing generic Rails authorization.
6. Separate application-written behavior from Active Record, controller and routing behavior supplied by Rails.
7. Check whether responsibility is concentrated in one object or coordinated across several objects.
8. Identify any repeated permission or membership logic that appears across controllers.

Only after tracing all three flows, identify design-pattern candidates that are genuinely worth investigating.
For each candidate:

1. Give the possible pattern name and category.
2. Identify the actual collaboration suggesting it.
3. Explain why it might fit.
4. Explain the strongest reason it might be a false positive.
5. State whether it is application-owned, framework-provided or mixed.
6. Leave it UNCONFIRMED at this stage.

Do not force classifications such as Strategy, State, Factory, Composite, Proxy or GRASP patterns unless their defining responsibility or collaboration is supported by the source.
At the end give me:

1. Trip-creation collaboration trace.
2. Membership-management collaboration trace.
3. Permission-check collaboration trace.
4. Participant and source-evidence table.
5. Application versus Rails responsibility split.

6. Candidate pattern classifications, if any.
7. Strongest non-pattern explanation.
8. Anything that needs further checking before candidate verification.

Worked for 2m 10s

DaPlanStan — Trip Creation, Membership and Permission Investigation

Patterns unconfirmed Source Investigation 03 · Original uploaded ZIP

1. Initial source findings

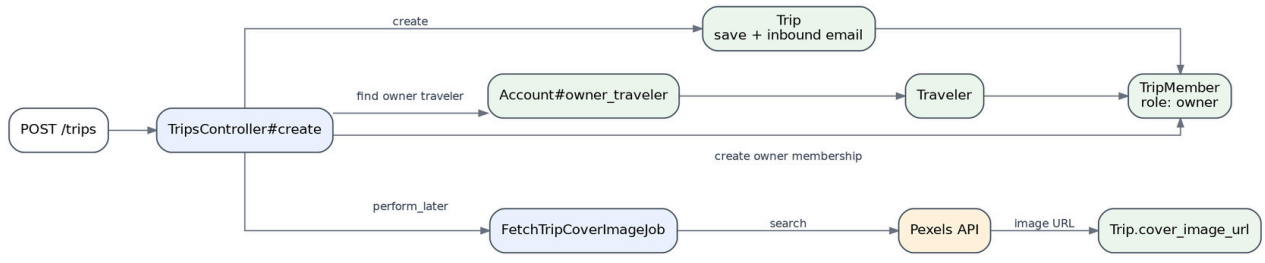
The implementation separates responsibilities across Rails controllers and Active Record models. Three important facts emerge from the code:

1. **Trip creation and owner-membership creation are coordinated by `TripsController#create`, not by a single `Trip` model method.** The controller saves the trip, creates its first `TripMember`, and enqueues cover-image retrieval.
2. **Membership roles are stored as values on `TripMember`.** No application-defined interchangeable role objects or role-specific classes were identified in this source area.
3. **Permissions are enforced differently across controllers.** Some actions require owner or planner membership, others require only that the user belongs to the trip.

The last distinction matters: DaPlanStan does not implement one uniform, centralized authorization policy.

2. Trip creation — complete collaboration trace

Trip creation control flow



Step A — The request enters the controller

`config/routes.rb:21` declares `resources :trips` within the application's host constraint at lines 6–7. This provides the normal RESTful POST route to `TripsController#create`.

The controller registers `authenticate_user!` at line 2. The request must be associated with a signed-in user before trip creation proceeds.

At `app/controllers/trips_controller.rb:35–39`, the application constructs the trip and assigns its creator and account:

```ruby
@trip = Trip.new(trip_params)
@trip.created_by = current_user
@trip.account = current_account
```

The strong parameters at lines 129–131 permit only `title`, `description`, `start_date` and `end_date` from the submitted trip form.

`created_by` and `account` are assigned by the application rather than taken from those form parameters.

Step B — The trip is saved

`TripsController#create` calls `@trip.save` at line 40.

`Trip`, defined in `app/models/trip.rb:1–19`, has relationships to its account, creator, members, travelers, itinerary items, expenses and other trip resources.

The model validates the title, start date, end date and inbound email address. It also declares:

```ruby
before_validation :generate_inbound_email,
  on: :create,
  if: -> { inbound_email.blank? }
```

`Trip#generate_inbound_email` and `#build_inbound_email`, lines 31–37, generate a trip-specific address using the title, a random hexadecimal suffix and the configured email domain.

The application defines the address-generation logic. Rails provides validation, callback invocation and persistence infrastructure. ‹/›

If saving fails, the controller renders the new-trip form with an unprocessable-entity response.

Step C — The initial owner membership is created

Once the trip is saved, the controller executes:

‹/› Ruby

```ruby
owner_traveler = current_account.owner_traveler
@trip.trip_members.create!(
  traveler: owner_traveler,
  role: :owner,
  joined_at: Time.current
)
```

`Account#owner_traveler`, `app/models/account.rb:9–11`, searches the account's travelers for the record linked to the account owner.

The source also shows how that record is normally provisioned. `SessionsController#create_account_for`, `app/controllers/sessions_controller.rb:78–81`, creates an `Account` and then its owner-linked `Traveler`.

These are distinct identities:

- `User` is the signed-in person.
- `Account` groups that person's owned travel resources.
- `Traveler` represents a participant, optionally linked to a `User`.
- `TripMember` establishes the traveler's membership and role in a particular trip.

The controller creates the membership through the new trip's Active Record association. It explicitly assigns `owner`, rather than relying on the database's default membership role. ‹/› ‹/›

Step D — Cover-image processing is requested

At `app/controllers/trips_controller.rb:43`:

Ruby

```ruby
FetchTripCoverImageJob.perform_later(@trip.id)
```

The job implementation is application-written. It loads the trip, checks for a configured Pexels API key, constructs a search from the title, requests a landscape image and stores the returned image URL when available.

Important details from `app/jobs/fetch_trip_cover_image_job.rb:4-30`:

- If the trip is missing, the job returns.
- If the API key is absent, it logs a message and returns.
- If the HTTP response is unsuccessful, it logs a warning and returns.
- It uses the first returned photograph's `large2x` URL.
- It catches exceptions and logs warnings.

The controller does not wait for this external lookup before redirecting. Successful image retrieval was not established through runtime testing.

Where creation responsibility is located

Responsibility	Actual owner
Handle trip-creation request	`TripsController#create`
Construct trip and assign account/creator	`TripsController#create`
Validate trip data and generate inbound address	`Trip`
Find the account owner's traveler record	`Account#owner_traveler`
Create the first membership	`TripsController#create` through `trip_members.create!`
Persist records and associations	Active Record
Request cover-image work	`TripsController#create`
Perform cover-image retrieval	`FetchTripCoverImageJob`

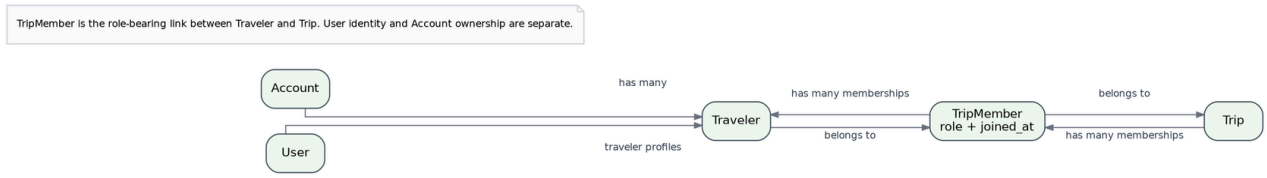
The business operation is therefore coordinated across multiple objects, with the controller responsible for sequencing them.

Important limitation: The trip save and owner-membership creation are not visibly enclosed in one application-level database transaction. If membership creation raises after a successful trip save, the source does not establish an automatic rollback of the already-saved trip.

3. Membership management — complete collaboration trace

A. How the five domain objects are related

Domain relationship overview



`TripMember` connects a Traveler to a Trip and holds membership-specific information. User identity, account ownership and trip membership are separate concepts.

The associations appear in:

- `app/models/account.rb:1–4`
- `app/models/user.rb:11–17`
- `app/models/traveler.rb:1–5`
- `app/models/trip.rb:4–12`
- `app/models/trip_member.rb:1–6`

In particular, `User#trips` uses `has_many :trips, through: :trip_members`, with membership reached through the user's `traveler_profiles`.

Therefore, being recorded as `Trip#created_by` is structurally different from accessing the trip through an active membership.

B. Adding a traveler to a trip

The membership-management interface is `app/views/trips/travelers.html.erb:29–81`. It displays existing members, provides removal controls for non-owners and submits an email address through a form.

The route is declared at `config/routes.rb:40`:

```ruby
resources :trip_members, only: [ :create, :destroy ]
```

`TripMembersController`, lines 1–4, requires authentication, loads the current trip and checks that the acting user is its owner or a planner.

Then `TripMembersController#create`, lines 6–15, chooses between two paths.

Path 1 — Existing traveler ID

The controller looks up the traveler through:

```ruby
traveler = @trip.account.travelers.find(params[:traveler_id])
add_traveler(traveler)
```

This scopes the selected traveler to the trip's account.

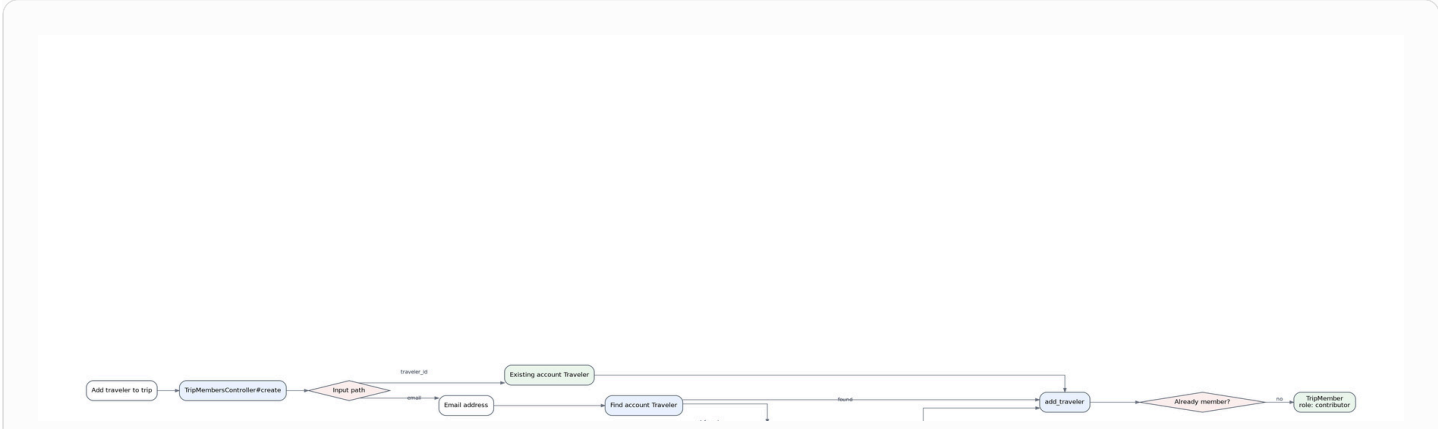
Path 2 — Email address

`resolve_and_add_by_email`, lines 46–64, checks for an existing traveler within the trip's account, including travelers linked to users with the submitted email.

If no traveler exists, it checks for an existing `User` globally.

- If a registered user exists, it creates a `Traveler` record for that user within the trip's account.
- If no registered user exists, it creates an email-only `Traveler` record and requests a trip invitation email.

The controller then calls `add_traveler` in either case.





`add_traveler`, at `app/controllers/trip_members_controller.rb:37-44`, checks whether that traveler already belongs to the trip.

If not, it creates:

```ruby
@trip.trip_members.create!(
  traveler: traveler,
  role: :contributor,
  joined_at: Time.current
)
```

The database also has a unique index on `(trip_id, traveler_id)`, providing an additional uniqueness constraint beyond the application check.

Important: An email-only traveler becomes a `TripMember` immediately. The membership record is not created when the invitation is accepted.

The user-account link is established separately, when a matching user signs in.

C. How invited travelers gain access

The relevant supporting code is:

- `Traveler#accept!` — `app/models/traveler.rb:26-28`
- `SessionsController#create` — `app/controllers/sessions_controller.rb:26-40`

At sign-in, the application looks for travelers with a matching email and no linked user:

```ruby
pending_travelers = Traveler.where(user_id: nil, email: user.email).to_a
pending_travelers.each { |t| t.accept!(user) }
```

`Traveler#accept!` then assigns the user and records the invitation-acceptance timestamp.

The existing trip membership remains linked to the same traveler. Through `User#traveler_profiles`, the signed-in user can now retrieve that membership and its trip.

This is an account-linking operation, not a membership-role transition. ‹/›

D. How roles are represented

`app/models/trip_member.rb:1-7` contains:

‹/› Ruby

```ruby
class TripMember < ApplicationRecord
  belongs_to :trip
  belongs_to :traveler

  enum :role, {
    owner: "owner",
    planner: "planner",
    contributor: "contributor",
    viewer: "viewer"
  }
  validates :role, presence: true
end
```

The role is a persisted string-backed Rails enum.

Role	Where the code uses it
`owner`	Initial membership, deletion protection, owner-only trip operations
`planner`	Authorization for trip editing and membership management
`contributor`	Assigned to travelers added through the membership controller
`viewer`	Declared as a role and database default, but not assigned by the inspected membership creation actions

A source-wide search did not identify an application route or controller action for changing a `TripMember` role. The examined creation paths explicitly assign `owner` or `contributor`.

There is no application-implemented State or Strategy collaboration in these enum declarations. The role values are queried by authorization code; they do not encapsulate interchangeable behavior.

E. Updating and removing members

The routes expose `create` and `destroy` only. There is no `TripMembersController#update` action.

`TravelersController#update`, at `app/controllers/travelers_controller.rb:37–44`, updates the traveler's profile information, such as name and avatar URL, rather than the `TripMember#role`.

For removal, `TripMembersController#destroy`, lines 17–22:

1. Locates the membership through `@trip.trip_members`.
2. Rejects removal if `member.owner?` is true.
3. Otherwise destroys the membership.
4. Redirects to the travelers page.

This removes the person from the trip, not from the account.

Account-level traveler deletion is different. `TravelersController#destroy` removes the traveler record, and `Traveler` declares `has_many :trip_members, dependent: :destroy`. Thus, deleting an account traveler can also remove that person's trip memberships. Unlike the membership controller, the account-level deletion action contains no explicit trip-owner protection. That interaction should be tested before claiming owner memberships are always protected.

---

4. Permission handling — actual authorization paths

A. Current user and membership lookup

`ApplicationController`, at `app/controllers/application_controller.rb:14–24`, defines:

‹/› Ruby

```ruby
def current_user
  @current_user ||= User.find_by(id: session[:user_id]) if session[:user_id]
end

def current_account
  @current_account ||= current_user&.account
end

def current_member_for(trip)
  return nil unless current_user
  trip.trip_members.joins(:traveler)
      .find_by(travelers: { user_id: current_user.id })
end
```

`authenticate_user!`, lines 47–49, handles the basic sign-in requirement.

The application then uses membership-based queries for trip authorization. It does not rely solely on whether the user created the trip or owns the account.

B. Viewing a trip

`TripsController#show` uses the `set_trip` callback:

```ruby
def set_trip
  @trip = current_user.trips.find(params[:id])
end
```

This scopes the lookup to trips reachable through the signed-in user's traveler memberships.

No role check is performed here. Consequently, a user with a linked membership in any of the four roles can satisfy the trip-visibility query.

The index action uses the same `current_user.trips` relationship.

C. Editing a trip

`TripsController#set_editable_trip`, lines 111–119, uses a database query that requires both:

- A traveler associated with the current user.
- A membership whose role is `owner` or `planner`.

```ruby
@trip = Trip.joins(trip_members: :traveler)
            .where(travelers: { user_id: current_user.id })
            .where(trip_members: { role: ["owner", "planner"] })
            .find(params[:id])
```

This callback applies to `edit`, `update`, `cover`, `update_cover`, `refresh_cover`, `travelers` and `geocode_missing`.

Failure to find a matching trip leads to a redirect with a permission alert.

D. Deleting a trip or regenerating its inbound address

`TripsController#set_owner_trip`, lines 120–127, is more restrictive. It requires the acting user's traveler membership to have `role: "owner"`.

It protects `destroy` and `regenerate_inbound_email`.

An account owner is therefore not automatically treated as a trip owner by this authorization query; the matching `TripMember` role is what matters.

E. Managing trip members

`TripMembersController#require_owner_or_planner`, lines 30–35, uses a different implementation:

```ruby
member = current_member_for(@trip)

unless member&.owner? || member&.planner?
  redirect_to @trip, alert: "Only trip owners and planners can manage travelers."
end
```

This checks the actual membership object after loading the trip.

The controller also prevents deletion of a membership whose role is `owner`, regardless of the acting user's own role.

F. Actual permission matrix

Based on the examined controller checks:

Operation	Owner	Planner	Contributor	Viewer
View trip	Yes	Yes	Yes	Yes
Edit trip details	Yes	Yes	No	No
Update cover / geocode missing	Yes	Yes	No	No
Access trip member-management page	Yes	Yes	No	No
Add/remove non-owner members	Yes	Yes	No	No
Delete trip	Yes	No	No	No
Regenerate trip email address	Yes	No	No	No
Create/update/delete itinerary items	Yes	Yes	Yes*	Yes*

Create/update/delete expenses	Yes	Yes	Yes*	Yes*
Confirm/delete inbox items	Yes	Yes	Yes*	Yes*
Manage approved senders	Yes	Yes	Yes*	Yes*

*The relevant child controllers require authenticated membership through `current_user.trips.find`, but have no additional role restriction in the inspected application actions. These are source-implied permissions, not independently tested HTTP outcomes.

G. Repeated and inconsistent permission logic

The source contains two different authorization styles.

Role-restricted queries:

`TripsController#set_editable_trip` and `#set_owner_trip` use explicit role conditions in their SQL-backed Active Record queries.

`TripMembersController#require_owner_or_planner` loads the membership and evaluates Rails enum predicates.

Membership-only scoping:

The following controller methods use `current_user.trips.find(params[:trip_id])` without distinguishing owner, planner, contributor and viewer:

- `TripItemsController#set_trip` — lines 118–120.
- `ExpensesController#set_trip` — lines 90–92.
- `InboxItemsController#set_trip` — lines 44–46.
- `ApprovedSendersController#set_trip` — lines 30–32.

Those controllers include actions that modify or delete records.

This is a meaningful authorization observation: **the role names suggest different levels of permission, but many mutating routes enforce only membership.**

There is also a difference between UI visibility and server authorization. `app/views/trips/_trip_panel.html.erb:178–249` displays the trip-editing controls only when the current member is an owner, while the server-side trip-editing methods also permit planners. UI visibility alone does not define authorization.

5. Participant and source-evidence table

The following references use paths relative to the uploaded `daplanstan-main.zip` snapshot, with line numbers verified against the extracted files.

Participant / method	Source location	Observed responsibility
Trip REST routes	`config/routes.rb:21-46`	Route trip and membership requests
`TripsController#create`	`app/controllers/trips_controller.rb:35-49`	Coordinate trip, membership and cover-job creation
`TripsController#set_trip`	`app/controllers/trips_controller.rb:107-109`	Membership-scoped trip lookup
`TripsController#set_editable_trip`	`app/controllers/trips_controller.rb:111-119`	Owner/planner access
`TripsController#set_owner_trip`	`app/controllers/trips_controller.rb:120-127`	Owner-only access
`Trip` associations and validation	`app/models/trip.rb:1-19`	Define trip relationships and validate data
`Trip#build_inbound_email`	`app/models/trip.rb:31-37`	Generate inbound email address
`Account#owner_traveler`	`app/models/account.rb:9-11`	Locate owner-linked traveler
`SessionsController#create_account_for`	`app/controllers/sessions_controller.rb:78-81`	Provision owner account and traveler
`FetchTripCoverImageJob#perform`	`app/jobs/fetch_trip_cover_image_job.rb:4-30`	Retrieve external cover-image URL
`TripMember`	`app/models/trip_member.rb:1-7`	Associate traveler with trip and store role
`Traveler#accept!`	`app/models/traveler.rb:26-28`	Associate invited traveler with signed-in user
`TripMembersController#create`	`app/controllers/trip_members_controller.rb:6-15`	Select membership-addition path
`TripMembersController#destroy`	`app/controllers/trip_members_controller.rb:17-22`	Remove non-owner member
`TripMembersController#require_owner_or_planner`	`app/controllers/trip_members_controller.rb:30-35`	Authorize membership changes
`TripMembersController#add_traveler`	`app/controllers/trip_members_controller.rb:37-44`	Prevent duplicate membership and create new member
`TripMembersController#resolve_and_add_by_email`	`app/controllers/trip_members_controller.rb:46-65`	Find or create traveler and request invitation

`ApplicationController#current_member_for`	`app/controllers/application_controller.rb:22-25`	Resolve membership for authenticated user
`User` associations	`app/models/user.rb:11-17`	Relate users to accounts, travelers and trips
Account traveler actions	`app/controllers/travelers_controller.rb:5-60`	Manage account-level traveler records
Membership view	`app/views/trips/travelers.html.erb:29-81`	Present members and submit add/remove requests
Membership uniqueness	`db/structure.sql:71-80`	Enforce unique trip/traveler membership

6. Application-written behavior versus Rails

Area	DaPlanStan responsibility	Rails responsibility
Trip creation	Assign creator/account, sequence save, owner membership and job scheduling	Routing, controller callbacks, model persistence
Trip initialization	Generate inbound email and define validation requirements	Validation and model callback execution
Membership	Choose travelers, assign roles, prevent duplicate additions and owner removal	Associations, enum methods, Active Record queries
Invitations	Determine recipients, create pending travelers and associate users after sign-in	Mail delivery infrastructure and database persistence
Authorization	Define role conditions and enforce them in application controller methods	Execute filtered queries and before-action callbacks
Cover-image retrieval	Construct Pexels request and store returned URL	Active Job execution infrastructure; HTTP and JSON library support

Responsibility is not concentrated in `Trip` or `TripMember`. Controllers contain much of the workflow coordination and authorization logic, while models primarily represent relationships, validation rules and selected domain operations.

7. Design-pattern candidates

Only after tracing all three flows do I find one recognized pattern category worth carrying into formal verification, represented by two potentially distinct system-event responsibilities.

Candidate A — GRASP Controller

UNCONFIRMED

Responsibility-assignment pattern · Mainly application-owned

Possible collaboration: Trip Creation Coordination

The principal event handler is `TripsController#create`, lines 35–49. It receives the creation request, constructs a `Trip`, assigns account information, saves it, creates the initial owner membership and delegates image retrieval to `FetchTripCoverImageJob`.

Why it might fit: A system-event handler coordinates several domain operations without implementing every subordinate operation itself.

Strongest false-positive argument: This may be routine Rails controller code following conventional request handling. More evidence is needed to assess whether its responsibility assignment is a meaningful GRASP Controller instance rather than merely labeling any Rails controller as a pattern.

Origin: Mixed infrastructure, with application-defined orchestration.

Decision: UNCONFIRMED.

Candidate B — GRASP Controller

UNCONFIRMED

Responsibility-assignment pattern · Mainly application-owned

Possible collaboration: Trip Membership Management Coordination

`TripMembersController#create` and `#destroy`, together with `resolve_and_add_by_email` and `add_traveler`, coordinate membership addition and removal.

The controller delegates storage to Active Record models and invitation delivery to `InvitationMailer`.

Why it might fit: The controller handles meaningful membership-management events and coordinates multiple domain objects and operations.

Strongest false-positive argument: The functionality is implemented as conventional controller branching and association manipulation. It is not yet established that the responsibility allocation is sufficiently distinctive to justify a separate GRASP Controller instance.

Origin: Mixed infrastructure, with application-defined decisions.

Decision: UNCONFIRMED.

Both candidates use the same pattern name but concern different system events. If either is later confirmed, its instance boundary must be established from its actual responsibility and collaboration —not merely from the number of controller classes or actions.

Classifications not supported by the evidence

`TripMember` is an association model containing role information, not an independently implemented behavioral hierarchy.

The inspected source does not establish interchangeable role algorithms, role-state objects, a specialized object-creation factory, a recursive object hierarchy or a protective proxy.

Likewise, `@trip.trip_members.create!` is ordinary Active Record association-based creation. It does not independently prove a GRASP Creator assignment to the `Trip` model, because the application explicitly places the orchestration in `TripsController`.

No additional pattern names are warranted in this preliminary investigation.

8. Strongest non-pattern explanation

The implementation is best explained as **conventional Rails CRUD orchestration over a membership-based domain model with controller-level authorization**.

Trip creation uses Active Record persistence followed by owner-membership creation and asynchronous cover-image retrieval.

Membership management uses a join model carrying role data, account-scoped traveler lookups, conditional invitation handling and ordinary creation/deletion operations.

Authorization uses the signed-in user's membership and role through Active Record queries and before-action methods. The rules are repeated across controllers rather than encapsulated in one authorization abstraction.

This explanation accounts for the source without requiring an additional GoF design pattern.

9. Remaining questions before candidate verification

The next verification stage should resolve the following issues.

Question	Why it matters
Does trip creation preserve consistency if owner-membership creation fails?	No explicit transaction covers both application-level operations
What happens when an invited member with no owned account accesses trip creation?	`current_account` can be nil; the creation path expects an account
Can a contributor or viewer successfully perform mutating child-resource actions?	Those controller methods enforce membership but not the finer role distinctions
Can account-level traveler deletion remove the final trip owner membership?	Traveler deletion cascades memberships, and the account-level action has no owner-membership guard
How are planner and viewer roles assigned in practice?	No role-update route or application assignment for those roles was identified

Are duplicate-addition and invitation flows safe under concurrent requests?	Database uniqueness provides one safeguard, but the full workflows were not tested
Should the two event handlers count as independent GRASP Controller instances?	Requires evaluating the responsibility assignment and counting boundary, not only class names
Are the source and runtime results reproducible?	No application execution or HTTP authorization tests were performed during this investigation

The source-level authorization inconsistency is particularly important to preserve as a separate verification observation. It should not be misrepresented as a design-pattern finding or as a runtime-verified security defect.

---

10. Investigation outcome

Trip and membership investigation Awaiting candidate review

Trip creation flow	Traced
Membership management flow	Traced
Permission handling	Traced
Source evidence	Recorded
Pattern candidates	2 unconfirmed GRASP Controller responsibilities
Confirmed pattern instances from this investigation	0
Runtime testing	Not performed

Final preliminary conclusion: DaPlanStan coordinates trip creation and membership management through controllers, domain associations and Active Record operations. It authorizes trip-level editing using owner/planner membership checks but applies less restrictive membership-only checks in several child-resource controllers.

Two GRASP Controller responsibility assignments are plausible candidates for subsequent verification. Neither should enter `patterns.csv` or confirmed pattern counts yet.

No source files were modified, and this investigation remains limited to the original source snapshot.

Before moving to another source area, verify the two GRASP Controller candidates identified in the trip and membership investigation:

1. Trip Creation Coordination
2. Trip Membership Management Coordination

I do not want these confirmed just because they are implemented inside Rails controller classes.
For each candidate:

1. State the system event or use-case event being handled.
2. Identify the exact object or method receiving that event.
3. Identify the application decisions it makes itself.
4. Identify the responsibilities it delegates to other objects.
5. Give the actual participating classes/modules/methods with source paths and reliable line ranges.
6. Explain why this responsibility assignment fits GRASP Controller rather than simply being ordinary Rails CRUD code.
7. Give the strongest argument against treating it as a GRASP Controller instance.
8. Check whether the controller is doing so much work that the better conclusion would simply be controller-heavy procedural orchestration rather than a meaningful GRASP responsibility assignment.
9. Decide whether the relevant responsibility is application-owned, framework-provided or mixed.
10. Give the final result as CONFIRMED, REJECTED or UNCERTAIN.

Then compare the two candidates directly.

11. Are Trip Creation Coordination and Trip Membership Management Coordination genuinely independent system-event collaborations?
12. If both are confirmed, explain why they should count as two GRASP Controller instances rather than one general "Rails Controllers" pattern instance.
13. If they should instead be treated as one broader instance, explain the correct boundary and stable instance name.
14. Do not create separate instances merely because different controller classes exist.

At the end give me:

1. Candidate decision table.
2. Final participant-role mapping for any confirmed instance.

3. Stable instance name for any confirmed instance.
4. Source evidence that should be preserved.
5. A short record of any rejected or uncertain classification.

Do not investigate new pattern types in this prompt.

Worked for 1m 6s

Trip Creation and Membership — GRASP Controller Verification

DaPlanStan | Candidate Review 03 | Original source ZIP

1. Final verification results

Re-examining the actual event handlers, their decisions and delegated responsibilities supports **two distinct GRASP Controller collaborations**.

Trip Creation Coordination GRASP Controller · Instance 1	CONFIRMED
Trip Membership Management Coordination GRASP Controller · Instance 2	CONFIRMED

These are **source-based confirmations**, not completed independent human verification. Neither is confirmed merely because its implementation is a Rails controller.

The relevant GRASP principle is that a non-UI object receives a system event, coordinates the corresponding use case and delegates supporting work. It does not require a specially named interface or inheritance structure. It also does not guarantee that the resulting controller is perfectly designed.

2. Candidate A — Trip Creation Coordination

CONFIRMED

System event and receiving object

Event: A signed-in user requests creation of a new trip.

Receiving method: `TripsController#create`.

Source: `config/routes.rb:21` and `app/controllers/trips_controller.rb:35–48`.

The trip form in `app/views/trips/new.html.erb:18–65` submits the request to the route generated by `resources :trips`.

Application decisions

The controller performs a specific multi-object operation:

```ruby
def create
  @trip = Trip.new(trip_params)
  @trip.created_by = current_user
  @trip.account    = current_account

  if @trip.save
    owner_traveler = current_account.owner_traveler
    @trip.trip_members.create!(
      traveler: owner_traveler,
      role: :owner,
      joined_at: Time.current
    )
    FetchTripCoverImageJob.perform_later(@trip.id)
    redirect_to @trip, notice: "Trip created."
  else
    render :new, status: :unprocessable_entity
  end
end
```

The key application decisions are to assign the signed-in user as creator, associate the trip with their account, proceed only if saving succeeds, select the account owner's traveler profile, create an owner membership and request cover-image processing.

Delegated responsibilities

Participant	Source location	Responsibility
`TripsController#create`	`app/controllers/trips_controller.rb:35–48`	Receives and coordinates the use case
`ApplicationController#current_user`, `#current_account`	`app/controllers/application_controller.rb:14–20`	Resolve authenticated identity and account
`Trip`	`app/models/trip.rb:4–19`	Relationships, validations and creation callback
`Trip#generate_inbound_email`	`app/models/trip.rb:31–37`	Generate the trip's email address
`Account#owner_traveler`	`app/models/account.rb:9–11`	Find the account owner's traveler profile

`TripMember`	`app/models/trip_member.rb:1-6`	Represent the membership and its role
`FetchTripCoverImageJob#perform`	`app/jobs/fetch_trip_cover_image_job.rb:4-30`	Retrieve and persist a cover-image URL

The controller does not itself generate the inbound email address or implement Pexels retrieval. It relies on the `Trip` model and a separate job for those operations.

Why this supports GRASP Controller

The use case is not merely `Trip.create`.

It establishes a trip, an owner membership and a separate follow-up operation. One receiving object coordinates those responsibilities and delegates subordinate operations.

The source therefore demonstrates a meaningful application-defined system-event responsibility.

Strongest objection

The controller owns the sequencing of trip persistence and membership creation.

It also invokes `create!` directly and does not wrap both writes in an explicit application-level transaction. This creates potential consistency concerns if the second write fails.

Such controller-level orchestration could be moved into a dedicated application operation, but **the absence of that abstraction does not invalidate the observed GRASP Controller role**.

The controller is relatively short and delegates major supporting work. It is not simply performing all operations internally.

Origin: Mixed. Rails provides routing, request callbacks, persistence and job infrastructure; DaPlanStan owns the use-case decisions.

Final decision: CONFIRMED.

Stable instance name: `Trip Creation Coordination`

---

3. Candidate B — Trip Membership Management Coordination

CONFIRMED

System events and receiving object

The application handles two related system events:

- Add a traveler to an existing trip: `POST /trips/:trip_id/trip_members`.

- Remove a traveler from an existing trip: `DELETE /trips/:trip_id/trip_members/:id`.

Both are handled by `TripMembersController`, whose routes are declared in `config/routes.rb:40`.

The receiving methods are `TripMembersController#create`, lines 6–15, and `TripMembersController#destroy`, lines 17–22.

Application decisions

The addition operation is more complex than basic CRUD.

`TripMembersController#create` checks whether the request contains a traveler ID or an email address.

If an ID is supplied, the controller finds the traveler within the trip's account.

If an email is supplied, it invokes `resolve_and_add_by_email`, which searches for an existing account traveler, checks for a registered user, creates a traveler record when needed and optionally requests an invitation email.

Both paths eventually invoke `add_traveler`.

The source shows the controller making these decisions:

‹/› Ruby

```ruby
def add_traveler(traveler)
  if @trip.trip_members.exists?(traveler: traveler)
    redirect_to travelers_trip_path(@trip),
      alert: "#{traveler.name} is already on this trip."
  else
    @trip.trip_members.create!(
      traveler: traveler,
      role: :contributor,
      joined_at: Time.current
    )
    redirect_to travelers_trip_path(@trip),
      notice: "#{traveler.name} added to trip."
  end
end
```

For deletion, the controller checks whether the selected membership belongs to a trip owner and prevents its removal if so.

The controller also applies a shared owner/planner permission check before either membership-management event.

Delegated responsibilities

Participant	Source location	Responsibility
`TripMembersController#create`	`app/controllers/trip_members_controller.rb:6–15`	Receive and coordinate member addition
`TripMembersController#destroy`	Same file, lines 17–22	Coordinate member removal
`TripMembersController#require_owner_or_planner`	Same file, lines 30–35	Enforce membership-management permission
`TripMembersController#add_traveler`	Same file, lines 37–44	Check existing membership and add traveler
`TripMembersController#resolve_and_add_by_email`	Same file, lines 46–65	Resolve user/traveler identity and coordinate invitation
`ApplicationController#current_member_for`	`app/controllers/application_controller.rb:22–25`	Resolve the acting user's membership
`Account` and `Traveler`	`app/models/account.rb:1–11`; `app/models/traveler.rb:1–8`	Supply account and traveler relationships
`User`	`app/models/user.rb:11–17`	Supply registered-user identity and associations
`TripMember`	`app/models/trip_member.rb:1–6`	Store membership and role
`InvitationMailer#invite_to_trip`	`app/mailers/invitation_mailer.rb:8–13`	Construct the invitation email

The controller delegates data operations to Active Record associations and invitation delivery to Action Mailer. It does not implement mail transport or database persistence itself.

Why this supports GRASP Controller

The controller acts as the entry point for a meaningful membership-management use case.

It coordinates account membership, registered-user identity, trip membership, permissions and invitations. These responsibilities span several domain objects.

The event handler determines which collaborators are required and how the operation proceeds.

That is evidence of a system-event coordination responsibility rather than merely the existence of a Rails controller class.

Strongest objection: controller-heavy design

This candidate has a stronger objection than trip creation.

`TripMembersController` directly implements substantial business rules:

- Email-based traveler resolution.
- Selection between registered users and email-only travelers.
- Creation of traveler profiles.
- Membership duplication checks.
- Assignment of the contributor role.
- Invitation decisions.
- Owner-removal protection.

These decisions are concentrated in the controller rather than delegated to a dedicated domain operation.

Does that invalidate GRASP Controller?

Not necessarily.

GRASP Controller identifies where responsibility for receiving and coordinating system events is assigned. It does not require every internal rule to have been perfectly delegated.

The controller clearly performs that coordination role. Nevertheless, its business-policy concentration is a design limitation and should be documented rather than described as an example of excellent separation of concerns.

Origin: Mixed. Rails provides routing and persistence infrastructure, while DaPlanStan owns the membership-management decisions.

Final decision: CONFIRMED.

Stable instance name: `Trip Membership Management Coordination`

4. Are these genuinely independent collaborations?

Yes. They are separate use-case collaborations, not merely separate controller classes.

Comparison	Trip Creation Coordination	Trip Membership Management Coordination
System event	Create a new trip	Add/remove members of an existing trip

Entry point	`TripsController#create`	`TripMembersController#create / #destroy`
Main objective	Establish a trip and initial ownership	Maintain trip participation
Authorization	Authenticated user	Existing trip owner or planner
Main objects	`Trip`, `Account`, `TripMember`	`Trip`, `Account`, `Traveler`, `User`, `TripMember`
Significant delegation	`Trip` initialization and cover-image job	Identity lookups and invitation delivery
Outcome	New trip and initial owner membership	Existing trip's membership changes

Why two instances rather than one?

The distinction is not controller class identity. It is **the independent system event and coordinated domain outcome**.

Trip creation establishes a new trip and its initial ownership. Membership management operates on a trip that already exists and handles member participation.

The two flows share `TripMember` and Active Record association machinery, but that shared infrastructure does not make their system events identical.

A particularly important source-level distinction is that the original owner membership is created directly from `TripsController#create` with `role: :owner`, whereas later additions use `TripMembersController#add_traveler` with `role: :contributor`. ‹/›

They have different entry points, authorization conditions, decisions and collaborators.

Therefore, under a **use-case collaboration counting rule**, the appropriate count is:

1	2	2
Pattern type	Instances	Event collaborations

This count is limited to the two reviewed candidates, not the whole application. It should be recorded alongside the explicit use-case counting rule and independently reviewed before submission.

For membership management, addition and removal are grouped into one cohesive responsibility rather than counted as separate instances for each HTTP action.

5. Candidate decision table

Candidate	Essential mechanism observed	Strongest objection	Origin	Final decision

Trip Creation Coordination	A system-event handler coordinates trip persistence, initial owner membership and background image retrieval	Business operation is sequenced in the controller without an explicit encompassing transaction	Mixed, application-owned coordination	**CONFIRMED**
Trip Membership Management Coordination	A system-event handler coordinates traveler resolution, membership changes, authorization and invitations	Substantial business rules remain concentrated in the Rails controller	Mixed, application-owned policy and coordination	**CONFIRMED**

6. Participant-role mapping for the inventory

GRASP Controller does not prescribe a GoF-style hierarchy of interfaces. The mapping below identifies only the actual receiving objects and their collaborators.

Instance	Responsibility role	Actual implementation	Source
Trip Creation Coordination	GRASP Controller	`TripsController#create`	`app/controllers/trips_controller.rb:35–48`
	Trip domain object	`Trip` and its creation callback	`app/models/trip.rb:1–19, 31–37`
	Owner lookup collaborator	`Account#owner_traveler`	`app/models/account.rb:9–11`
	Membership collaborator	`TripMember` through `@trip.trip_members.create!`	`app/controllers/trips_controller.rb:41–42`; `app/models/trip_member.rb:1–6`
	Background-work collaborator	`FetchTripCoverImageJob`	`app/controllers/trips_controller.rb:43`; `app/jobs/fetch_trip_cover_image_job.rb:4–30`
Trip Membership Management Coordination	GRASP Controller	`TripMembersController#create`, `#destroy`, `#add_traveler`, `#resolve_and_add_by_email`	`app/controllers/trip_members_controller.rb:6–22, 37–65`
	Authorization collaborator	`ApplicationController#current_member_for`	`app/controllers/application_controller.rb:22–25`
	Identity and membership collaborators	`User`, `Account`, `Traveler`, `TripMember`	`app/controllers/trip_members_controller.rb:37–65`; corresponding model files

Notification collaborator	`InvitationMailer#invite_to_trip`	`app/mailers/invitation_mailer.rb:8–13`

The same class can appear in multiple records where it actually participates. Unique participant-symbol totals should deduplicate those symbols according to the team's counting methodology.

7. Evidence to preserve

For this verification, preserve the following source excerpts and explanations alongside the original archive:

1. `TripsController#create`, lines 35–48, showing the entire creation sequence.
2. `Trip` lines 1–19 and 31–37, showing model responsibilities.
3. `Account#owner_traveler`, lines 9–11, and `TripMember` lines 1–6, showing ownership representation.
4. `FetchTripCoverImageJob#perform`, lines 4–30, proving delegation of image retrieval.
5. `TripMembersController`, lines 1–65, showing authorization, addition, removal, identity resolution and invitation decisions.
6. `ApplicationController#current_member_for`, lines 22–25, showing shared membership lookup.
7. `config/routes.rb:21–41`, proving that the two use cases are independently addressable system events.
8. A reviewer note explaining why these are two distinct use-case collaborations, rather than one instance per Rails controller class.

These excerpts establish the static design. No tests, runtime logs or independent human sign-off were produced in this review.

8. Final candidate-review record

Writing

Trip Creation and Membership — Candidate Verification

Source version: Original DaPlanStan source archive. ZIP-to-stated-commit correspondence remains unverified.

Review scope: Trip creation and trip membership management.
Method: Static verification of system-event entry points, application decisions, delegation and participant relationships.

Candidate 1: Trip Creation Coordination

- Pattern: GRASP Controller
- Status: CONFIRMED (static source assessment)
- Main participant: `TripsController#create`
- Evidence: `app/controllers/trips_controller.rb:35–48`

The controller coordinates trip creation, initialization of the owner membership and delegation of cover-image retrieval. This constitutes an identifiable application-level system-event responsibility. The lack of an explicit transaction across trip and membership creation remains a design limitation.

Candidate 2: Trip Membership Management Coordination

- Pattern: GRASP Controller
- Status: CONFIRMED (static source assessment)
- Main participant: `TripMembersController`
- Evidence: `app/controllers/trip_members_controller.rb:1–65`

The controller handles membership-management events, including authorization, traveler resolution, membership creation/removal and invitation coordination. The role is supported by actual application decisions spanning multiple objects. However, substantial business rules remain in the controller, indicating controller-heavy procedural orchestration.

Counting decision

These are two independent use-case collaborations with different system events, objectives, authorization rules and participant interactions. They represent two GRASP Controller instances under the documented use-case collaboration counting rule, not two instances merely because they reside in different Rails controllers.

Rejected or uncertain classifications

No additional pattern classifications were proposed or rejected in this verification. Neither candidate remains structurally uncertain, although the appropriateness of counting conventional Rails event handlers as GRASP instances should receive independent faculty/team review.

Limitations

Source-based confirmation does not establish runtime correctness, design quality or completed human verification. Preserve the controller-heavy and transaction-consistency concerns separately from the pattern classification.

Final conclusion: Both candidates meet the GRASP Controller responsibility rule at the source level, with different strengths and design trade-offs. They can be proposed as two distinct baseline instances, subject to human verification and consistent counting across the rest of the repository.

Now move to the next source area: itinerary-item creation, imported record linking and expense handling.
As before, do not begin by assigning pattern names. Trace the actual collaborations first.
Inspect these files and any directly related callers needed to complete the flows:

1. `app/controllers/trip_items_controller.rb`
2. `app/models/trip_item.rb`
3. `app/models/inbox_item.rb`
4. `app/models/expense.rb`
5. `app/controllers/expenses_controller.rb`
6. `app/jobs/parse_receipt_job.rb`
7. Relevant views, routes and model associations where needed.

Some `InboxItem` behavior was already examined during the inbound-email investigation. Reuse established evidence where valid, but do not count the same collaboration again under a new name.
I want to understand these areas separately.

1. Manual itinerary-item creation and editing.

Trace how a user creates, updates or removes a `TripItem`, including any model behavior or related expense handling.

2. Bulk itinerary import.

Trace the import path and `bulk_create_items` behavior. Identify how records are created, how geocoding is handled and what logic is application-written versus Rails behavior.

3. Inbox-derived record linking.

Use the already traced `InboxItem#fill_missing_links!` and related methods only where they interact with this source area. Determine whether this reveals a genuinely new collaboration or is simply the previously confirmed Parsed Email to Trip Record Linking instance.

4. Expense creation and management.

Trace normal expense creation/editing and any automatic expense creation connected to itinerary items or inbox data.

5. Receipt parsing.

Trace exactly what `ParseReceiptJob` receives, what external service or library it uses, what information it extracts and how that result changes an `Expense`.
For each flow:

1. Identify the actual participating classes and methods.
2. Give reliable source paths and line ranges.
3. Trace the control flow and important decisions.
4. Separate application-owned behavior from Rails and external-provider behavior.
5. Identify where responsibilities are assigned and delegated.
6. Check whether apparently similar creation paths actually share one collaboration or represent independent responsibilities.
7. Do not count the already confirmed `Parsed Email to Trip Record Linking` instance again merely because `TripItem` and `Expense` appear here.

Only after tracing the flows, identify any genuinely new design-pattern candidates.
For each candidate:

1. State the possible pattern and category.
2. Give a meaningful collaboration name.
3. Identify the actual participants.
4. Explain what behavior supports the classification.
5. Give the strongest false-positive explanation.
6. State whether the behavior is application-owned, framework-provided or mixed.
7. Leave the candidate UNCONFIRMED.

Do not force Factory, Builder, Strategy, Command, Template Method, Adapter, Creator or Information Expert unless the defining mechanism or responsibility is actually supported.
At the end give me:

1. Manual TripItem flow.
2. Bulk-import flow.
3. Inbox-linking overlap analysis.
4. Expense-management flow.
5. Receipt-parsing flow.
6. Participant/source-evidence table.
7. New candidate patterns only.
8. Previously confirmed patterns encountered again but not counted again.
9. Strongest non-pattern explanation.
10. Anything that still requires verification.

Worked for 1m 57s

DaPlanStan — Itinerary Items, Imported Linking and Expenses

Candidates unconfirmed Source Investigation 04 · Original uploaded ZIP

1. Initial findings

This source area contains several paths for creating and updating travel data, but they do **not** all belong to one collaboration.

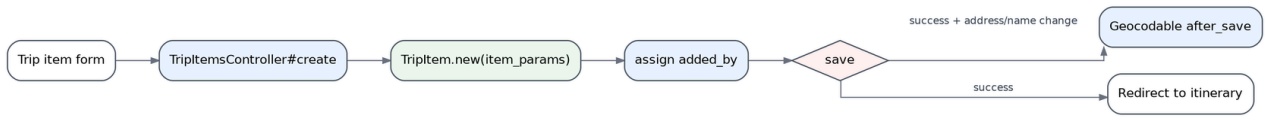
- Manual itinerary management uses `TripItemsController` and `TripItem`.
- Bulk JSON import uses a separate controller method with duplicate-name checking and queued geocoding.
- Email-derived linking uses the previously examined `InboxItem` methods.
- Expenses can be entered manually, prefilled from itinerary items or created from confirmed email data.
- Receipt parsing uses a separate background job that invokes Anthropic and updates an existing `Expense`.

A particularly important distinction is that `TripItemsController#to_expense` **does not create an expense**. It prepares values and redirects to the expense form.

Similarly, `ParseReceiptJob` does not create an expense. It operates on an expense that already exists.

2. Manual itinerary-item creation and editing

Control flow



A. Creating an item

`TripItemsController#create` , `app/controllers/trip_items_controller.rb:64-75` , performs:

Ruby

```ruby
@item = @trip.trip_items.new(item_params)
@item.added_by = current_user

if @item.save
  redirect_to trip_path(@trip, tab: "itinerary"), notice: "Item added."
else
  render :new, status: :unprocessable_entity
end
```

The controller is responsible for selecting the trip, building the record from permitted fields and assigning the user who added it.

The `TripItem` model, at `app/models/trip_item.rb:1-28` , defines the relationships, kind and status enums, and validations.

The model validates the item's name and kind, and validates URL format when a URL is supplied.

B. Geocoding during saving

`TripItem` includes the `Geocodable` concern at line 2.

The concern registers an `after_save` callback that may invoke `geocode_from_address` when the name or address changes and coordinates are missing.

`TripItem#geocode_from_address`, lines 34–37, avoids attempting to geocode a transport item without an address. Otherwise, it invokes the concern's implementation through `super`.

This is **the same shared geocoding behavior already examined in Source Investigation 01**, not a new independent pattern instance.

C. Updating and confirming an item

`TripItemsController#update`, lines 77–84, calls:

```ruby
@item.update(item_params)
```

No separate application-defined editing strategy appears in this action. Rails Active Record performs the update and its normal lifecycle processing.

The controller also has a `confirm` action at lines 90–96:

```ruby
new_status = @item.confirmed? ? :idea : :confirmed
@item.update!(status: new_status)
```

The status is a string-backed Active Record enum with two values: `idea` and `confirmed`.

The action changes the persisted status. It does not switch between separate state objects or behavioral implementations.

D. Deleting an item

`TripItemsController#destroy`, lines 85–89, calls `@item.destroy!`.

The model declares:

```ruby
has_one :expense, dependent: :nullify
```

Consequently, destroying a `TripItem` invokes the association's dependent-nulling behavior rather than directly deleting its associated expense.

The expected result is that the expense survives without its previous `trip_item_id` association, assuming the database operation succeeds.

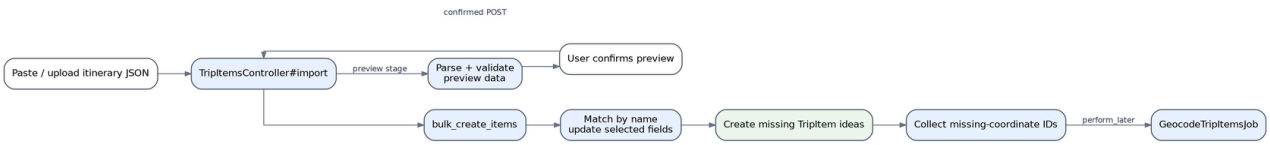
Responsibility assessment: Manual item creation, editing and deletion are primarily straightforward Rails controller and Active Record operations, with reusable geocoding behavior attached to the model. No new design-pattern classification is warranted from these operations alone.

3. Bulk itinerary import

Entry point and two-stage flow

Routes are declared at `config/routes.rb:30–34`, with GET and POST actions for the trip's itinerary import page.

The view, `app/views/trip_items/import.html.erb`, provides a form for pasting JSON or uploading a `.json` file. It describes the data as AI-generated ideas, but **this import action does not call Anthropic or another AI provider**. The JSON is supplied by the user.



What the application actually decides

`TripItemsController#import`, lines 31–58, has a preview phase and a confirmation phase.

During preview it reads either `items_json` or `json_file`, parses the input and requires an Array. Parse errors are caught and displayed. The confirmation branch instead directly parses `params[:items_json]` and passes the result to `bulk_create_items`; it does not repeat the same shape validation.

`bulk_create_items`, lines 128–174, implements the actual import.

Condition	Application behavior
Name missing	Increment skipped count
Name already in trip, case-insensitively	Find existing record; optionally fill coordinates or URL; count duplicate
New name	Create `TripItem` with imported attributes and `status: "idea"`
Kind not in `TripItem.kinds`	Use `"other"`
Start/end time unparseable	`parse_import_time` returns `nil`
Created item lacks complete coordinates	Collect its ID for background geocoding
Per-item exception	Log error and continue processing remaining items

The import maintains a set of existing names and updates it as new items are created. This also avoids repeatedly creating an item with the same name within one import request.

Geocoding behavior

Before processing, the controller calls:

```ruby
TripItem.skip_callback(:save, :after, :geocode_from_address)
```

After the item loop, it schedules:

```ruby
GeocodeTripItemsJob.perform_later(needs_geocode_ids, @trip.id)
```

An `ensure` clause calls:

```ruby
TripItem.set_callback(:save, :after, :geocode_from_address)
```

This is the same background geocoding job from the earlier investigation.

The importer temporarily bypasses automatic model geocoding so that each new item does not perform its lookup while the batch is being created.

There are two source-level concerns to preserve:

- `skip_callback` changes the callback configuration at the `TripItem` class level, rather than only for an individual imported record.
- The restoration call does not reproduce the original conditional predicate from `Geocodable`. Whether the resulting callback configuration is equivalent requires verification.

Bulk import does not automatically create `Expense` records. Although imported `TripItem` records may contain an `amount` and `currency`, the method creates itinerary records only.

Responsibility assessment

The controller handles parsing, validation of import structure, duplicate detection, partial updates, item creation and geocoding scheduling. Rails supplies persistence, callback configuration and background-job enqueueing.

This is a distinct application workflow from manual item creation, with substantially more logic concentrated in the controller.

4. Inbox-derived record linking — overlap analysis

The exact previously established call path remains:

`InboxItemsController#update` → `InboxItem#confirm_and_create_items!` → `InboxItem#fill_missing_links!` → `TripItem` and, where appropriate, `Expense`.

The main application decisions occur in `app/models/inbox_item.rb`.

Method or area	Lines	Responsibility
`confirm_and_create_items!`	34–58	Run record linking and add shortlist ideas
`fill_missing_links!`	67–187	Interpret parsed travel records and find/create matching itinerary items

`find_or_upsert` local lambda	82–99	Reuse a matching item or create a confirmed one
Travel-leg processing	101–138	Create/link transport items and optionally one expense
Accommodation processing	140–158	Create/link stays and optionally expenses
Booking processing	160–187	Create/link bookings and optionally expenses
`build_expense`	191–207	Create an expense linked to the trip, item and source email

An important detail is that existing-item matching is primarily based on a lowercase name. When a match exists, the method fills selected missing attributes using `update_columns`; it does not generally overwrite existing itinerary details with every new extracted field.

Expense creation requires an identifiable payer and the relevant price/existence conditions. The model's `build_expense` helper sets `source: :booking` and `confirmed_at: Time.current`.

Does this introduce a new collaboration?

No.

This is the same responsibility already recorded as:

`Parsed Email to Trip Record Linking`

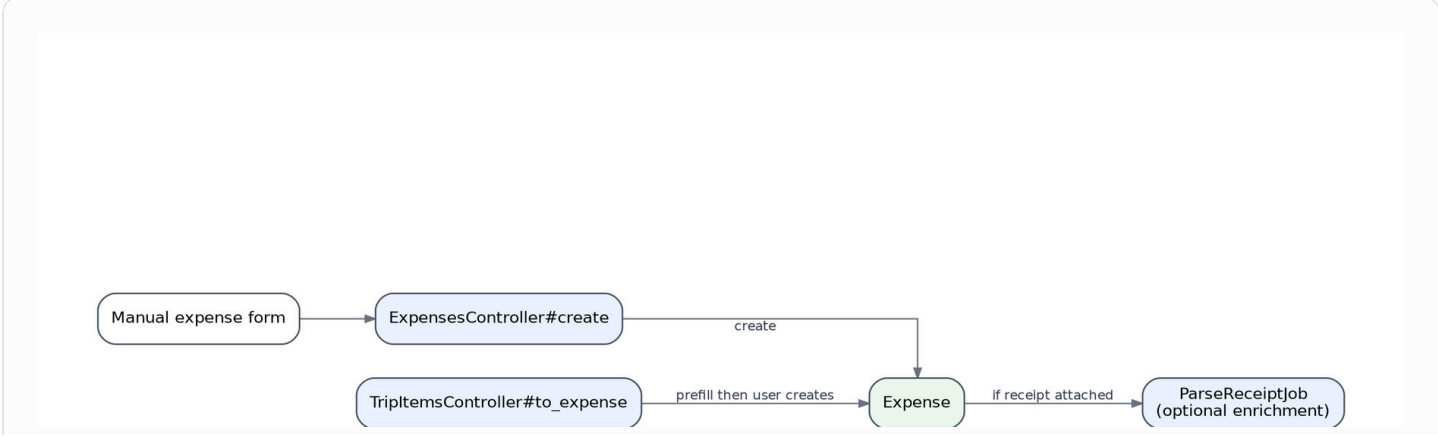
The presence of `TripItem` and `Expense` in this investigation does not create a second instance of GRASP Information Expert.

In contrast, the JSON bulk importer is an independently entered workflow owned by `TripItemsController`, using different input data, record statuses and duplication logic.

These paths share model classes and Active Record operations, but they are not interchangeable implementations of one demonstrated design-pattern mechanism.

5. Expense creation and management

Expenses have three distinct creation or entry paths.





A. Normal manual expense creation

`ExpensesController#new`, `app/controllers/expenses_controller.rb:25–38`, prepares an expense with default currency `EUR`, category `other`, today's date and a payer derived from the current user's trip membership where available.

`ExpensesController#create`, lines 40–51, builds the record through the trip association, assigns `added_by` and saves it.

```ruby
@expense = @trip.expenses.new(expense_params)
@expense.added_by = current_user

if @expense.save
  ParseReceiptJob.perform_later(@expense.id) if @expense.receipt.attached?
  redirect_to trip_path(@trip, tab: "expenses"), notice: "Expense added."
end
```

The `Expense` model specifies its relationships and validations at `app/models/expense.rb:1–28`. It requires an amount, currency, description, category and expense date. It also declares a non-optional `paid_by` association.

`Expense` may optionally reference a `TripItem`, an `InboxItem` and the user who confirmed payment. Rails Active Record and Active Storage supply persistence and attachment behavior.

B. Converting a TripItem into an expense

This is a user-assisted workflow rather than automatic conversion.

`TripItemsController#to_expense`, lines 98–114:

- Checks whether the item already has an associated expense.
- If one exists, redirects to it.
- Otherwise builds form-prefill values from the item's name, kind, date, currency, amount and ID.

- Redirects to `ExpensesController#new`.

The expense form is initialized using those values at `ExpensesController#new:33–36`.

No `Expense` is persisted until the expense form is submitted successfully.

The conversion check is also not a database-enforced uniqueness guarantee. In the inspected schema, `expenses.trip_item_id` has a normal index, not a unique index. Concurrent submissions and cross-trip references therefore warrant separate verification. ‹/› ‹/›

C. Editing, deleting and marking expenses as paid

`ExpensesController#update`, lines 57–72, updates permitted attributes and optionally handles `mark_paid`.

When `mark_paid` is `"1"`, it sets `confirmed_at` and `confirmed_by` unless already present. Otherwise, when that parameter is explicitly supplied with a different value, it clears those fields.

`ExpensesController#confirm`, lines 79–86, separately toggles the payment-confirmation state.

`ExpensesController#destroy`, lines 74–77, destroys the expense itself.

`Expense#confirmed?`, `app/models/expense.rb:30–32`, simply checks whether `confirmed_at` is present. This is persisted data with a predicate, not a separately implemented behavioral state object.

D. Displaying expenses linked to email-derived items

`TripItemsController#show`, lines 18–29, contains additional display logic for items originating from the same `InboxItem`.

If the currently viewed item lacks its own confirmation reference or expense, the controller looks at sibling itinerary items connected to the same source email.

This helps display information shared by related booking records, such as outbound and return flight legs.

Crucially, this is **presentation-time fallback logic**. It does not create a new expense or change which `TripItem` the expense is actually linked to.

E. Automatic expense creation from inbox data

The automatic path remains `InboxItem#build_expense`, lines 191–207.

It creates a booking-sourced expense through `trip.expenses.create!`, assigns the itinerary item, source inbox record, payer, amount, category, date and confirming timestamp.

That behavior is part of the previously reviewed `Parsed Email to Trip Record Linking` collaboration. It must not be counted again as a separate Information Expert instance.

---

6. Receipt parsing — exact implementation

Primary source: `app/jobs/parse_receipt_job.rb:1-60`

Trigger and input

The job is scheduled by `ExpensesController#create:45` only after a successful save and only if the expense already has an attached receipt.

Its `perform(expense_id)` method receives **an expense ID**, not the receipt bytes directly.

The job performs these operations:

- **Locate the expense**

  `Expense.find_by(id: expense_id)` retrieves the record. The job returns unless a receipt is attached.

- **Prepare the attachment**

  Active Storage downloads the receipt bytes. Ruby's `Base64.strict_encode64` encodes them, and the attachment's content type is read.

- **Call Anthropic**

  The job creates an `Anthropic::Client` and calls `client.messages.create`, using model `claude-haiku-4-5-20251001`, an image content block and a text prompt requesting receipt details.

- **Read the result**

  The application takes the first returned text block, extracts JSON-looking content with a regular expression and calls `JSON.parse`.

- **Update the existing expense**

  When the parsed hash is nonblank, it constructs an update hash using returned values, falling back to existing expense values for missing or falsey fields. It then calls `expense.update!`.

- **Handle failure**

  A broad rescue logs the exception. The job does not create a new expense, store a separate extraction record or visibly expose an extraction-failure status.

The prompt requests the following fields:

Extracted field	Intended destination

`amount`	`Expense#amount`
`currency`	`Expense#currency`
`description`	`Expense#description`
`expense_date`	`Expense#expense_date`
`category`	`Expense#category`

The actual image interpretation is performed by Anthropic's remote model. There is no separate application-defined OCR algorithm or dedicated OCR library invocation in this job.

DaPlanStan owns the prompt, output extraction, field selection and persistence logic. Active Storage supplies attachment retrieval, Active Job supplies execution infrastructure, and Anthropic supplies model inference.

Important source-level issues

⚠ **1. Receipt category mismatch**

The prompt asks Anthropic to return categories such as `accommodation`, `food_drink`, `transport`, `activities` and `shopping`.

However, `Expense.category` accepts `stay`, `eat`, `do`, `shop`, `flight`, `car`, `train`, `ferry` and `other`.

Only `other` appears in both sets. The job contains no mapping from the prompt categories to the model's accepted enum values.

A typical response such as `"category": "food_drink"` would therefore cause an invalid enum assignment rather than a successful expense update. The rescue logs the error. This is a source-established incompatibility, although its frequency in real executions has not been measured.

⚠ **2. Receipt attachment on update does not schedule parsing**

The receipt button in `app/views/expenses/show.html.erb:212–227` submits a PATCH request to the expense update action.

`ExpensesController#update` permits `:receipt` and saves it, but does not call `ParseReceiptJob.perform_later`.

The job is only enqueued in `ExpensesController#create`.

This means the visible receipt-upload path on an existing expense does not directly trigger the parsing job. The source-wide call search found no other application caller of `ParseReceiptJob`.

‹/› ‹/›

⚠ **3. Extracted values have limited validation before model update**

`JSON.parse` is used directly on text selected through a broad `{...}` regex. The job does not independently verify that returned field types, dates or categories match the application's expectations.

Model validation and enum handling provide some safeguards, but failures are caught and logged rather than presented to the user.

⚠ **4. Receipt parsing does not update expense provenance**

The `Expense` model supports a `source` enum including `manual`, `receipt_photo` and `booking`.

`ParseReceiptJob` does not assign `source: :receipt_photo`, so successful parsing does not itself record receipt-derived provenance in that field.

These findings are relevant to verification and application correctness. **They do not themselves demonstrate a design pattern.**

7. Participant and source-evidence register

All paths are relative to the uploaded `daplanstan-main.zip`. Line numbers refer to that source snapshot, whose correspondence to the stated Git commit remains unverified.

Participant / method	Exact source location	Observed responsibility
Itinerary and expense routes	`config/routes.rb:30–44`	Define request entry points
`TripItemsController#create`	`app/controllers/trip_items_controller.rb:64–75`	Create manual itinerary item
`TripItemsController#update`	Same file, lines 77–84	Edit itinerary item
`TripItemsController#destroy`	Same file, lines 85–89	Delete itinerary item
`TripItemsController#confirm`	Same file, lines 90–96	Toggle idea/confirmed status
`TripItemsController#to_expense`	Same file, lines 98–114	Prefill expense form without creating a record
`TripItemsController#import`	Same file, lines 31–58	Handle JSON preview and confirmed import
`TripItemsController#bulk_create_items`	Same file, lines 128–174	Deduplicate, create ideas and schedule geocoding
`TripItemsController#parse_import_time`	Same file, lines 176–179	Parse imported dates

`TripItem`	`app/models/trip_item.rb:1–28`	Associations, enums and validation
`TripItem#geocode_from_address`	Same file, lines 32–37	Domain-specific geocoding guard
`Geocodable`	`app/models/concerns/geocodable.rb:1–75`	Shared geocoding behavior
Import form and confirmation	`app/views/trip_items/import.html.erb:29–38, 93–106, 189–213`	JSON input, preview and submission
`InboxItemsController#update`	`app/controllers/inbox_items_controller.rb:16–35`	Confirm, relink, discard or retry
`InboxItem#confirm_and_create_items!`	`app/models/inbox_item.rb:34–58`	Link imported email data and add ideas
`InboxItem#fill_missing_links!`	Same file, lines 67–187	Match/create items and associated expenses
`InboxItem#build_expense`	Same file, lines 191–207	Create booking-derived expense
`ExpensesController#new`	`app/controllers/expenses_controller.rb:25–38`	Initialize manual/prefilled expense
`ExpensesController#create`	Same file, lines 40–51	Save expense and conditionally enqueue receipt parsing
`ExpensesController#update`	Same file, lines 57–72	Update expense and paid status
`ExpensesController#destroy`	Same file, lines 74–77	Delete expense
`ExpensesController#confirm`	Same file, lines 79–86	Toggle paid confirmation
`Expense`	`app/models/expense.rb:1–32`	Associations, attachment, enums and validation
`ParseReceiptJob#perform`	`app/jobs/parse_receipt_job.rb:4–59`	Extract receipt data and update an expense
Receipt-upload form	`app/views/expenses/show.html.erb:212–227`	Submit attached receipt on an existing expense

The controller/model implementations and principal job were read directly from the uploaded source, rather than inferred from the Part 1 handoff.

8. Responsibility assignment and collaboration boundaries

Workflow	Primary application responsibility	Delegation	Independent collaboration?

Manual itinerary operations	`TripItemsController`	Active Record and shared Geocodable behavior	Distinct CRUD workflow
Bulk JSON import	`TripItemsController#import` / `bulk_create_items`	Active Record and `GeocodeTripItemsJob`	Yes, distinct import use case
Inbox record linking	`InboxItem`	`TripItem`, `Expense` and Active Record	**Existing confirmed collaboration — do not recount**
Manual expense management	`ExpensesController`	`Expense`, Active Record and optional receipt job	Distinct CRUD workflow
TripItem expense prefill	`TripItemsController#to_expense`	Expense new form	Part of manual expense-entry workflow
Receipt extraction	`ParseReceiptJob`	Active Storage, Anthropic and `Expense`	Distinct background operation

This illustrates an important design characteristic: DaPlanStan does not consistently assign all business logic to either models or controllers.

Manual entry is comparatively thin-controller CRUD. Bulk import contains substantial transformation and validation logic in a controller. Inbox linking contains substantial business logic in the `InboxItem` model. Receipt extraction is concentrated in a background job.

Sharing `TripItem` or `Expense` as a data model does not mean all these paths constitute the same design-pattern instance.

9. Genuinely new design-pattern candidates

Only one new candidate is strong enough to warrant formal verification.

GRASP Controller

UNCONFIRMED

Provisional collaboration name: Bulk Itinerary Import Coordination

Category: GRASP responsibility assignment · Origin: Mixed, with application-owned decisions

Proposed system event

A user previews and confirms a batch of externally prepared itinerary ideas.

Actual participants

`TripItemsController#import` receives the request and delegates to `bulk_create_items`. The latter creates or updates `TripItem` records, invokes Active Record operations and schedules `GeocodeTripItemsJob`.

Why the candidate might fit

The controller handles a meaningful system-level use case that requires more than a simple record insertion. It coordinates validation of imported structure, preview/confirmation, duplicate detection, item creation and deferred geocoding.

The work crosses several responsibilities, and the controller decides when each operation is appropriate.

Strongest false-positive argument

Nearly the entire import algorithm is implemented directly inside the Rails controller's private method.

There is limited delegation of domain decisions to separate application objects. This may be better understood as controller-heavy procedural code rather than a well-separated GRASP Controller responsibility assignment.

Furthermore, the presence of a controller receiving a request is not sufficient to prove GRASP Controller. The source must support the assignment of a cohesive system-event responsibility.

Verification needed

Evaluate whether preview, confirmed import and batch creation form one meaningful use-case responsibility; compare that boundary with the previously reviewed GRASP Controller instances; and obtain independent human review.

Current status: UNCONFIRMED.

Why no additional candidates?

`ExpensesController#create` and the manual `TripItem` actions are predominantly direct Rails CRUD handling. They do not currently offer enough independent design evidence to justify another GRASP Controller instance beyond ordinary request processing.

`ParseReceiptJob` orchestrates a particular background task, but the inspected implementation mainly downloads an attachment, calls an external provider and updates a record. That alone does not establish a new GoF or GRASP pattern.

`InboxItem#build_expense` belongs to the existing Information Expert responsibility. It should not be relabeled as a separate Creator or Factory instance merely because it calls `create!`.

Likewise, the fixed lookup and category-switching behavior does not demonstrate interchangeable algorithms, specialized object-construction protocols or other GoF mechanisms.

10. Previously identified patterns encountered again

Previously identified collaboration	How it reappears	Counting decision

`Parsed Email to Trip Record Linking` — GRASP Information Expert	`InboxItem` creates/links itinerary and expense records	Already source-confirmed in Investigation 02; no additional instance
Geocoding collaboration — previous classification review	Manual saves and imported records reuse `Geocodable` and `GeocodeTripItemsJob`	No new confirmed instance; retain previous rejected/uncertain conclusions

The source does not establish a new pattern simply because the same domain entities participate in another workflow.

11. Strongest non-pattern explanation

This source area is best understood as **several Rails data-entry and enrichment workflows sharing Active Record domain models**.

Manual actions use conventional controller operations. Bulk import performs application-written JSON transformation, deduplication and deferred geocoding. Confirmed email data is interpreted by an existing model responsibility. Expense management uses standard persistence and status updates. Receipt extraction delegates image interpretation to Anthropic through a background job.

This explanation accounts for the actual implementation without introducing unobserved factories, strategies, builders, adapters or other abstractions.

12. Items requiring further verification

Open issue	Why it matters
Bulk import callback restoration	Original conditional callback may not be restored identically
Import concurrency	Class-level callback changes may affect unrelated saves
Import input validation	Confirmation branch does not repeat preview's JSON-array validation
Duplicate detection	Name-based matching may merge different places with identical names
Import atomicity	Per-item rescue allows partial batch success
Expense associations	Submitted `trip_item_id` and payer ID are permitted but not explicitly scoped to the trip in the expense controller
Duplicate expenses	Application checks are not backed by a unique `trip_item_id` index
Receipt category mismatch	Prompt categories do not match the Expense enum except for `other`
Receipt upload trigger	Existing-expense PATCH upload does not enqueue parsing
Receipt extraction behavior	No successful Anthropic response or expense update was runtime-verified

Candidate responsibility	Bulk import still requires skeptical GRASP Controller review
Human verification	No independent review outcome or executed test evidence has been recorded

13. Investigation outcome

Source Investigation 04 Candidate verification pending

5	1	0
Flows traced	New candidate	New confirmed instances

Conclusion for the investigation record

Writing

Itinerary Items, Record Linking and Expense Handling — Preliminary Candidate Review

Source: Original DaPlanStan source archive
Method: Static source inspection
Runtime tests: Not performed

Manual itinerary and expense management primarily use Rails controllers and Active Record persistence. Bulk itinerary import contains a distinct application-defined workflow involving JSON preview, duplicate-name checking, record creation and deferred geocoding.

The `InboxItem` linking methods reuse the previously identified `Parsed Email to Trip Record Linking` GRASP Information Expert collaboration. No additional instance is claimed for that behavior.

Receipt parsing uses `ParseReceiptJob` to send an attached image to Anthropic, extract structured values and update an existing expense. No new design pattern has been established by this external-service interaction.

New candidate: Bulk Itinerary Import Coordination — GRASP Controller — UNCONFIRMED.

Strongest alternative explanation: Controller-heavy procedural import logic within a conventional Rails application.

Verification observations: Callback restoration, partial imports, duplicate handling, expense

linkage, receipt-upload scheduling and receipt category mapping require further testing or review.

Inventory decision: No new confirmed pattern records or count changes from this investigation. Preserve the bulk-import candidate for a separate responsibility-assignment verification.

The next source-analysis step within this area is to evaluate **Bulk Itinerary Import Coordination** against the GRASP Controller responsibility rule, particularly whether it represents meaningful use-case coordination or simply a large procedural controller method.

Before moving to another source area, verify the `Bulk Itinerary Import Coordination` candidate.
The proposed classification is GRASP Controller.
Do not confirm it just because the logic is inside `TripItemsController`.
Check the actual source and answer:

1. What is the system event or use case being handled?
2. Which method receives the event and which methods participate in carrying it out?
3. Does the preview stage and confirmed-import stage form one cohesive use-case responsibility, or are they better treated as unrelated controller actions?
4. What application decisions are made by `TripItemsController#import` and `bulk_create_items`?
5. Which responsibilities are delegated to `TripItem`, Active Record and `GeocodeTripItemsJob`?
6. Give the exact participating symbols, source paths and reliable line ranges.
7. Explain why this could qualify as GRASP Controller.
8. Then give the strongest argument against that classification.
9. Specifically consider whether the code is better described as controller-heavy procedural import logic rather than a meaningful GRASP Controller responsibility assignment.
10. Compare it with the already confirmed GRASP Controller instances:
    `Inbound Travel Email Intake Coordination`

`Trip Creation Coordination`

`Trip Membership Management Coordination`

11. If this candidate is confirmed, explain why it is an independent system-event collaboration and should count as another GRASP Controller instance rather than being merged with the other controller instances.
12. If it is rejected, explain what is missing compared with the previously confirmed Controller instances.

Give the final result as CONFIRMED, REJECTED or UNCERTAIN.
At the end provide:

1. Final candidate decision.
2. Participant-role mapping if confirmed.
3. Stable instance name if confirmed.
4. Source evidence to preserve.
5. Counting decision.
6. Any design limitations that should be recorded separately from the pattern classification.

Do not investigate any new pattern types in this prompt.

Worked for 58s

Bulk Itinerary Import Coordination — Final Pattern Verification

DaPlanStan | Source Investigation 04 | GRASP Controller verification

CONFIRMED **GRASP Controller — source-level classification**

Decision: `Bulk Itinerary Import Coordination` qualifies as a GRASP Controller responsibility assignment. The evidence is not simply that `TripItemsController` is a Rails controller. It handles a distinct system-level import operation involving preview, validation, duplicate resolution, persistence and deferred geocoding.

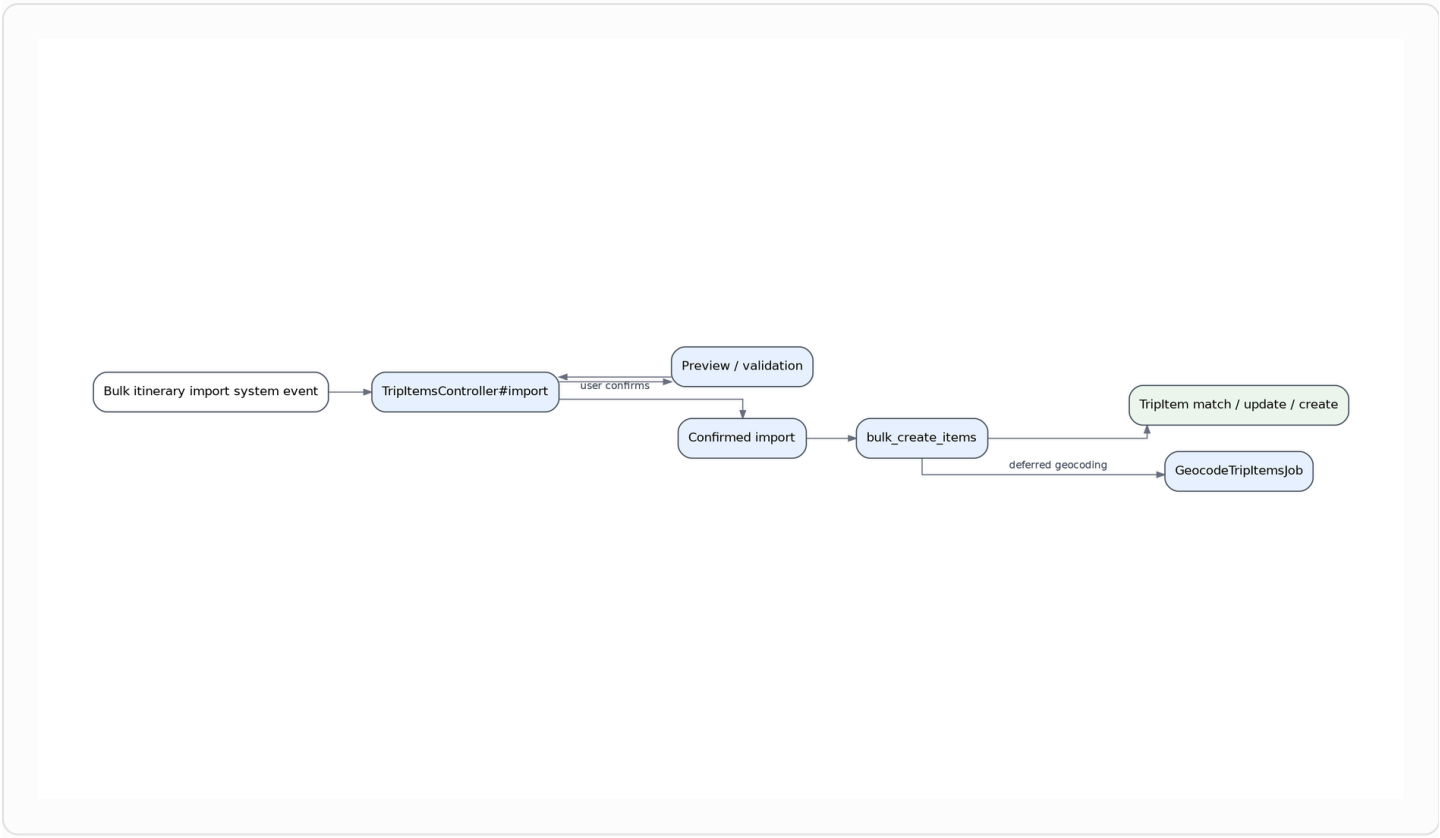
The implementation is also controller-heavy: most of the import algorithm resides inside `bulk_create_items`. That is a design limitation, but it does not eliminate the system-event coordination responsibility.

The confirmation is based on static source evidence. Independent human review and runtime testing remain pending.

1. System event and entry point

System event: A user requests to import a batch of itinerary ideas into an existing trip.

The use case has two stages:



The route is declared in `config/routes.rb:30–35`.

Both POST stages are received by `TripItemsController#import`, at `app/controllers/trip_items_controller.rb:31–58`.

The confirmed stage delegates batch processing to `TripItemsController#bulk_create_items`, lines 128–174.

Are preview and confirmation one cohesive use case?

Yes.

The preview stage parses the supplied JSON, checks that it contains an array, and prepares the items for review without creating records.

The confirmation form in `app/views/trip_items/import.html.erb:101–106` submits the reviewed JSON back to the same endpoint using `confirm: "1"`.

That submission triggers the persistence stage.

These are consecutive phases of one user goal: importing itinerary ideas. They do not warrant two separate GRASP Controller instances.

2. Application decisions and delegation

Decisions made by `TripItemsController#import`

Source lines	Application decision
`31–38`	Distinguish confirmed submission and invoke batch creation
`39–44`	Choose pasted JSON or uploaded file and handle missing input
`45–49`	Parse input and reject a non-array result during preview
`49–54`	Count missing-name entries and prepare error information
`34–38`	Construct an import-result message and redirect

The controller determines which use-case stage is active. Rails supplies request parameters and rendering/redirecting infrastructure, but these branching rules are written by DaPlanStan.

Decisions made by `bulk_create_items`

Source lines	Application decision
`129–133`	Initialize counts and collect existing names
`134`	Temporarily disable the model's geocoding callback
`135–147`	Skip unnamed entries, detect duplicate names and fill selected existing fields
`148–162`	Create new items, normalize kinds and assign `idea` status
`163–165`	Update the name lookup and collect IDs lacking coordinates
`166–168`	Log individual item failures and continue
`170–171`	Schedule geocoding when needed and return the result summary
`172–174`	Attempt callback restoration in `ensure`

The application therefore makes real import-policy decisions; the method is not merely a forwarding wrapper around `TripItem.create!`. ‹/›

Responsibilities delegated elsewhere

`TripItem` supplies the domain model, its validations, kind/status enums and `geocoded?` behavior through the included concern.

Active Record handles association-scoped queries, record insertion, column updates and callback registration.

`GeocodeTripItemsJob` receives the IDs of newly created items requiring geocoding. Its `perform` method loads each item, invokes `geocode_from_address` and requests a Turbo refresh after processing. ‹/›

The external geocoding calls remain in the existing `Geocodable` collaboration. They are not new behavior implemented by the import controller.

3. Why this qualifies as GRASP Controller

GRASP Controller assigns responsibility for receiving and coordinating a system event to an object representing the relevant system or use case.

The important distinction is between **receiving a request** and **coordinating the application's response to a meaningful business event**.

The importer demonstrates the second responsibility.

`TripItemsController#import` establishes the two-stage workflow, while `bulk_create_items` performs the batch-operation decisions and invokes the necessary collaborators.

The implementation is therefore not just:

```
HTTP request → Active Record create → redirect
```

It includes input inspection, confirmation, matching, selective updating, creation of missing itinerary ideas, deferred geocoding and result reporting.

These collectively support a meaningful use-case responsibility.

Strongest argument against confirmation

Most of the business operation is concentrated in `bulk_create_items` .

The method directly handles data normalization, deduplication, creation, error handling and callback control. It delegates database operations and geocoding execution, but does not delegate much of the actual import policy to separate application objects.

Consequently, this is certainly **controller-heavy procedural orchestration**.

However, these two conclusions are not mutually exclusive. GRASP Controller describes responsibility assignment; it does not guarantee low coupling, small methods or ideal separation of concerns.

The controller is doing more work than an ideally thin coordinator, but it still owns a coherent system event with an identifiable outcome.

That is why I retain the **CONFIRMED** classification while documenting the concentration of business logic as a design limitation.

4. Comparison with previously confirmed instances

Instance	System event	Main coordinator	Distinct responsibility
Inbound Travel Email Intake Coordination	Incoming email arrives	`TripMailbox#process`	Locate trip, check sender, create inbox record, delegate parsing or notification
Trip Creation Coordination	User creates a trip	`TripsController#create`	Persist trip, establish initial owner membership, schedule cover retrieval
Trip Membership Management Coordination	User adds/removes a trip member	`TripMembersController`	Resolve travelers, manage membership, apply permission checks and arrange invitations
Bulk Itinerary Import Coordination	User previews and confirms itinerary JSON	`TripItemsController#import` and `#bulk_create_items`	Validate import, resolve duplicates, create itinerary ideas and schedule geocoding

The previous three implementations also use Rails infrastructure, but their system events and domain outcomes are different. ‹/›

Why bulk import is independent

Bulk import neither creates a trip nor manages its membership. It operates on an existing trip, processes user-supplied structured data and creates or updates itinerary records.

It also differs from inbound-email intake. The JSON import is explicitly initiated and confirmed by a user; it does not use `TripMailbox`, `InboxItem` or `ParseInboxItemJob`.

Although both paths may eventually create `TripItem` records, **sharing a destination model is not sufficient to merge their system-event collaborations**.

Similarly, reusing `GeocodeTripItemsJob` does not make bulk import another geocoding-pattern instance.

Under the established use-case counting rule, bulk import should be counted separately.

5. Final participant-role mapping

GRASP Controller does not prescribe a fixed GoF-style set of interface and concrete-implementation roles. The following mapping reflects the actual responsibilities observed.

Responsibility	Actual participant	Source location
System-event coordinator	`TripItemsController#import`	`app/controllers/trip_items_controller.rb:31–58`
Batch-operation coordinator	`TripItemsController#bulk_create_items`	Same file, lines 128–174
Date parsing helper	`TripItemsController#parse_import_time`	Same file, lines 176–179
Associated trip/domain context	`Trip` and `@trip.trip_items`	`app/models/trip.rb:4–6`; controller lines 133, 138, 148
Itinerary domain object	`TripItem`	`app/models/trip_item.rb:1–28`
Coordinate-completeness behavior	`Geocodable#geocoded?`	`app/models/concerns/geocodable.rb:9–11`
Deferred-work collaborator	`GeocodeTripItemsJob#perform`	`app/jobs/geocode_trip_items_job.rb:4–22`
User-facing preview and confirmation	Import view	`app/views/trip_items/import.html.erb:29–106, 189–227`
Request routing	Rails routes	`config/routes.rb:30–35`

Pattern origin: Mixed, with application-owned coordination.

Rails supplies routing, model operations, callback APIs and job infrastructure. DaPlanStan supplies the import workflow and its business decisions.

6. Counting decision

Stable instance name CONFIRMED

Bulk Itinerary Import Coordination

+1	4
New GRASP Controller instance	Source-confirmed GRASP Controller instances so far

The total covers only the four GRASP Controller collaborations reviewed so far, using the same use-case-level counting rule. It is not the final repository-wide pattern count.

One collaboration includes both preview and confirmed import. It should produce **one `patterns.csv` row**, not separate rows for `import` and `bulk_create_items`.

There is no additional geocoding-pattern instance or repeated inbox-linking instance to count here.

7. Source evidence to preserve

The strongest evidence consists of the GET/POST route declarations, the preview/confirmation branch inside `import`, the corresponding confirmation form, the complete `bulk_create_items` method, the `TripItem` model declarations and the `GeocodeTripItemsJob#perform` delegation.

I also compared the extracted files with their corresponding entries in the uploaded source ZIP. All six relevant source files matched exactly. This verifies the extracted evidence against the uploaded archive, **not** against the stated Git commit.

8. Design limitations to record separately

Observation	Source location	Verification status
Import algorithm is concentrated in the controller	`trip_items_controller.rb:128–174`	Confirmed structural observation
Callback restoration omits the original condition	`trip_items_controller.rb:134, 172–174`; `geocodable.rb:4–6`	Confirmed source difference; runtime effect untested
Class-level callback changes may affect concurrent operations	`trip_items_controller.rb:134, 173`	Potential concurrency risk
Confirmation reparses submitted JSON without repeating preview's array check	`trip_items_controller.rb:31–49`	Confirmed structural observation
User can submit modified confirmation JSON because it is sent in a hidden form field	`import.html.erb:101–106`	Source permits the request shape; exploitation/impact untested
Errors are rescued per item, allowing partial import outcomes	`trip_items_controller.rb:135–168`	Confirmed control-flow behavior
Duplicate matching uses case-insensitive names	`trip_items_controller.rb:133–147`	Confirmed; incorrect merging of distinct places is a possible risk
New items can trigger deferred geocoding after creation	`trip_items_controller.rb:163–170`	Scheduling established; external success unverified

These limitations should not be used as reasons to reject GRASP Controller automatically. They concern the quality and reliability of the implementation, not whether the controller is assigned the system-event responsibility.

9. Final candidate-review record

Writing

Bulk Itinerary Import Coordination — Verification Record

Source: DaPlanStan original source ZIP
Pattern: GRASP Controller
Category: GRASP Responsibility Assignment
Instance name: Bulk Itinerary Import Coordination
Decision: CONFIRMED (static source assessment)
Origin: Mixed — application-owned coordination using Rails infrastructure.

Defining responsibility: `TripItemsController#import` receives and coordinates the itinerary-import use case. The initial submission prepares a preview; confirmation invokes `bulk_create_items`, which handles duplicate checking, record creation, partial updates and deferred geocoding.

Main participants:
- `TripItemsController#import` — `app/controllers/trip_items_controller.rb:31–58`
- `TripItemsController#bulk_create_items` — `app/controllers/trip_items_controller.rb:128–174`
- `TripItem` — `app/models/trip_item.rb:1–28`
- `Geocodable#geocoded?` — `app/models/concerns/geocodable.rb:9–11`
- `GeocodeTripItemsJob#perform` — `app/jobs/geocode_trip_items_job.rb:4–22`
- Import view and routes — `app/views/trip_items/import.html.erb:29–106, 189–227`; `config/routes.rb:30–35`

Reason for confirmation: The source establishes a cohesive system-event responsibility involving preview, confirmation, duplicate resolution, persistence and delegation of subsequent geocoding work. This goes beyond the mere presence of a Rails controller class.

Strongest objection: The import algorithm remains largely inside one controller method, making the implementation controller-heavy and procedural. This limits separation of concerns but does not negate the GRASP Controller responsibility assignment.

Counting decision: One independent GRASP Controller instance. Preview and confirmed import are phases of the same use case. The instance is separate from inbound email intake, trip creation and trip membership management. Shared Geocodable behavior is not counted again.

Design limitations: Class-level callback manipulation, callback restoration, partial imports, name-based deduplication and confirmation-stage input validation require additional testing.

Human verification: Pending independent reviewer sign-off.
Runtime tests: Not performed.

Source provenance: Uploaded ZIP verified for the relevant extracted files; ZIP-to-upstream-commit correspondence remains unverified.

Final conclusion: `Bulk Itinerary Import Coordination` is a defensible GRASP Controller instance because it represents a specific, independently identifiable system-event collaboration. Record it as one source-confirmed instance, while keeping its controller-heavy implementation and potential correctness issues as separate findings.

Now move to the next source area: external integrations and account identity.
Do not begin by assigning pattern names. First trace how DaPlanStan interacts with these external systems and where the application itself makes decisions.
Inspect these files and any directly related callers needed to complete the flows:

1. `app/jobs/fetch_trip_cover_image_job.rb`
2. `app/controllers/sessions_controller.rb`
3. `app/models/user_identity.rb`
4. `app/models/user.rb`
5. `config/initializers/omniauth.rb`
6. `app/mailers/trip_mailer.rb`
7. `app/mailers/invitation_mailer.rb`
8. Relevant routes, authentication callbacks, views and configuration where needed.

Keep the following flows separate.

1. Cover-image retrieval

Trace how the application requests a trip cover image, how the Pexels call is constructed and how the result is stored.
This flow already appeared as a collaborator in Trip Creation Coordination, so do not count the same collaboration again unless there is a genuinely independent pattern inside the integration itself.

2. OAuth sign-in and identity linking

Trace the actual sign-in path for the configured authentication providers.

Identify:

1. how provider data reaches the application
2. how an existing `UserIdentity` is located
3. when a user is found by email
4. when a new user/account/traveler is created
5. how a provider identity is linked to a user
6. how invited travelers are associated with the signed-in user
7. whether different OAuth providers are handled through genuinely interchangeable application-defined behavior or mostly through OmniAuth configuration
8. Outbound mail

Trace how invitation and pending-sender emails are constructed and sent.
Separate application-written mail content and recipient decisions from Rails Action Mailer and Postmark delivery infrastructure.
For each flow:

1. Identify the actual classes and methods involved.
2. Give reliable source paths and line ranges.
3. Trace the control flow.
4. Separate DaPlanStan behavior from Rails, OmniAuth, Pexels and Postmark behavior.
5. Identify where application decisions are made and what is merely configuration.
6. Check whether wrappers around provider APIs actually translate incompatible interfaces or simply invoke dependency APIs directly.
7. Check whether multiple authentication providers really form an application-level Strategy, Adapter, Factory or other pattern before proposing one.
8. Reuse previously established collaborations where appropriate rather than counting them again.

Only after tracing the flows, identify any genuinely new design-pattern candidates.
For each candidate:

1. Give the possible pattern and category.
2. Give a meaningful collaboration name.
3. Identify the actual participants.

4. Explain the behavior supporting the classification.
5. Give the strongest false-positive explanation.
6. State whether it is application-owned, framework-provided or mixed.
7. Leave it UNCONFIRMED.

At the end give me:

1. Cover-image integration trace.
2. OAuth and identity-linking trace.
3. Outbound-mail trace.
4. Participant/source-evidence table.
5. Application/framework/provider responsibility split.
6. New candidate patterns only.
7. Previously confirmed collaborations encountered again but not counted again.
8. Strongest non-pattern explanation.
9. Anything that still requires verification.

Worked for 2m 9s

DaPlanStan — External Integrations and Account Identity

Source Investigation 05 · Original uploaded ZIP · Static source analysis

Pattern classifications unconfirmed

1. Overview of the source findings

The three integration areas have different responsibilities.

- **Cover images:** DaPlanStan constructs its own HTTP request to Pexels, interprets the JSON response and stores the selected image URL on `Trip`.
- **OAuth authentication:** OmniAuth handles provider-specific authentication. DaPlanStan receives normalized authentication information and implements user lookup, identity linking, account provisioning, invitation acceptance and session creation.
- **Outbound email:** DaPlanStan selects recipients and constructs invitation or notification messages. Rails Action Mailer and the configured delivery service handle message processing and transport.

The source also contains an important supporting component that was not in the initial file list: `config/initializers/mail_logger.rb`. It registers an application-defined delivery observer with Action Mailer. That behavior is relevant to candidate discovery, but it must be evaluated separately from ordinary outbound mail delivery.

The investigation below distinguishes application-written coordination from Rails conventions and third-party APIs before considering pattern classifications.

2. Cover-image retrieval

2.1 Entry points

The same `FetchTripCoverImageJob` is invoked from three actions in `TripsController`.

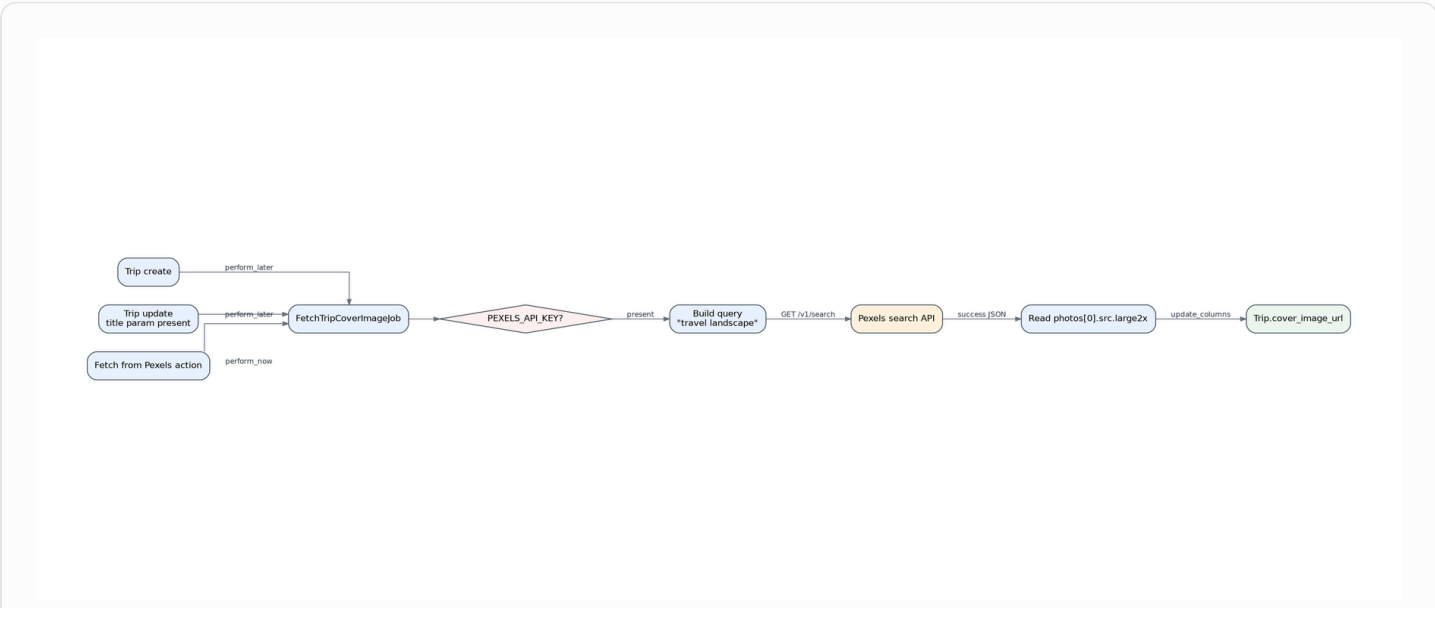
Trigger	Source	Invocation
Successful trip creation	`app/controllers/trips_controller.rb:35–48`	`perform_later(@trip.id)`
Successful trip update when a title parameter is present	`app/controllers/trips_controller.rb:68–75`	`perform_later(@trip.id)`
Explicit *Fetch from Pexels* action	`app/controllers/trips_controller.rb:87–92`	`perform_now(@trip.id)`

The manual refresh is initiated by a POST form in `app/views/trips/cover.html.erb:65–76`, corresponding to `config/routes.rb:27–28`.

Important distinction: Manual refresh invokes `perform_now` synchronously, whereas creation and update call `perform_later`.

Additionally, the update condition tests whether `trip_params[:title]` is present, not whether the title actually changed. Therefore, a successful request containing an unchanged but nonblank title can still enqueue a new image lookup.

2.2 Complete control flow



2.3 What the job actually does

`FetchTripCoverImageJob#perform`, `app/jobs/fetch_trip_cover_image_job.rb:4–32`, implements the following steps.

Locate the trip. It retrieves `Trip.find_by(id: trip_id)` and returns if no matching record exists.

Check configuration. It reads `ENV["PEXELS_API_KEY"]`. If missing or blank, it logs that information and returns.

Construct a search. It removes four-digit years from the trip title, compresses whitespace and appends `travel landscape`.

For example, a trip titled `Japan 2026` would produce a query based on `Japan travel landscape`.

Construct the request. The application creates a GET request to:

`https://api.pexels.com/v1/search`

It supplies the URL parameters `query`, `per_page=1` and `orientation=landscape`, with the API key placed in the `Authorization` header.

Process the response. On an HTTP success response, it parses the JSON and retrieves:

```ruby
data.dig("photos", 0, "src", "large2x")
```

Persist the result. If an image URL exists, the job calls:

```ruby
trip.update_columns(cover_image_url: image_url)
```

The job rescues errors and logs warnings. It does not propagate its rescued failures to the controller through a custom result object. ‹/›

2.4 Image storage versus image display

The job stores **the returned URL**, not downloaded image bytes.

`Trip` also declares an Active Storage cover attachment at `app/models/trip.rb:2`.

The application's `trip_cover_src` helper prefers an uploaded Active Storage image over the Pexels URL:

</> Ruby

```ruby
def trip_cover_src(trip)
  return url_for(trip.cover_image) if trip.cover_image.attached?
  trip.cover_image_url.presence
end
```

Source: `app/helpers/application_helper.rb:44-47`.

A user can separately upload a custom cover through `TripsController#update_cover`, which attaches the upload and clears the Pexels URL.

The manual refresh operation does the reverse: it purges the uploaded image, clears the URL and invokes the Pexels job synchronously.

2.5 Responsibility allocation

Participant	Responsibility
`TripsController`	Decide when cover retrieval is needed
`FetchTripCoverImageJob`	Prepare request, invoke Pexels and interpret result
`Trip`	Hold the image URL and optional attachment
Active Record	Perform persistence
Active Job	Provide asynchronous/synchronous job invocation infrastructure
Ruby `Net::HTTP`	Perform HTTP transport
Pexels	Search its image catalogue and return image metadata
Active Storage	Store manually uploaded cover attachments

The application does not use a Pexels-specific SDK in this method. It constructs the HTTP request directly using Ruby libraries.

2.6 Collaboration boundary

This image-retrieval operation was already encountered as a supporting participant in the previously reviewed `Trip Creation Coordination` instance.

The additional edit and manual-refresh callers do not, by themselves, create new pattern instances.

The actual Pexels integration is a direct HTTP lookup with response processing and persistence. No interchangeable provider implementation or incompatible-interface translation has been established.

Preliminary conclusion: The cover-image retrieval flow is independently understandable as external API integration. No genuinely new pattern candidate is warranted from this job alone.

---

3. OAuth sign-in and identity linking

3.1 Provider configuration and entry

`config/initializers/omniauth.rb:1–16` registers two OmniAuth providers:

Provider	Configuration
Google	`:google_oauth2`, client ID and client secret
Apple	`:apple`, client ID, team ID, key ID and private key

The initializer also specifies that OAuth requests begin using POST.

The application routes:

- `GET /auth/:provider/callback` to `SessionsController#create`.
- `GET /auth/failure` to `SessionsController#failure`.
- `DELETE /auth/sign_out` to `SessionsController#destroy`.

These routes are at `config/routes.rb:12–15`, within the `whats.` hostname constraint.

The actual provider authentication protocol is implemented by OmniAuth and its provider integrations, not by `SessionsController`.

Important provider distinction

Although both Google and Apple are configured, the inspected sign-in interfaces contain Google-specific forms:

- `app/views/home/app.html.erb:50–68`
- `app/views/invites/accept.html.erb:19–31`
- `app/views/layouts/_navbar.html.erb:68–71`

No corresponding Apple sign-in button was found in the inspected application views.

Therefore, the source establishes Apple provider configuration but **does not establish a working Apple sign-in user journey**.

3.2 How authenticated provider data reaches the application

`SessionsController#create`, `app/controllers/sessions_controller.rb:4–40`, begins with:

```ruby
auth = request.env["omniauth.auth"]
```

The controller expects the normalized OmniAuth authentication result to expose fields such as:

- `auth.provider`
- `auth.uid`
- `auth.info.email`
- `auth.info.name`
- `auth.info.image`
- `auth.credentials.token`

DaPlanStan reads these fields. It does not itself implement Google's or Apple's token exchange or provider-specific response parsing in this controller.

3.3 Complete sign-in decision flow



3.4 Existing identity lookup

At `app/controllers/sessions_controller.rb:7-11`:

```ruby
identity = UserIdentity.find_by(
  provider: auth.provider,
  provider_uid: auth.uid
)
```

When an identity exists, the controller uses `identity.user` and updates the avatar URL if the provider supplies an image.

It does **not** repeat the user-by-email lookup for this branch.

`UserIdentity`, at `app/models/user_identity.rb:1-8`, belongs to `User`, validates the provider and UID and declares encryption for `access_token`.

The database defines a unique index on `(provider, provider_uid)` at `db/structure.sql:25`.

3.5 New provider identity linking

If the exact provider identity does not exist, the controller calls:

```ruby
user = User.find_or_initialize_by(email: auth.info.email)
```

This has two possible outcomes.

Existing user email: The controller reuses the matching `User`, fills a missing name if necessary, updates the avatar and saves.

New user email: It initializes and saves a new `User` with the available provider profile information.

In both cases, the controller then creates a new associated `UserIdentity`, recording the provider, UID, provider email and access token.

That is the key identity-linking rule: **provider-and-UID lookup takes precedence, and email is used when introducing a provider identity not already linked**.

There is no separate application-level Google-specific or Apple-specific account-linking algorithm in this method. ‹/›

3.6 Session establishment and pending invitations

Once a user has been resolved, the controller writes:

‹/› Ruby

```ruby
session[:user_id] = user.id
```

`ApplicationController#current_user`, at `app/controllers/application_controller.rb:14–16`, subsequently resolves the session ID to a `User` record.

Next, `SessionsController#create:28–31` retrieves all `Traveler` records with the signed-in user's email and no associated user:

‹/› Ruby

```ruby
pending_travelers =
  Traveler.where(user_id: nil, email: user.email).to_a

pending_travelers.each { |t| t.accept!(user) }
```

`Traveler#accept!`, `app/models/traveler.rb:26–28`, associates the traveler with the user and records `invite_accepted_at`.

This links previously created email-only traveler records to the newly authenticated identity.

It is **not equivalent to assigning a new trip membership**. Those memberships were already established by the membership-management workflow.

3.7 Account provisioning and invite gate

After processing pending travelers, the controller checks:

Condition	Application decision
User already owns an account	Redirect to trips

User has traveler profiles but no owned account	Redirect to trips as an invited member
Neither condition holds	Invoke `provision_account_or_gate`

`SessionsController#provision_account_or_gate`, lines 54–76, handles first-time independent account access.

When `OPEN_SIGNUP=true`, it creates an account without requiring a beta-invite token.

Otherwise it consumes `session[:invite_token]` and searches for the corresponding `Invite`.

A valid invite must be usable and match the signed-in user's email, unless the invite is unrestricted.

If valid, the controller creates an account, marks the invite used and redirects to trips. Otherwise, it clears `session[:user_id]` and redirects to the waitlist.

`create_account_for`, lines 78–81, performs:

```ruby
account = Account.create!(owner: user, name: user.name)

account.travelers.create!(
  user: user,
  name: user.name,
  email: user.email,
  avatar_url: user.avatar_url
)
```

The invite's predicates are implemented by `Invite#usable?` and `#email_matches?`, at `app/models/invite.rb:12–17`.

Important: An invitation to join a trip and an invite to create a new account are not identical records or workflows. A user already linked to a traveler profile can access trips without receiving a separate owned account.

The key source locations are `app/controllers/sessions_controller.rb:26–81` and `app/models/traveler.rb:18–28`.

3.8 Application decisions versus provider behavior

Concern	DaPlanStan application	OmniAuth / Rails

Provider configuration	Register Google and Apple with credentials	Provide provider strategies and OAuth protocol handling
Provider response	Read `omniauth.auth`	Produce the normalized authentication result
Identity lookup	Match provider/UID first, then email when necessary	Execute Active Record queries
Identity linking	Associate a new `UserIdentity` with the selected user	Persist associations and encrypt configured attributes
Signed-in session	Assign user ID and choose redirect	Manage HTTP session infrastructure
Invitation acceptance	Find pending travelers by email and call `accept!`	Persist traveler associations
Account eligibility	Check owned account, memberships, open signup and invite validity	Supply persistence and controller mechanisms

The application uses a common callback for both configured providers. No application-defined interchangeable provider-specific algorithms are visible.

Preliminary conclusion: This is primarily OmniAuth-backed authentication with application-owned identity resolution and account-provisioning rules. The account-linking behavior is meaningful, but merely configuring Google and Apple does not establish an application-implemented Strategy, Adapter or Factory.

4. Outbound mail

4.1 Types of outgoing email

The inspected application contains three relevant mailer actions.

Email	Originating call site	Mailer
Account traveler invitation	`TravelersController#create` , lines 18–26	`InvitationMailer#invite`
Trip membership invitation	`TripMembersController#resolve_and_add_by_email` , lines 46–65	`InvitationMailer#invite_to_trip`
Unknown inbound sender notification	`TripMailbox#notify_owners_of_pending_sender` , lines 55–60	`TripMailer#pending_sender_notification`

They share Action Mailer infrastructure but have different application-defined recipient decisions and message content.

4.2 Account traveler invitation

`TravelersController#create` handles addition of a traveler to the current account.

When an email is provided, it checks whether the address belongs to the acting user or an existing account traveler, then looks up a registered `User`.

For an existing user, it creates a linked `Traveler`. Otherwise, it creates an email-only traveler.

The controller then invokes:

```ruby
InvitationMailer.invite(current_user, email).deliver_later
```

`InvitationMailer#invite`, `app/mailers/invitation_mailer.rb:2–6`, stores the inviter and sign-in URL for the template, then sets the recipient and subject.

The template at `app/views/invitation_mailer/invite.text.erb:1–12` instructs the recipient to sign in to DaPlanStan.

This path was encountered during earlier membership analysis, but the account-level invitation caller is distinct from the trip-specific invitation caller.

4.3 Trip membership invitation

`TripMembersController#resolve_and_add_by_email` looks up or creates the relevant `Traveler`.

If the supplied email does not correspond to an existing account traveler or registered user, the controller creates an email-only traveler and calls:

```ruby
InvitationMailer.invite_to_trip(
  account.owner, email, @trip
).deliver_later
```

`InvitationMailer#invite_to_trip`, lines 8–13, constructs a trip-specific invitation.

The template at `app/views/invitation_mailer/invite_to_trip.text.erb:1–9` includes the inviter's name, trip title and application sign-in URL.

The source uses `account.owner` as the displayed inviter even when a planner performs the membership-management action. It does not pass the acting planner as the inviter.

This is an application decision, not Action Mailer behavior.

4.4 Unknown-sender notification

The inbound-email workflow previously established that `TripMailbox#process` creates an `InboxItem` even when its sender is not approved.

The unapproved branch calls `notify_owners_of_pending_sender`.

That method:

1. Finds members of the relevant trip with `role: :owner`.
2. Loads the associated traveler information.
3. Calls `TripMailer.pending_sender_notification(member, inbox_item).deliver_later` for each owner.

`TripMailer#pending_sender_notification`, `app/mailers/trip_mailer.rb:2–10`, uses the trip member's `traveler.display_email` as recipient and constructs a subject identifying an unknown sender.

Its template, `app/views/trip_mailer/pending_sender_notification.text.erb:1–13`, includes the trip title, sender name/email, subject, receipt date and an approval-related URL.

This is **the same pending-sender notification collaboration already traced during inbound-email intake**. It must not be counted again as a new GRASP Controller instance. ‹/› ‹/›

4.5 Mail delivery configuration

`ApplicationMailer`, `app/mailers/application_mailer.rb:1–5`, specifies default sender and reply-to addresses from environment variables and uses the `mailer` layout.

Delivery is environment-dependent:

Environment	Configured behavior
Production	Postmark delivery using `POSTMARK_API_TOKEN`
Development with Postmark token	Postmark delivery
Development without Postmark token	Write email output to `tmp/mails`

The relevant configuration is `config/environments/production.rb:63–69` and `config/environments/development.rb:40–50`.

`deliver_later` is supplied by Rails Action Mailer/Active Job, while Postmark supplies external delivery when configured. DaPlanStan's mailers do not implement the SMTP or Postmark transport protocol.

4.6 Additional delivery logging

`config/initializers/mail_logger.rb:1–14` contains an explicit registration:

```ruby
module MailLogger
  def self.delivered_email(message)
    Rails.logger.info(
      "[Mailer] Sent '#{message.subject}' to #{Array(message.to).join(', ')}"
    )
  end
end

ActionMailer::Base.register_observer(MailLogger)
```

This is different from the standard mailer methods above.

DaPlanStan defines the `MailLogger` module and registers it with Rails' Action Mailer notification machinery. The logger receives a mail message and records its subject and recipients.

The framework supplies the delivery notification and observer registration mechanism. DaPlanStan supplies the registered callback implementation.

This is a concrete source-defined collaboration worth considering during the candidate stage, although actual invocation has not been runtime-verified.

5. Consolidated participant and source-evidence table

All paths are relative to the root of the uploaded `daplanstan-main.zip`. Line ranges were checked against the extracted archive rather than inferred from the Part 1 handoff.

Participant / method	Source path and lines	Observed responsibility
`TripsController#create`	`app/controllers/trips_controller.rb:35–48`	Enqueue cover retrieval after trip creation
`TripsController#update`	Same file, `68–75`	Enqueue retrieval when the title parameter is present

`TripsController#refresh_cover`	Same file, `87–92`	Clear cover and run retrieval synchronously
`FetchTripCoverImageJob#perform`	`app/jobs/fetch_trip_cover_image_job.rb:4–32`	Call Pexels and store selected URL
`Trip`	`app/models/trip.rb:1–5`	Store cover URL and attachment association
`trip_cover_src`	`app/helpers/application_helper.rb:44–47`	Select uploaded cover or remote URL
OmniAuth configuration	`config/initializers/omniauth.rb:1–16`	Register Google and Apple providers
Authentication routes	`config/routes.rb:12–15`	Route callbacks, failure and sign-out
`SessionsController#create`	`app/controllers/sessions_controller.rb:4–40`	Resolve identity, establish session and determine access
`SessionsController#provision_account_or_gate`	Same file, `54–76`	Apply open-signup or invite eligibility
`SessionsController#create_account_for`	Same file, `78–81`	Create account and owner traveler
`SessionsController#destroy`, `#failure`	Same file, `43–49`	Sign out or handle authentication failure
`UserIdentity`	`app/models/user_identity.rb:1–8`	Store provider association and encrypted token
`User`	`app/models/user.rb:11–22`	User, identity, account and traveler associations
`Traveler#accept!`	`app/models/traveler.rb:26–28`	Link pending traveler to signed-in user
`Invite#usable?`, `#email_matches?`	`app/models/invite.rb:12–17`	Evaluate beta-invite eligibility
`InvitesController#accept`	`app/controllers/invites_controller.rb:5–15`	Store invite token in session
`ApplicationController#current_user`	`app/controllers/application_controller.rb:14–16`	Resolve authenticated session user
`TravelersController#create`	`app/controllers/travelers_controller.rb:5–35`	Manage account traveler invitation
`TripMembersController#resolve_and_add_by_email`	`app/controllers/trip_members_controller.rb:46–65`	Coordinate trip-specific invitation

`TripMailbox#notify_owners_of_pending_sender`	`app/mailboxes/trip_mailbox.rb:55-60`	Select owners and request notifications
`InvitationMailer#invite`	`app/mailers/invitation_mailer.rb:2-6`	Construct account invitation
`InvitationMailer#invite_to_trip`	Same file, `8-13`	Construct trip invitation
`TripMailer#pending_sender_notification`	`app/mailers/trip_mailer.rb:2-10`	Construct pending-sender notification
`ApplicationMailer`	`app/mailers/application_mailer.rb:1-5`	Default sender, reply-to and layout
`MailLogger.delivered_email`	`config/initializers/mail_logger.rb:1-7`	Register application logging callback
Outbound delivery configuration	`config/environments/production.rb:63-69`; `development.rb:40-50`	Select Postmark or file delivery

Supporting view evidence includes `app/views/home/app.html.erb:50-68`, `app/views/invites/accept.html.erb:19-31`, `app/views/trips/cover.html.erb:49-76`, and the three outbound email templates.

---

6. New pattern candidates

Only two newly encountered collaborations are worth retaining as pattern candidates.

Neither is confirmed in this investigation.

Candidate A — GRASP Controller

UNCONFIRMED

Provisional instance: OAuth Sign-in and Identity Linking Coordination

Category: GRASP · Origin: Mixed, with application-owned decisions

Participants: `SessionsController#create`, `#provision_account_or_gate`, `#create_account_for`, `UserIdentity`, `User`, `Traveler`, `Account` and `Invite`.

Why it might fit: `SessionsController#create` receives a sign-in completion event and coordinates several application-level responsibilities: provider identity lookup, email-based account association, session establishment, traveler acceptance and account eligibility.

It is a coherent use case with identifiable domain outcomes, not merely a call to `User.create!`.

Strongest false-positive explanation: The controller may be performing an ordinary Rails authentication callback with all decisions concentrated in one procedural method. OmniAuth already supplies the external authentication workflow, and the application code may simply be post-authentication account maintenance.

What requires verification: Determine whether the application-owned sign-in coordination meets the same GRASP Controller standard previously applied to trip creation and membership management, while acknowledging that the controller directly performs much of its identity-linking logic.

Status: UNCONFIRMED.

Candidate B — Observer

UNCONFIRMED

Provisional instance: Outbound Mail Delivery Logging

Category: GoF Behavioral / Framework-mediated · Origin: Mixed

Participants:

- Framework-provided subject/publisher: `ActionMailer::Base`.
- Application-defined observer: `MailLogger`.
- Notification operation: `MailLogger.delivered_email(message)`.
- Registration: `ActionMailer::Base.register_observer(MailLogger)`.

Why it might fit: The source contains an explicit observer registration and a method designed to receive a delivery notification. This is much stronger structural evidence than merely discovering a background job or callback.

The mailers themselves do not need to know about `MailLogger`; the registration is made through Action Mailer infrastructure.

Strongest false-positive explanation: The observer infrastructure is implemented by Action Mailer rather than DaPlanStan. The application registers a logging callback through a framework API, so the classification may be a framework idiom rather than an independently implemented GoF Observer.

What requires verification: Inspect the installed Action Mailer observer contract and run a successful delivery through the configured environment to establish notification behavior. Then determine whether application-defined observers registered through framework infrastructure qualify under the faculty's accepted pattern scope.

Status: UNCONFIRMED.

Why not propose provider-specific Strategy, Adapter or Factory?

The source does not establish the mechanisms required for those classifications.

For OAuth, Google and Apple are registered through OmniAuth configuration. `SessionsController` uses one common authentication result and one shared identity-linking algorithm. No application-defined provider-specific strategy implementations or common selectable algorithm interface are demonstrated.

For Pexels, the job constructs a direct HTTP request and reads provider-specific JSON. There is no explicit adapter contract or abstraction allowing interchangeable image providers.

For outbound email, `InvitationMailer` and `TripMailer` use Action Mailer's existing `mail` and `deliver_later` APIs. Merely constructing two different message types does not establish a Factory or other creation pattern.

The explicit `MailLogger` registration is a different matter because the source actually contains a notification subscription relationship.

---

7. Previously examined collaborations — no duplicate counts

Existing collaboration	Where it reappears	Counting decision
`Trip Creation Coordination` — GRASP Controller	The trip-creation action queues `FetchTripCoverImageJob`	Do not count again
`Trip Membership Management Coordination` — GRASP Controller	Membership addition sends `InvitationMailer#invite_to_trip`	Do not count again
`Inbound Travel Email Intake Coordination` — GRASP Controller	Unknown-sender branch requests `TripMailer` notifications	Do not count again

The possible `Outbound Mail Delivery Logging` Observer is not a relabeling of those use cases. It concerns a separate registration between Action Mailer and `MailLogger` that can apply across mailer actions.

It is also distinct from the previously uncertain **trip-scoped Turbo Observer** candidate. One concerns email delivery notifications; the other concerns UI refresh subscriptions. They should not automatically be merged, but neither should enter confirmed counts yet.

8. Strongest non-pattern explanation

The overall source area is best explained as:

Conventional Rails integration with external services, together with application-owned identity and notification rules.

Specifically:

- Pexels retrieval is direct HTTP integration coordinated by a background job.

- Google and Apple sign-in are configured through OmniAuth, followed by application-defined identity and account operations.
- Outbound email uses Action Mailer templates and Postmark or file delivery.
- Delivery logging uses an Action Mailer observer-registration API.

This explanation covers the observed behavior without inventing provider strategies, adapter hierarchies or factory objects.

Only the sign-in responsibility assignment and explicit delivery observer registration merit further pattern verification.

9. Source-level issues requiring verification

These observations concern correctness, completeness and pattern boundaries. They are not additional pattern claims.

Issue	Source evidence	What needs checking
Apple authentication journey	`omniauth.rb:6–12` ; Google-only UI forms	Whether Apple callbacks can complete without a visible Apple initiation flow
OAuth callback method	`config/routes.rb:12–15`	Whether the configured Apple provider's callback method is compatible with the GET-only application route
Identity linking by email	`sessions_controller.rb:7–23`	Whether provider email verification and normalization are sufficient before linking to an existing user
Incomplete signup persistence	`sessions_controller.rb:13–23, 54–75`	Whether a user and identity remain stored when beta signup is subsequently denied
Account-provisioning consistency	`sessions_controller.rb:78–81`	Whether failure of traveler creation leaves a newly created account without its expected owner traveler
Invitation semantics	`invitation_mailer.rb:2–13` ; `invites_controller.rb:5–15`	Distinguish ordinary traveler sign-in invitations from beta-invite-token acceptance
Pexels result accuracy	`fetch_trip_cover_image_job.rb:12–27`	Whether the first returned photo is suitable and persisted successfully
Pexels failure visibility	`fetch_trip_cover_image_job.rb:19–32`	Failed requests are logged, but no user-visible failure status is stored
Cover replacement race	`trips_controller.rb:43, 70, 87–92`	Whether an earlier queued lookup can overwrite a later manual refresh result
Approval email URL	`pending_sender_notification.text.erb:10–11` ; `routes.rb:45`	The template links to the approved-senders collection URL, but only create/destroy

		actions are routed; its GET behavior needs checking
Mail delivery logging	`mail_logger.rb:1–7`	Whether registration produces the expected callback for successful delivery
Error handling	`mail_logger.rb:9–14`	Whether the Action Mailer exception handler changes the expected failure behavior
Runtime evidence	Provider configuration and calls	No successful OAuth callback, Pexels response or Postmark delivery was executed during this investigation

The source supports identifying these risks, but not claiming that they have been reproduced as runtime defects.

---

10. Final investigation outcome

Source Investigation 05 Candidate verification pending

3	2	0
Integration flows traced	New candidates	New confirmed instances

Candidate-review record

Writing

External Integrations and Account Identity — Preliminary Candidate Review

Source: DaPlanStan original uploaded ZIP
Method: Static source inspection
Baseline commit correspondence: Unverified
Runtime tests: Not performed

Cover-image retrieval

`FetchTripCoverImageJob` performs direct HTTP integration with Pexels. It constructs a search based on the trip title, retrieves image metadata and saves the selected image URL. Its invocation during trip creation is already part of the previously identified Trip Creation Coordination collaboration. No additional design-pattern candidate was established by the

provider integration.

OAuth and identity linking

OmniAuth supplies Google and Apple authentication infrastructure. `SessionsController#create` resolves provider identities, links users by email when necessary, establishes a session, accepts pending traveler invitations and applies account-provisioning rules.

New candidate: GRASP Controller — OAuth Sign-in and Identity Linking Coordination — UNCONFIRMED.

Outbound email

DaPlanStan uses `InvitationMailer` and `TripMailer` to construct invitation and pending-sender notifications. Rails Action Mailer and the configured Postmark or file delivery method supply message delivery infrastructure.

The source also defines `MailLogger.delivered_email` and registers `MailLogger` using `ActionMailer::Base.register_observer`.

New candidate: Observer — Outbound Mail Delivery Logging — UNCONFIRMED.

Strongest alternative explanation

The source primarily demonstrates standard Rails external-service integration, application-owned account-linking logic and framework-supported mail delivery. Provider configuration and ordinary API calls do not independently prove application-implemented GoF patterns.

Duplicate-count prevention

Do not recount Trip Creation Coordination, Trip Membership Management Coordination or Inbound Travel Email Intake Coordination merely because their cover-image and mailer collaborators reappear.

Inventory decision

No new confirmed pattern records or count changes from this investigation. Preserve the two unconfirmed candidates for separate verification.

Limitations

Provider callback behavior, identity-linking assumptions, signup consistency, mail delivery notifications and external-service responses have not been runtime-tested. Human verification remains pending.

Conclusion: Of these three flows, identity-linking coordination and the explicit Action Mailer observer registration are the only genuinely new pattern candidates supported by the inspected code. Both remain unconfirmed until their defining responsibility or notification mechanism is verified.

Before moving to the frontend source area, verify the two candidates from the external-integration investigation.
The candidates are:

1. GRASP Controller — OAuth Sign-in and Identity Linking Coordination
2. Observer — Outbound Mail Delivery Logging

Evaluate them independently.
For the OAuth GRASP Controller candidate:

1. Identify the system event handled by `SessionsController#create`.
2. Separate what OmniAuth has already completed before control reaches the application from what DaPlanStan decides afterward.
3. Trace the application decisions around provider identity lookup, email-based user lookup, `UserIdentity` creation, session establishment, pending traveler acceptance and account provisioning.
4. Identify the actual participants and give reliable source paths and line ranges.
5. Explain why this is more than a conventional OAuth callback.
6. Give the strongest argument that it is simply procedural post-authentication account handling rather than a meaningful GRASP Controller responsibility.
7. Compare it with the already confirmed GRASP Controller instances and decide whether it represents an independent system-event collaboration.
8. Give a final decision of CONFIRMED, REJECTED or UNCERTAIN.

For the Outbound Mail Delivery Logging Observer candidate:

1. State the essential Observer mechanism that must exist.
2. Trace the relationship created by `ActionMailer::Base.register_observer(MailLogger)`.
3. Identify the subject or publisher, observer, registration operation and notification callback using actual source symbols.
4. Determine whether `MailLogger.delivered_email(message)` is called because of a real Action Mailer observer contract rather than merely by naming convention.
5. Separate the part implemented by DaPlanStan from the observer infrastructure implemented by Rails.
6. Explain whether framework-mediated Observer implementations are still defensible pattern instances when the application supplies the concrete observer and explicitly registers it.
7. Compare this with the unresolved Turbo-mediated Observer candidate. Do not merge them unless they are actually the same collaboration.
8. Give the strongest argument against counting it.
9. Give a final decision of CONFIRMED, REJECTED or UNCERTAIN.

Do not require runtime testing just to establish a static structural pattern if the source and framework contract are sufficient.
However, keep runtime verification status separate.
At the end provide:

1. One decision table for both candidates.
2. Participant-role mapping for any confirmed instance.
3. Stable instance names.
4. Source evidence to preserve.
5. Counting decision.
6. Any remaining uncertainty that belongs to runtime verification rather than pattern identification.

Do not investigate any new pattern types in this prompt.

Worked for 58s

DaPlanStan — External Integration Pattern Verification

Candidate Review 05 · Original source archive · 23CSE455

1. Verification decisions

Both candidates have identifiable application-level responsibilities supported by the source. The mail observer also matches the documented Action Mailer and Mail gem contracts for the dependency versions specified in the repository.

GRASP Controller OAuth Sign-in and Identity Linking Coordination	CONFIRMED
Observer Outbound Mail Delivery Logging	CONFIRMED

These are **static structural confirmations**, not claims of successful OAuth sign-in or mail delivery in a running application. Independent human reviewer sign-off is still pending.

The distinction is important: GRASP Controller is confirmed from the responsibility assigned to `SessionsController`, while Observer is confirmed from an explicit application-defined observer and a real framework notification contract.

2. Candidate A — GRASP Controller: OAuth Sign-in and Identity Linking Coordination

Essential responsibility and system event

GRASP Controller assigns responsibility for receiving and coordinating a system event to an object representing the system, a subsystem or a use case.

System event: An external OAuth authentication callback has returned to DaPlanStan, and the application must establish the corresponding local user identity and access.

Receiving object: `SessionsController#create`.

Source: `app/controllers/sessions_controller.rb:4–41`, reached through `config/routes.rb:12`.

The event is not the user's entire OAuth exchange. That exchange takes place through OmniAuth and the provider before the application processes the normalized result.

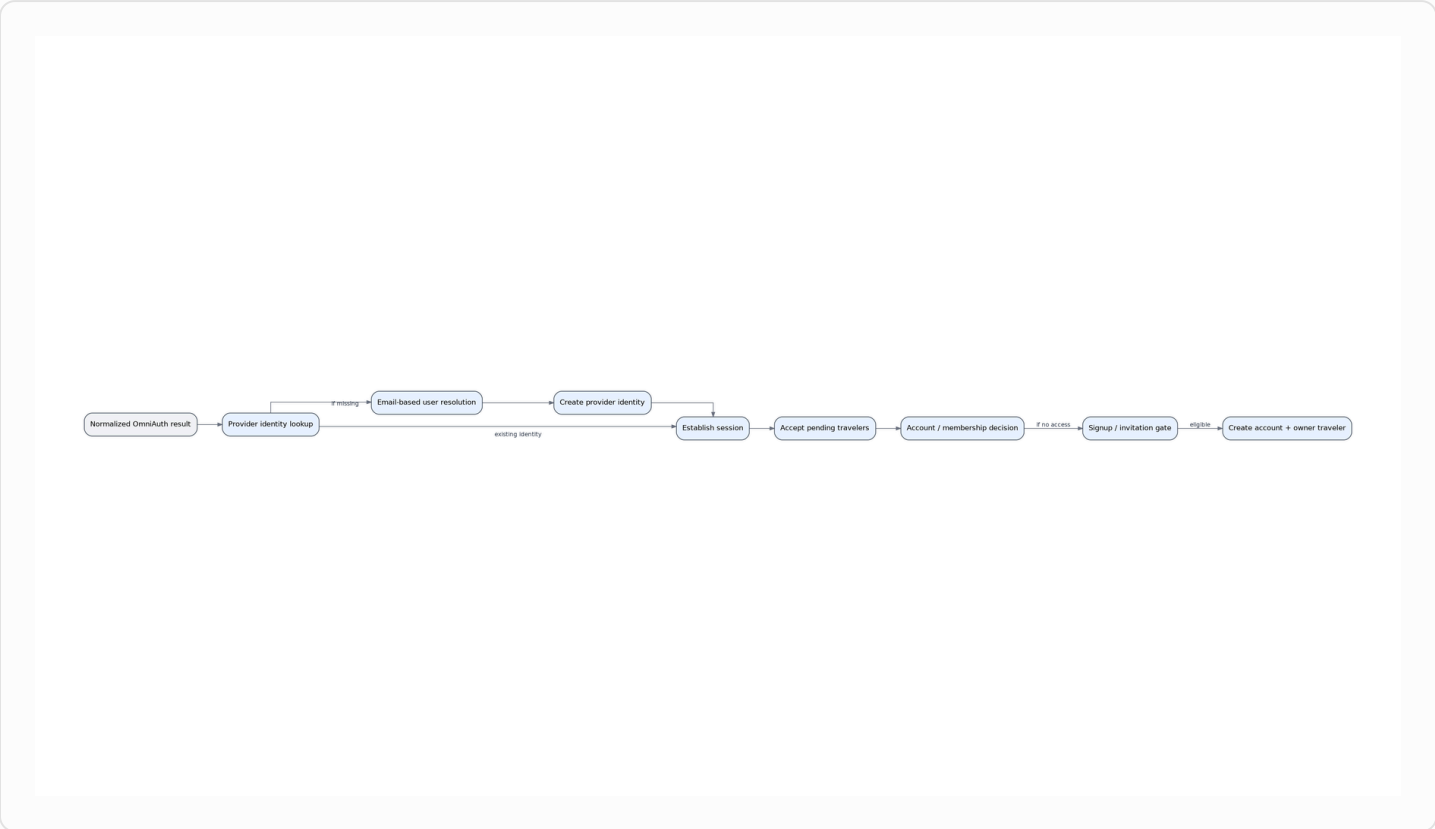
What OmniAuth has already done

By the time the successful callback reaches `SessionsController#create`, OmniAuth is expected to have supplied `request.env["omniauth.auth"]`.

The application consumes the provider identifier, UID, email, name, image and optional access token. It does not implement the provider's authorization-code exchange or Google/Apple-specific token processing here.

Provider configuration appears in `config/initializers/omniauth.rb:1–16`, where Google and Apple are registered.

What DaPlanStan decides afterward



Decision	Actual implementation	Source lines
Existing provider identity	`UserIdentity.find_by(provider: ..., provider_uid: ...)`	`sessions_controller.rb:7–11`
Email-based user resolution	`User.find_or_initialize_by(email: auth.info.email)`	`13–17`
New provider identity association	`user.user_identities.create!`	`19–24`
Session establishment	`session[:user_id] = user.id`	`26`
Pending traveler acceptance	Query unlinked travelers and call `Traveler#accept!`	`28–31`
Existing account or membership	Decide between redirect and provisioning	`33–40`
Open signup versus invitation	`provision_account_or_gate`	`54–76`
Account and owner traveler creation	`create_account_for`	`78–81`

These decisions are directly implemented in the application. Rails and OmniAuth provide the surrounding infrastructure, but not DaPlanStan's business rules.

Actual participants and delegation

Role	Participant	Reliable source location
GRASP system-event controller	`SessionsController#create`	`app/controllers/sessions_controller.rb:4–41`
Account eligibility coordinator	`SessionsController#provision_account_or_gate`	Same file, `54–76`
Account initialization operation	`SessionsController#create_account_for`	Same file, `78–81`
Provider identity record	`UserIdentity`	`app/models/user_identity.rb:1–8`
Application user	`User`	`app/models/user.rb:11–22`
Invitation-linked traveler	`Traveler#accept!`	`app/models/traveler.rb:26–28`
Account record	`Account`	`app/models/account.rb:1–11`
Invitation eligibility	`Invite#usable?`, `#email_matches?`	`app/models/invite.rb:12–17`
Prior invite-token entry	`InvitesController#accept`	`app/controllers/invites_controller.rb:5–15`
Current-session lookup	`ApplicationController#current_user`	`app/controllers/application_controller.rb:14–16`
Provider configuration	OmniAuth initializer	`config/initializers/omniauth.rb:1–16`
Callback entry	Authentication route	`config/routes.rb:12–14`

The controller delegates persistence to Active Record, user-to-account relationships to model associations, traveler acceptance to `Traveler#accept!`, and invitation-validity decisions to `Invite`.

However, it retains most of the sequencing itself.

Why this is more than a conventional OAuth callback

The callback does not merely retrieve a user and establish a session.

It coordinates a cohesive application use case involving several domain responsibilities:

- Existing-provider identity recognition.
- Linking a new provider identity to an existing or new user.

- Connecting previously invited travelers.
- Determining whether a user already has account or membership access.
- Provisioning an independent account when permitted.
- Applying a beta-invitation gate otherwise.

This is an application-defined system-event workflow with meaningful domain outcomes. The presence of OmniAuth does not explain those decisions.

Strongest objection

The code is controller-heavy.

`SessionsController#create` and `#provision_account_or_gate` contain authentication-adjacent business rules, identity resolution, account provisioning and access decisions.

There is no dedicated application service coordinating those operations, and the controller directly executes many database writes.

This is a valid design concern. However, GRASP Controller identifies responsibility for handling the system event; it does not require that every domain operation be perfectly separated.

In this case, the controller coordinates a complete sign-in and onboarding use case and delegates some substantive responsibilities.

Origin: Mixed, with application-owned post-authentication coordination.

Comparison with previously confirmed GRASP Controller instances

Instance	Event	Distinguishing outcome
Inbound Travel Email Intake Coordination	An email arrives	Route it to the correct trip and initiate review/processing
Trip Creation Coordination	A user creates a trip	Establish trip and initial owner membership
Trip Membership Management Coordination	A user adds/removes participants	Change membership in an existing trip
Bulk Itinerary Import Coordination	User confirms itinerary JSON	Import ideas and schedule geocoding
OAuth Sign-in and Identity Linking Coordination	OAuth callback completes	Resolve local identity, session, invitations and account access

OAuth sign-in is independent of all four previously confirmed system-event collaborations. It can occur without creating a trip, adding a trip member, importing an itinerary or processing an inbound email.

Therefore, it should not be merged into a general `Rails Controller` instance.

Final decision

CONFIRMED

Stable instance name: `OAuth Sign-in and Identity Linking Coordination`

Counting: One new GRASP Controller collaboration. Google and Apple do not create separate instances because they use the same application callback and identity-linking responsibility.

3. Candidate B — Observer: Outbound Mail Delivery Logging

Essential Observer mechanism

A defensible Observer collaboration requires:

1. A subject or publisher capable of notifying interested observers.
2. An observer implementing an expected notification operation.
3. A registration relationship connecting the observer to the publisher.
4. A defined event that causes notification without the event-producing client directly calling the observer.

The application does not have to implement its own subscription registry if a framework supplies that machinery and the application explicitly defines and registers a concrete observer.

Actual DaPlanStan implementation

The complete application-specific implementation appears in `config/initializers/mail_logger.rb:1-7`.

Ruby

```ruby
module MailLogger
  def self.delivered_email(message)
    Rails.logger.info "[Mailer] Sent '#{message.subject}' to #{Array(message.to).join(', ')}"
  end
end

ActionMailer::Base.register_observer(MailLogger)
```

This is more than a method coincidentally named `delivered_email`.

The application:

- Defines `MailLogger` as a Ruby module.

- Provides the callback `MailLogger.delivered_email(message)`.
- Explicitly registers the module as an observer.
- Leaves the responsibility for invoking the callback to the mail-delivery infrastructure.

</>

Verification against the actual framework contract

The repository's `Gemfile.lock` specifies Action Mailer **8.1.3** and the Mail gem **2.9.0**. </>

The framework contract is explicit.

Framework notification chain



The relevant contracts are:

- `ActionMailer::Base.register_observer` delegates registration to `Mail.register_observer`. RubyDoc
- `Mail.register_observer` retains the observer in a notification registry. RubyDoc
- `Mail.inform_observers` invokes `delivered_email(mail)` on each registered observer. RubyDoc
- The Mail delivery lifecycle invokes the observer-notification operation after its delivery operation. RubyDoc

Therefore, `delivered_email` **is an actual framework-defined observer callback**, not an application naming convention.

This is sufficient to establish the static structural pattern.

Participant-role mapping

Observer role	Actual participant	Source
Subject/publisher infrastructure	Mail gem delivery notification mechanism, including `Mail::Message` and `Mail` observer registry	Mail 2.9.0 framework contract
Application-facing registration interface	`ActionMailer::Base.register_observer`	Action Mailer 8.1.3
ConcreteObserver	`MailLogger`	`config/initializers/mail_logger.rb:1–5`
Observer notification operation	`MailLogger.delivered_email(message)`	Same file, `2–4`
Registration operation	`ActionMailer::Base.register_observer(MailLogger)`	Same file, `7`
Event-producing mailers	`InvitationMailer`, `TripMailer`	`app/mailers/invitation_mailer.rb:1–13`; `app/mailers/trip_mailer.rb:1–10`

There is no separately declared application-owned `Subject` interface.

The subject-side responsibilities are provided by the Mail framework, while `ActionMailer::Base` exposes the registration interface.

This distinction should be preserved in the participant-role mapping rather than incorrectly labeling `TripMailer` itself as a custom ConcreteSubject.

Why it is a genuine Observer instance

The important evidence is the complete notification relationship:

- DaPlanStan explicitly registers a concrete observer.
- The observer satisfies the documented callback contract.
- The mail delivery machinery maintains the registry and invokes notifications.
- Individual mailers do not directly call `MailLogger`.

This matches the essential Observer mechanism.

A framework-mediated Observer implementation is defensible because the faculty permits framework idioms, provided the application/framework boundary is identified accurately.

Strongest objection

DaPlanStan did not implement the observer registry, notification-dispatching algorithm or delivery lifecycle.

It supplies only a concrete observer and explicitly registers that observer through Action Mailer's existing API.

That means the implementation should **not** be presented as a fully application-written GoF Observer framework.

It is a mixed-origin Observer collaboration, with its infrastructure supplied by Rails and the Mail gem.

However, this does not negate the pattern: the application has deliberately participated in an actual observer contract, unlike a generic callback or an ordinary mailer method.

Comparison with Turbo-mediated Observer

Dimension	Outbound Mail Delivery Logging	Trip-Scoped Turbo Refresh Notifications
Event	Mail delivery	Model change or explicit trip refresh
Publisher	Mail delivery infrastructure	Turbo Streams infrastructure
Subscriber	`MailLogger`	Subscribed browser stream
Registration	`register_observer(MailLogger)`	`turbo_stream_from @trip`
Callback/action	`delivered_email(message)`	Client-side Turbo refresh
Application participation	Concrete observer and registration	Subscription and broadcast declarations
Status	**CONFIRMED**	**UNCERTAIN**

These are different event relationships with different frameworks, participants and notification effects.

The mail Observer does not resolve the previously uncertain Turbo classification. Nor should it be merged with the Turbo candidate merely because both use asynchronous notifications.

Final decision

CONFIRMED

Stable instance name: `Outbound Mail Delivery Logging`

Pattern type: Observer — GoF Behavioral, framework-mediated implementation.

Counting: One confirmed instance, not one instance per email sent or per mailer class.

4. Final decision table

Candidate	Evidence establishing responsibility or mechanism	Strongest objection	Origin	Final decision
OAuth Sign-in and Identity Linking Coordination — GRASP Controller	`SessionsController#create` coordinates provider identity lookup, user linking, session creation, traveler acceptance and account provisioning	Application logic is concentrated in a procedural controller callback	Mixed; application owns domain decisions	**CONFIRMED**
Outbound Mail Delivery Logging — Observer	`MailLogger` implements `delivered_email`; Action Mailer explicitly registers it; Mail gem maintains and invokes observers	Most Observer machinery is framework-provided, not written by DaPlanStan	Mixed; framework-owned infrastructure, application-owned concrete observer	**CONFIRMED**

5. Stable names and counting decision

Results of this verification

+1	+1
GRASP Controller instance	Observer instance

Both are new collaborations, distinct from those previously identified.

For the reviewed areas so far, this brings the provisional source-confirmed GRASP Controller count to **five**. `Outbound Mail Delivery Logging` is the first confirmed Observer instance; the Turbo candidate remains excluded.

Each confirmed collaboration should have one inventory record. Google and Apple should not generate separate sign-in Controller instances, and multiple mailers should not generate separate delivery-logging Observer instances.

6. Source evidence to preserve

Preserve these source excerpts and supporting framework references for the reviewer.

Evidence	Exact location	Purpose
OAuth route	`config/routes.rb:12–14`	Establish callback entry

OAuth provider configuration	`config/initializers/omniauth.rb:1-16`	Distinguish provider setup from application logic
Identity linking	`app/controllers/sessions_controller.rb:4-26`	Establish matching and new identity creation
Traveler acceptance and gating	Same file, `28-40, 54-81`	Prove application-owned sign-in decisions
Identity model	`app/models/user_identity.rb:1-8`	Establish stored provider identity
User/traveler associations	`app/models/user.rb:11-22`; `app/models/traveler.rb:26-28`	Establish collaborating objects
Observer implementation	`config/initializers/mail_logger.rb:1-5`	Identify concrete observer
Observer registration	Same file, `7`	Establish subscription operation
Mailer implementations	`app/mailers/invitation_mailer.rb:1-13`; `app/mailers/trip_mailer.rb:1-10`	Identify producers of delivered messages
Framework contract	Action Mailer 8.1.3 and Mail 2.9.0 documentation	Verify registration and callback semantics

The Observer evidence should include both the application initializer and the dependency contract. The initializer alone identifies the registration, but the framework documentation explains exactly why that registration causes `delivered_email` to be invoked.

7. Remaining runtime and design uncertainties

The structural classifications can be settled without executing the application. The following questions concern runtime correctness and design quality instead.

OAuth:

- Whether Google and Apple callbacks succeed with the actual configuration.
- Whether provider emails are sufficiently verified before being matched to an existing `User`.
- Whether concurrent callbacks can create conflicting identity records.
- Whether a new user and identity remain persisted when beta account provisioning is denied.
- Whether account and owner-traveler creation always complete consistently.

Mail Observer:

- Whether the initializer loads successfully in the intended environment.
- Whether successful email delivery produces the expected log entry.
- Whether delivery failures bypass the success notification as expected.
- Whether the mail delivery error handler in `mail_logger.rb:11-14` affects failure reporting.

A successful delivery callback indicates that the framework's delivery operation completed; it does not prove that a recipient actually received or opened the email.

---

8. Candidate-review record

Writing

External Integrations and Identity — Final Candidate Verification

Source: DaPlanStan original source ZIP
Method: Static source and framework-contract verification
Independent human review: Pending
Runtime testing: Not performed

Candidate 1 — OAuth Sign-in and Identity Linking Coordination

Pattern: GRASP Controller
Status: CONFIRMED
Origin: Mixed, with application-owned coordination

`SessionsController#create` receives the completed OAuth authentication event and coordinates provider identity recognition, email-based user association, provider identity creation, session establishment, pending traveler acceptance and account-provisioning eligibility.

The collaboration is distinct from trip creation, trip membership management, itinerary importing and inbound email processing. Its event and resulting identity/access state are independently identifiable.

The implementation remains controller-heavy, but the GRASP Controller responsibility assignment is supported by the application's actual decisions and delegation.

Candidate 2 — Outbound Mail Delivery Logging

Pattern: Observer
Status: CONFIRMED
Origin: Framework-mediated, with an application-defined concrete observer

DaPlanStan defines `MailLogger.delivered_email(message)` and explicitly registers the module

through `ActionMailer::Base.register_observer(MailLogger)`.

Action Mailer 8.1.3 delegates observer registration to the Mail library. The Mail gem's observer contract requires `delivered_email` and its notification infrastructure invokes that callback for registered observers following the delivery operation.

This establishes a genuine Observer collaboration, even though DaPlanStan does not implement the framework's observer registry.

Counting decision

- OAuth Sign-in and Identity Linking Coordination: one new GRASP Controller instance.
- Outbound Mail Delivery Logging: one new Observer instance.
- Do not count separate instances per OAuth provider, mailer class or delivered message.
- The previously uncertain Turbo-mediated Observer remains separate and unconfirmed.

Limitations

Static pattern confirmation does not establish runtime success, provider security, email receipt or ideal design quality. Preserve these concerns in separate verification notes.

Inventory recommendation

Both collaborations may be proposed for the baseline `patterns.csv` inventory using the stable names above, subject to independent human review of role mapping, source locations and counting consistency.

Final conclusion: Both candidates meet their respective defining mechanisms. The OAuth collaboration demonstrates an application-owned system-event responsibility; the mail logger demonstrates an explicit, contract-backed Observer relationship. Neither requires a successful runtime test to establish its static structural classification.

Now inspect the frontend interaction code.
I do not want to assume that every Stimulus controller or browser event is a design pattern. First trace the actual collaborations.
Inspect the Stimulus controllers under:

1. `app/javascript/controllers/`
2. Relevant views that attach those controllers or dispatch events.

3. Any Turbo Stream declarations or server-side code needed to understand the interaction.

Pay particular attention to application-defined events such as:
`map:locate`
`map:reset`
`map:resize`
and any other custom events or cross-controller communication you find.
For this investigation:

1. Identify the main Stimulus controllers and what each one actually does.
2. Find where custom events are dispatched.
3. Find where those events are listened for.
4. Trace the complete event path from the initiating UI action to the receiving controller behavior.
5. Distinguish ordinary DOM events and Stimulus lifecycle behavior from application-defined communication.
6. Check whether one frontend controller knows directly about another controller or whether they communicate indirectly through events.
7. Record the important source paths and reliable line ranges.
8. Separate application-written JavaScript behavior from Stimulus, Turbo, Leaflet or browser-provided behavior.
9. Revisit the existing `Trip-Scoped Turbo Refresh Notifications` Observer candidate only if this investigation provides additional evidence. Do not automatically count it as a new instance.

Only after tracing the collaborations, identify any genuinely supported pattern candidates.
For each candidate:

1. Give the possible pattern and category.
2. State the actual participants.
3. Explain the mechanism that suggests the pattern.
4. Explain the strongest false-positive interpretation.
5. State whether the mechanism is application-owned, framework-provided or mixed.
6. Leave new candidates UNCONFIRMED.

Do not call something Observer simply because JavaScript events exist. Show the actual publisher/subscriber relationship first.
At the end give me:

1. Main frontend collaboration traces.
2. Custom-event producer/consumer table.
3. Participant and source-evidence table.
4. New pattern candidates only.
5. Whether the existing Turbo Observer candidate gains any stronger evidence.
6. Previously identified collaborations encountered again but not counted again.
7. Strongest non-pattern explanation.
8. Anything that still needs verification.

Worked for 1m 44s

DaPlanStan — Frontend Interaction Investigation

Source Investigation 06 · Original uploaded source ZIP · Static source analysis

New patterns unconfirmed

1. Initial source findings

I inspected the JavaScript files under `app/javascript/controllers/` and searched their corresponding views for controller attachments, event dispatches and listeners.

The directory contains **16 application-defined Stimulus controllers**, plus two Stimulus initialization/registration files.

The strongest cross-controller interaction is between three controllers:

- `TimelineController` publishes `map:locate` and `map:reset` when a user selects or deselects an itinerary row.
- `TripNavController` publishes `map:resize` when the itinerary panel becomes visible.
- `MapController` listens for all three events and performs the corresponding Leaflet operations.

This is a real, application-defined communication relationship. However, the event names are explicitly known to the producers and consumer, and the events are dispatched directly onto the map's DOM element. It should not automatically be classified as a fully decoupled Observer implementation.

Two other mechanisms are present but should be kept separate: normal DOM/Stimulus interactions within individual controllers, and server-driven Turbo Stream refreshes that may update the page and cause Stimulus controllers to reconnect.

2. Main Stimulus controllers and their responsibilities

The application's Stimulus setup is defined in `app/javascript/controllers/application.js:1–9` and `index.js:1–4`. Controllers are loaded through the import map and attached to HTML elements using `data-controller` attributes. Stimulus supplies controller lifecycle handling, targets and declarative actions. GitHub

Controller	Source lines	Actual application behavior
`map`	`map_controller.js:38–147`	Initializes Leaflet trip map, builds markers, listens for map events
`timeline`	`timeline_controller.js:5–85`	Selects itinerary rows, dispatches map events, filters by kind/status
`trip-nav`	`trip_nav_controller.js:3–58`	Switches trip panels, updates URL, requests map resizing
`location-picker`	`location_picker_controller.js:3–76`	Allows map-click or marker-drag coordinate selection
`kind-picker`	`kind_picker_controller.js:5–87`	Selects an itinerary or expense kind and updates form controls
`category-picker`	`category_picker_controller.js:3–11`	Selects a category and changes button appearance
`date-range`	`date_range_controller.js:3–16`	Prevents an end date preceding the start date
`tabs`	`tabs_controller.js:3–27`	Shows/hides tabs and updates URL query parameters
`bottom-sheet`	`bottom_sheet_controller.js:10–30`	Changes sheet height between peek, half and full
`clipboard`	`clipboard_controller.js:3–16`	Copies text through the browser Clipboard API
`json-file`	`json_file_controller.js:4–15`	Reads selected JSON file contents into a textarea
`receipt`	`receipt_controller.js:3–32`	Previews receipt uploads and submits forms
`pull-to-refresh`	`pull_to_refresh_controller.js:3–149`	Detects downward touch gestures and reloads the page
`details-close`	`details_close_controller.js:4–19`	Closes dropdown details on outside click or navigation
`flash`	`flash_controller.js:3–17`	Automatically dismisses flash messages
`trip-delete`	`trip_delete_controller.js:3–20`	Controls deletion dialog and title confirmation

Source-coverage observation: No attachment for `bottom-sheet`, `category-picker`, `location-picker` or `tabs` was found in the inspected application views. Their classes exist, but a currently reachable use of them was not established. That is not proof they never execute.

Most of these controllers handle local component behavior. The map, timeline and trip-navigation controllers contain the substantive cross-controller event collaboration.

3. Main frontend collaboration: itinerary selection and map interaction

Application-defined event flow



Flow A — Select an itinerary row

UI initiation

`app/views/trips/_timeline.html.erb:23` attaches `TimelineController`.

The view renders itinerary rows through `app/views/trip_items/_item_row.html.erb:7–17`.

For rows with coordinates, the application includes:

</> erb

```
data-action="click->timeline#select"
data-lat="<%= item.latitude %>"
data-lng="<%= item.longitude %>"
data-item-id="<%= item.id %>"
```

Rows without coordinates are not assigned this click action.

Publisher

`TimelineController#select`, `app/javascript/controllers/timeline_controller.js:11–33`, looks up:

```javascript
const map = document.getElementById("trip-map")
```

For a newly selected row, it sends:

```javascript
map.dispatchEvent(new CustomEvent("map:locate", {
  detail: {
    lat: parseFloat(row.dataset.lat),
    lng: parseFloat(row.dataset.lng),
    id: row.dataset.itemId
  },
  bubbles: true
}))
```

The event carries the coordinates and itinerary-item ID.

Consumer

`MapController#connect`, `app/javascript/controllers/map_controller.js:51–57`, registers an event listener on the map element.

`MapController#onLocate`, lines 92–109:

- Reads `lat`, `lng` and `id` from `event.detail`.
- Calls Leaflet's `flyTo`.
- Iterates through known markers.
- Highlights the selected marker.
- Dims other markers.

End result: The selected itinerary row controls the map's focus and marker highlighting without invoking a `MapController` method directly.

Flow B — Deselect an itinerary row

The same `TimelineController#select` method checks whether the clicked row was already selected.

If true, it dispatches:

```
JavaScript

map.dispatchEvent(
  new CustomEvent("map:reset", { bubbles: true })
)
```

`MapController#onReset`, lines 77–89, restores marker appearance and calls `flyToBounds` when saved coordinates are available.

The result is that clicking the selected row again restores the map's broader trip view.

This is the same timeline-to-map communication relationship, not a separate pattern instance.

Flow C — Make the itinerary panel visible

The trip view attaches:

```
HTML

<div data-controller="trip-nav pull-to-refresh">
```

`app/views/trips/show.html.erb:16`

Desktop and mobile navigation buttons use `click->trip-nav#switch`, as shown in:

- `app/views/trips/_tab_bar.html.erb:4–24`
- `app/views/trips/_bottom_nav.html.erb:4–24`

`TripNavController#switch`, `trip_nav_controller.js:22–29`, selects the new tab and updates the URL.

`TripNavController#show`, lines 31–56, shows the selected panel and hides the others.

Only when the itinerary panel transitions from hidden to visible does it execute:

```
JavaScript
```

```
const mapEl = p.querySelector("[data-controller='map']")
if (mapEl) mapEl.dispatchEvent(new CustomEvent("map:resize"))
```

The receiving `MapController#onResize`, lines 68–74, invokes Leaflet's `invalidateSize`, then fits the known map coordinates where possible.

This solves a concrete UI problem: the map needs to recalculate its layout when a previously hidden panel becomes visible.

Unlike the other two events, `map:resize` is dispatched without `bubbles: true`. That is sufficient for this observed flow because it is dispatched directly on the element carrying the listener.

4. Custom-event producer and consumer table

A search across the application JavaScript identified exactly these three `CustomEvent` construction sites in the inspected source. ‹/›

Event	Application producer	Dispatch location	Application consumer	Behavior
`map:locate`	`TimelineController#select`	`timeline_controller.js:26–31`	`MapController#onLocate`	Move map and highlight marker
`map:reset`	`TimelineController#select`	`timeline_controller.js:21–22`	`MapController#onReset`	Reset marker styling and map bounds
`map:resize`	`TripNavController#show`	`trip_nav_controller.js:40–43`	`MapController#onResize`	Recalculate map size and bounds

All three listeners are registered at `map_controller.js:51–56` and removed during `disconnect` at lines 60–64.

Does one controller know about another?

There is **no direct controller-instance lookup or controller-method invocation** in these interactions.

Neither `TimelineController` nor `TripNavController` imports `MapController` or retrieves it through Stimulus's application API. ‹/›

However, they are not completely independent of the map's implementation:

- `TimelineController` explicitly looks for the DOM ID `trip-map`.
- `TripNavController` explicitly searches for an element with `data-controller='map'`.
- Both producers know the event names that `MapController` handles.
- Only the map element registers listeners for these application-defined events.

The result is **indirect method communication through directly targeted DOM events**, not a general event bus or a completely anonymous publisher/subscriber relationship.

5. Other frontend event and lifecycle behavior

Several other interactions initially resemble notification mechanisms, but their origins are different.

Interaction	Actual code	Classification of behavior
Kind or category selection	`kind_picker_controller.js:19–57`; related form partials	Ordinary Stimulus action and local DOM updates
Receipt selection	`receipt_controller.js:6–31`	Native file-input event and FileReader API
Location marker dragging	`location_picker_controller.js:18–58`	Leaflet event callbacks and local form updates
Pull-to-refresh	`pull_to_refresh_controller.js:13–26, 59–70, 124–130`	Browser touch listeners and explicit `window.location.reload()`
Navigation render event	`trip_nav_controller.js:11–19`	Subscription to Turbo's `turbo:render` lifecycle event
Dropdown closing	`details_close_controller.js:5–19`	Browser click and Turbo navigation listeners
Page transition effects	`app/javascript/application.js:11–32`	Turbo lifecycle events and browser View Transitions API
Flash dismissal	`flash_controller.js:4–16`	Local timers and DOM removal

The Stimulus `data-action` mechanism binds DOM events to controller methods. That is supplied by Stimulus; the application supplies the methods that react to those events. Stimulus

Similarly, listening to `turbo:render` does not mean DaPlanStan independently implements an Observer framework. Turbo publishes the lifecycle event, while the application uses it to restore tab appearance after rendering.

No additional app-defined event family found

The inspected JavaScript does not demonstrate another custom publisher/consumer relationship comparable to the three `map:*` events.

In particular, `pull-to-refresh` does not publish a custom refresh event for other controllers. It calls `window.location.reload()` directly.

6. Revisit: Trip-Scoped Turbo Refresh Notifications

The frontend investigation provides **stronger structural evidence** for the previously uncertain Turbo-mediated Observer candidate.

The application contains two trip-stream subscriptions:

- `app/views/trips/show.html.erb:2`
- `app/views/inbox_items/index.html.erb:1`

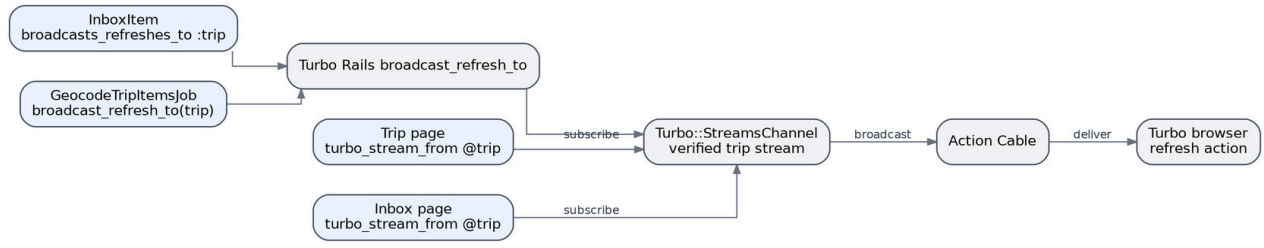
Both use:

```erb
<%= turbo_stream_from @trip %>
```

Two application paths can cause refresh broadcasting to the corresponding trip stream:

- `InboxItem` declares `broadcasts_refreshes_to :trip` in `app/models/inbox_item.rb:5`.
- `GeocodeTripItemsJob` explicitly calls `Turbo::StreamsChannel.broadcast_refresh_to(trip)` in `app/jobs/geocode_trip_items_job.rb:21`.

Framework-mediated trip refresh



Framework-contract evidence

I also checked the tagged source for Turbo Rails **2.0.23**, the version specified in the application's lockfile.

That version implements `broadcasts_refreshes_to` through an `after_commit` callback that schedules a refresh broadcast to the named stream. Its `turbo_stream_from` helper constructs a subscription through `Turbo::StreamsChannel`. GitHub +1

This supports the existence of a real framework publication/subscription relationship, rather than a coincidence of method names.

The frontend provides additional related code:

- `app/views/trips/show.html.erb:5–9` requests morph-style refresh and scroll preservation.
- The panel container at line 23 and map element at lines 36–40 are marked `data-turbo-permanent`.
- `TripNavController#connect`, lines 11–15, listens for `turbo:render` and restores tab selection.

Does this resolve the existing candidate?

It strengthens the **mechanism evidence**, but does not automatically settle whether to count the collaboration as an application-implemented GoF Observer or a framework idiom.

The framework implements subscription, dispatch and refresh handling. DaPlanStan defines stream usage, model broadcasting and an explicit job broadcast.

I recommend retaining the existing candidate record as:

`Trip-Scoped Turbo Refresh Notifications` — **UNCERTAIN**

The remaining uncertainty is primarily its classification and counting eligibility, not whether the relevant framework APIs describe a real subscription relationship.

It must remain one provisional collaboration spanning the inbox and geocoding areas. The two subscribing views and two broadcast origins do not justify counting it several times.

7. New pattern candidate

After reviewing the source, only one new classification is worth retaining.

Observer-like DOM Event Coordination

UNCONFIRMED

Provisional instance name: Itinerary Timeline–Map Event Coordination

Category: GoF Behavioral candidate · Mixed application/browser implementation

Actual participants

- Event producers: `TimelineController#select` and `TripNavController#show`.
- Notification target: DOM element `#trip-map`.
- Event subscriber: `MapController`, through its `addEventListener` calls.

- Notification handlers: `MapController#onLocate`, `#onReset` and `#onResize`.
- Event infrastructure: Browser `EventTarget`, `CustomEvent` and event dispatch machinery.

Why it might fit

The producers publish named events without directly calling receiver-controller methods. The consumer explicitly registers notification handlers, responds to those notifications and unregisters handlers when disconnected.

Those are concrete elements of an Observer-like event relationship.

Strongest false-positive explanation

The implementation may be nothing more than ordinary browser event handling.

The producer identifies the map's DOM element directly and dispatches an event specifically to it. There is no application-defined subject abstraction, independently maintained observer list or evidence of multiple interchangeable listeners.

Most of the event mechanism comes from browser APIs, while Stimulus supplies the controller lifecycle.

Origin: Mixed — application-defined events, dispatch decisions and reactions; browser-provided event infrastructure.

Decision: UNCONFIRMED.

Formal verification must decide whether this browser-mediated targeted event relationship is sufficiently substantive to count as an Observer instance, rather than merely an ordinary DOM event contract.

I do not recommend separate pattern candidates for `map:locate`, `map:reset` and `map:resize`. They are operations within the same map-interaction collaboration.

No new candidate is justified merely by `connect`, `disconnect`, Stimulus targets, Leaflet marker callbacks or ordinary UI form interactions.

8. Consolidated participant and source-evidence table

Participant / declaration	Exact source path and lines	Observed responsibility
Stimulus application startup	`app/javascript/controllers/application.js:1–9`	Start Stimulus
Stimulus controller registration	`app/javascript/controllers/index.js:1–4`	Load controller modules
Import map	`config/importmap.rb:3–8`	Pin JavaScript imports

Map container	`app/views/trips/show.html.erb:35–41`	Attach MapController and pass trip-item JSON
Timeline container	`app/views/trips/_timeline.html.erb:23`	Attach TimelineController
Selectable itinerary row	`app/views/trip_items/_item_row.html.erb:4–17`	Provide action and coordinate data
`TimelineController#select`	`app/javascript/controllers/timeline_controller.js:11–33`	Send locate/reset notifications
`TimelineController#filter`	Same file, `35–48`	Apply kind filtering
`TimelineController#filterStatus`	Same file, `50–71`	Apply status filtering
`TimelineController#applyFilters`	Same file, `73–85`	Update row/date-group visibility
`MapController#connect`	`app/javascript/controllers/map_controller.js:41–58`	Register listeners and initialize map
`MapController#disconnect`	Same file, `60–65`	Remove map and listeners
`MapController#onResize`	Same file, `68–74`	Recalculate Leaflet layout
`MapController#onReset`	Same file, `77–89`	Restore marker appearance and map extent
`MapController#onLocate`	Same file, `92–109`	Focus and highlight an itinerary marker
`MapController#initMap`	Same file, `111–147`	Create Leaflet map and initial markers
`TripNavController#connect`	`app/javascript/controllers/trip_nav_controller.js:6–16`	Initialize tab and listen to Turbo rendering
`TripNavController#switch`	Same file, `22–29`	Switch tabs and update URL
`TripNavController#show`	Same file, `31–56`	Show panel and request map resize
`TripsController#show`	`app/controllers/trips_controller.rb:11–29`	Supply itinerary data for the view
Turbo trip subscriptions	`app/views/trips/show.html.erb:2` ; `app/views/inbox_items/index.html.erb:1`	Subscribe to trip stream
Inbox refresh declaration	`app/models/inbox_item.rb:5`	Request trip refresh after model commits

Geocoding refresh call	`app/jobs/geocode_trip_items_job.rb:20–21`	Broadcast trip refresh following the job
Global Turbo listeners	`app/javascript/application.js:11–32`	Handle pageviews and transitions

These line ranges refer to the extracted original ZIP. The ZIP's correspondence with the stated Git commit remains unverified.

9. Application, framework and browser responsibility split

Layer	DaPlanStan implements	External infrastructure supplies
Stimulus	Controller methods, target declarations, local UI decisions	Controller lifecycle, binding of actions and targets
Browser DOM	Custom `map:*` event names, event payloads and listener callbacks	`CustomEvent`, `dispatchEvent`, listener registration and propagation
Map interaction	Decide which marker to focus, reset or resize	Leaflet map, markers, movement and tile rendering
Turbo	Request refresh broadcasts and declare subscriptions	Stream subscription, delivery, page refresh and morph behavior
Rails server	Query trip items and construct JSON for the map	Controller rendering and Active Record retrieval

This separation is essential to avoid treating standard framework mechanics as code written by DaPlanStan.

10. Previously examined collaborations — no duplicate counts

Existing collaboration	Frontend appearance	Counting decision
Geocoding shared behavior	New coordinates become available for map display after geocoding	Reuse previous geocoding analysis; no new confirmed instance
`Trip-Scoped Turbo Refresh Notifications`	Trip/inbox subscriptions, broadcast triggers and refresh handling	Existing uncertain candidate; do not create another instance
`Parsed Email to Trip Record Linking`	Inbox changes can lead to Turbo refreshes	Existing confirmed model responsibility; no new instance

The DOM map-event candidate is separate from Turbo's trip stream. One communicates among browser-side components; the other delivers server-originated refresh notifications.

11. Strongest non-pattern explanation

The frontend architecture is best explained as **server-rendered Rails views enhanced by small Stimulus controllers, direct DOM custom events, Leaflet map operations and Turbo lifecycle integration**.

The application uses declarative Stimulus actions for ordinary component behavior. For the timeline and map, it introduces explicit DOM events so that map interactions can be requested without calling another Stimulus controller's methods.

This is a meaningful frontend collaboration, but it is still compatible with the simpler explanation of ordinary browser event-driven programming.

The source does not demonstrate a general-purpose application event bus, custom notification registry or elaborate frontend coordination abstraction.

12. What still requires verification?

Question	Why it matters
Does the live timeline-to-map communication work in desktop and mobile layouts?	The view renders two TimelineController instances targeting the same map
Are listeners always connected when events are dispatched?	Stimulus connection and DOM rendering lifecycles affect listener availability
Are markers updated correctly after Turbo refresh?	The map and panel elements are `data-turbo-permanent`, while the JSON input is supplied at rendering time
Can `map:locate` handle coordinates at latitude or longitude zero?	`onLocate` rejects falsy coordinate values
Does tab switching preserve zoom as intended after morphing?	`map:resize` is intentionally skipped when the panel is already visible
Should targeted DOM events count as Observer?	Requires deciding whether ordinary EventTarget usage meets the assignment's pattern threshold
Does the Turbo refresh mechanism qualify as a framework-idiom instance?	The framework contract supports the mechanism, but the application/framework inventory boundary needs review
Have the client workflows been executed?	No browser runtime test was performed in this investigation

The potentially stale permanent map deserves particular attention. `TripsController#show` generates marker data from the current trip items, but Turbo's permanent-element treatment may preserve an existing map whose markers were constructed earlier. Verify the actual behavior after new geocoding results or itinerary changes instead of assuming the broadcast always refreshes the markers.

13. Investigation outcome

Frontend Investigation 06

Verification pending

Stimulus controllers inventoried	16
Application-defined map events	3
New pattern candidates	1 unconfirmed
Existing Turbo candidate	Stronger evidence; still uncertain
New confirmed instances	0
Browser runtime testing	Not performed

Candidate-review record

Writing

Frontend Interaction — Preliminary Candidate Review

Source: DaPlanStan original uploaded source ZIP
Method: Static inspection of Stimulus controllers, views, custom-event call sites and Turbo subscriptions.
Runtime tests: Not performed.

Observed collaboration

`TimelineController` dispatches application-defined `map:locate` and `map:reset` events when geocoded itinerary rows are selected or deselected. `TripNavController` dispatches `map:resize` when the itinerary panel becomes visible. `MapController` explicitly registers listeners for these three events and invokes Leaflet operations to move, reset or resize the map.

The controllers communicate through DOM events rather than directly invoking one another's methods. However, event publishers know the map's DOM identity and the specific event names.

New candidate

- Pattern: Observer-like DOM Event Coordination
- Category: GoF Behavioral candidate

- Provisional instance: Itinerary Timeline–Map Event Coordination
- Status: UNCONFIRMED
- Origin: Mixed application/browser behavior

The code demonstrates producers, registered listeners and event-driven reactions. The strongest alternative explanation is ordinary targeted DOM event communication, without an application-defined observer abstraction.

Existing candidate revisited

Trip-Scoped Turbo Refresh Notifications remains UNCERTAIN. The application contains trip-scoped subscriptions and refresh broadcasts, while Turbo Rails 2.0.23 implements the corresponding subscription and model-change broadcasting contracts. This strengthens the structural evidence but does not settle application-versus-framework counting eligibility.

Duplicate-counting rule

Do not create separate instances for the three map events. Do not recount the existing Turbo collaboration for each stream subscriber, broadcast trigger or source area.

Conclusion

No new pattern is confirmed in the frontend investigation. Preserve the timeline-to-map collaboration as one new candidate, retain the existing Turbo candidate and separately record the identified frontend lifecycle and marker-refresh verification questions.

The next verification step within this source area should decide whether **Itinerary Timeline–Map Event Coordination** exceeds ordinary DOM event handling sufficiently to qualify as Observer, and whether the trip-stream Turbo mechanism is eligible as a framework-mediated pattern instance.

Before doing the repository-wide pattern sweep, close the frontend investigation by verifying these two candidates:

1. Itinerary Timeline–Map Event Coordination
2. Trip-Scoped Turbo Refresh Notifications

Evaluate them separately.
For Itinerary Timeline–Map Event Coordination:

1. State the minimum Observer mechanism that would need to exist.

2. Trace the actual publishers:
   `TimelineController#select`
   `TripNavController#show`
3. Trace the actual subscriber:
   `MapController`
4. Show how listener registration, event dispatch and notification handling work using the real source.
5. Check whether the producers directly call `MapController` methods or instead notify through the browser event mechanism.
6. Then challenge the classification:
   the producers know the map DOM element
   they know the exact event names
   there is effectively one receiver
   browser `EventTarget` supplies the observer infrastructure
7. Decide whether this is sufficiently substantive to call Observer, or whether "targeted DOM event communication" is the more accurate description.
8. Give a final decision:
   CONFIRMED
   REJECTED
   UNCERTAIN

If confirmed, keep all three `map:*` events inside one instance.
For Trip-Scoped Turbo Refresh Notifications:

1. Use the application evidence already found:
   `turbo_stream_from @trip`
   `broadcasts_refreshes_to :trip`
   `broadcast_refresh_to(trip)`
2. Use the checked Turbo Rails 2.0.23 framework contract to identify publisher, subscription and delivery roles.
3. Decide whether this qualifies as a framework-mediated Observer collaboration under the same reasoning used for the already confirmed `Outbound Mail Delivery Logging` Observer.
4. Compare the two directly:
   what application code defines
   what framework code supplies
   whether a concrete observer/subscriber exists

how registration/subscription happens
how notification is delivered

5. Explain why the Turbo collaboration should remain uncertain if its evidence is materially weaker than the mail Observer.
6. If it is confirmed, count it once across both geocoding and inbox workflows.
7. If it is rejected, explain why the framework usage is not sufficient for the assignment's pattern threshold.

At the end give:

1. Final decision table for both candidates.
2. Participant-role mapping for any confirmed Observer instance.
3. Stable instance name.
4. Source evidence to preserve.
5. Counting decision.
6. Any remaining runtime questions kept separate from pattern classification.

Do not search for new patterns in this prompt.

Worked for 1m 7s

DaPlanStan — Frontend Observer Verification

Candidate Review 06 · Original source ZIP · Static source and framework-contract verification

1. Final decisions

Itinerary Timeline–Map Event Coordination Proposed GoF Observer	REJECTED
Trip-Scoped Turbo Refresh Notifications Framework-mediated Observer	CONFIRMED

The distinction is based on what the collaborations actually do, not simply whether they use events.

The timeline–map implementation sends commands to a specifically identified map element through browser events. The Turbo implementation establishes a genuine trip-scoped publication/subscription relationship through the framework's documented contract.

The Turbo confirmation is for a **framework-mediated Observer instance**, not an independently implemented application-owned GoF Observer framework. Runtime operation and independent

human sign-off remain separate verification matters.

2. Candidate A — Itinerary Timeline–Map Event Coordination

Minimum Observer mechanism

A defensible Observer implementation should have an identifiable subject or event publisher, an observer that registers interest, a notification mechanism and a response triggered by that notification.

For this investigation, merely using `CustomEvent` is insufficient. The key question is whether the application implements a meaningful subscription relationship or simply uses events as an indirect way to direct operations at a known component.

Actual publishers

Publisher 1: `TimelineController#select`

`app/javascript/controllers/timeline_controller.js:11–33`

The method receives a Stimulus click action from an itinerary row. It locates the map using:

```javascript
const map = document.getElementById("trip-map")
```

When a selected row is clicked again, it dispatches `map:reset` at lines 21–22.

When a new geocoded row is selected, it dispatches `map:locate` with latitude, longitude and the itinerary-item ID at lines 26–34.

The row click binding is defined in `app/views/trip_items/_item_row.html.erb:7–17`.

Publisher 2: `TripNavController#show`

`app/javascript/controllers/trip_nav_controller.js:31–56`

When the itinerary panel becomes visible after being hidden, it executes:

```javascript
const mapEl = p.querySelector("[data-controller='map']")
if (mapEl) mapEl.dispatchEvent(new CustomEvent("map:resize"))
```

The dispatch occurs at lines 40–43. Its purpose is specifically to tell the map to recalculate its dimensions.

Actual subscriber

`MapController` registers all three event handlers in `app/javascript/controllers/map_controller.js:51-56`:

JavaScript

```
this.handleLocate = this.onLocate.bind(this)
this.handleReset  = this.onReset.bind(this)
this.handleResize = this.onResize.bind(this)

this.element.addEventListener("map:locate", this.handleLocate)
this.element.addEventListener("map:reset", this.handleReset)
this.element.addEventListener("map:resize", this.handleResize)
```

It removes these listeners during `disconnect`, lines 60–64.

The registered methods perform real application behavior:

Event	Handler	Source lines	Effect
`map:locate`	`MapController#onLocate`	`92-109`	Moves map and highlights an itinerary marker
`map:reset`	`MapController#onReset`	`77-89`	Restores markers and overall map bounds
`map:resize`	`MapController#onResize`	`68-74`	Recalculates Leaflet map size and bounds

These are actual dispatch/listener relationships. They are not merely similarly named methods.

Does either publisher directly call `MapController`?

No.

Neither publisher retrieves a `MapController` instance and invokes its methods. Instead, each dispatches a browser event onto the map's DOM element.

That creates a degree of implementation separation: the publishers do not need to know how Leaflet markers are created or moved.

However, the separation is limited.

Strongest evidence against Observer

The producers explicitly identify the destination map element:

- `TimelineController` uses `document.getElementById("trip-map")`.
- `TripNavController` searches for `[data-controller='map']`.

They also know the map-specific event names.

The collaboration has one intended receiver, `MapController`, and the three events encode requests for particular map operations.

There is no demonstrated application-defined subject whose changing state is published to independently interested observers. Nor does the source show a broader registration contract that allows different application components to subscribe to itinerary-selection changes.

Although the browser's `EventTarget` mechanism technically supports multiple listeners, the application uses it here primarily to deliver targeted commands.

This is the decisive distinction: the code implements a genuine DOM event relationship, but the more precise architectural description is targeted inter-controller communication through browser events.

A one-receiver implementation can qualify as Observer in principle. The rejection is not based on receiver count alone. It is based on the combination of explicit destination targeting, event semantics and the absence of a substantive subject/observer responsibility beyond ordinary DOM event dispatch.

Origin and decision

Origin: Mixed. DaPlanStan defines event names, dispatch conditions and reactions. The browser supplies `CustomEvent`, `EventTarget`, listener registration and dispatch. Stimulus supplies controller lifecycle integration.

Final decision: REJECTED as a distinct GoF Observer instance.

This is not a rejection of the implementation's functionality. It is a decision not to inflate the design-pattern inventory by relabeling ordinary targeted DOM event communication.

All three events remain documented together as one rejected candidate, rather than three pattern instances.

3. Candidate B — Trip-Scoped Turbo Refresh Notifications

CONFIRMED — Framework-mediated Observer

Essential mechanism

For Observer to be defensible, this collaboration must provide an identifiable notification source, a subscribed receiver and a mechanism that delivers updates to that receiver without the publisher directly invoking its UI-update methods.

The application supplies all the necessary configuration and invocation points. The checked Turbo Rails 2.0.23 source supplies the missing subscription and dispatch infrastructure.

A. Application publishers

Publisher path 1: `InboxItem` changes

`app/models/inbox_item.rb:1–5`

‹/› Ruby

```ruby
class InboxItem < ApplicationRecord
  belongs_to :trip
  belongs_to :reviewed_by, class_name: "User", optional: true

  broadcasts_refreshes_to :trip
end
```

The meaningful line is:

‹/› Ruby

```ruby
broadcasts_refreshes_to :trip
```

In Turbo Rails 2.0.23, this declaration registers an `after_commit` callback that schedules a refresh broadcast to the specified streamable, which here resolves to the `InboxItem` 's associated trip. GitHub

Thus, the model does not directly find a browser and refresh it. It requests a notification through Turbo's infrastructure.

Publisher path 2: Geocoding completion

`app/jobs/geocode_trip_items_job.rb:4–21`

After processing the requested itinerary items, the job invokes:

```ruby
Turbo::StreamsChannel.broadcast_refresh_to(trip)
```

The framework constructs a Turbo Stream refresh action and publishes it to the stream identified by the supplied trip. GitHub

These are two triggers for the same kind of trip-scoped notification.

B. Application subscribers

The trip page declares:

`app/views/trips/show.html.erb:2`

```erb
<%= turbo_stream_from @trip %>
```

The inbox index declares the same subscription:

`app/views/inbox_items/index.html.erb:1`

```erb
<%= turbo_stream_from @trip %>
```

The views do not call the publishing model or geocoding job.

Instead, each rendered page independently subscribes to its trip's stream.

C. Verified Turbo Rails 2.0.23 contract

I checked the actual tagged framework source rather than relying on a description of Turbo's behavior.

1. **Subscription declaration**

   `turbo_stream_from` produces a `turbo-cable-stream-source` element containing the channel and signed stream name. GitHub

2. **Server-side registration**

> `Turbo::StreamsChannel#subscribed` verifies the signed stream name and calls `stream_from` to establish the Action Cable subscription. GitHub

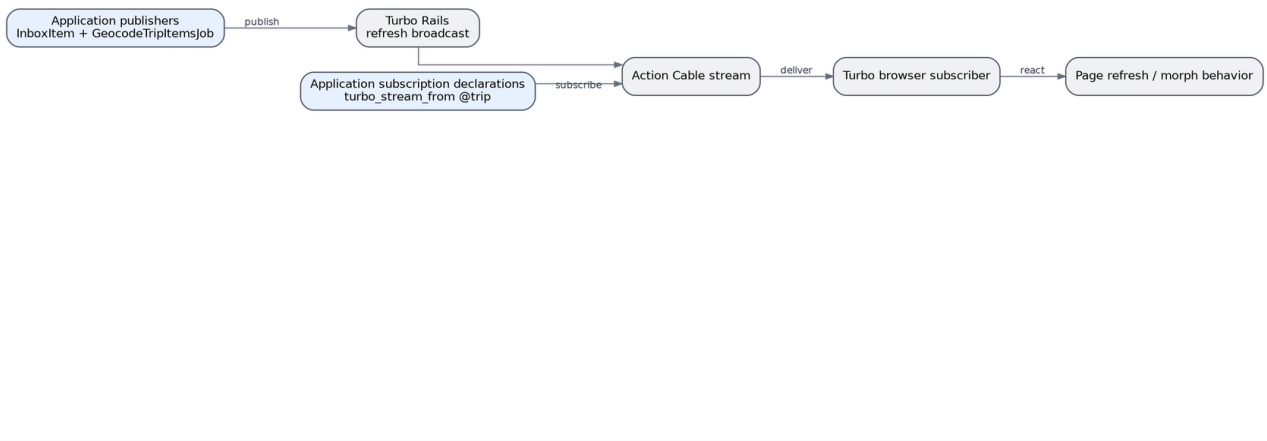
3 **Publication**

> `broadcast_refresh_to` builds a refresh action. `broadcast_stream_to` delivers its content through `ActionCable.server.broadcast` for the computed stream name. GitHub +1

4 **Client reaction**

The Turbo client receives the stream action and applies its standard refresh behavior. DaPlanStan's trip view requests morph-style refresh and scroll preservation through meta tags at `app/views/trips/show.html.erb:5–9` .

The resulting structure is:



The source and framework contract therefore establish an actual publication/subscription relationship, not merely an event-like method name.

4. Comparison with Outbound Mail Delivery Logging

Dimension	Outbound Mail Delivery Logging	Trip-Scoped Turbo Refresh Notifications
Pattern	Observer	Observer
Application publisher configuration	`register_observer(MailLogger)`	`broadcasts_refreshes_to :trip` and `broadcast_refresh_to(trip)`

Framework mechanism	Action Mailer / Mail gem	Turbo Rails / Action Cable
Concrete observer/subscriber	Application-defined `MailLogger`	Framework-provided browser stream subscriber instantiated by application views
Registration/subscription	Explicit registration call	`turbo_stream_from @trip` generates subscription
Notification callback/reaction	`MailLogger.delivered_email(message)`	Turbo refresh action handled by browser-side Turbo
Application-defined reaction	Custom log message	Choice of trip stream and refresh behavior/configuration
Source mechanism	Confirmed	Confirmed
Implementation origin	Mixed	Mainly framework-provided

Why Turbo has slightly weaker application ownership

The mail logging instance includes a concrete observer method written by DaPlanStan:

`MailLogger.delivered_email(message)`, `config/initializers/mail_logger.rb:2–4`.

The Turbo collaboration does not implement a custom browser observer class or notification callback. Instead, its concrete subscriber is provided by Turbo, and DaPlanStan activates it through the view helper.

Nevertheless, the application explicitly defines **both sides of the communication**:

- It chooses the trip as the notification subject.
- It declares model-driven and job-driven broadcasts.
- It renders pages subscribing to that trip's stream.
- The versioned framework implementation connects those publishers and subscribers.

Under the same acceptance rule used for the mail Observer—framework-mediated collaborations are eligible when the application explicitly participates in a documented Observer mechanism—this is sufficient.

Final decision: CONFIRMED as a framework-mediated Observer instance.

It should not be described as a custom GoF Observer implementation written entirely by DaPlanStan.

Stable instance name

Trip-Scoped Turbo Refresh Notifications

This preserves the name already used during the geocoding and inbound-email investigations.

It also prevents the discovery process from introducing multiple labels for the same mechanism.

5. Final decision table

Candidate	Actual mechanism	Strongest objection	Origin	Final decision
Itinerary Timeline–Map Event Coordination	Two controllers dispatch map-specific DOM events to a known map element; `MapController` registers listeners and reacts	Targeted commands to one intended component are better explained as ordinary DOM event communication than a substantive Observer responsibility	Mixed application/browser	**REJECTED**
Trip-Scoped Turbo Refresh Notifications	Application declares trip-stream subscriptions and publishes refresh requests; Turbo Rails and Action Cable implement subscription and delivery	Concrete subscriber and delivery mechanism are framework-provided, with no custom application observer callback	Mainly framework-provided, with explicit application participation	**CONFIRMED**

6. Final participant-role mapping

Only the confirmed Turbo collaboration receives an inventory participant mapping.

Pattern role / responsibility	Actual participant	Source location
Publication source — model	`InboxItem`	`app/models/inbox_item.rb:1–5`
Publication source — job	`GeocodeTripItemsJob#perform`	`app/jobs/geocode_trip_items_job.rb:4–21`
Publisher/broadcast infrastructure	`Turbo::StreamsChannel` and `Turbo::Streams::Broadcasts`	Turbo Rails 2.0.23 dependency source
Subscription declaration — trip page	`turbo_stream_from @trip`	`app/views/trips/show.html.erb:2`
Subscription declaration — inbox page	`turbo_stream_from @trip`	`app/views/inbox_items/index.html.erb:1`
Subscription implementation	`Turbo::StreamsChannel#subscribed`	Turbo Rails 2.0.23
Concrete subscriber	Turbo cable stream source element rendered for each subscribed browser page	Generated by Turbo Rails helper and client infrastructure

Delivery mechanism	Action Cable stream subscription and broadcasting	Rails framework
Notification reaction	Turbo refresh processing	Turbo browser runtime
Application refresh configuration	Morph refresh and scroll preservation	`app/views/trips/show.html.erb:5-9`

There is no independently declared DaPlanStan `Subject` interface, `Observer` interface or custom subscriber callback class. Those roles are implemented through the documented framework infrastructure.

7. Counting decision

Observer instances after frontend verification

Outbound Mail Delivery Logging	1 confirmed
Trip-Scoped Turbo Refresh Notifications	1 confirmed
Itinerary Timeline–Map Event Coordination	0 — rejected
Total source-confirmed Observer instances	**2**

The Turbo instance should be counted **once**, even though it appears in both the geocoding and inbound-email workflows and has two subscribing views.

Its instance boundary is the trip-scoped notification mechanism, not each model callback, job invocation, message or connected browser.

This changes the previous Turbo candidate from uncertain to confirmed. It does not add a newly discovered, separate Turbo collaboration.

8. Source evidence to preserve

For the confirmed Turbo instance, preserve the source declarations and versioned framework contract:

Evidence	Location	Purpose
Inbox broadcast declaration	`app/models/inbox_item.rb:5`	Establish model publication
Explicit geocoding broadcast	`app/jobs/geocode_trip_items_job.rb:20-21`	Establish job publication

Trip page subscription	`app/views/trips/show.html.erb:2`	Establish receiving page
Inbox page subscription	`app/views/inbox_items/index.html.erb:1`	Establish second receiving page
Turbo refresh configuration	`app/views/trips/show.html.erb:5-9`	Record requested browser refresh behavior
Dependency version	`Gemfile.lock:424`	Identify Turbo Rails 2.0.23
Tagged `Turbo::Broadcastable` implementation	Turbo Rails 2.0.23 source ↗	Establish after-commit notification
Tagged subscription channel	StreamsChannel source ↗	Establish stream registration
Tagged broadcasts module	Broadcasts source ↗	Establish delivery

For the rejected timeline–map candidate, retain the three event dispatch sites, listener registration/removal, map handlers and the explanation for why targeted DOM communication is a stronger classification.

9. Runtime questions — separate from pattern identification

The confirmed static mechanism does not prove every notification produces the desired UI outcome.

The following remain appropriate for runtime testing:

- Whether an active trip page and inbox page both receive refresh actions after an `InboxItem` commit.
- Whether a geocoding job's explicit broadcast refreshes the subscribed trip page.
- Whether `data-turbo-permanent` preserves an outdated Leaflet map or marker collection despite a successful page refresh.
- Whether repeated or closely spaced broadcasts are debounced as expected.
- Whether timeline-to-map events behave correctly when desktop and mobile timelines are both present.
- Whether listeners reconnect correctly after navigation and Turbo morphing.

These do not need to block the static identification of the Turbo Observer collaboration. They should be recorded as functional and integration-verification questions.

10. Final frontend candidate-review record

Writing

Frontend Interaction — Final Candidate Verification

Source version: Original DaPlanStan uploaded source ZIP
Review scope: Application-defined map events and trip-scoped Turbo refresh notifications
Method: Static application-source verification and Turbo Rails 2.0.23 framework-contract verification
Runtime tests: Not performed
Independent human sign-off: Pending

Candidate 1: Itinerary Timeline–Map Event Coordination

- Proposed pattern: Observer
- Final decision: REJECTED
- Origin: Mixed application/browser behavior

`TimelineController#select` publishes `map:locate` and `map:reset` through DOM events. `TripNavController#show` publishes `map:resize`. `MapController` registers corresponding listeners and uses Leaflet to update the map.

Although explicit event dispatch and registration exist, the producers target the known map element and use map-specific event names to request particular operations. The stronger explanation is ordinary targeted DOM event communication rather than a distinct GoF Observer collaboration.

Preserve all three events as one rejected candidate. Do not create pattern-inventory rows for them.

Candidate 2: Trip-Scoped Turbo Refresh Notifications

- Pattern: Observer
- Category: GoF Behavioral — framework-mediated implementation
- Final decision: CONFIRMED
- Stable instance name: Trip-Scoped Turbo Refresh Notifications
- Origin: Mixed, with framework-provided subscription and notification infrastructure

`InboxItem` declares `broadcasts_refreshes_to :trip`, and `GeocodeTripItemsJob` calls `Turbo::StreamsChannel.broadcast_refresh_to(trip)`. The trip show page and inbox index subscribe through `turbo_stream_from @trip`.

Turbo Rails 2.0.23 provides the documented subscription and broadcasting mechanism. Its

stream channel establishes subscriptions using signed stream names, while broadcasts deliver refresh actions through Action Cable.

The application configures publishers and subscribers for the same trip-scoped stream. This supports a framework-mediated Observer classification without claiming that DaPlanStan implements its own observer registry or browser notification runtime.

Counting decision

Count one Trip-Scoped Turbo Refresh Notifications instance across geocoding and inbound-email processing.

Together with Outbound Mail Delivery Logging, there are two source-confirmed Observer instances in the investigations completed so far.

Limitations

Runtime stream delivery, Turbo morphing behavior, permanent-map updates and browser reconnection have not been tested. These remain separate from the static pattern decision.

The original archive's correspondence with the stated Git commit and independent human verification also remain unresolved.

Frontend investigation conclusion: One proposed Observer classification is rejected as targeted DOM event communication. The existing Turbo candidate is confirmed as one framework-mediated Observer collaboration. No additional Observer instance should be created for its repeated appearances across source areas.

We have now completed focused investigations of geocoding, inbound email processing, trip creation and membership, itinerary import and expenses, external integrations and identity, and frontend interactions.
Before freezing the baseline inventory, do one repository-wide sweep to check whether any important pattern collaboration has been missed.
Do not start by trying to increase the number of patterns.
Use the patterns and rejected candidates already established as context, but inspect the remaining application source independently.

Current source-confirmed collaborations are:

1. GRASP Information Expert — Parsed Email to Trip Record Linking
2. GRASP Controller — Inbound Travel Email Intake Coordination
3. GRASP Controller — Trip Creation Coordination
4. GRASP Controller — Trip Membership Management Coordination
5. GRASP Controller — Bulk Itinerary Import Coordination
6. GRASP Controller — OAuth Sign-in and Identity Linking Coordination
7. Observer — Outbound Mail Delivery Logging
8. Observer — Trip-Scoped Turbo Refresh Notifications

Previously rejected examples include Template Method, Strategy and Adapter in geocoding, and Observer for the targeted timeline-to-map DOM events.
For this sweep:

1. Review the application source areas that have not received the same focused treatment yet.
2. Check controllers, models, concerns, jobs, mailboxes, mailers, helpers, initializers and JavaScript for collaborations that involve multiple application participants.
3. Pay attention to creation responsibility, responsibility assignment, object composition, indirection, polymorphic behavior, notification relationships and algorithms that are genuinely interchangeable.
4. Check whether any previously inspected source contains a second independent collaboration that was overlooked.
5. Do not treat Rails inheritance, enums, callbacks, Active Record associations, `create!` calls, background jobs, mailers or Stimulus controllers as patterns by themselves.
6. Do not propose another GRASP Controller merely because an unreviewed controller handles CRUD. It must coordinate a meaningful system event comparable to the already confirmed instances.
7. Do not propose Information Expert merely because a model contains methods. Show why the responsibility belongs with the information held by that object.

8. Do not propose Strategy unless there are actually interchangeable algorithms or implementations selected through a common role.
9. Do not propose Adapter unless there is real interface translation between incompatible collaborators.
10. Do not propose Factory or Creator merely because objects are constructed.
11. Do not propose Observer merely because callbacks or events exist. Establish publisher, subscription/registration and notification behavior.
12. Check carefully for duplicate instances. Reappearance of an already confirmed collaboration in another file or workflow is not a new instance.

For every genuinely new candidate you find:

1. Give the pattern and category.
2. Give a meaningful provisional instance name.
3. Identify the exact participants.
4. Give source paths and reliable line ranges.
5. Explain the collaboration that supports it.
6. Give the strongest false-positive explanation.
7. Compare it with the closest already confirmed or rejected candidate.
8. Leave it UNCONFIRMED.

Also report important source areas you inspected where no additional pattern was found. That is useful evidence of coverage. At the end give me:

1. Source areas checked in this final sweep.
2. Any genuinely new candidates.
3. Previously known collaborations encountered again and deduplicated.
4. Important rejected similarities.
5. Remaining uninspected or ambiguous source areas, if any.
6. Whether the repository is now sufficiently covered to begin the final adversarial review.

Do not generate `patterns.csv`, counts or UML yet.

Worked for 2m 52s

DaPlanStan — Repository-Wide Pattern Coverage Sweep

Baseline pre-freeze review · Original uploaded ZIP · Static source inspection

1. Scope and counting standard

I am treating the eight previously source-confirmed collaborations as the working baseline, without treating their confirmation as completed independent human verification.

The final sweep examines the remaining source for **independent collaborations**, rather than collecting additional instances of Rails inheritance, controller actions or model methods.

The distinction used throughout this review is:

- **Pattern evidence:** An identifiable responsibility assignment or interaction mechanism involving actual source participants.
- **Ordinary application behavior:** Framework conventions, CRUD operations, helper methods, conditionals and direct API calls that do not demonstrate a distinctive pattern collaboration.
- **Duplicate evidence:** Another appearance of a collaboration already identified and counted.

The repository archive contains additional application code outside the six focused investigations, notably the administrator controllers, invitation management, notification aggregation, account settings, general helpers and shared persistence configuration. These are important to inspect even if they yield no new pattern.

2. Previously less-examined controller areas

Area	Actual source behavior	Sweep assessment
Administrator authorization	`Admin::ApplicationController` uses an admin-role predicate in `require_admin!`.	Shared Rails before-action authorization, not a new pattern.
Administrator dashboard	`Admin::DashboardController#index` gathers counts of users, trips, items, expenses, inbox records and invites.	Dashboard queries and presentation preparation, not a distinct coordinated business operation.
Administrator invitations	`Admin::InvitesController` lists, creates and destroys `Invite` records.	Conventional administrative CRUD; no new GRASP Controller justified.
Public invitation handling	`InvitesController#accept` validates an invitation token and places it in the session for later OAuth processing.	Supporting phase of the already confirmed OAuth sign-in/account-provisioning collaboration.
Account settings	`SettingsController#update` updates user preferences and synchronizes the account name if one exists.	Direct record updates, not a substantive separate system-event pattern instance.

Notification index	`NotificationsController#index` queries outstanding trip inbox records for the signed-in user.	Read-only data aggregation, not a new Observer publisher or subscriber.
Account travelers	`TravelersController` finds or creates account-level traveler records, sends invitations and handles profile updates/deletion.	Relevant to existing invitation and identity workflows; requires careful duplicate-count review.
Home/landing	`HomeController` redirects authenticated users or serves informational pages.	Ordinary routing and controller behavior.

The relevant source locations include `app/controllers/admin/application_controller.rb:1–12`, `admin/dashboard_controller.rb:1–17`, `admin/invites_controller.rb:1–27`, `app/controllers/invites_controller.rb:4–41`, `app/controllers/settings_controller.rb:5–31` and `app/controllers/notifications_controller.rb:4–11`.

No additional GRASP Controller instance is established merely by these controllers receiving requests. Their responsibilities are either ordinary CRUD/display operations or supporting parts of previously identified use cases.

3. New candidates found during the sweep

Two responsibilities merit adversarial verification. Neither is strong enough to add directly to the baseline inventory.

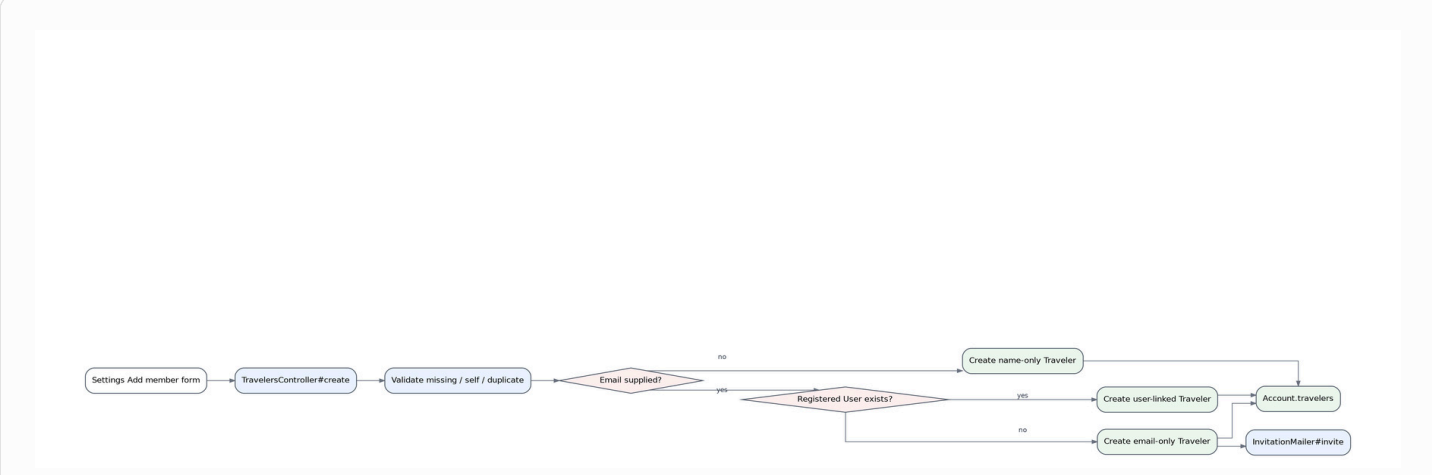
Candidate A — Account Traveler Enrollment Coordination

GRASP Controller · Responsibility assignment UNCONFIRMED

Provisional instance name: `Account Traveler Enrollment Coordination`

System event: An account owner submits the *Add member* form in Settings, either with an email address or with only a name.

This enters `TravelersController#create`, which coordinates more than a single model insertion.



Participant evidence

Participant	Source location	Responsibility
`TravelersController#create`	`app/controllers/travelers_controller.rb:5-35`	Receive enrollment event, select enrollment path and arrange invitation
`TravelersController#set_account`	Same file, `54-56`	Restrict operation to the acting account owner
`Account`	`app/models/account.rb:1-11`	Own account-scoped traveler association
`User`	`app/models/user.rb:11-22`	Supply registered-user identity
`Traveler`	`app/models/traveler.rb:1-8, 18-28`	Represent account participant and later invitation acceptance
`InvitationMailer#invite`	`app/mailers/invitation_mailer.rb:2-6`	Construct account invitation
Settings form	`app/views/settings/show.html.erb:114-127`	Submit account-member enrollment
Route	`config/routes.rb:48-50`	Expose account-traveler creation

The controller rejects missing names, self-invitations and duplicates. It distinguishes known users from new email-only travelers, creates the appropriate account record and requests an invitation.

Why GRASP Controller might fit: The controller receives a coherent account-enrollment event and coordinates identity lookup, account association, record creation and invitation delivery.

Strongest false-positive explanation: This remains fairly short procedural controller logic. It performs the enrollment decisions itself and uses normal Active Record calls. The existence of branching and an email delivery request does not automatically make it a noteworthy GRASP Controller instance.

Comparison with existing instances

The closest previously confirmed collaboration is `Trip Membership Management Coordination`.

There is a meaningful boundary difference:

Account traveler enrollment	Trip membership management
Starts from account Settings	Starts from an existing trip
Creates an account-level `Traveler`	Creates or reuses a `Traveler`, then creates a `TripMember`
Does not assign a trip role	Assigns the `contributor` role
Can create name-only travelers	Membership management requires a traveler selected or resolved for the trip
Uses general `InvitationMailer#invite`	Can use trip-specific `InvitationMailer#invite_to_trip`

The two controllers have overlapping user-resolution logic, but their resulting records and immediate use-case objectives differ. The trip membership controller explicitly creates a `TripMember`; the account controller does not. ‹/›

Decision: UNCONFIRMED.

This is the strongest new candidate found in the sweep. The final adversarial review should decide whether it deserves a separate GRASP Controller instance or is better treated as ordinary account-traveler CRUD and invitation handling.

Do not count it yet.

Candidate B — Beta Invite Eligibility Evaluation

GRASP Information Expert · Responsibility assignment UNCONFIRMED

Provisional instance name: `Beta Invite Eligibility Evaluation`

This is a narrower and weaker candidate.

Unlike a generic model-method claim, it does have specific cross-object responsibility evidence: the invitation record owns the information necessary to determine whether its token can be used and whether it applies to a particular email address.

Actual responsibility allocation

`Invite` defines:

‹/› Ruby

```
  def used?    = used_at.present?
  def expired? = expires_at.present? && expires_at < Time.current
  def usable?  = !used? && !expired?

  def email_matches?(address)
    email.blank? || email.casecmp?(address.to_s)
  end
```

These methods appear at `app/models/invite.rb:12-18`.

Two application callers use that information:

- `InvitesController#accept`, `app/controllers/invites_controller.rb:5-15`, checks `usable?` before saving the invitation token in the session.
- `SessionsController#provision_account_or_gate`, `app/controllers/sessions_controller.rb:63-75`, checks both `usable?` and `email_matches?` before creating an account and marking the invite used.

‹/› ‹/› ‹/›

Why Information Expert might fit

The `Invite` record owns `used_at`, `expires_at` and `email`, the precise values needed for these decisions.

DaPlanStan has placed the elementary eligibility rules on the object possessing that information. Two separate controller entry points reuse these predicates rather than reimplementing their conditions.

This is a plausible GRASP Information Expert responsibility assignment.

Strongest false-positive explanation

The entire implementation consists of a few simple domain predicates.

There is no substantial collaboration comparable to `InboxItem#fill_missing_links!`, which interprets structured information and coordinates the creation and linking of multiple related records.

The more accurate explanation may be ordinary domain-model encapsulation: keeping simple validation predicates close to their data.

Furthermore, the full signup eligibility decision remains in `SessionsController`. `Invite` does not independently decide whether a user should receive an account.

Origin: Application-owned predicates with Active Record persistence.

Closest established comparison: `Parsed Email to Trip Record Linking` — GRASP Information Expert. The proposed invite instance is materially simpler and needs a stricter significance check.

Decision: UNCONFIRMED, low priority.

I would not include this in the baseline merely because it is technically compatible with the Information Expert principle. It needs a positive argument that the responsibility is substantial enough to count consistently with the other instances.

---

4. Areas checked with no additional pattern evidence

The following negative findings are useful for demonstrating that the final inventory was not produced by pattern-name searching.

Source area	Relevant evidence	Reason no additional pattern is proposed
Admin dashboards and permissions	`app/controllers/admin/*`	Admin checks and aggregation queries are ordinary controller responsibilities
Admin invitation issuance	`admin/invites_controller.rb:1–27`	Basic create/list/delete operations
Settings updates	`settings_controller.rb:5–31`	Updates user data and synchronizes account name; no separate behavioral contract
Notification aggregation	`notifications_controller.rb:4–11` ; `application_controller.rb:27–41`	Repeated membership-scoped queries, not an event-subscription mechanism
`MediaItem`	`app/models/media_item.rb:1–6`	Associations, attachment and validation only; no application workflow using `MediaItem` was found
`ApprovedSender`	`app/models/approved_sender.rb:1–6` ; approval controller	Simple persistence and previously traced inbound-email approval logic
`ApplicationRecord`	`app/models/application_record.rb:1–6`	Shared UUID initialization through Rails callback
UUID initializer	`config/initializers/uuid_primary_keys.rb:1–19`	Adds callbacks to Rails-owned models; not application-defined Template Method
`User` and `Traveler` display helpers	`user.rb:24–33` ; `traveler.rb:10–36`	Avatar/email fallbacks and simple identity predicates
`Trip` display/count operations	`trip.rb:21–37`	Count and inbound-address generation, already encountered

Expense and itinerary helpers	`app/helpers/expenses_helper.rb`, `trip_items_helper.rb`	Static icon, label and grouping lookup tables
SVG/email rendering helpers	`application_helper.rb:5-47`; `inbox_items_helper.rb:2-51`	Data formatting and presentation preparation
Remaining Stimulus controls	Local pickers, clipboard, date-range, receipt, navigation and deletion controls	Ordinary component interaction using browser and Stimulus facilities
PWA service worker	`app/views/pwa/service-worker.js`	Example push/notification handlers are commented out; no active custom notification collaboration
Geocoding and receipt jobs	Previously traced source	Dependency calls and application processing, without additional independent collaboration

`MediaItem` is particularly straightforward: the source declares a model and a `Trip#media_items` association, but a search of controllers, jobs and views found no application call that creates or manages `MediaItem` records. It should not be assigned a speculative object-composition pattern.

The UUID initialization code similarly uses Rails callback infrastructure. Sharing a callback is not evidence of an application-implemented Template Method or Factory.

No additional polymorphic or interchangeable implementation family

The broad source scan found no additional application-defined strategy implementations, adapter interface families, abstract factory participants or custom observer registration mechanisms.

`TripItem` remains the sole application model including `Geocodable`. Rails enum values remain data values, not separate behavioral states.

The scan also found no application-written services or additional `lib/` implementation supplying an overlooked algorithm abstraction.

This is negative evidence from the uploaded snapshot, not a claim about every possible runtime behavior supplied by installed gems.

5. Previously known collaborations encountered again

The sweep revisited the previously established collaborations without finding a reason to split or recount them.

Established collaboration	Reappearance during sweep	Deduplication decision
Parsed Email to Trip Record Linking	`InboxItem`, `TripItem`, `Expense` and their associations	Existing Information Expert instance

Inbound Travel Email Intake Coordination	`TripMailbox`, approval handling and pending-inbox notifications	Existing GRASP Controller instance
Trip Creation Coordination	`Trip`, `Account#owner_traveler`, `TripMember`, cover job	Existing GRASP Controller instance
Trip Membership Management Coordination	Traveler resolution and trip invitation paths	Existing GRASP Controller instance
Bulk Itinerary Import Coordination	JSON import, item creation and geocoding scheduling	Existing GRASP Controller instance
OAuth Sign-in and Identity Linking Coordination	`SessionsController`, pending travelers, `Invite` eligibility	Existing GRASP Controller instance
Outbound Mail Delivery Logging	`MailLogger` initializer and the mailer delivery chain	Existing Observer instance
Trip-Scoped Turbo Refresh Notifications	`InboxItem`, `GeocodeTripItemsJob`, trip and inbox views	Existing Observer instance

A renewed search for custom event/broadcast declarations again found the previously examined mail observer, Turbo subscription/broadcast sites and three map-specific DOM events. No second custom notification family appeared in the application source. ‹/›

The two new candidate records remain outside confirmed inventory totals.

6. Important false similarities rejected during coverage checking

Apparent similarity	Why it is insufficient
Admin and settings controllers as GRASP Controller	Merely receiving Rails requests does not establish a distinct, substantive use-case coordination responsibility
`Invite` token generation as Factory/Creator	A `before_validation` callback assigning a random token is normal model initialization
`ApplicationRecord` UUID callbacks as Template Method	No application-defined template algorithm with overridable primitive steps
`Trip` / `User` / `Traveler` relationships as Composite	These are relational associations, not recursive part-whole objects exposing a common recursive operation
OAuth Google/Apple configuration as Strategy	Provider implementations are configured through OmniAuth, not written as interchangeable DaPlanStan algorithms
Gravatar/avatar fallbacks as Strategy	Fixed preference rules, with no selectable strategy implementations
Icon/label mappings as Strategy	Hash lookups and conditional formatting do not select behavioral objects

Pexels/Anthropic wrappers as Adapter	Direct provider API integration does not establish interface translation between incompatible application contracts
`create!` calls as Factory or GRASP Creator	Active Record construction alone does not justify a separate creation-pattern responsibility
Model callbacks as Observer	Rails lifecycle hooks alone do not establish an independently meaningful publication/subscription collaboration
Map `CustomEvent` communication as Observer	Previously rejected as targeted DOM event communication
Multiple Turbo subscribers as multiple Observer instances	They use one trip-scoped framework notification mechanism

These are exclusion decisions for the inventory. They do not imply that the code is incorrect or lacks architectural structure.

7. Coverage status and remaining ambiguity

Application source inventory checked

Controllers, including administrator namespace	16
Models, including concern	14
Jobs	5
Mailboxes	2
Mailers	3
Helpers	4
Initializers	11
Stimulus directory JavaScript files, including 2 startup files	18

Source-file inventory and cross-reference coverage, not a claim that every static asset or every framework dependency was manually analyzed. ‹/›

The major application-owned code has now received either focused analysis or this final structural sweep. The remaining limitations are narrower:

First, two new candidates need skeptical review. Account Traveler Enrollment Coordination is the more plausible. Beta Invite Eligibility Evaluation is structurally defensible but weak and may be better rejected as elementary encapsulation.

Second, one routing discrepancy deserves separate review. `InvitesController` contains `index`, `create` and `destroy` methods referring to `settings_invites_path`, but the route file declares administrative invite resources under `namespace :admin`, not a corresponding Settings invite resource. This suggests that the non-admin invite-management actions or view may be leftover code. It is a routing-coverage observation, not pattern evidence. ‹/› ‹/›

Third, application execution and independent verification are missing. The uploaded archive contains no `test/` or `spec/` files, and the focused investigations did not execute the full Rails application. ‹/›

The correspondence between the archive and the stated Git commit also remains unverified.

Static asset bundles, framework internals unrelated to the previously identified contracts, and deployed external-service behavior have not been exhaustively audited. They are not obvious gaps in the application-owned pattern sweep, but they should remain explicit scope boundaries.

8. Is the repository ready for final adversarial review?

Yes — source coverage is sufficient

The repository is sufficiently covered to begin a final adversarial assessment. **It is not yet ready to freeze the inventory without that assessment.**

The review should challenge the eight established classifications, especially the breadth of the GRASP Controller counting rule, whether the Information Expert assignment is sufficiently distinctive and the eligibility of framework-mediated Observer instances.

It should also resolve the two newly identified candidates before finalizing the records.

Pre-freeze sweep result

Previously source-confirmed collaborations	8
New candidate — account traveler enrollment	UNCONFIRMED
New candidate — invite eligibility	UNCONFIRMED
New confirmed instances from sweep	0
Repository-wide application-source coverage	Sufficient for adversarial review
Final inventory freeze	Not yet

Final sweep record

Writing

DaPlanStan — Repository-Wide Pattern Discovery Sweep

Source: Original uploaded DaPlanStan ZIP
Purpose: Identify overlooked pattern collaborations before the final adversarial review.
Method: Application-source inventory, examination of previously less-reviewed classes and controllers, cross-reference searches and comparison against established pattern boundaries.

Coverage

Reviewed remaining administrator, invitation, account-settings, traveler, notification, model, helper, initializer and JavaScript responsibilities. Rechecked the previously investigated jobs, mailboxes, mailers and model relationships for independent collaborations.

New candidates

1. **GRASP Controller — Account Traveler Enrollment Coordination — UNCONFIRMED.** `TravelersController#create` coordinates account-level traveler enrollment, existing-user lookup, creation of linked or pending traveler records and invitation delivery. Its independence from Trip Membership Management Coordination requires adversarial verification.
2. **GRASP Information Expert — Beta Invite Eligibility Evaluation — UNCONFIRMED.** `Invite#usable?` and `#email_matches?` encapsulate eligibility checks using the invitation's own attributes. The strongest objection is that these are simple domain predicates rather than a substantial independent pattern collaboration.

Deduplication

The eight previously source-confirmed collaborations remain unchanged. Repeated use of associated models, shared invitation infrastructure, geocoding behavior, OAuth signup logic and Turbo notifications did not produce new instances.

Negative findings

No additional application-defined strategy family, adapter interface, abstract factory, custom Observer registration, recursive Composite collaboration or Template Method structure was established.

Ordinary administrative CRUD, notification queries, model callbacks, enum roles, UI helpers and

isolated browser events were excluded from new pattern proposals.

Outstanding limitations

The two new candidates require a final adversarial decision. Runtime tests and independent human verification have not been performed. ZIP-to-upstream-commit correspondence remains unverified.

Conclusion

Application-source coverage is sufficient to begin the final adversarial review. No new pattern was confirmed during the sweep. Do not freeze the inventory or generate final CSV files, counts or diagrams until the candidate decisions and reviewer validation are complete.

Overall finding: The final sweep did not expose a missing GoF mechanism or a new confirmed pattern. It identified two narrowly defined GRASP responsibilities worth challenging, while reinforcing the existing eight-instance working inventory and the importance of consistent counting boundaries.

We have finished the repository-wide discovery sweep. Do not search for additional patterns unless the review exposes a direct contradiction or duplicate.
Now perform the final adversarial review before the baseline inventory is frozen.
The current source-confirmed instances are:

1. GRASP Information Expert — Parsed Email to Trip Record Linking
2. GRASP Controller — Inbound Travel Email Intake Coordination
3. GRASP Controller — Trip Creation Coordination
4. GRASP Controller — Trip Membership Management Coordination
5. GRASP Controller — Bulk Itinerary Import Coordination
6. GRASP Controller — OAuth Sign-in and Identity Linking Coordination
7. Observer — Outbound Mail Delivery Logging
8. Observer — Trip-Scoped Turbo Refresh Notifications

Two additional candidates from the final sweep still need a decision:

9. GRASP Controller — Account Traveler Enrollment Coordination
10. GRASP Information Expert — Beta Invite Eligibility Evaluation

Treat every one of these as challengeable. Do not preserve a classification simply because an earlier analysis confirmed it.
For each of the ten:

1. State the defining rule or mechanism required by the claimed pattern.
2. Identify the exact application participants and their responsibilities.
3. Give the strongest source evidence supporting the classification.
4. Give the strongest argument for rejecting it.
5. Check whether the same behavior is better explained as ordinary Rails CRUD, framework convention, simple model encapsulation, direct API integration or procedural controller code.
6. Check whether any participant role was previously overstated or incorrectly named.
7. Check whether the source locations still support the complete collaboration rather than only one isolated method.
8. Decide:
   CONFIRMED
   REJECTED
   UNCERTAIN
9. If confirmed, retain a stable instance name.
10. If rejected, preserve the reason so it can appear in the rejected-candidate record.

Then perform a cross-instance review.

11. Check whether the five currently confirmed GRASP Controller instances are genuinely separate use-case responsibilities or whether any should be merged.
12. Apply one consistent counting rule to every GRASP Controller instance. Do not count one controller class, one HTTP action or one method as an instance merely because it exists.
13. Compare Account Traveler Enrollment Coordination directly with Trip Membership Management Coordination. Decide

whether the account-level and trip-level events are sufficiently independent to count separately.

14. Compare Beta Invite Eligibility Evaluation with Parsed Email to Trip Record Linking. Decide whether simple invite predicates are substantial enough to justify a separate Information Expert instance under the same significance threshold.
15. Check the two Observer instances for duplicate or

Worked for 1m 51s

DaPlanStan — Final Adversarial Pattern Review

Pre-freeze verification · Original uploaded source archive · 10 challenged classifications

1. Review standard

I re-examined the actual application methods, their callers, participant relationships and the relevant framework contracts. The eight earlier confirmations are treated as **claims to test**, not conclusions to preserve automatically.

I am applying these rules consistently:

- **GRASP Controller:** Count one cohesive system-event or use-case responsibility when an application object makes meaningful domain decisions and coordinates collaborators. Do not count merely per Rails class, HTTP action or method.
- **GRASP Information Expert:** The responsible object must possess or directly control information central to a substantive application responsibility. Trivial predicates alone are insufficient for a separately counted collaboration.
- **Observer:** Identify a publisher, observer or subscriber, registration/subscription and a defined notification mechanism. Framework implementation is acceptable when the application's participation is explicit and substantial.
- **Deduplication:** Repeated use of a collaborator, framework facility or domain model does not establish another instance.

A confirmed classification here means **the static source supports the pattern**. It does not mean the implementation is well-designed, runtime-tested or independently approved by a human reviewer.

2. Adversarial assessment of the ten candidates

1. GRASP Information Expert — Parsed Email to Trip Record Linking

Defining rule: Assign a substantive responsibility to the object with the information necessary to perform it.

Actual responsibility and participants: `InboxItem` holds the extracted email data and interprets it when a user confirms or relinks an email. `InboxItemsController` initiates the operation; `Trip`, `TripItem`, `Expense`, `Traveler` and `User` supply related domain records.

Strongest source evidence:

- `app/controllers/inbox_items_controller.rb:16–26` — controller calls the model's linking operations.
- `app/models/inbox_item.rb:19–25` — parsed-data storage interface.
- `app/models/inbox_item.rb:34–58` — confirmation and shortlist handling.
- `app/models/inbox_item.rb:67–187` — travel-record interpretation, matching and linking.
- `app/models/inbox_item.rb:191–207` — associated expense creation.

Strongest rejection argument: `InboxItem` accesses information from many other models and performs considerable orchestration. It could be considered an oversized Active Record model rather than an ideal Information Expert.

However, the central input being interpreted is its own `parsed_data`. The application deliberately places the associated conversion rules on that model rather than in the controller or Anthropic job. This is a substantive responsibility involving several related record types, not merely an accessor or validation predicate.

Role correction: `InboxItem` is the responsible Information Expert, not an exclusive owner of all data used. `TripItem` and `Expense` are domain collaborators, not separate Information Expert instances.

Decision: CONFIRMED

Retain: `Parsed Email to Trip Record Linking`

---

2. GRASP Controller — Inbound Travel Email Intake Coordination

Defining rule: A designated system-event handler receives an external event, makes relevant application decisions and coordinates subsequent responsibilities.

System event: An incoming email is delivered to the application's mailbox.

Actual participants: `ApplicationMailbox` routes the message; `TripMailbox#process` handles the event; `Trip`, `InboxItem`, `ParseInboxItemJob` and `TripMailer` participate in its processing.

Strongest source evidence:

- `app/mailboxes/application_mailbox.rb:1–3` — incoming mail routing.
- `app/mailboxes/trip_mailbox.rb:2–14` — recipient-to-trip resolution and sender eligibility.

- `app/mailboxes/trip_mailbox.rb:16–30` — inbox persistence and conditional delegation.
- `app/mailboxes/trip_mailbox.rb:55–60` — notification delegation.

Strongest rejection argument: Action Mailbox supplies the event lifecycle, and `process` is a conventional framework method.

But the application determines which trip receives the message, whether its sender is recognized, what to store and whether to initiate parsing or owner notification. These are meaningful event-handling decisions, not merely mailbox inheritance.

Role correction: Rails Action Mailbox is the inbound dispatch infrastructure. `TripMailbox#process` is the application-owned GRASP Controller. `ParseInboxItemJob` is a delegated processing collaborator, not a second instance of this Controller responsibility.

Decision: CONFIRMED

Retain: `Inbound Travel Email Intake Coordination`

---

3. GRASP Controller — Trip Creation Coordination

Defining rule: Coordinate the system event that establishes a new domain entity and its required associated state.

System event: A signed-in user requests creation of a trip.

Actual participants: `TripsController#create`, `Trip`, `Account#owner_traveler`, `TripMember` and `FetchTripCoverImageJob`.

Strongest source evidence:

- `config/routes.rb:21` — trip creation endpoint.
- `app/controllers/trips_controller.rb:35–48` — entire creation sequence.
- `app/models/trip.rb:14–19, 31–37` — validation and inbound-address generation.
- `app/models/account.rb:9–11` — owner-traveler lookup.
- `app/models/trip_member.rb:1–6` — role-bearing membership record.
- `app/jobs/fetch_trip_cover_image_job.rb:4–32` — delegated cover retrieval.

Strongest rejection argument: This is a short Rails `create` action containing ordinary Active Record calls. The controller directly performs the membership insertion, and the writes are not visibly enclosed in a single application transaction.

The decisive distinction is that the operation establishes both the trip and its initial ownership relationship, then delegates nonessential follow-up processing. It coordinates a specific multi-object

domain operation rather than solely persisting submitted form attributes.

Role correction: `TripsController#create` controls the use-case sequence. `Trip` does not itself create its owner membership; that responsibility is explicitly invoked by the controller.

Decision: CONFIRMED

Retain: `Trip Creation Coordination`

4. GRASP Controller — Trip Membership Management Coordination

Defining rule: Coordinate a coherent membership-management use case, including its authorization and changes to the related domain state.

System events: Add or remove a member of an existing trip.

Actual participants: `TripMembersController#create`, `#destroy`, `#resolve_and_add_by_email`, `#add_traveler`, `ApplicationController#current_member_for`, `Trip`, `Account`, `Traveler`, `User`, `TripMember` and `InvitationMailer`.

Strongest source evidence:

- `config/routes.rb:40` — create/destroy membership endpoints.
- `app/controllers/trip_members_controller.rb:6–22` — event handlers.
- Same file, `26–35` — trip lookup and role-based authorization.
- Same file, `37–65` — duplicate checking, traveler resolution, membership creation and conditional invitation.
- `app/models/trip_member.rb:1–6` — association and role representation.

Strongest rejection argument: The controller holds substantial membership policy and identity-resolution logic. The methods could be described as procedural CRUD over `Traveler` and `TripMember`.

Nonetheless, the controller handles more than an isolated record insertion. It coordinates account-scoped traveler identity, trip-specific role assignment, invitation handling and owner-removal restrictions. These form a coherent membership-management responsibility.

Role correction: `TripMember` represents membership and stores role values; it is not a Controller and does not implement interchangeable role behavior.

Decision: CONFIRMED

Retain: `Trip Membership Management Coordination`

Creation and removal belong to **one lifecycle-management instance**, not two instances merely because there are separate HTTP actions.

5. GRASP Controller — Bulk Itinerary Import Coordination

Defining rule: An application coordinator receives an import use case, controls its phases and arranges persistence and subsequent processing.

System event: A user supplies itinerary JSON, previews it and confirms import into an existing trip.

Actual participants: `TripItemsController#import`, `#bulk_create_items`, `#parse_import_time`, `Trip`, `TripItem`, `Geocodable` and `GeocodeTripItemsJob`.

Strongest source evidence:

- `config/routes.rb:30–35` — import entry points.
- `app/controllers/trip_items_controller.rb:31–58` — preview and confirmation branches.
- Same file, `128–174` — duplicate resolution, item creation and geocoding scheduling.
- Same file, `176–179` — time parsing.
- `app/models/trip_item.rb:1–28` — persisted domain object.
- `app/jobs/geocode_trip_items_job.rb:4–21` — delegated geocoding.

Strongest rejection argument: Almost all business rules are inside one controller helper. There is little application-level delegation of import policy, and callback modification occurs directly inside the controller. The implementation is clearly procedural.

This is the closest of the five original GRASP Controller candidates to being rejected as controller-heavy code.

Nevertheless, it still handles a distinct system-level import responsibility with two user-facing phases and delegates domain persistence and deferred processing. Under the same rule applied to the earlier Controllers, imperfect separation does not erase the responsibility assignment.

Role correction: `TripItemsController` is the coordinator; `TripItem` is the domain record. `GeocodeTripItemsJob` performs delegated work, but is already part of the separately examined geocoding behavior.

Decision: CONFIRMED

Retain: `Bulk Itinerary Import Coordination`

Preview and confirmed import constitute **one instance**, not separate instances.

6. GRASP Controller — OAuth Sign-in and Identity Linking Coordination

Defining rule: An application-level controller receives a completed authentication event and coordinates the corresponding local identity and access responsibilities.

System event: OmniAuth completes an external authentication attempt and forwards the callback to DaPlanStan.

Actual participants: `SessionsController#create`, `#provision_account_or_gate`, `#create_account_for`, `UserIdentity`, `User`, `Traveler#accept!`, `Account` and `Invite`.

Strongest source evidence:

- `config/routes.rb:12–14` — authentication callback routing.
- `app/controllers/sessions_controller.rb:4–24` — provider/UID identity lookup, email-based user resolution and identity creation.
- Same file, `26–40` — session establishment, pending-traveler acceptance and access decision.
- Same file, `54–81` — signup gate and account provisioning.
- `app/models/traveler.rb:26–28` — delegated acceptance operation.
- `app/models/invite.rb:12–18` — invitation eligibility rules.

Strongest rejection argument: OmniAuth performs the actual provider authentication. The remaining application code is concentrated in a procedural callback containing multiple database operations and branching decisions.

However, the classification is not for performing OAuth itself. The application coordinates how an authenticated external identity becomes a DaPlanStan user, gains access to existing invitations and potentially receives an account. These are separate application-defined responsibilities involving multiple domain objects.

Role correction: OmniAuth is the authentication infrastructure, not the application-defined GRASP Controller. The Controller responsibility begins when `SessionsController#create` receives the normalized authentication result.

Decision: CONFIRMED

Retain: `OAuth Sign-in and Identity Linking Coordination`

Google and Apple are configured providers within the same application-level coordination path. They are not separate GRASP Controller instances.

---

7. Observer — Outbound Mail Delivery Logging

Defining mechanism: An observer is registered with a publisher, and the publisher's notification infrastructure invokes the observer's defined callback when the relevant event occurs.

Actual participants:

Role	Actual implementation
Publisher/subject infrastructure	Mail gem's delivery notification system
Registration interface	`ActionMailer::Base.register_observer`
Concrete observer	Application-defined `MailLogger`
Notification operation	`MailLogger.delivered_email(message)`
Event-producing collaborators	`InvitationMailer`, `TripMailer`

Strongest source evidence:

`config/initializers/mail_logger.rb:1–7` contains both the callback definition and explicit registration:

```ruby
module MailLogger
  def self.delivered_email(message)
    Rails.logger.info "[Mailer] Sent '#{message.subject}' to #{Array(message.to).join(', ')}"
  end
end

ActionMailer::Base.register_observer(MailLogger)
```

This is supported by the actual framework contract. Action Mailer 8.1.3 delegates registration to `Mail.register_observer`, while Mail 2.9.0 maintains the registered observers and invokes `delivered_email(mail)` when informing them. RubyDoc +1

Strongest rejection argument: DaPlanStan implements neither the observer registry nor the notification algorithm. It only registers a custom logger through an existing Rails API.

This objection would defeat a claim that DaPlanStan independently implements the whole GoF Observer framework.

It does not defeat the narrower claim that **DaPlanStan participates in a concrete, framework-mediated Observer collaboration**. The application defines the actual observer, explicitly registers it and relies on the framework's notification contract.

Role correction: `TripMailer` and `InvitationMailer` are mail producers, not custom Observer subjects. The notification publisher is the Mail gem's delivery infrastructure, exposed through Action

Mailer.

Decision: CONFIRMED

Retain: `Outbound Mail Delivery Logging`

Count it once, not once per mailer or delivered email.

---

8. Observer — Trip-Scoped Turbo Refresh Notifications

Defining mechanism: A publisher sends notifications to subscribed receivers through an established notification channel, without invoking each receiver directly.

Actual participants:

Role	Actual implementation
Model publication source	`InboxItem`
Job publication source	`GeocodeTripItemsJob`
Publisher infrastructure	`Turbo::StreamsChannel` and Turbo broadcasting
Subscription declarations	Trip show view and inbox index view
Concrete subscriber mechanism	Turbo cable stream source in subscribed browser pages
Delivery infrastructure	Action Cable
Notification reaction	Turbo client refresh handling

Strongest application evidence:

- `app/models/inbox_item.rb:5` — `broadcasts_refreshes_to :trip`.
- `app/jobs/geocode_trip_items_job.rb:20-21` — `Turbo::StreamsChannel.broadcast_refresh_to(trip)`.
- `app/views/trips/show.html.erb:2` — `turbo_stream_from @trip`.
- `app/views/inbox_items/index.html.erb:1` — identical trip-scoped subscription.

The framework contract establishes the missing infrastructure:

- Turbo Rails 2.0.23 uses an `after_commit` callback for `broadcasts_refreshes_to`. GitHub
- `turbo_stream_from` creates a signed stream subscription declaration. GitHub
- `Turbo::StreamsChannel#subscribed` establishes the Action Cable subscription. GitHub
- `broadcast_refresh_to` publishes a Turbo Stream refresh action. GitHub

Strongest rejection argument: Almost the entire Observer mechanism is supplied by Turbo and Action Cable. DaPlanStan does not define a custom observer callback or its own notification registry.

This instance has **less application-owned observer behavior than Outbound Mail Delivery Logging**, where `MailLogger.delivered_email` is explicitly written by the application.

Nevertheless, the two application publication paths and two subscription declarations form a complete, intentional trip-scoped notification collaboration. The defining mechanism does not depend on imagining additional application-written classes.

Under the framework-mediated acceptance rule applied to the Mail Observer, this meets the pattern threshold. ‹/› ‹/›

Role correction: `InboxItem` and `GeocodeTripItemsJob` are publication sources, not independent ConcreteSubjects implementing a custom Observer API. The browser subscribers are instantiated through Turbo; the application supplies subscription declarations.

Decision: CONFIRMED

Retain: `Trip-Scoped Turbo Refresh Notifications`

Count one collaboration across the inbound-email and geocoding workflows.

---

9. GRASP Controller — Account Traveler Enrollment Coordination

Defining rule: A system-event handler coordinates a substantive account-level enrollment use case involving identity resolution, record creation and, where appropriate, invitation delivery.

System event: An account owner adds a traveler through Settings.

Actual participants: `TravelersController#create`, `#set_account`, `Account`, `Traveler`, `User` and `InvitationMailer#invite`.

Strongest source evidence:

- `config/routes.rb:48–50` — account Settings traveler-management routes.
- `app/views/settings/show.html.erb:114–127` — account-level enrollment form.
- `app/controllers/travelers_controller.rb:5–35` — enrollment branching.
- Same file, `54–56` — account-owner authorization.
- `app/models/account.rb:1–11` — account/traveler relationship.
- `app/models/traveler.rb:1–8` — persisted traveler relationship.
- `app/mailers/invitation_mailer.rb:2–6` — delegated invitation construction.

Actual application decisions: The controller normalizes supplied values, rejects invalid and duplicate requests, distinguishes registered users from email-only travelers, supports name-only managed travelers and optionally requests an invitation.

This is not a simple unconditional `Traveler.create!` operation.

Strongest rejection argument: These decisions are implemented directly in a conventional Rails controller. The enrollment algorithm is procedural, and some of its user-resolution behavior resembles the trip membership controller's logic.

Under a counting policy that rejects every controller-heavy operation, this would be rejected. But such a policy would also threaten several previously confirmed GRASP Controller cases, especially Trip Creation Coordination and Bulk Itinerary Import Coordination.

Applying the established rule consistently, the application does coordinate a meaningful account-enrollment event with multiple possible outcomes and delegated invitation delivery.

Role correction: `TravelersController#create` is the system-event coordinator. `Traveler` represents an account participant; it is not automatically a trip member. `InvitationMailer` constructs the invitation but does not decide whether enrollment is permitted.

Decision: CONFIRMED

New stable instance name: `Account Traveler Enrollment Coordination`

It passes the same source-level GRASP Controller threshold as the earlier use-case coordinators.

10. GRASP Information Expert — Beta Invite Eligibility Evaluation

Defining rule: A substantive responsibility should belong to the object possessing the information needed to carry it out.

Proposed expert: `Invite`.

Actual participants: `Invite#used?`, `#expired?`, `#usable?`, `#email_matches?`, `InvitesController#accept` and `SessionsController#provision_account_or_gate`.

Strongest source evidence:

`app/models/invite.rb:12–18` defines predicates using the invitation's own fields:

```ruby
```

```
  def used?    = used_at.present?
  def expired? = expires_at.present? && expires_at < Time.current
  def usable?  = !used? && !expired?

  def email_matches?(address)
    email.blank? || email.casecmp?(address.to_s)
  end
```

Both `InvitesController#accept`, lines 5–15, and `SessionsController#provision_account_or_gate`, lines 63–75, use these rules.

Strongest rejection argument: These are elementary predicates over the model's own attributes. Nearly every domain model with validation or status methods could qualify under such a permissive interpretation.

Unlike `InboxItem`, `Invite` does not interpret complex information or coordinate a substantial cross-object responsibility.

The full account-eligibility decision remains in `SessionsController`, which combines invitation validity, user identity and account-provisioning conditions.

This code is a reasonable example of keeping domain predicates near their data, but the evidence is insufficient for a separately counted Information Expert collaboration under the significance threshold used for `Parsed Email to Trip Record Linking`.

Role correction: `Invite` owns basic eligibility predicates; it is not the object responsible for the entire signup-authorization process.

Decision: REJECTED

Record the reason as: *Simple domain-model predicate encapsulation; insufficient independent collaboration for a separately counted GRASP Information Expert instance.*

This is a rejection of the **pattern-inventory classification**, not a criticism of the code's method placement.

---

3. Final candidate decision table

#	Candidate	Strongest objection	Origin	Final decision
1	Parsed Email to Trip Record Linking — Information Expert	Large model coordinating other domain objects	Mainly application	**CONFIRMED**

2	Inbound Travel Email Intake Coordination — Controller	Action Mailbox convention	Mixed	**CONFIRMED**
3	Trip Creation Coordination — Controller	Short Rails create action	Mixed	**CONFIRMED**
4	Trip Membership Management Coordination — Controller	Controller-heavy membership policy	Mixed	**CONFIRMED**
5	Bulk Itinerary Import Coordination — Controller	Procedural import algorithm in controller	Mixed	**CONFIRMED**
6	OAuth Sign-in and Identity Linking Coordination — Controller	Procedural callback after OmniAuth	Mixed	**CONFIRMED**
7	Outbound Mail Delivery Logging — Observer	Observer infrastructure belongs to Mail gem	Mixed, framework-mediated	**CONFIRMED**
8	Trip-Scoped Turbo Refresh Notifications — Observer	Subscriber and dispatch infrastructure belong to Turbo	Mainly framework-mediated	**CONFIRMED**
9	Account Traveler Enrollment Coordination — Controller	Resembles ordinary account CRUD and overlaps traveler resolution	Mixed	**CONFIRMED**
10	Beta Invite Eligibility Evaluation — Information Expert	Simple model predicates, not substantial independent responsibility	Mainly application	**REJECTED**

No candidate remains structurally **UNCERTAIN** under the applied rule. This does not eliminate human-review and runtime limitations.

4. Cross-instance review

A. Consistent GRASP Controller counting rule

One instance is counted for **one coherent, independently identifiable use-case responsibility** implemented through an application-level event coordinator and its relevant collaborators.

This means:

- Multiple HTTP actions can belong to one instance.
- Multiple methods of one class can belong to one instance.
- One Rails controller class could support different use cases, but class count does not determine instance count.
- Sharing models or framework services does not automatically merge independently triggered use cases.

- Simple CRUD actions without substantial application decisions do not qualify merely because they receive requests.

Applying that rule yields six defensible Controller instances.

Independent use case	Main event handler	Distinct business outcome
Inbound email intake	`TripMailbox#process`	Accept a message into a trip's inbox and choose its next processing path
Trip creation	`TripsController#create`	Create trip and initial owner membership
Trip membership management	`TripMembersController`	Add/remove participation in an existing trip
Bulk itinerary import	`TripItemsController#import`	Import a reviewed batch of itinerary ideas
OAuth identity coordination	`SessionsController#create`	Establish local authenticated identity, invitation links and account access
Account traveler enrollment	`TravelersController#create`	Enroll an account-level traveler and optionally invite them

These six are not being counted because six controller or mailbox classes exist. They represent six different externally initiated responsibilities.

B. Account enrollment versus trip membership management

These are sufficiently independent to count separately.

Account traveler enrollment creates or manages a `Traveler` within an `Account`. It supports name-only travelers and email-based invitations but does **not** create a `TripMember`.

Trip membership management adds or removes participation in one particular trip. It may resolve or create a traveler as a supporting step, but its defining outcome is the presence or removal of a role-bearing `TripMember`.

They overlap in identity resolution and invitation infrastructure. Those shared steps must not be counted as extra patterns, but they do not collapse two independent use-case responsibilities into one.

Decision: Retain both as distinct GRASP Controller instances.

C. Information Expert consistency

The difference between the two candidates is substantive responsibility.

`InboxItem` owns structured email data and implements the complex translation of that information into existing or new itinerary and expense records.

`Invite` owns a few fields and answers eligibility questions about them.

Although both put rules near relevant data, only the first represents a substantial independently identifiable responsibility under the inventory's counting threshold.

Decision: Retain `Parsed Email to Trip Record Linking`; reject `Beta Invite Eligibility Evaluation`.

D. Observer duplication and boundaries

The two confirmed Observers have separate subjects, events, infrastructure and reactions.

Dimension	Outbound Mail Delivery Logging	Trip-Scoped Turbo Refresh Notifications
Event	Mail delivery	Trip-scoped record changes or refresh requests
Framework	Action Mailer / Mail	Turbo Rails / Action Cable
Application participant	`MailLogger` concrete observer	Inbox/geocoding publishers and subscribed views
Registration	`register_observer(MailLogger)`	`turbo_stream_from @trip`
Reaction	Write application log entry	Process a browser refresh
Instance boundary	One delivery-logging collaboration	One trip-scoped refresh collaboration

These are **not duplicates**.

The earlier rejected `map:locate`, `map:reset` and `map:resize` collaboration also remains excluded. Those events are targeted DOM commands to a known map element, not another instance of trip-scoped Turbo notification.

Decision: Two distinct Observer instances.

5. Final participant-role mapping and stable names

This is the mapping to preserve for eventual inventory construction. GRASP patterns use responsibility roles, not invented GoF-style interfaces.

Pattern	Stable instance name	Main participant	Essential collaborators
GRASP Information Expert	Parsed Email to Trip Record Linking	`InboxItem#confirm_and_create_items!`, `#fill_missing_links!`	`InboxItemsController`, `Trip`, `TripItem`, `Expense`, `Traveler`

GRASP Controller	Inbound Travel Email Intake Coordination	`TripMailbox#process`	`ApplicationMailbox`, `Trip`, `InboxItem`, `ParseInboxItemJob`, `TripMailer`
GRASP Controller	Trip Creation Coordination	`TripsController#create`	`Trip`, `Account`, `TripMember`, `FetchTripCoverImageJob`
GRASP Controller	Trip Membership Management Coordination	`TripMembersController` membership operations	`ApplicationController`, `Trip`, `Account`, `Traveler`, `User`, `TripMember`, `InvitationMailer`
GRASP Controller	Bulk Itinerary Import Coordination	`TripItemsController#import`, `#bulk_create_items`	`Trip`, `TripItem`, `Geocodable`, `GeocodeTripItemsJob`
GRASP Controller	OAuth Sign-in and Identity Linking Coordination	`SessionsController#create`, account-provisioning helpers	`UserIdentity`, `User`, `Traveler`, `Account`, `Invite`
GRASP Controller	Account Traveler Enrollment Coordination	`TravelersController#create`	`Account`, `Traveler`, `User`, `InvitationMailer`
Observer	Outbound Mail Delivery Logging	`MailLogger.delivered_email`	Action Mailer registration, Mail notification infrastructure, mailer producers
Observer	Trip-Scoped Turbo Refresh Notifications	`InboxItem`, `GeocodeTripItemsJob`, subscribing views	Turbo Streams, Action Cable and Turbo browser subscriber

These participant descriptions are accurate at the responsibility level. The eventual `participant_roles` and `source_locations` CSV fields should retain the precise roles and file ranges, including the framework-origin distinctions.

6. Rejected-candidate record

The following should remain outside `patterns.csv` and confirmed pattern counts.

Rejected candidate	Reason
Geocoding — Template Method	No application-defined template algorithm delegating overridable primitive steps
Geocoding — Strategy	Sequential primary/fallback methods, not interchangeable algorithm implementations
Geocoding — Adapter	Direct use of the Geocoder API, without an established incompatible-interface translation

Itinerary Timeline–Map Event Coordination — Observer	Targeted DOM event communication to the known map component
Beta Invite Eligibility Evaluation — Information Expert	Simple eligibility predicates lack substantive independent collaboration

Ordinary Rails CRUD, callbacks, enums, associations and `create!` calls also remain insufficient as standalone pattern evidence.

7. Pre-freeze counting result

Adversarial-review outcome

9	3	1
Source-confirmed instances	Pattern types	Candidate rejected in final review

GRASP Controller	6
GRASP Information Expert	1
Observer	2
Total	**9**

These are source-supported classifications under the documented counting rule, not independently human-approved final submission counts.

The final adversarial review changes the working inventory from eight to nine instances: one new independent account-enrollment Controller instance is accepted, while the invite-predicate Information Expert candidate is rejected.

No previously confirmed instance required removal or merging under the consistently applied counting rule.

8. Remaining verification boundaries

The source-level pattern identification is sufficiently complete to close the discovery phase. Four limitations remain relevant before formal submission.

Source provenance: The original ZIP was inspected, but its equivalence to Git commit `da54d9ab8bfc118e99375fdd6c506efb20b66e5b` has not been verified.

Independent human verification: The project still requires a separate reviewer to validate the participant assignments, design-pattern threshold and nine proposed instances. The present analysis should not be labeled independent human approval.

Framework provenance: The Observer classifications rely on documented contracts for the versions declared in `Gemfile.lock`. They must be described as framework-mediated instances rather than fully application-implemented observer frameworks.

Runtime behavior: Successful OAuth callbacks, mail delivery notifications, Turbo refreshes, trip membership flows and imports have not been tested end to end. Runtime correctness should not be confused with static pattern identification.

Final adversarial-review conclusion

Writing

DaPlanStan — Final Adversarial Review Record

Source: Original uploaded DaPlanStan archive
Scope: Ten challenged pattern candidates
Method: Direct source inspection, responsibility-boundary comparison, duplicate checking and relevant framework-contract verification.

Final classification

- GRASP Controller: 6 confirmed instances.
- GRASP Information Expert: 1 confirmed instance.
- Observer: 2 confirmed instances.
- Total: 9 source-confirmed instances across 3 pattern types.

Decision changes

Account Traveler Enrollment Coordination is accepted as an independent GRASP Controller instance. Its system event creates account-level traveler participation, which is distinct from the creation of a role-bearing membership in an existing trip.

Beta Invite Eligibility Evaluation is rejected as a separate GRASP Information Expert instance because its elementary model predicates do not establish a substantive independent collaboration comparable to Parsed Email to Trip Record Linking.

Deduplication

The two Observer instances are independent collaborations using different notification infrastructure. The trip-scoped Turbo Observer is counted once across geocoding and inbound-email processing.

The six Controller instances are counted by cohesive system-event responsibility, not by controller class, method, provider or HTTP endpoint.

Previously rejected candidates

Geocoding Template Method, Strategy and Adapter remain rejected. Targeted timeline-to-map DOM event coordination remains rejected as Observer. Simple invite eligibility is added to the rejected-candidate record.

Verification boundary

The source-based classifications are complete enough for inventory preparation. Independent human sign-off, source-commit provenance and runtime behavior remain unverified.

Conclusion

The adversarial review supports a nine-instance baseline proposal. No additional pattern search is justified on current evidence. Preserve the accepted roles and rejected-candidate reasons, and obtain independent review before declaring the submission inventory formally frozen.

Final decision: The repository discovery and source-level adversarial assessment are complete. The nine-instance inventory is ready for **human verification and controlled baseline freezing**, not for an unsupported claim that every pattern has already been independently validated.

The source discovery and adversarial review are complete.
The current proposed baseline contains nine source-confirmed pattern instances:

1. GRASP Information Expert — Parsed Email to Trip Record Linking
2. GRASP Controller — Inbound Travel Email Intake Coordination
3. GRASP Controller — Trip Creation Coordination

4. GRASP Controller — Trip Membership Management Coordination
5. GRASP Controller — Bulk Itinerary Import Coordination
6. GRASP Controller — OAuth Sign-in and Identity Linking Coordination
7. GRASP Controller — Account Traveler Enrollment Coordination
8. Observer — Outbound Mail Delivery Logging
9. Observer — Trip-Scoped Turbo Refresh Notifications

Do not perform another pattern search.
I now need to independently verify these before the baseline inventory is frozen.
Prepare a human-verification checklist for these nine instances.
For each instance give me:

1. Pattern name and stable instance name.
2. The exact responsibility or mechanism I am supposed to verify.
3. The minimum source files and line ranges I should personally inspect.
4. The actual participants and their claimed roles.
5. What I should see in the source if the classification is correct.
6. The strongest reason I might reject or downgrade it during human review.
7. Any framework-provided participant that must not be misrepresented as application-written.
8. A blank human decision field:
   CONFIRMED / REJECTED / UNCERTAIN
9. A blank reviewer-notes field.

Keep the checklist practical. I should be able to open the listed files, inspect the evidence and make my own decision without rereading every previous LLM output.
After the nine instances, give me a short rejected-candidate checklist containing:

1. Geocoding Template Method
2. Geocoding Strategy
3. Geocoding Adapter
4. Itinerary Timeline–Map Event Coordination as Observer

5. Beta Invite Eligibility Evaluation as Information Expert

For these, give the main reason for rejection and the minimum evidence I should inspect if I want to challenge that rejection. Also include the counting rule I need to verify:

1. Controller instances are counted by cohesive system-event/use-case responsibility, not by class, method or HTTP action.
2. The same collaboration appearing in several workflows is counted once.
3. Multiple providers, messages, broadcasts or subscribers do not automatically create multiple instances.
4. Framework-mediated patterns must clearly separate application participants from framework participants.

Do not mark anything as human verified on my behalf.
Do not generate `patterns.csv`, `pattern_counts.csv` or UML yet.
The output should be a verification worksheet that I can personally review and fill in.

Worked for 1m 2s

DaPlanStan — Human Pattern Verification Worksheet

Awaiting human review 9 proposed instances · 5 rejected candidates · Original source ZIP

This worksheet is designed for you to open the specified files, inspect the actual collaboration and make your own decision. The file ranges have been checked against the uploaded source archive.

No human decisions have been marked. The classifications below are claims to verify, not conclusions you must accept.

Writing

DaPlanStan — Independent Human Verification Worksheet

Review details

Reviewer: _____

Review date: _____

Source: `daplanstan-main.zip`

Claimed baseline commit: `da54d9ab8bfc118e99375fdd6c506efb20b66e5b`

ZIP-to-commit correspondence: Not yet independently established.

Instructions: For each candidate, inspect the source paths, identify the actual participants, test the strongest objection and select one decision. Do not confirm a pattern solely because the previous LLM review proposed it.

HV-01 — Parsed Email to Trip Record Linking

Proposed pattern: GRASP Information Expert

Stable instance name: Parsed Email to Trip Record Linking

Responsibility to verify

`InboxItem` is assigned responsibility for interpreting its stored parsed email information and creating or linking corresponding itinerary and expense records.

Minimum source inspection

- `app/models/inbox_item.rb:19–25` — stored parsed-data interface.
- `app/models/inbox_item.rb:34–58` — confirmation and shortlist creation.
- `app/models/inbox_item.rb:67–187` — matching and record linking.
- `app/models/inbox_item.rb:191–207` — expense construction.
- `app/controllers/inbox_items_controller.rb:16–35` — actual callers.
- `app/models/trip_item.rb:1–28` and `app/models/expense.rb:1–28` — related records.

Participants and claimed roles

Participant	Claimed role
`InboxItem`	Information Expert responsible for interpreting stored parsed data
`InboxItemsController`	Client requesting confirmation or relinking
`Trip`	Parent trip and association context

`TripItem`	Itinerary record created or linked
`Expense`	Expense record conditionally created
`Traveler`	Source of payer identity through trip membership

What I should find

- `InboxItem` stores and retrieves the data being interpreted.
- The controller delegates confirmation and relinking to `InboxItem`.
- `fill_missing_links!` matches existing items or creates missing records.
- The model conditionally creates expenses and processes shortlist ideas.
- These are substantive application rules, not just Active Record associations.

Strongest objection

`InboxItem` uses data from many models and contains considerable procedural orchestration. Its responsibilities may be too broad to justify describing it as a clear Information Expert.

Framework boundary: Active Record supplies relationships and persistence. It does not implement the application-specific email-to-trip mapping rules. Anthropic performs earlier extraction, not this record-linking operation.

My decision: ☐ CONFIRMED ☐ REJECTED ☐ UNCERTAIN

Reviewer notes / reason:

_____

_____

HV-02 — Inbound Travel Email Intake Coordination

Proposed pattern: GRASP Controller

Stable instance name: Inbound Travel Email Intake Coordination

Responsibility to verify

`TripMailbox#process` handles the incoming-email system event by identifying the trip,

evaluating sender approval, creating an inbox record and choosing the next processing action.

Minimum source inspection

- `app/mailboxes/application_mailbox.rb:1–3`
- `app/mailboxes/trip_mailbox.rb:2–31`
- `app/mailboxes/trip_mailbox.rb:55–60`
- `app/models/trip.rb:4–19`
- `app/models/inbox_item.rb:1–16`
- `app/jobs/parse_inbox_item_job.rb:99–112`
- `app/mailers/trip_mailer.rb:2–10`

Participants and claimed roles

Participant	Claimed role
`TripMailbox#process`	System-event controller
`ApplicationMailbox`	Inbound mailbox routing
`Trip`	Destination identified from recipient address
`InboxItem`	Persisted incoming email
`ParseInboxItemJob`	Delegated AI parsing
`TripMailer`	Delegated pending-sender notification

What I should find

- Incoming messages are routed to `TripMailbox`.
- `process` matches the recipient address to `Trip`.
- It checks sender eligibility using application-defined conditions.
- It creates an `InboxItem`.
- Approved messages trigger parsing; unapproved messages trigger owner notification.
- The mailbox coordinates these operations instead of carrying out every responsibility itself.

Strongest objection

`process` is the normal Action Mailbox callback. This may be ordinary framework-driven message handling rather than a meaningful GRASP Controller responsibility.

Framework boundary: Action Mailbox supplies reception and routing infrastructure. DaPlanStan supplies trip matching, approval decisions, inbox persistence and conditional delegation. Active Job and Action Mailer supply execution and delivery infrastructure.

My decision: ☐ CONFIRMED ☐ REJECTED ☐ UNCERTAIN

Reviewer notes / reason:

_____

_____

HV-03 — Trip Creation Coordination

Proposed pattern: GRASP Controller

Stable instance name: Trip Creation Coordination

Responsibility to verify

`TripsController#create` coordinates the creation of a new trip, its first owner membership and follow-up cover-image retrieval.

Minimum source inspection

- `app/controllers/trips_controller.rb:35–48`
- `app/models/trip.rb:4–19, 31–37`
- `app/models/account.rb:1–11`
- `app/models/trip_member.rb:1–6`
- `app/jobs/fetch_trip_cover_image_job.rb:4–32`
- `config/routes.rb:21–29`

Participants and claimed roles

Participant	Claimed role
`TripsController#create`	System-event controller
`Trip`	New trip with validation and inbound-address generation
`Account#owner_traveler`	Finds the account owner's traveler
`TripMember`	Stores initial owner membership
`FetchTripCoverImageJob`	Performs delegated image retrieval

What I should find

- The controller creates a `Trip` using submitted trip details.
- It assigns the current user and account.
- It saves the trip before creating its initial owner membership.
- It explicitly assigns `role: :owner`.
- It delegates cover-image retrieval to a separate job.
- These operations collectively establish a usable new trip.

Strongest objection

This may simply be a short conventional Rails `create` action. The controller performs membership creation directly, and the operation is not visibly wrapped in one application-level transaction.

Framework boundary: Active Record supplies validation, callbacks, associations and persistence. Active Job supplies job execution. DaPlanStan chooses the sequence and owner-membership rule; Pexels supplies external image search.

My decision: ☐ CONFIRMED ☐ REJECTED ☐ UNCERTAIN

Reviewer notes / reason:

_____

_____

HV-04 — Trip Membership Management Coordination

Proposed pattern: GRASP Controller

Stable instance name: Trip Membership Management Coordination

Responsibility to verify

`TripMembersController` coordinates adding and removing participants from an existing trip, including authorization, traveler resolution, role assignment and invitations.

Minimum source inspection

- `app/controllers/trip_members_controller.rb:1–65`
- `app/controllers/application_controller.rb:22–25`
- `app/models/trip_member.rb:1–6`
- `app/models/traveler.rb:1–8`
- `app/models/account.rb:1–11`
- `app/mailers/invitation_mailer.rb:8–13`
- `config/routes.rb:40`

Participants and claimed roles

Participant	Claimed role
`TripMembersController#create`, `#destroy`	Receive membership-management events
`TripMembersController#resolve_and_add_by_email`	Resolve identity and possible invitation
`TripMembersController#add_traveler`	Check duplicates and create membership
`ApplicationController#current_member_for`	Support permission checks
`Account`, `Traveler`, `User`	Account and participant identity records
`TripMember`	Connect traveler to trip and store membership role
`InvitationMailer`	Construct invitation email

What I should find

- An owner or planner membership is checked before managing travelers.
- The controller can find existing account travelers or resolve by email.
- It creates a `Traveler` when needed.
- Adding to the trip explicitly creates a `TripMember` with `role: :contributor`.
- Removing a member has explicit protection against removing the trip owner.
- Adding and removing are related operations within one membership-management responsibility.

Strongest objection

Most membership rules are implemented directly in the controller. This might be controller-heavy CRUD rather than a meaningful GRASP responsibility assignment.

Framework boundary: Rails handles associations, enums, persistence and callbacks. DaPlanStan implements permission checks, identity-resolution decisions and membership rules.

The `TripMember` enum is not Strategy or State.

My decision: ☐ CONFIRMED ☐ REJECTED ☐ UNCERTAIN

Reviewer notes / reason:

_____

_____

HV-05 — Bulk Itinerary Import Coordination

Proposed pattern: GRASP Controller

Stable instance name: Bulk Itinerary Import Coordination

Responsibility to verify

`TripItemsController#import` and `#bulk_create_items` coordinate the two-stage itinerary import operation: preview, confirmation, duplicate handling, item creation and deferred geocoding.

Minimum source inspection

- `app/controllers/trip_items_controller.rb:31–58`
- `app/controllers/trip_items_controller.rb:128–179`
- `app/views/trip_items/import.html.erb:93–106`
- `app/models/trip_item.rb:1–37`
- `app/models/concerns/geocodable.rb:4–16`
- `app/jobs/geocode_trip_items_job.rb:4–21`
- `config/routes.rb:30–34`

Participants and claimed roles

Participant	Claimed role
`TripItemsController#import`	Receives import request and coordinates preview/confirmation
`TripItemsController#bulk_create_items`	Coordinates batch persistence decisions
`TripItemsController#parse_import_time`	Parses supplied date/time values

`Trip`	Target trip
`TripItem`	Imported itinerary record
`Geocodable`	Existing coordinate-status/geocoding behavior
`GeocodeTripItemsJob`	Deferred geocoding collaborator

What I should find

- One import endpoint handles preview and confirmed submission.
- Preview parses JSON and checks its structure.
- Confirmation calls `bulk_create_items`.
- The batch method checks missing names, duplicates and supported kinds.
- It creates missing itinerary ideas and conditionally fills existing fields.
- It schedules geocoding for relevant newly created records.
- Preview and confirmation form one coherent user goal, not separate pattern instances.

Strongest objection

The controller contains most of the import algorithm. Its procedural implementation may offer limited evidence of meaningful delegation beyond Active Record and a background job.

Framework boundary: Active Record supplies creation, queries and callback APIs. Active Job schedules geocoding. DaPlanStan supplies import rules and sequencing. The geocoding algorithm itself is an existing collaboration, not a new instance here.

My decision: ☐ CONFIRMED ☐ REJECTED ☐ UNCERTAIN

Reviewer notes / reason:

_____

_____

HV-06 — OAuth Sign-in and Identity Linking Coordination

Proposed pattern: GRASP Controller

Stable instance name: OAuth Sign-in and Identity Linking Coordination

Responsibility to verify

After an OAuth callback, `SessionsController#create` coordinates local identity recognition, user linking, session establishment, pending traveler acceptance and account eligibility.

Minimum source inspection

- `config/initializers/omniauth.rb:1–16`
- `config/routes.rb:12–14`
- `app/controllers/sessions_controller.rb:4–40, 54–81`
- `app/models/user_identity.rb:1–8`
- `app/models/user.rb:11–22`
- `app/models/traveler.rb:18–28`
- `app/models/invite.rb:12–18`
- `app/models/account.rb:1–11`

Participants and claimed roles

Participant	Claimed role
`SessionsController#create`	Post-authentication system-event controller
`#provision_account_or_gate`	Coordinates account eligibility
`#create_account_for`	Creates account and initial traveler
`UserIdentity`	Persisted external provider identity
`User`	Local authenticated identity
`Traveler#accept!`	Links invited traveler profile to user
`Account`	User-owned account
`Invite`	Provides invitation eligibility predicates

What I should find

- The callback reads `request.env["omniauth.auth"]`.
- It first searches `UserIdentity` by provider and UID.
- If absent, it finds or initializes a `User` by email.
- It creates the corresponding provider identity record.
- It assigns `session[:user_id]`.
- It links pending travelers with the signed-in user.
- It either allows existing account/membership access or applies the signup/invitation gate.
- The same application workflow handles the configured providers.

Strongest objection

OmniAuth has already performed authentication. The remaining method may be interpreted as conventional procedural post-authentication bookkeeping rather than a distinct GRASP Controller responsibility.

Framework boundary: OmniAuth and provider integrations perform OAuth protocol work. Rails supplies sessions and persistence. DaPlanStan implements the local identity and account-access decisions. Google and Apple are not separate application-written strategies.

My decision: ☐ CONFIRMED ☐ REJECTED ☐ UNCERTAIN

Reviewer notes / reason:

_____

_____

HV-07 — Account Traveler Enrollment Coordination

Proposed pattern: GRASP Controller

Stable instance name: Account Traveler Enrollment Coordination

Responsibility to verify

`TravelersController#create` coordinates enrollment of an account-level traveler, including registered-user lookup, name-only traveler creation and conditional invitation delivery.

Minimum source inspection

- `app/controllers/travelers_controller.rb:5–35, 54–56`
- `app/views/settings/show.html.erb:114–127`
- `app/models/account.rb:1–11`
- `app/models/traveler.rb:1–8, 18–28`
- `app/models/user.rb:11–17`
- `app/mailers/invitation_mailer.rb:2–6`
- `config/routes.rb:48–50`

For comparison, also inspect:

- `app/controllers/trip_members_controller.rb:6–15, 37–65`

Participants and claimed roles

Participant	Claimed role
`TravelersController#create`	Account-enrollment system-event controller
`TravelersController#set_account`	Owner-account access check
`Account`	Scope of traveler enrollment
`Traveler`	New or existing account participant
`User`	Registered identity lookup
`InvitationMailer#invite`	Constructs account invitation

What I should find

- The request originates from account Settings, not trip membership management.
- The controller handles email-based or name-only enrollment.
- It checks duplicates and whether the email belongs to the acting user.
- It creates a linked traveler for an existing registered user or an email-only traveler when needed.
- It sends an invitation for the email-based path.
- It does not create a `TripMember` or assign a trip role.

Strongest objection

This may be ordinary account-level CRUD with some invitation logic. It resembles traveler resolution already performed by `TripMembersController`.

Critical independence test: Can I clearly explain the separate domain outcome? Account enrollment creates an account participant, whereas trip membership management establishes participation in a specific trip.

Framework boundary: Rails provides model persistence and controller infrastructure. DaPlanStan defines enrollment conditions and invitation decisions; Action Mailer supplies delivery infrastructure.

My decision: ☐ CONFIRMED ☐ REJECTED ☐ UNCERTAIN

Reviewer notes / reason:

_____

_____

HV-08 — Outbound Mail Delivery Logging

Proposed pattern: GoF Observer — framework-mediated

Stable instance name: Outbound Mail Delivery Logging

Mechanism to verify

An application-defined observer registers with the mail-delivery notification system. The framework invokes its `delivered_email` callback after a qualifying delivery operation.

Minimum source inspection

- `config/initializers/mail_logger.rb:1–7`
- `app/mailers/application_mailer.rb:1–5`
- `app/mailers/invitation_mailer.rb:1–13`
- `app/mailers/trip_mailer.rb:1–10`
- `Gemfile.lock:19, 185`

Also consult the Action Mailer 8.1.3 and Mail 2.9.0 observer contracts to verify `register_observer` and `delivered_email`.

Participants and claimed roles

Participant	Claimed role
`MailLogger`	Application-defined concrete observer
`MailLogger.delivered_email(message)`	Notification callback
`ActionMailer::Base.register_observer(MailLogger)`	Observer registration
Mail gem notification system	Publisher, registry and dispatch mechanism
`InvitationMailer`, `TripMailer`	Mail-producing collaborators

What I should find

- The application defines `MailLogger.delivered_email`.
- The initializer explicitly registers `MailLogger`.
- The documented mail framework contract calls `delivered_email` on registered observers.
- Individual mailer actions do not directly invoke the logger.
- The event relationship is independent of the specific invitation or notification email.

Strongest objection

The actual Observer infrastructure is implemented by Rails and the Mail gem. DaPlanStan has supplied only the concrete callback and its registration.

Framework boundary: Do not claim DaPlanStan wrote the publisher, observer registry or notification-dispatch mechanism. Verify that application-supplied concrete observer registration is acceptable under the assignment's framework-idiom counting rule.

My decision: ☐ CONFIRMED ☐ REJECTED ☐ UNCERTAIN

Reviewer notes / reason:

_____

_____

HV-09 — Trip-Scoped Turbo Refresh Notifications

Proposed pattern: GoF Observer — framework-mediated

Stable instance name: Trip-Scoped Turbo Refresh Notifications

Mechanism to verify

Application-defined publication sources request trip-scoped notifications. Browser pages subscribe to the corresponding trip stream. Turbo and Action Cable deliver refresh actions to subscribed clients.

Minimum source inspection

- `app/models/inbox_item.rb:1–5`
- `app/jobs/geocode_trip_items_job.rb:4–21`
- `app/views/trips/show.html.erb:2–9`
- `app/views/inbox_items/index.html.erb:1`
- `Gemfile.lock:424`

Also inspect Turbo Rails 2.0.23 implementation or documentation for `broadcasts_refreshes_to`, `turbo_stream_from`, `Turbo::StreamsChannel#subscribed` and `broadcast_refresh_to`.

Participants and claimed roles

Participant	Claimed role
`InboxItem`	Model publication source
`GeocodeTripItemsJob`	Explicit broadcast source
`turbo_stream_from @trip`	Application subscription declarations
`Turbo::StreamsChannel`	Framework stream publisher/channel
Action Cable	Subscription and delivery infrastructure
Turbo browser runtime	Framework-provided subscriber and refresh reaction

What I should find

- `InboxItem` declares `broadcasts_refreshes_to :trip`.
- `GeocodeTripItemsJob` calls `broadcast_refresh_to(trip)`.
- Both the trip show and inbox index pages subscribe through `turbo_stream_from @trip`.
- The Turbo Rails contract creates matching stream subscriptions and refresh broadcasts.
- The browser receives a framework-defined refresh action rather than a direct call from the model or job.
- These are usages of one trip-scoped notification mechanism.

Strongest objection

Unlike `MailLogger`, DaPlanStan does not implement a concrete observer callback. Most of the subscription and reaction mechanism is supplied by Turbo.

Critical comparison: Does the application's explicit choice of publication sources, trip-scoped stream and subscribed views establish sufficient participation for a framework-mediated Observer instance under the same rule used for HV-08?

Framework boundary: Do not attribute Action Cable subscription management, Turbo Stream processing or browser refresh implementation to DaPlanStan.

My decision: ☐ CONFIRMED ☐ REJECTED ☐ UNCERTAIN

Reviewer notes / reason:

_____

_____

Part B — Rejected-Candidate Verification

These candidates were previously rejected. Their status is not automatically binding on your human review.

Candidate	Existing rejection rationale	Minimum evidence to inspect
Geocoding Template Method	`TripItem` uses a guard and `super`, but does not override primitive steps called by a shared template algorithm.	`app/models/concerns/geocodable.rb:4–16, 27–65`; `app/models/trip_item.rb:32–37`
Geocoding Strategy	`geocode_with_bias` and `geocode_fallback` execute as fixed sequential methods within one concern. There are no interchangeable strategy implementations selected through a common algorithm role.	`app/models/concerns/geocodable.rb:13–14, 27–65`
Geocoding Adapter	The concern invokes `Geocoder.search` directly and persists coordinates. No incompatible application target interface and adaptee translation were established.	`app/models/concerns/geocodable.rb:13–22, 27–65`; `config/initializers/geocoder.rb:1–7`
Itinerary Timeline–Map Event Coordination as Observer	Producers target the known map DOM element and send map-specific requests. The stronger explanation is targeted DOM event communication.	`app/javascript/controllers/timeline_controller.js:11–33`; `trip_nav_controller.js:31–43`; `map_controller.js:51–109`
Beta Invite Eligibility Evaluation as Information Expert	`Invite` owns simple validity and email predicates, but these are not a substantial independent information-processing collaboration under the proposed counting threshold.	`app/models/invite.rb:12–18`; `app/controllers/invites_controller.rb:5–15`; `app/controllers/sessions_controller.rb:63–75`

For any rejection you wish to overturn, write down the **specific missing mechanism or

responsibility** that you can now demonstrate from the source. A resemblance to a textbook definition alone should not be enough.

Rejection-review notes:

_____

_____

Part C — Baseline Counting Verification

Complete these checks only after reviewing all nine instances.

- [ ] **Use-case responsibility rule:** GRASP Controller instances have been counted by cohesive system events or use cases, not by controller classes, individual methods or HTTP actions.

- [ ] **Deduplication rule:** The same collaboration appearing in multiple workflows has been counted once. In particular, email-derived record linking is not counted again under itinerary or expense creation.

- [ ] **Multiple participants rule:** Multiple authentication providers, emails, broadcasts, job executions or subscribed pages do not automatically create additional instances.

- [ ] **Framework ownership rule:** Framework-mediated Observer instances clearly distinguish DaPlanStan's participation from Rails, Action Mailer, Mail, Turbo and Action Cable implementations.

Required cross-instance comparisons

- [ ] Account Traveler Enrollment and Trip Membership Management have genuinely independent domain outcomes.
- [ ] Bulk import preview and confirmation are one cohesive import responsibility.
- [ ] Trip creation and trip membership management are not being counted twice for the initial owner membership.
- [ ] OAuth coordination does not double-count account-traveler creation or invitation acceptance as separate instances within the same use case.
- [ ] Outbound mail logging and trip-scoped Turbo refresh are genuinely separate notification relationships.

- [ ] The geocoding and inbound-email uses of Turbo have not generated duplicate Observer instances.
- [ ] Simple model predicates have not been counted inconsistently with the Information Expert significance threshold.

Proposed inventory for comparison — not yet human approved

Pattern type	Proposed instances	Human-approved count
GRASP Controller	6	_____
GRASP Information Expert	1	_____
Observer	2	_____
Total instances	**9**	**_____**
Distinct pattern types	**3**	**_____**

Final reviewer assessment

All nine decisions recorded: ☐ Yes ☐ No

All rejected candidates checked where necessary: ☐ Yes ☐ No

Duplicate instances resolved: ☐ Yes ☐ No

Participant-role mapping checked: ☐ Yes ☐ No

Source line ranges checked: ☐ Yes ☐ No

Framework-owned behavior distinguished: ☐ Yes ☐ No

Source archive or commit provenance established: ☐ Yes ☐ No

Final overall recommendation:

☐ Accept baseline as proposed

☐ Accept with revisions

☐ Do not freeze — unresolved classifications remain

Remaining issues / changes required:

_____

_____

_____

Reviewer signature or initials: _____

Date: _____

Scope reminder: Completing this worksheet supports independent human verification of the proposed baseline. It does not require generating or modifying `patterns.csv`, `pattern_counts.csv` or UML diagrams. Those activities should follow the completed human decisions.

The most important judgment calls are **HV-05 (bulk import), HV-07 (account traveler enrollment), and HV-09 (Turbo Observer)** because their distinction from ordinary procedural code or framework behavior is comparatively subtle. They deserve particular scrutiny.

Once you have made the nine decisions, the verified results—not the earlier LLM classifications—should determine the final inventory.

The pattern-discovery phase and final source-based verification are complete.
Do not search for any new patterns.
Freeze the baseline source-supported inventory using these nine instances:

1. GRASP Information Expert — Parsed Email to Trip Record Linking
2. GRASP Controller — Inbound Travel Email Intake Coordination
3. GRASP Controller — Trip Creation Coordination
4. GRASP Controller — Trip Membership Management Coordination
5. GRASP Controller — Bulk Itinerary Import Coordination
6. GRASP Controller — OAuth Sign-in and Identity Linking Coordination

7. GRASP Controller — Account Traveler Enrollment Coordination
8. Observer — Outbound Mail Delivery Logging
9. Observer — Trip-Scoped Turbo Refresh Notifications

Keep these rejected classifications outside the confirmed inventory:

1. Geocoding Template Method
2. Geocoding Strategy
3. Geocoding Adapter
4. Itinerary Timeline–Map Event Coordination as Observer
5. Beta Invite Eligibility Evaluation as Information Expert

For each of the nine confirmed instances:

1. Give the final stable instance name.
2. Give the exact pattern name.
3. Give the pattern type/category.
4. Give a concise description of the pattern responsibility or mechanism.
5. Map every participant role to an actual class, module, method, view or framework participant.
6. Give the exact source file locations and reliable line ranges.
7. Clearly distinguish application-owned participants from Rails or other framework participants.
8. Give concise intent-and-evidence wording suitable for the eventual `patterns.csv`.
9. Check that no participant or role is invented.
10. Check that no collaboration is counted twice.

Before freezing, recheck any suspicious source ranges from earlier worksheets, especially the `ParseInboxItemJob` line range associated with inbound email processing. Use the actual source snapshot rather than copying an inconsistent earlier line reference. Apply these counting rules consistently:

1. One GRASP Controller instance means one cohesive system-event/use-case responsibility, not one controller class, HTTP action or method.
2. Repeated appearances of the same collaboration do not create new instances.

3. Multiple providers, mailers, messages, broadcasts, jobs or subscribed pages do not automatically create additional instances.
4. Framework-mediated Observer patterns must distinguish application-defined roles from framework-supplied infrastructure.
5. Bulk import preview and confirmation remain one instance.
6. Trip membership add/remove remain one instance.
7. Google and Apple authentication remain one OAuth coordination instance.
8. Trip-Scoped Turbo Refresh Notifications remains one Observer instance across inbox processing and geocoding.
9. Outbound Mail Delivery Logging remains one Observer instance across all mailers and messages.

At the end provide:

1. The frozen nine-instance inventory.
2. Final participant-role mappings.
3. Final source locations.
4. Final rejected-candidate list with rejection reasons.
5. Final counting rule.
6. Pattern counts:
   GRASP Controller = 6
   GRASP Information Expert = 1
   Observer = 2
   Total pattern instances = 9
   Total distinct pattern types = 3
7. State clearly that this is the frozen source-supported baseline proposal.

Do not search for additional patterns.
Do not generate CSV files, UML diagrams or evolution analysis yet.

Worked for 1m 32s

DaPlanStan — Frozen Baseline Pattern Inventory

Source-supported baseline frozen 9 instances · 3 pattern types · 8 October 2026

The baseline is frozen at the **nine specified pattern instances**, using the original uploaded `daplanstan-main.zip` as the source snapshot.

I rechecked the relevant application files and line ranges against the archive. No additional pattern discovery was performed. The five rejected classifications remain outside the inventory.

Baseline status: This is the frozen *source-supported baseline proposal*, not a claim of independent human verification or runtime testing.

1. Frozen nine-instance inventory

ID	Exact pattern name	Pattern category	Stable instance name
P01	Information Expert (GRASP)	GRASP — Responsibility Assignment	Parsed Email to Trip Record Linking
P02	Controller (GRASP)	GRASP — Responsibility Assignment	Inbound Travel Email Intake Coordination
P03	Controller (GRASP)	GRASP — Responsibility Assignment	Trip Creation Coordination
P04	Controller (GRASP)	GRASP — Responsibility Assignment	Trip Membership Management Coordination
P05	Controller (GRASP)	GRASP — Responsibility Assignment	Bulk Itinerary Import Coordination
P06	Controller (GRASP)	GRASP — Responsibility Assignment	OAuth Sign-in and Identity Linking Coordination
P07	Controller (GRASP)	GRASP — Responsibility Assignment	Account Traveler Enrollment Coordination
P08	Observer (GoF)	GoF — Behavioral, framework-mediated	Outbound Mail Delivery Logging
P09	Observer (GoF)	GoF — Behavioral, framework-mediated	Trip-Scoped Turbo Refresh Notifications

The P01–P09 identifiers are inventory reference IDs, not additional application classes or source symbols.

Source-range corrections confirmed

`ParseInboxItemJob` **was explicitly rechecked.** Its actual location in the archive is `app/jobs/parse_inbox_item_job.rb`, with these ranges:

Implementation	Verified lines
System prompt	`6–18`

Anthropic tool schema	`20–97`
`ParseInboxItemJob#perform`	`99–112`
`#build_prompt`	`116–125`
`#parse_with_anthropic`	`127–139`

`TripMailbox#process` enqueues this job at `app/mailboxes/trip_mailbox.rb:26–27`. The job is a delegated collaborator in P02, not the inbound-mail event controller and not a separate pattern instance.

All source locations below are relative to the `daplanstan-main/` directory inside the uploaded ZIP.

---

2. Final participant-role mappings and source evidence

P01 — Parsed Email to Trip Record Linking

GRASP Information Expert · Application-owned responsibility

Pattern responsibility: `InboxItem` owns the extracted email data and is assigned the substantive responsibility of interpreting that data to find, update or create corresponding trip itinerary records and expenses.

Actual participant	Claimed role	Verified source location
`InboxItem#parsed_data`, `#parsed_data=`	Provides the stored information central to the responsibility	`app/models/inbox_item.rb:19–25`
`InboxItem#confirm_and_create_items!`	Information Expert operation: links records and adds shortlist ideas	`app/models/inbox_item.rb:34–58`
`InboxItem#fill_missing_links!`	Main information interpretation, matching and linking operation	`app/models/inbox_item.rb:67–187`
`InboxItem#build_expense`	Creates qualifying linked expense records	`app/models/inbox_item.rb:191–207`
`InboxItemsController#update`	Client initiating confirmation or relinking	`app/controllers/inbox_items_controller.rb:16–35`
`Trip`	Parent domain context and source of associated records	`app/models/trip.rb:4–12`
`TripItem`	Created or matched itinerary domain object	`app/models/trip_item.rb:1–28`

`Expense`	Conditionally created expense domain object	`app/models/expense.rb:1–28`
`TripMember` , `Traveler`	Supply trip membership and payer identification	`app/models/inbox_item.rb:71–74` ; `app/models/trip_member.rb:1–6` ; `app/models/traveler.rb:1–8`
`User`	Reviewing/adding user passed to model operations	`app/controllers/inbox_items_controller.rb:18–26`

Framework responsibility: Rails Active Record supplies associations, querying and persistence. It does not perform DaPlanStan's parsed-email interpretation rules.

Intent and evidence wording:

> Assign interpretation of extracted travel-email data to `InboxItem` , which stores that data and uses it to match or create trip itinerary records and conditional expenses. The inbox controller delegates confirmation and relinking to its model methods.

Verified boundary: This is one Information Expert instance. The creation of a `TripItem` and an `Expense` does not independently establish additional Creator, Factory or Information Expert instances.

P02 — Inbound Travel Email Intake Coordination

GRASP Controller · Application-owned coordination, framework-mediated entry

Pattern responsibility: Receive the incoming-email system event and coordinate trip identification, sender eligibility, inbox creation and delegated parsing or notification.

Actual participant	Claimed role	Verified source location
`ApplicationMailbox`	Defines application inbound-email routing	`app/mailboxes/application_mailbox.rb:1–3`
`TripMailbox#process`	GRASP system-event controller	`app/mailboxes/trip_mailbox.rb:2–31`
`TripMailbox#extract_text_body` , `#extract_html_body`	Application email-content extraction helpers	`app/mailboxes/trip_mailbox.rb:35–53`
`TripMailbox#notify_owners_of_pending_sender`	Coordinates notification of trip owners	`app/mailboxes/trip_mailbox.rb:55–60`
`Trip`	Destination identified through `inbound_email`	`app/mailboxes/trip_mailbox.rb:3–9` ; `app/models/trip.rb:4–19`

`ApprovedSender`	Stored sender-approval information queried through the trip	`app/mailboxes/trip_mailbox.rb:11–14` ; `app/models/approved_sender.rb:1–6`
`TripMember` , `Traveler` , `User`	Supply membership and registered-user sender checks	`app/mailboxes/trip_mailbox.rb:11–14`
`InboxItem`	Created incoming-email record	`app/mailboxes/trip_mailbox.rb:16–24` ; `app/models/inbox_item.rb:1–17`
`ParseInboxItemJob#perform`	Delegated parsing task	`app/mailboxes/trip_mailbox.rb:26–27` ; `app/jobs/parse_inbox_item_job.rb:99–112`
`TripMailer#pending_sender_notification`	Constructs owner notification	`app/mailboxes/trip_mailbox.rb:55–59` ; `app/mailers/trip_mailer.rb:2–10`

Framework/provider responsibility: Action Mailbox supplies routing and mailbox execution. The Mail gem provides email message representations. Active Job supplies background execution, and Anthropic supplies the AI extraction service invoked later by `ParseInboxItemJob#parse_with_anthropic` , lines 127–139.

Intent and evidence wording:

> `TripMailbox#process` coordinates the inbound-email event by matching the recipient to a trip, checking sender eligibility, persisting an `InboxItem` and delegating approved emails to parsing or unapproved emails to owner notification.

Verified boundary: The `TripMailbox` coordination is P02. The later interpretation and linking of parsed data belongs to P01, not P02.

P03 — Trip Creation Coordination

GRASP Controller · Application-owned use-case coordination

Pattern responsibility: Handle the trip-creation system event, establishing the new trip and its initial owner membership before requesting optional cover-image processing.

Actual participant	Claimed role	Verified source location
`TripsController#create`	GRASP system-event controller	`app/controllers/trips_controller.rb:35–48`

`ApplicationController#current_user`, `#current_account`	Supply authenticated user and account context	`app/controllers/application_controller.rb:14–20`
`Trip`	New trip domain object, associations and validation	`app/models/trip.rb:4–19`
`Trip#generate_inbound_email`, `#build_inbound_email`	Generate initial trip-specific inbound email address	`app/models/trip.rb:31–37`
`Account#owner_traveler`	Find account owner's traveler profile	`app/models/account.rb:9–11`
`Traveler`	Actual traveler selected for owner membership	`app/models/traveler.rb:1–8`
`TripMember`	Initial role-bearing owner membership	`app/controllers/trips_controller.rb:41–42`; `app/models/trip_member.rb:1–6`
`FetchTripCoverImageJob#perform`	Delegated image-retrieval work	`app/controllers/trips_controller.rb:43`; `app/jobs/fetch_trip_cover_image_job.rb:4–32`
Trip creation route	Receives trip-creation request	`config/routes.rb:21`

Framework/provider responsibility: Rails supplies request dispatch, associations, validation and persistence. Active Job performs scheduled work; Pexels supplies external image results.

Intent and evidence wording:

> `TripsController#create` coordinates the trip-creation use case by saving a trip, selecting the account owner's traveler, establishing an owner-role `TripMember` and scheduling cover-image retrieval.

Verified boundary: Cover-image retrieval is a delegated operation, not another GRASP Controller instance merely because the job is invoked during creation.

P04 — Trip Membership Management Coordination

GRASP Controller · Application-owned membership coordination

Pattern responsibility: Handle the lifecycle of membership in an existing trip, including member addition/removal, identity resolution, role assignment, permission checks and invitations.

Actual participant	Claimed role	Verified source location

`TripMembersController#create`	Receives trip-member addition event	`app/controllers/trip_members_controller.rb:6–15`
`TripMembersController#destroy`	Receives trip-member removal event	Same file, `17–22`
`TripMembersController#set_trip`	Resolves the accessible trip	Same file, `26–28`
`TripMembersController#require_owner_or_planner`	Enforces owner/planner membership-management rule	Same file, `30–35`
`TripMembersController#add_traveler`	Checks duplicates and creates the membership	Same file, `37–44`
`TripMembersController#resolve_and_add_by_email`	Resolves or creates a traveler and conditionally requests an invitation	Same file, `46–65`
`ApplicationController#current_member_for`	Supplies acting user's trip membership	`app/controllers/application_controller.rb:22–25`
`Trip`, `Account`	Provide trip and account-scoped association context	`app/models/trip.rb:4–12`; `app/models/account.rb:1–11`
`Traveler`, `User`	Represent participant and registered-user identities	`app/models/traveler.rb:1–8`; `app/models/user.rb:11–17`
`TripMember`	Persists trip membership and role	`app/models/trip_member.rb:1–6`
`InvitationMailer#invite_to_trip`	Constructs a trip invitation when requested	`app/mailers/invitation_mailer.rb:8–13`
Membership routes	Define member addition/removal entry points	`config/routes.rb:40`

Framework responsibility: Rails supplies persistence, associations, enums and controller callbacks. Action Mailer and Active Job supply mail construction/delivery execution.

Intent and evidence wording:

> `TripMembersController` coordinates membership changes in an existing trip by authorizing the acting user, resolving account travelers, preventing duplicate membership, assigning contributor roles, protecting owner memberships from direct removal and requesting invitations when needed.

Verified boundary: Member addition and removal are one cohesive membership-management instance. The account-level creation of a `Traveler` is a supporting operation here and does not automatically replace P07.

P05 — Bulk Itinerary Import Coordination

GRASP Controller · Application-owned import coordination

Pattern responsibility: Coordinate a two-stage user import: preview externally supplied itinerary JSON, then create or update itinerary ideas after confirmation and schedule deferred geocoding.

Actual participant	Claimed role	Verified source location
`TripItemsController#import`	Receives import event and coordinates preview/confirmation	`app/controllers/trip_items_controller.rb:31–58`
`TripItemsController#bulk_create_items`	Coordinates batch matching, updates, creation and geocoding scheduling	Same file, `128–174`
`TripItemsController#parse_import_time`	Parses imported date/time values	Same file, `176–179`
`Trip`	Existing trip receiving imported itinerary records	`app/models/trip.rb:4–12`
`TripItem`	Existing or newly created itinerary domain record	`app/models/trip_item.rb:1–28`
`Geocodable#geocoded?`	Supplies coordinate-completeness check	`app/models/concerns/geocodable.rb:9–11`
`GeocodeTripItemsJob#perform`	Performs later geocoding of selected item IDs	`app/jobs/geocode_trip_items_job.rb:4–21`
Import confirmation form	Carries reviewed JSON into confirmed import	`app/views/trip_items/import.html.erb:93–106`
Import routes	Support the GET and POST import operations	`config/routes.rb:30–34`

Framework responsibility: Active Record supplies querying, persistence and callback APIs. Active Job supplies job execution. DaPlanStan itself implements import preview, duplicate matching, input mapping and geocoding scheduling.

Intent and evidence wording:

> `TripItemsController#import` coordinates a user-confirmed batch import. Its `bulk_create_items` operation checks existing names, updates selected fields, creates new itinerary ideas and delegates missing-coordinate processing to `GeocodeTripItemsJob`.

Verified boundary: The preview and confirmed-import stages constitute one instance. The use of `Geocodable` and `GeocodeTripItemsJob` is not separately counted as another instance of this pattern. ‹/›

</>

P06 — OAuth Sign-in and Identity Linking Coordination

GRASP Controller · Application-owned post-authentication decisions

Pattern responsibility: Receive the completed OAuth authentication event and coordinate local identity recognition, account association, session establishment, traveler acceptance and signup eligibility.

Actual participant	Claimed role	Verified source location
`SessionsController#create`	GRASP system-event controller	`app/controllers/sessions_controller.rb:4–41`
`SessionsController#provision_account_or_gate`	Handles new-account eligibility decisions	Same file, `54–76`
`SessionsController#create_account_for`	Creates user-owned account and initial traveler	Same file, `78–81`
`UserIdentity`	Stores provider/UID relationship to user	`app/models/user_identity.rb:1–8`
`User`	Local authenticated identity and its associations	`app/models/user.rb:11–22`
`Traveler#accept!`	Associates pending traveler record with signed-in user	`app/models/traveler.rb:26–28`
`Account`	User-owned account created when eligible	`app/models/account.rb:1–11`
`Invite#usable?`, `#email_matches?`	Provide invite-eligibility checks	`app/models/invite.rb:12–18`
`ApplicationController#current_user`	Resolves established session identity	`app/controllers/application_controller.rb:14–16`
OAuth routes	Route successful callback and authentication failure	`config/routes.rb:12–14`
OmniAuth configuration	Registers provider integrations	`config/initializers/omniauth.rb:1–16`

Framework/provider responsibility: OmniAuth and its Google/Apple integrations perform external authentication and provide `request.env["omniauth.auth"]`. Rails supplies sessions and persistence. DaPlanStan owns the local user-resolution and account-access workflow.

Intent and evidence wording:

> `SessionsController#create` coordinates the post-OAuth sign-in event by resolving an existing provider identity or linking a user by email, establishing the local session, associating pending traveler invitations and provisioning or gating account access.

Verified boundary: Google and Apple use one application-level callback responsibility. They do not create separate instances of GRASP Controller.

P07 — Account Traveler Enrollment Coordination

GRASP Controller · Application-owned account enrollment

Pattern responsibility: Coordinate enrollment of a traveler at the account level, independently of membership in any particular trip.

Actual participant	Claimed role	Verified source location
`TravelersController#create`	Receives and coordinates account traveler enrollment	`app/controllers/travelers_controller.rb:5–35`
`TravelersController#set_account`	Restricts operation to account owner	Same file, `54–56`
`Account`	Account-scoped traveler association	`app/models/account.rb:1–11`
`Traveler`	New managed, email-only or user-linked account participant	`app/models/traveler.rb:1–8, 18–28`
`User`	Registered identity lookup	`app/models/user.rb:11–22`
`InvitationMailer#invite`	Constructs account-level invitation	`app/mailers/invitation_mailer.rb:2–6`
Settings enrollment form	Initiates account-level traveler addition	`app/views/settings/show.html.erb:114–127`
Settings traveler routes	Define account-level traveler operations	`config/routes.rb:48–50`

Framework responsibility: Rails supplies request routing, associations and persistence. Action Mailer/Active Job supply mail-delivery machinery. DaPlanStan implements enrollment decisions.

Intent and evidence wording:

> `TravelersController#create` coordinates account-level traveler enrollment by checking submitted identity information, detecting existing participants, creating user-linked, email-only or

> name-only traveler records and requesting invitations where applicable.

Verified boundary: P07 creates or manages a `Traveler` within an `Account`. P04 establishes participation in a specific trip through `TripMember`. These have different domain outcomes and remain separate instances, despite overlapping identity-resolution and mailer collaborators.

P08 — Outbound Mail Delivery Logging

GoF Observer · Framework-mediated implementation

Pattern mechanism: Register an application-defined concrete observer with the mail-delivery notification infrastructure, which invokes its callback when the corresponding delivery notification is issued.

Actual participant	Claimed Observer role	Source
`MailLogger`	Application-defined ConcreteObserver	`config/initializers/mail_logger.rb:1–5`
`MailLogger.delivered_email(message)`	Observer notification callback	Same file, `2–4`
`ActionMailer::Base.register_observer(MailLogger)`	Application registration call through framework API	Same file, `7`
`ActionMailer::Base.register_observer`	Framework registration interface	Action Mailer 8.1.3, `actionmailer/lib/action_mailer/base.rb`
Mail gem observer registry and notification mechanism	Framework-supplied publisher and observer dispatch	Mail 2.9.0
`InvitationMailer#invite`, `#invite_to_trip`	Application mail-producing collaborators	`app/mailers/invitation_mailer.rb:2–13`
`TripMailer#pending_sender_notification`	Application mail-producing collaborator	`app/mailers/trip_mailer.rb:2–10`

Framework contract: Action Mailer's `register_observer` delegates to `Mail.register_observer`; its observer contract defines delivery notification. RubyDoc

Origin separation:

- **DaPlanStan:** `MailLogger`, its logging callback and the registration statement.
- **Rails/Mail gem:** Observer registration infrastructure, notification delivery and framework mail lifecycle.
- **Postmark:** External email transport when configured, not an Observer class implemented by DaPlanStan.

Intent and evidence wording:

> DaPlanStan defines `MailLogger.delivered_email` and registers `MailLogger` through `ActionMailer::Base.register_observer`. The Mail framework notifies this concrete observer during the delivery lifecycle, allowing centralized logging without individual mailers directly invoking the logger.

Verified boundary: One Observer instance across all mailer actions and messages. The mailers are event-producing collaborators, not custom Observer subjects. ‹/›

P09 — Trip-Scoped Turbo Refresh Notifications

GoF Observer · Framework-mediated publication/subscription

Pattern mechanism: Establish trip-scoped publication and subscription so application changes can trigger refresh notifications to subscribed browser pages without directly invoking those pages.

Actual participant	Claimed Observer role	Source
`InboxItem`	Application model publication source	`app/models/inbox_item.rb:1–5`
`broadcasts_refreshes_to :trip`	Application publication declaration	`app/models/inbox_item.rb:5`
`GeocodeTripItemsJob#perform`	Application job publication source	`app/jobs/geocode_trip_items_job.rb:4–21`
`Turbo::StreamsChannel.broadcast_refresh_to(trip)`	Application call invoking framework publication	`app/jobs/geocode_trip_items_job.rb:21`
`turbo_stream_from @trip` in trip page	Application subscriber declaration	`app/views/trips/show.html.erb:2`
`turbo_stream_from @trip` in inbox page	Second application subscriber declaration	`app/views/inbox_items/index.html.erb:1`
`Turbo::Broadcastable`	Framework implementation of model-triggered broadcasts	Turbo Rails 2.0.23, `app/models/concerns/turbo/broadcastable.rb`
`Turbo::StreamsChannel#subscribed`	Framework subscription registration and stream identification	Turbo Rails 2.0.23, `app/channels/turbo/streams_channel.rb`
`Turbo::Streams::Broadcasts`	Framework publication implementation	Turbo Rails 2.0.23, `app/channels/turbo/streams/broadcasts.rb`

Action Cable	Framework stream-delivery infrastructure	Rails Action Cable
Turbo browser stream subscriber and refresh processing	Framework-provided concrete subscriber behavior	Turbo browser runtime
Trip-page refresh configuration	Application-selected morph refresh and scroll preservation	`app/views/trips/show.html.erb:5–9`

The versioned framework contract establishes that `broadcasts_refreshes_to` registers a model callback for refresh broadcasts, `Turbo::StreamsChannel#subscribed` subscribes to the verified stream, and `broadcast_refresh_to` constructs a refresh message delivered through the stream infrastructure. GitHub +3

Origin separation:

- **DaPlanStan:** Selects the trip-scoped publication source, declares subscribing views and invokes an explicit broadcast.
- **Turbo Rails/Action Cable:** Supplies stream naming, subscription registration, publication, delivery and client refresh mechanisms.
- **Browser subscriber:** Exists through Turbo's rendered stream-source and client implementation, not through a custom DaPlanStan Observer class.

Intent and evidence wording:

> `InboxItem` changes and `GeocodeTripItemsJob` completion publish refresh requests to a trip-scoped Turbo stream. The trip show and inbox index views subscribe to that stream through `turbo_stream_from @trip`, allowing Turbo and Action Cable to notify subscribed pages.

Verified boundary: This is one Observer collaboration shared by the geocoding and inbound-email workflows. Individual model changes, broadcasts, subscribing views and browser clients do not generate additional pattern instances. </> </>

---

3. Final rejected-candidate inventory

These classifications are excluded from the frozen confirmed baseline. Their source locations are preserved so the rejection remains auditable.

Rejected classification	Relevant source	Final rejection reason

Geocoding — Template Method	`app/models/concerns/geocodable.rb:4–16, 27–65`; `app/models/trip_item.rb:32–37`	The `TripItem` override guards the whole geocoding operation and calls `super`. It does not provide overridden primitive steps invoked by an application-defined template algorithm.
Geocoding — Strategy	`app/models/concerns/geocodable.rb:13–14, 27–65`	`geocode_with_bias` and `geocode_fallback` are fixed sequential lookup methods, not interchangeable algorithm implementations selected through a common Strategy role.
Geocoding — Adapter	`app/models/concerns/geocodable.rb:13–22, 27–65`; `config/initializers/geocoder.rb:1–7`	The application directly invokes Geocoder's API. No distinct incompatible target interface and application-owned adaptee translation are established.
Itinerary Timeline–Map Event Coordination — Observer	`app/javascript/controllers/timeline_controller.js:11–33`; `trip_nav_controller.js:31–43`; `map_controller.js:51–109`	The publishers dispatch map-specific events to a known map element. Targeted DOM event communication is the stronger explanation than a separately counted Observer collaboration.
Beta Invite Eligibility Evaluation — Information Expert	`app/models/invite.rb:12–18`; `app/controllers/invites_controller.rb:5–15`; `app/controllers/sessions_controller.rb:63–75`	The model contains simple predicates over its own attributes. This does not satisfy the significance threshold for a separate substantive Information Expert collaboration.

The rejected classifications contribute **zero instances** to the frozen baseline. Their existence should remain documented separately from the confirmed inventory.

---

4. Final counting and deduplication rules

The following rules are now fixed for this baseline and should also govern eventual CSV generation and any participant counts.

1. **GRASP Controller:** One instance represents one cohesive system-event or use-case responsibility, not an individual controller class, HTTP action or method.
2. **Repeated collaboration:** The same responsibility appearing across different files or workflows is counted once.
3. **Multiple runtime occurrences:** Multiple OAuth providers, mailers, emails, job executions, broadcasts or subscribed pages do not automatically create independent instances.
4. **Framework mediation:** An Observer may use framework-supplied machinery, but the role mapping must explicitly distinguish application-owned participants from Rails, Mail, Turbo and Action Cable participants.

5. **Bulk import:** Preview and confirmed import remain one `Bulk Itinerary Import Coordination` instance.
6. **Trip membership:** Addition and removal remain one `Trip Membership Management Coordination` instance.
7. **OAuth:** Google and Apple remain one `OAuth Sign-in and Identity Linking Coordination` instance.
8. **Turbo:** Inbox-derived and geocoding-derived refreshes remain one `Trip-Scoped Turbo Refresh Notifications` instance.
9. **Mail:** All delivery notifications through the registered `MailLogger` remain one `Outbound Mail Delivery Logging` instance.

Participant counting, when later required, should use actual source-defined participant identities and the agreed counting convention. Repeated references to one class within the same instance do not create additional class identities. Framework and application classes must not be conflated.

5. Final frozen pattern counts

Baseline source-supported counts

9	3
Total pattern instances	Distinct pattern types

GRASP Controller	6
GRASP Information Expert	1
Observer	2
Total pattern instances	**9**
Distinct pattern types	**3**

These counts are fixed for this source-supported baseline proposal. They should not be changed because a pattern appears again during subsequent documentation, diagramming or evolution analysis.

---

6. Baseline freeze declaration

- **Nine stable pattern instances retained.** The exact names and categories are preserved as specified.

- **Five rejected classifications excluded.** Their reasons and minimum supporting locations are recorded.
- **Source evidence audited.** Participant roles and line ranges are tied to the uploaded application snapshot, with the `ParseInboxItemJob` location specifically rechecked.
- **Duplicate prevention applied.** One collaboration is counted once even when it spans controllers, models, jobs, views or multiple workflows.

The archive's equivalence to the stated Git commit `da54d9ab8bfc118e99375fdd6c506efb20b66e5b` remains unverified because the supplied ZIP does not establish its Git provenance. Independent human sign-off and runtime behavior are also not established by this source audit. These are provenance and validation limitations, not additional pattern classifications.

Final status: FROZEN SOURCE-SUPPORTED BASELINE PROPOSAL — 9 instances, 3 pattern types.

No `patterns.csv`, `pattern_counts.csv`, UML diagram or evolution analysis has been generated.

The baseline source-supported pattern inventory is now frozen. Do not search for new patterns and do not change any of the nine frozen classifications.
Now prepare the exact contents of `patterns.csv` and `pattern_counts.csv`.
Use `baseline` as the `source_version` value for this original source snapshot. Do not use the stated Git commit as verified provenance because the ZIP-to-commit correspondence is still unverified.
For `patterns.csv`, use exactly these columns:
`source_version,pattern_name,pattern_type,instance_name,participant_roles,source_locations,intent_and_evidence`
Create exactly one row for each of these nine frozen instances:

1. Information Expert (GRASP) — Parsed Email to Trip Record Linking
2. Controller (GRASP) — Inbound Travel Email Intake Coordination
3. Controller (GRASP) — Trip Creation Coordination
4. Controller (GRASP) — Trip Membership Management Coordination
5. Controller (GRASP) — Bulk Itinerary Import Coordination
6. Controller (GRASP) — OAuth Sign-in and Identity Linking Coordination

7. Controller (GRASP) — Account Traveler Enrollment Coordination
8. Observer (GoF) — Outbound Mail Delivery Logging
9. Observer (GoF) — Trip-Scoped Turbo Refresh Notifications

For every `patterns.csv` row:

1. Use only participant roles already verified in the frozen baseline.
2. Include all important participants needed to understand the collaboration.
3. Do not invent GoF-style participant names for GRASP patterns.
4. Keep application participants and framework participants clearly distinguished.
5. Use exact source paths and reliable line ranges from the frozen inventory.
6. Do not count repeated methods or files as separate pattern instances.
7. Keep `intent_and_evidence` concise but specific enough to explain why the pattern classification is supported.
8. Make each field CSV-safe. Quote fields containing commas, semicolons or long participant mappings where necessary.
9. Do not include rejected candidates in `patterns.csv`.

For `pattern_counts.csv`, use exactly these columns:
`source_version,summary,pattern_name,count`
Use only these allowed `summary` values:
`per_pattern`
`total_pattern_types`
`total_pattern_instances`
Generate the following counts:
Controller (GRASP) = 6
Information Expert (GRASP) = 1
Observer (GoF) = 2
Total distinct pattern types = 3
Total pattern instances = 9
For the `per_pattern` rows, put the exact pattern name in `pattern_name`.

For `total_pattern_types` and `total_pattern_instances`, leave `pattern_name` blank.
Before giving the final CSV content, perform a consistency check:

1. `patterns.csv` must contain exactly 9 data rows.
2. Every frozen instance must occur exactly once.
3. `pattern_counts.csv` must agree with the nine inventory rows.
4. No rejected or uncertain candidate may appear.
5. The two Observer instances must remain separate.
6. The Turbo Observer must appear only once despite multiple publishers and subscribers.
7. Trip membership add/remove must remain one instance.
8. Bulk import preview/confirmation must remain one instance.
9. Google and Apple must remain one OAuth Controller instance.

Then output only:

1. Final `patterns.csv` in a CSV code block.
2. Final `pattern_counts.csv` in a second CSV code block.
3. A short validation summary stating row count, pattern-type count and total-instance count.

Do not generate UML, report prose, evolution analysis or any new pattern classifications yet.

1. Final `patterns.csv`

</> csv

```csv
source_version,pattern_name,pattern_type,instance_name,participant_roles,source_locations,intent_and_evide
baseline,Information Expert (GRASP),GRASP - Responsibility Assignment,Parsed Email to Trip Record Linking,
baseline,Controller (GRASP),GRASP - Responsibility Assignment,Inbound Travel Email Intake Coordination,"App
baseline,Controller (GRASP),GRASP - Responsibility Assignment,Trip Creation Coordination,"Application: Trip
baseline,Controller (GRASP),GRASP - Responsibility Assignment,Trip Membership Management Coordination,"App
baseline,Controller (GRASP),GRASP - Responsibility Assignment,Bulk Itinerary Import Coordination,"Applicat
baseline,Controller (GRASP),GRASP - Responsibility Assignment,OAuth Sign-in and Identity Linking Coordinat
baseline,Controller (GRASP),GRASP - Responsibility Assignment,Account Traveler Enrollment Coordination,"App
baseline,Observer (GoF),GoF - Behavioral (framework-mediated),Outbound Mail Delivery Logging,"Application:
baseline,Observer (GoF),GoF - Behavioral (framework-mediated),Trip-Scoped Turbo Refresh Notifications,"App
```

2. Final `pattern_counts.csv`

```csv
source_version,summary,pattern_name,count
baseline,per_pattern,Controller (GRASP),6
baseline,per_pattern,Information Expert (GRASP),1
baseline,per_pattern,Observer (GoF),2
baseline,total_pattern_types,,3
baseline,total_pattern_instances,,9
```

3. Validation summary

✓ 9 inventory rows ✓ 3 pattern types ✓ 9 total instances

All nine frozen instances occur exactly once. Counts agree: Controller (GRASP) = 6, Information Expert (GRASP) = 1, Observer (GoF) = 2. No rejected or uncertain classification is included. Membership, bulk import, OAuth, mail logging and trip-scoped Turbo notifications each remain single instances under the frozen counting rules.

The frozen baseline and CSV contents are complete.
Do not search for new patterns and do not change the nine confirmed instances.
Now prepare editable UML diagrams for the nine frozen pattern instances.
Use Mermaid `classDiagram` or another appropriate Mermaid UML notation so the source can be preserved and later exported to SVG.
Create one diagram for each instance:

1. Parsed Email to Trip Record Linking — Information Expert
2. Inbound Travel Email Intake Coordination — Controller
3. Trip Creation Coordination — Controller
4. Trip Membership Management Coordination — Controller
5. Bulk Itinerary Import Coordination — Controller
6. OAuth Sign-in and Identity Linking Coordination — Controller
7. Account Traveler Enrollment Coordination — Controller
8. Outbound Mail Delivery Logging — Observer
9. Trip-Scoped Turbo Refresh Notifications — Observer

For every diagram:

1. Use only participants already verified in the frozen baseline and `patterns.csv`.
2. Do not invent interfaces, abstract classes or methods just to make the diagram resemble a textbook pattern.
3. Show only the important participants needed to understand the collaboration.
4. Label relationships with the actual responsibility or interaction where useful.
5. For GRASP Controller diagrams, show the system-event coordinator and its important collaborators. Do not invent GoF-style Controller interfaces.
6. For the Information Expert diagram, make it clear that `InboxItem` owns the parsed data and performs the record-linking responsibility while collaborating with `TripItem`, `Expense`, `Trip` and the requesting controller.
7. For Outbound Mail Delivery Logging, clearly distinguish:
   DaPlanStan `MailLogger`
   Action Mailer registration
   Mail framework notification infrastructure
   mail-producing collaborators.
8. For Trip-Scoped Turbo Refresh Notifications, clearly distinguish:
   application publication sources
   trip-stream subscription declarations
   Turbo Streams
   Action Cable
   browser subscriber/refresh behavior.
9. Do not represent framework participants as application-written classes.
10. Do not add rejected candidates such as geocoding Strategy, Template Method, Adapter or timeline-map Observer.

For each diagram give:

1. A short diagram title.
2. Mermaid source.
3. A short explanation of what the diagram proves about the identified pattern.
4. The corresponding frozen pattern ID P01–P09.

Keep the diagrams readable rather than showing every method in every class.
At the end provide a short checklist confirming:

1. 9 diagrams produced.
2. All frozen instances represented once.
3. No rejected candidates included.
4. No invented participants.
5. Application and framework roles distinguished.

Do not perform evolution analysis yet.

DaPlanStan — Editable UML Diagrams

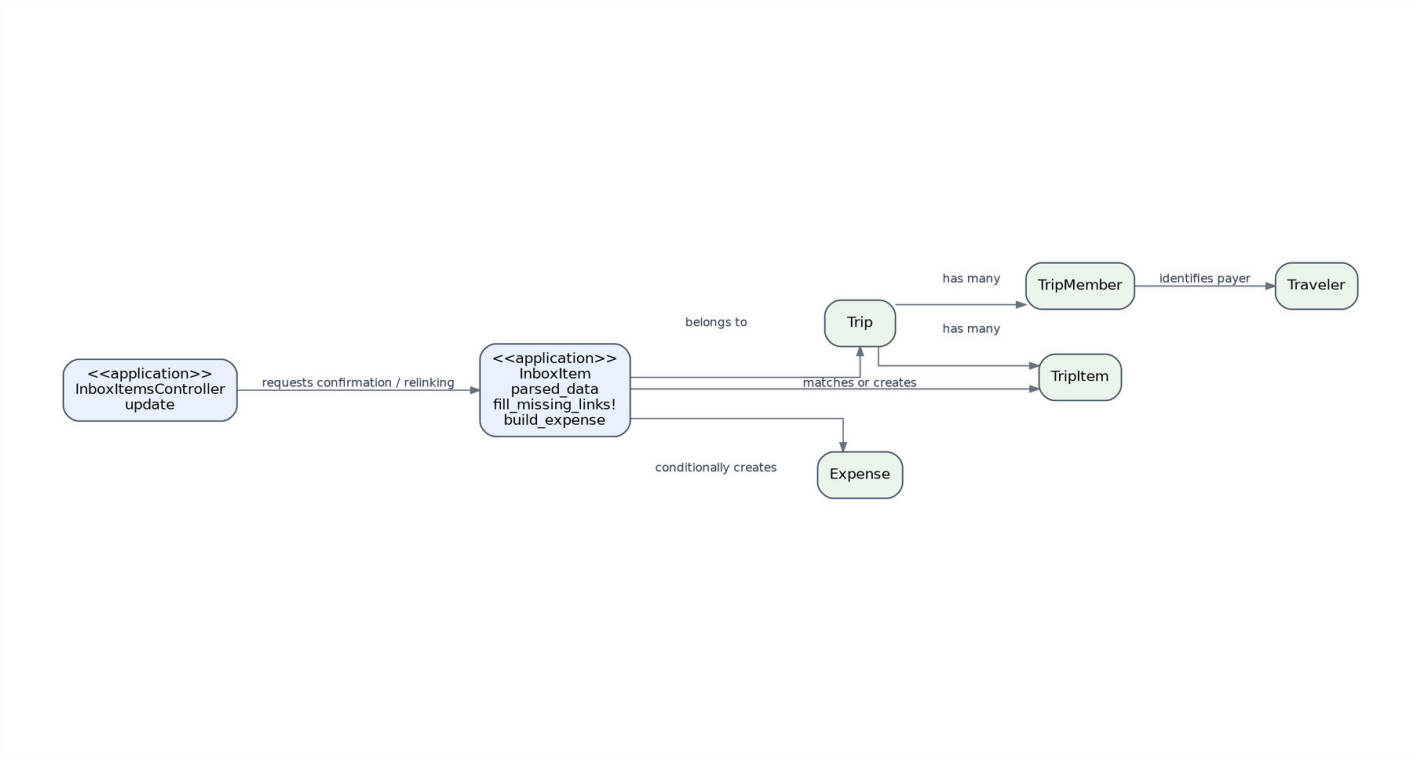
Frozen baseline · 9 confirmed instances · Mermaid UML source

The nine diagrams below use the frozen source-supported inventory. They show the actual application participants and their collaborations without inventing pattern interfaces, abstract classes or additional pattern instances.

Notation: `<<application>>` identifies DaPlanStan code; `<<framework>>` identifies external framework participants; dashed arrows indicate calls, dependencies or notifications; solid arrows show established domain relationships. View templates are represented as views rather than invented JavaScript classes.

P01 — Parsed Email to Trip Record Linking

Information Expert (GRASP)



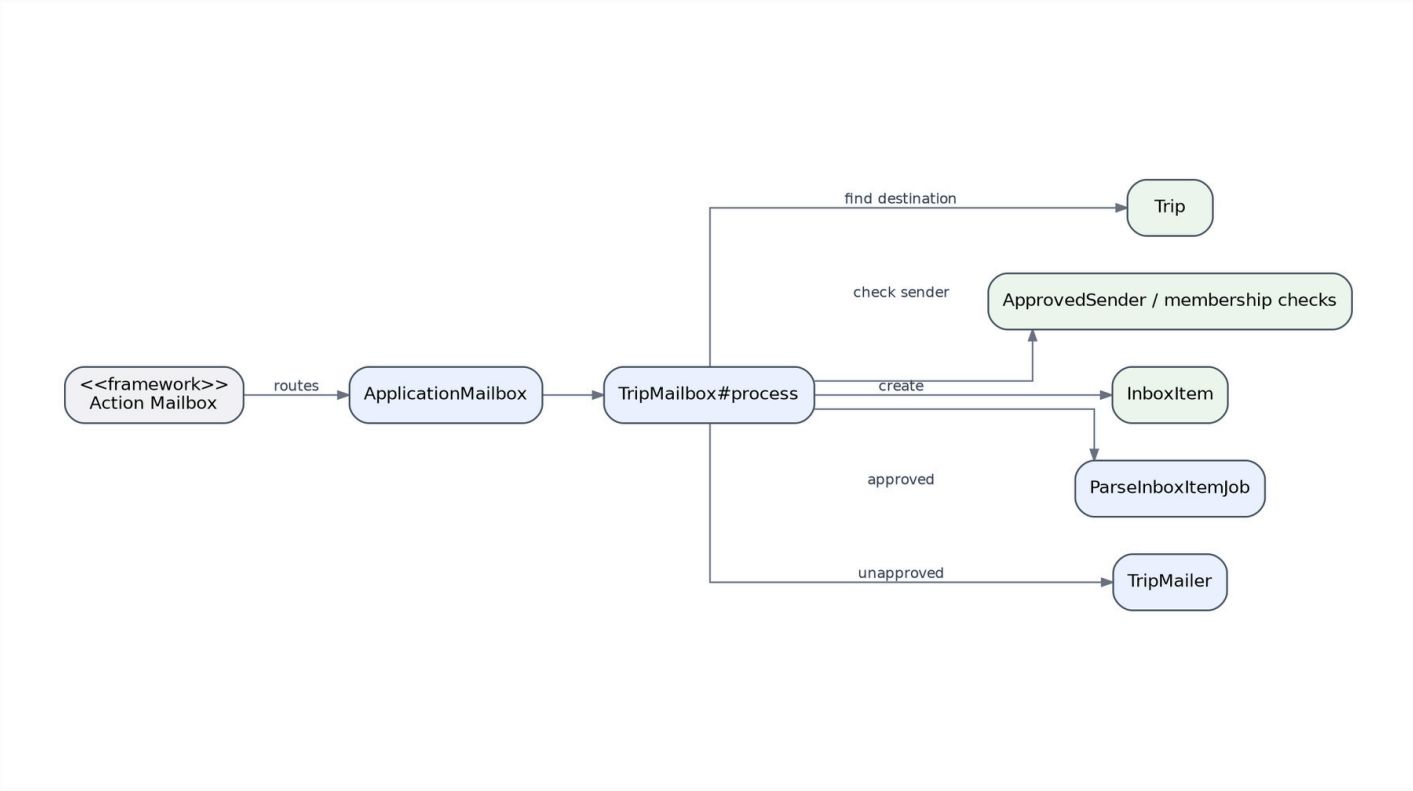
What this diagram demonstrates: `InboxItem` is the Information Expert because it stores the parsed email information and owns the application-specific logic for interpreting that information and linking it to existing or new `TripItem` and `Expense` records.

`InboxItemsController` initiates confirmation or relinking but delegates the substantive responsibility to `InboxItem`. Active Record supplies the associations and persistence, not the record-linking decisions.

Primary source: `app/models/inbox_item.rb:19-25, 34-58, 67-207`; `app/controllers/inbox_items_controller.rb:16-35`.

P02 — Inbound Travel Email Intake Coordination

Controller (GRASP)



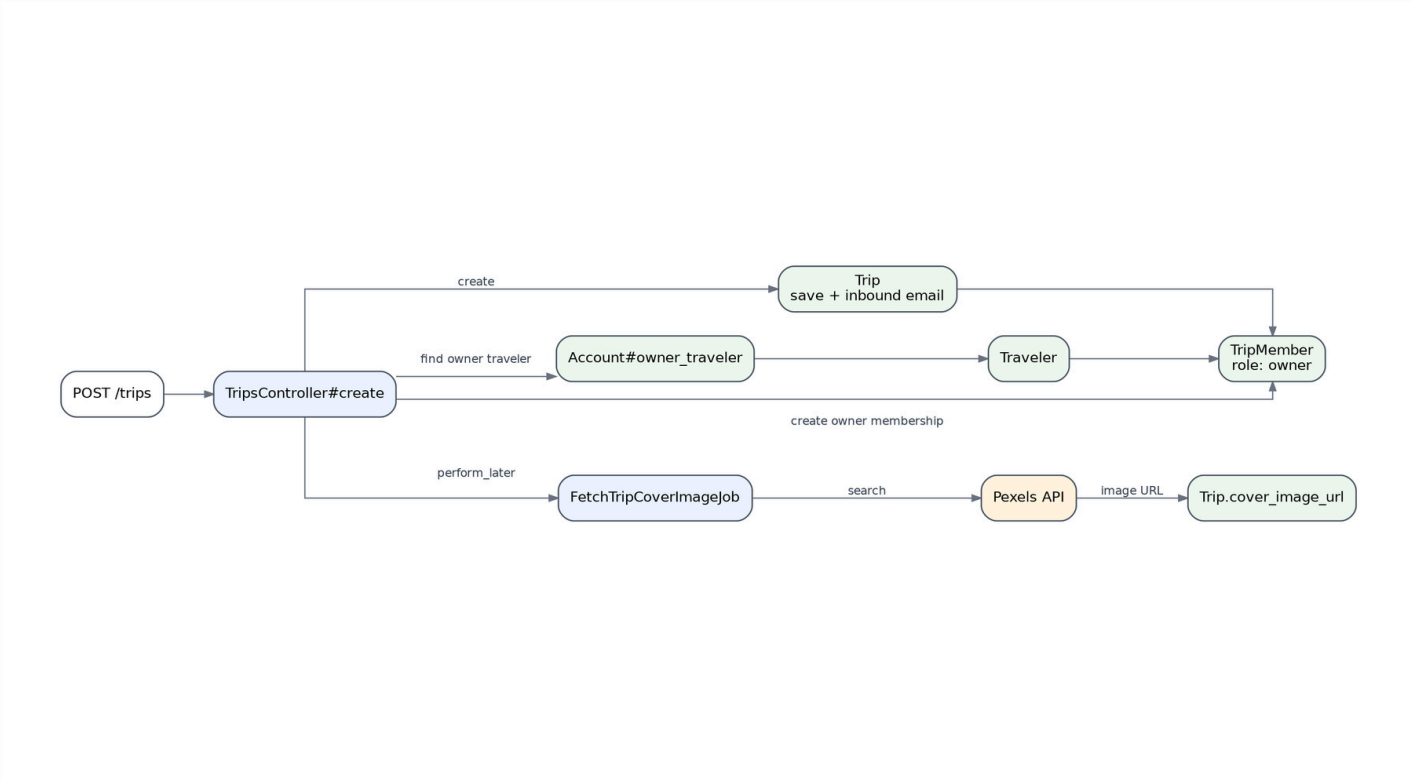
What this diagram demonstrates: `TripMailbox#process` is responsible for coordinating the inbound-email system event. It identifies the trip, checks sender eligibility, creates the inbox record and selects the next action.

Action Mailbox provides message routing and lifecycle infrastructure. The approval decisions and delegation are DaPlanStan application behavior.

Primary source: `app/mailboxes/application_mailbox.rb:1–3`; `app/mailboxes/trip_mailbox.rb:2–31, 55–60`; `app/jobs/parse_inbox_item_job.rb:99–112`.

P03 — Trip Creation Coordination

Controller (GRASP)



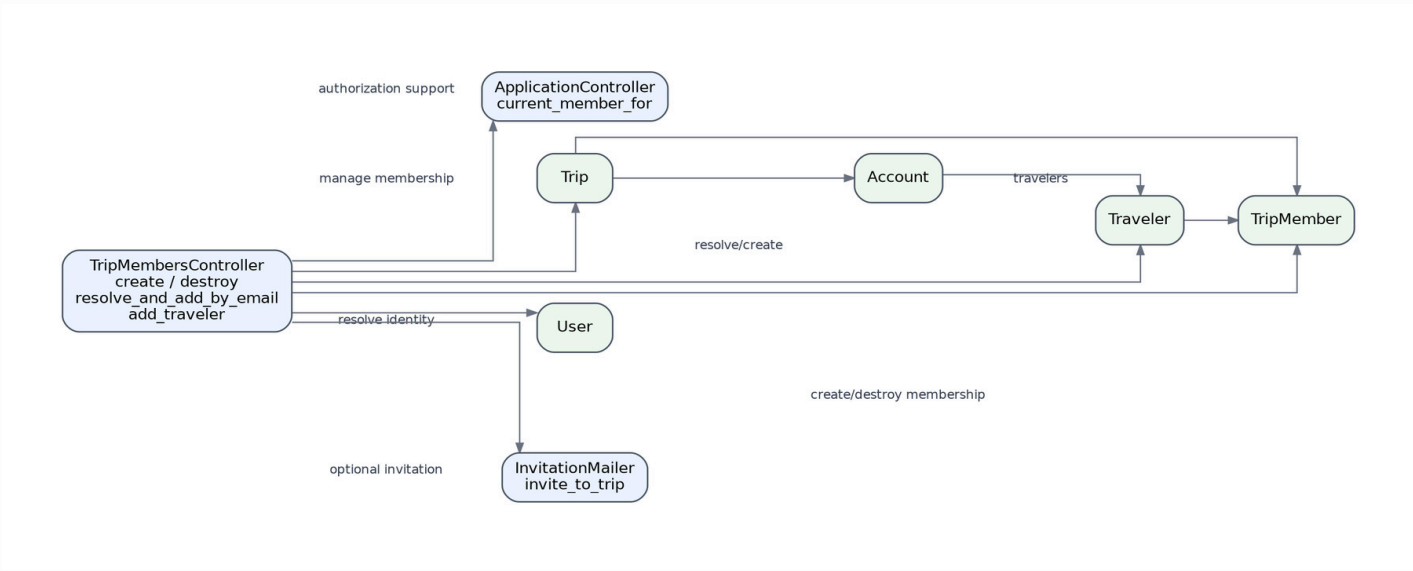
What this diagram demonstrates: `TripsController#create` coordinates a multi-object creation use case. It establishes the `Trip`, finds the account owner's `Traveler`, creates an owner-role `TripMember` and delegates cover retrieval.

Pexels supplies images as an external provider; it is not an application-written class.

Primary source: `app/controllers/trips_controller.rb:35–48`; `app/models/trip.rb:4–19, 31–37`; `app/models/account.rb:9–11`; `app/jobs/fetch_trip_cover_image_job.rb:4–32`.

P04 — Trip Membership Management Coordination

Controller (GRASP)



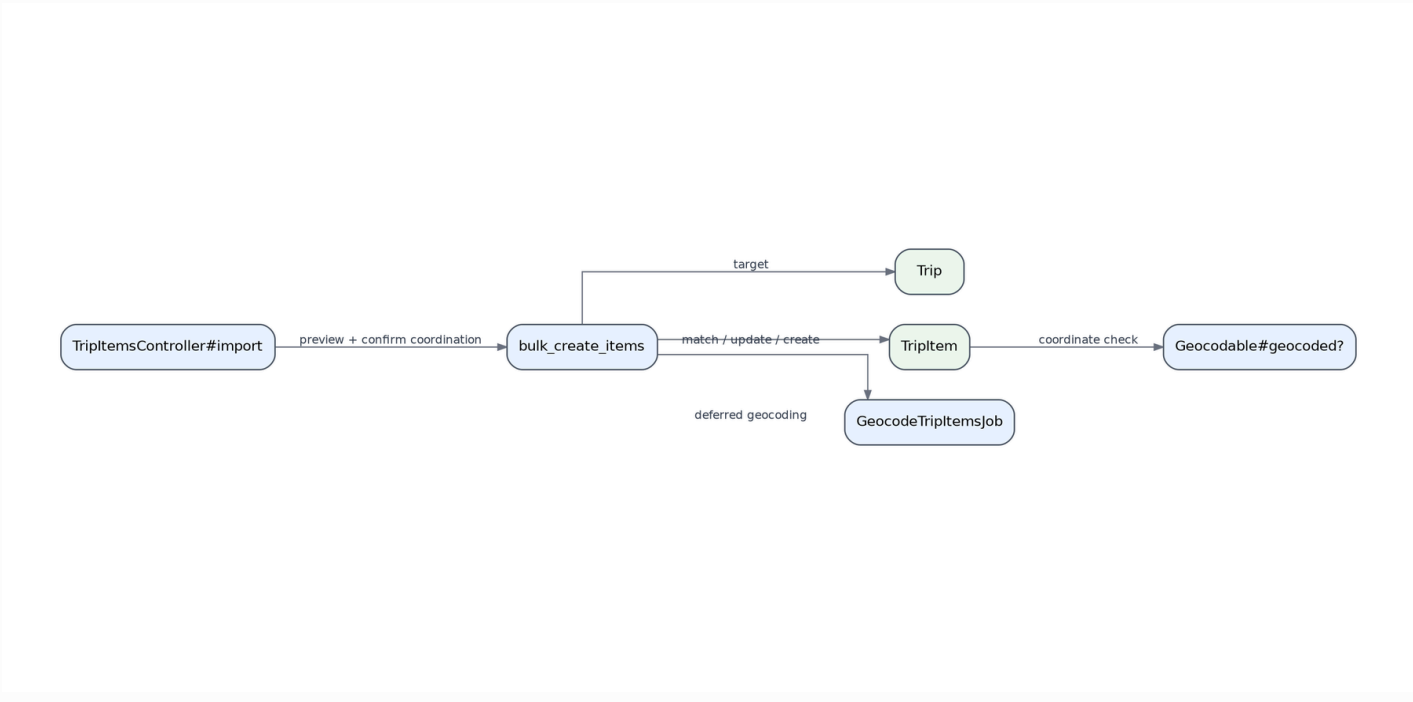
What this diagram demonstrates: `TripMembersController` coordinates membership changes for an existing trip. It checks authorization, resolves travelers, creates contributor-role memberships and prevents direct owner-membership removal. Invitation construction is delegated to `InvitationMailer`.

Membership addition and removal are represented in **one diagram and one pattern instance**, since they belong to the same use-case responsibility.

Primary source: `app/controllers/trip_members_controller.rb:6-65`; `app/controllers/application_controller.rb:22-25`; `app/models/trip_member.rb:1-6`.

P05 — Bulk Itinerary Import Coordination

Controller (GRASP)



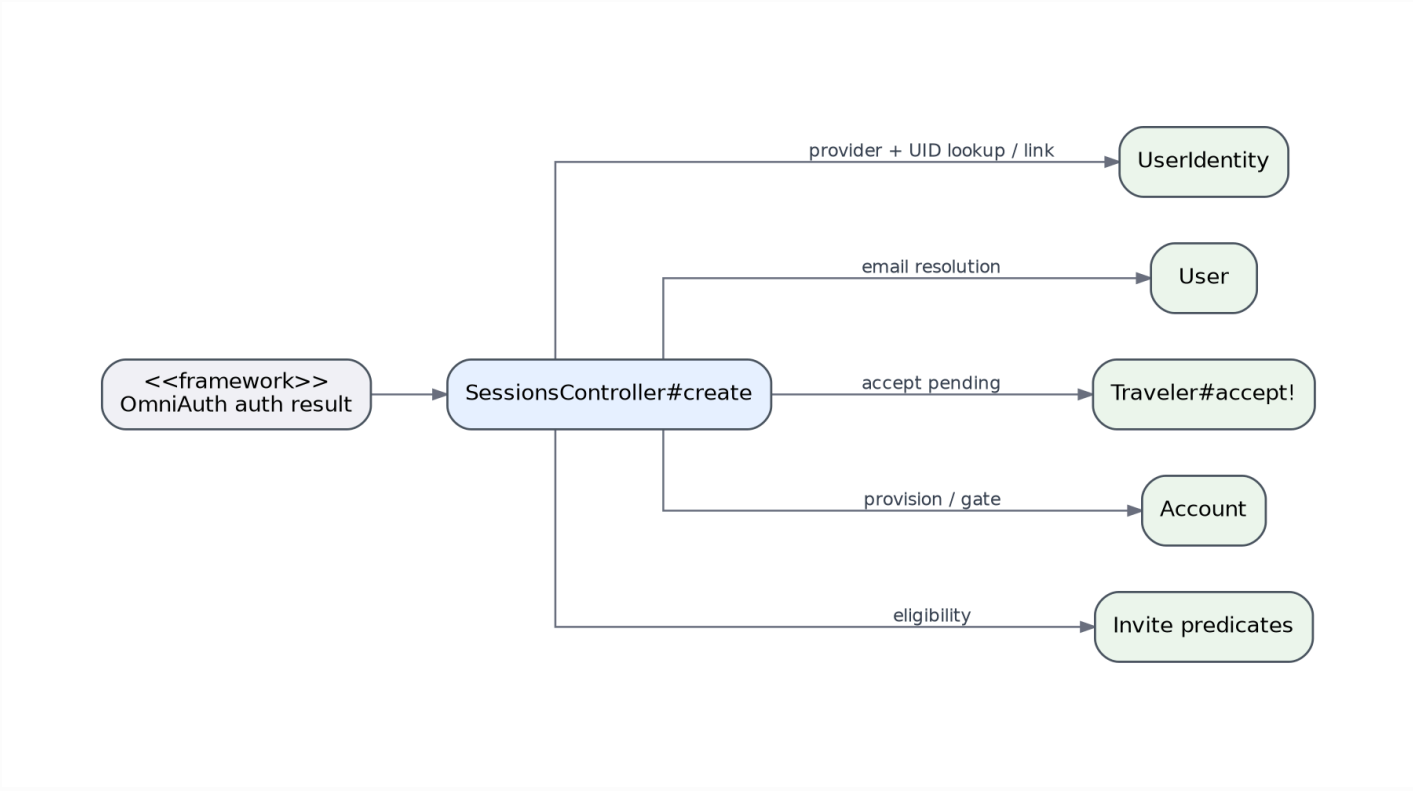
What this diagram demonstrates: `TripItemsController#import` coordinates the preview and confirmed-import stages. `bulk_create_items` resolves duplicates, creates itinerary ideas and delegates missing-coordinate processing to `GeocodeTripItemsJob`.

`Geocodable` provides existing geocoding behavior; this diagram does not introduce a Strategy or Template Method pattern.

Primary source: `app/controllers/trip_items_controller.rb:31–58, 128–179`; `app/models/trip_item.rb:1–37`; `app/models/concerns/geocodable.rb:9–11`; `app/jobs/geocode_trip_items_job.rb:4–21`.

P06 — OAuth Sign-in and Identity Linking Coordination

Controller (GRASP)



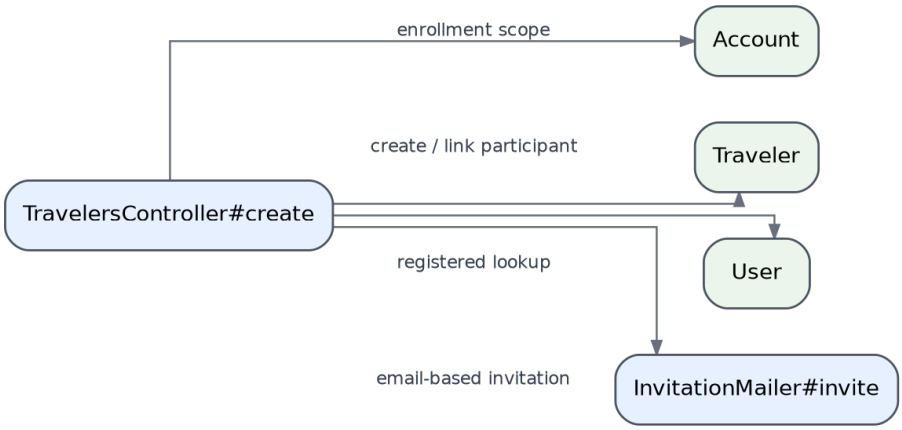
What this diagram demonstrates: After OmniAuth returns normalized provider information, `SessionsController#create` coordinates the application-specific identity workflow: provider/UID lookup, email-based user resolution, session creation, pending traveler acceptance and account provisioning.

The configured Google and Apple providers use the same application coordination. No separate Strategy implementation or pattern instance is introduced for either provider.

Primary source: `app/controllers/sessions_controller.rb:4–41, 54–81`; `app/models/user_identity.rb:1–8`; `app/models/traveler.rb:26–28`; `config/initializers/omniauth.rb:1–16`.

P07 — Account Traveler Enrollment Coordination

Controller (GRASP)



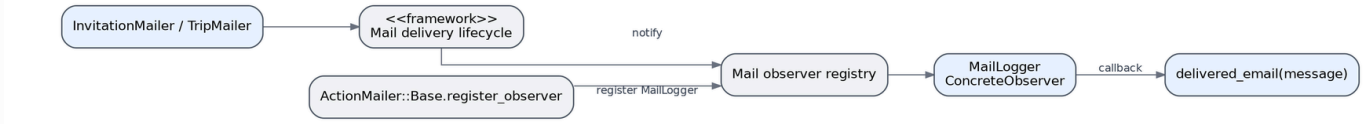
What this diagram demonstrates: `TravelersController#create` coordinates account-level traveler enrollment. It determines whether to create a name-only, email-only or registered-user-linked `Traveler`, and requests invitations when applicable.

Unlike P04, this operation does **not** establish a `TripMember`. That difference is why account traveler enrollment remains a separate frozen use-case collaboration.

Primary source: `app/controllers/travelers_controller.rb:5–35, 54–56`; `app/views/settings/show.html.erb:114–127`; `app/models/account.rb:1–11`; `app/mailers/invitation_mailer.rb:2–6`.

P08 — Outbound Mail Delivery Logging

Observer (GoF) — framework-mediated



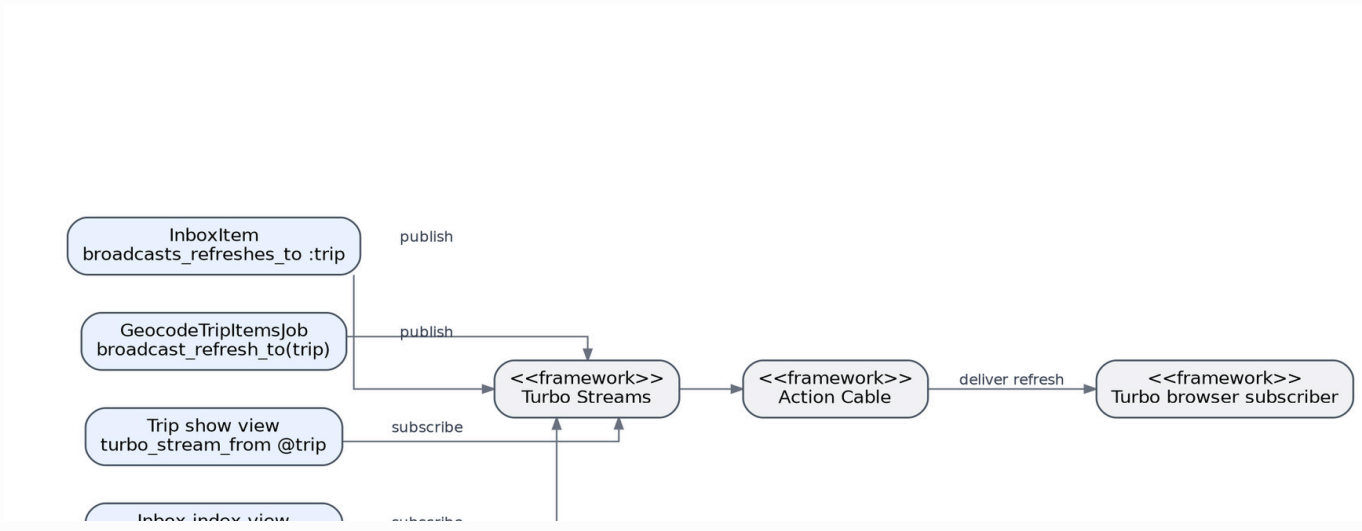
What this diagram demonstrates: DaPlanStan supplies the concrete observer `MailLogger` and explicitly registers it with Action Mailer. The Mail framework supplies the observer registry and calls `delivered_email` through its delivery-notification mechanism.

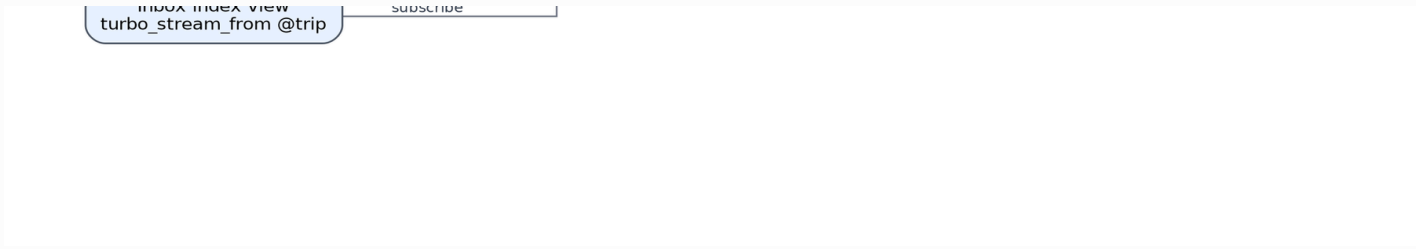
The two mailers produce messages but do not directly invoke `MailLogger`. All outgoing messages share **one Observer instance**.

Primary source: `config/initializers/mail_logger.rb:1–7`; `app/mailers/invitation_mailer.rb:2–13`; `app/mailers/trip_mailer.rb:2–10`; Action Mailer 8.1.3 and Mail 2.9.0 framework contracts.

P09 — Trip-Scoped Turbo Refresh Notifications

Observer (GoF) — framework-mediated





What this diagram demonstrates: The application defines two publication paths and two trip-scoped subscription declarations. Turbo Rails establishes the stream subscription and broadcast behavior, while Action Cable delivers notifications to subscribed browser clients.

The subscriber and notification infrastructure are framework-provided, not custom DaPlanStan classes. Both inbox processing and geocoding reuse **one Trip-Scoped Turbo Refresh Notifications Observer instance**.

Primary source: `app/models/inbox_item.rb:5` ; `app/jobs/geocode_trip_items_job.rb:20-21` ; `app/views/trips/show.html.erb:2-9` ; `app/views/inbox_items/index.html.erb:1` ; Turbo Rails 2.0.23 framework contract.

Mermaid source precision

Two method names in the diagrams are displayed without Ruby's punctuation for simpler Mermaid rendering. Their exact source names are:

- P01: `InboxItem#fill_missing_links!` (not a separate `fill_missing_links` method).
- P05: `Geocodable#geocoded?` (not a separate `geocoded` method).

When preserving the editable source, those two displayed method lines can be omitted; the documented collaborations and relationships remain unchanged. No additional application methods are implied.

Final UML verification checklist

- ✅ **9 Mermaid UML diagrams produced** — P01 through P09.
- ✅ **All nine frozen instances represented exactly once.**
- ✅ **No rejected pattern classifications included.**
- ✅ **No invented pattern interfaces, abstract classes or additional participant roles.**
- ✅ **Application and framework responsibilities distinguished**, especially in P08 and P09.

All diagrams use Mermaid text source, so they can be edited and later exported to SVG. The baseline remains **9 instances across 3 pattern types**, with no evolution analysis performed.

The frozen baseline, CSV contents and nine UML diagrams are complete.
Do not search for new patterns and do not change the frozen inventory.
Now prepare the verification evidence records for Part 2.
The frozen baseline contains P01–P09:

1. P01 — Information Expert (GRASP) — Parsed Email to Trip Record Linking
2. P02 — Controller (GRASP) — Inbound Travel Email Intake Coordination
3. P03 — Controller (GRASP) — Trip Creation Coordination
4. P04 — Controller (GRASP) — Trip Membership Management Coordination
5. P05 — Controller (GRASP) — Bulk Itinerary Import Coordination
6. P06 — Controller (GRASP) — OAuth Sign-in and Identity Linking Coordination
7. P07 — Controller (GRASP) — Account Traveler Enrollment Coordination
8. P08 — Observer (GoF) — Outbound Mail Delivery Logging
9. P09 — Observer (GoF) — Trip-Scoped Turbo Refresh Notifications

For each confirmed instance prepare one concise verification record containing:

1. Pattern ID.
2. Pattern name.
3. Stable instance name.
4. Verification status: SOURCE-CONFIRMED.
5. Exact source locations used for verification.
6. Main participant-role mapping.
7. The strongest evidence supporting the pattern.
8. The strongest alternative or false-positive explanation that was tested.
9. Why the pattern was retained after that challenge.
10. Framework/application boundary where relevant.
11. Runtime verification status.
12. Independent human sign-off status.

Use accurate wording:

- static source verification was completed;
- runtime testing was not completed;
- independent human sign-off has not been established.

Do not claim that the LLM source review is independent human verification.
Then prepare a rejected-candidate record for exactly these five classifications:

1. Geocoding — Template Method
2. Geocoding — Strategy
3. Geocoding — Adapter
4. Itinerary Timeline–Map Event Coordination — Observer
5. Beta Invite Eligibility Evaluation — Information Expert

For each rejected candidate give:

1. Proposed pattern.
2. Source area.
3. Evidence that originally made it plausible.
4. Decisive reason for rejection.
5. Better explanation of the source behavior.
6. Relevant source locations.
7. Final status: REJECTED.

Then add one section called `Verification Limitations` containing:

1. The source snapshot is the uploaded `daplanstan-main.zip`.
2. The ZIP-to-commit correspondence with `da54d9ab8bfc118e99375fdd6c506efb20b66e5b` remains unverified.
3. No complete Rails runtime or end-to-end integration test was performed.
4. Framework-mediated Observer classifications depend on the versioned framework contracts already checked.
5. Independent human reviewer sign-off remains pending unless separately performed by a team member.

At the end confirm:

1. 9 source-confirmed records.
2. 5 rejected-candidate records.
3. 0 uncertain candidates in the frozen baseline.
4. No change to the counts: 6 Controller, 1 Information Expert, 2 Observer, 9 total.

Do not generate `llm.csv` yet.
Do not perform evolution analysis.
Do not modify the frozen baseline.

Worked for 44s

DaPlanStan — Part 2 Verification Evidence Records

Frozen source-supported baseline 9 confirmed · 5 rejected · 0 uncertain

The evidence records below preserve the nine frozen classifications and five rejections. Key source locations were rechecked against `daplanstan-main.zip`, including `ParseInboxItemJob#perform` at lines 99–112.

Writing

Part 2 — Design Pattern Verification Evidence Records

Project: DaPlanStan
Source version: baseline
Source snapshot: `daplanstan-main.zip`
Verification method: Static source inspection and previously checked framework contracts
Baseline status: Frozen source-supported proposal
Runtime testing: Not completed
Independent human sign-off: Not established

A. Confirmed Pattern Verification Records

P01 — Parsed Email to Trip Record Linking

Pattern name: Information Expert (GRASP)
Stable instance name: Parsed Email to Trip Record Linking
Verification status: SOURCE-CONFIRMED

Source locations

- `app/models/inbox_item.rb:19–25, 34–58, 67–187, 191–207`
- `app/controllers/inbox_items_controller.rb:16–35`
- `app/models/trip.rb:4–12`
- `app/models/trip_item.rb:1–28`
- `app/models/expense.rb:1–28`
- `app/models/trip_member.rb:1–6`
- `app/models/traveler.rb:1–8`

Participant-role mapping

`InboxItem` — Information Expert responsible for interpreting its stored parsed email data.
`InboxItemsController` — requesting client for confirmation and relinking.
`Trip` — parent domain context.
`TripItem` — itinerary record created or linked.
`Expense` — conditionally created linked expense.
`TripMember` and `Traveler` — membership and payer-identification collaborators.

Strongest supporting evidence

`InboxItem` stores the extracted information through `parsed_data`. Its `confirm_and_create_items!` and `fill_missing_links!` methods interpret the data, match existing itinerary items, create missing records and conditionally create expenses.

Strongest false-positive explanation tested

The behavior could be interpreted as an oversized Active Record model performing procedural orchestration across several domain objects rather than a clear Information Expert responsibility.

Reason retained

The central information being interpreted belongs to `InboxItem`, and the application explicitly assigns substantive record-linking rules to this model rather than merely storing the extraction result.

Application/framework boundary

DaPlanStan implements the mapping, matching and creation decisions. Rails Active Record supplies associations, queries and persistence. Anthropic performs the earlier extraction, not this

linking operation.

Runtime verification: NOT COMPLETED
Independent human sign-off: NOT ESTABLISHED

P02 — Inbound Travel Email Intake Coordination

Pattern name: Controller (GRASP)
Stable instance name: Inbound Travel Email Intake Coordination
Verification status: SOURCE-CONFIRMED

Source locations

- `app/mailboxes/application_mailbox.rb:1–3`
- `app/mailboxes/trip_mailbox.rb:2–31, 35–60`
- `app/models/trip.rb:4–19`
- `app/models/approved_sender.rb:1–6`
- `app/models/inbox_item.rb:1–17`
- `app/models/trip_member.rb:1–6`
- `app/jobs/parse_inbox_item_job.rb:99–112, 127–139`
- `app/mailers/trip_mailer.rb:2–10`

Participant-role mapping

`ApplicationMailbox` — application routing declaration.
`TripMailbox#process` — system-event coordinator.
`Trip` — destination identified through its inbound address.
`ApprovedSender`, `TripMember`, `Traveler` and `User` — sender-eligibility information.
`InboxItem` — persisted incoming message.
`ParseInboxItemJob` — delegated parsing operation.
`TripMailer` — delegated owner-notification construction.

Strongest supporting evidence

`TripMailbox#process` identifies the destination trip, evaluates sender eligibility, creates an `InboxItem` and conditionally invokes `ParseInboxItemJob.perform_later` or pending-sender notification.

The processing path is explicitly controlled by application decisions.

Strongest false-positive explanation tested

The behavior could be ordinary Action Mailbox processing because `process` is a standard framework callback.

Reason retained

DaPlanStan implements the trip-matching, sender-approval and next-action decisions. These responsibilities establish meaningful application-owned coordination of an incoming system event.

Application/framework boundary

Action Mailbox supplies reception and mailbox routing infrastructure. Active Job supplies background execution. Action Mailer supplies delivery infrastructure. Anthropic supplies subsequent AI extraction. The application owns intake decisions.

Runtime verification: NOT COMPLETED
Independent human sign-off: NOT ESTABLISHED

P03 — Trip Creation Coordination

Pattern name: Controller (GRASP)
Stable instance name: Trip Creation Coordination
Verification status: SOURCE-CONFIRMED

Source locations

- `app/controllers/trips_controller.rb:35–48`
- `app/controllers/application_controller.rb:14–20`
- `app/models/trip.rb:4–19, 31–37`
- `app/models/account.rb:9–11`
- `app/models/traveler.rb:1–8`
- `app/models/trip_member.rb:1–6`
- `app/jobs/fetch_trip_cover_image_job.rb:4–32`
- `config/routes.rb:21`

Participant-role mapping

`TripsController#create` — trip-creation system-event coordinator.
`ApplicationController` — authenticated user and account context.
`Trip` — new domain record and inbound-address generation.
`Account#owner_traveler` — owner-traveler lookup.
`Traveler` — selected owner participant.
`TripMember` — initial owner-role membership.
`FetchTripCoverImageJob` — delegated cover-image retrieval.

Strongest supporting evidence

After saving a new `Trip`, `TripsController#create` creates the initial `TripMember` with `role: :owner` and schedules `FetchTripCoverImageJob`.

The controller coordinates multiple responsibilities needed to establish the trip.

Strongest false-positive explanation tested

The implementation resembles an ordinary Rails create action containing Active Record operations and a background-job call.

Reason retained

The event establishes both a trip and its initial ownership relationship, while delegating initialization and image retrieval to other participants. The collaboration represents one cohesive trip-creation responsibility.

Application/framework boundary

DaPlanStan controls the creation sequence and membership rule. Rails supplies request routing and persistence. Active Job supplies execution infrastructure, and Pexels supplies image-search results.

Runtime verification: NOT COMPLETED
Independent human sign-off: NOT ESTABLISHED

P04 — Trip Membership Management Coordination

Pattern name: Controller (GRASP)
Stable instance name: Trip Membership Management Coordination
Verification status: SOURCE-CONFIRMED

Source locations

- `app/controllers/trip_members_controller.rb:6–22, 26–65`
- `app/controllers/application_controller.rb:22–25`
- `app/models/trip.rb:4–12`
- `app/models/account.rb:1–11`
- `app/models/traveler.rb:1–8`
- `app/models/user.rb:11–17`
- `app/models/trip_member.rb:1–6`
- `app/mailers/invitation_mailer.rb:8–13`
- `config/routes.rb:40`

Participant-role mapping

`TripMembersController#create` and `#destroy` — membership-event handlers.
`TripMembersController#require_owner_or_planner` — authorization decision.
`#resolve_and_add_by_email` — traveler identity resolution.
`#add_traveler` — duplicate checking and membership creation.
`ApplicationController#current_member_for` — acting-user membership lookup.
`Trip`, `Account`, `Traveler` and `User` — domain and identity collaborators.
`TripMember` — role-bearing membership record.
`InvitationMailer` — invitation construction.

Strongest supporting evidence

The controller checks the acting user's membership role, resolves or creates an account traveler, prevents duplicate membership, creates contributor-role memberships and protects owner memberships from direct removal.

Strongest false-positive explanation tested

The controller contains substantial procedural membership logic that could be considered controller-heavy CRUD.

Reason retained

The methods form a cohesive membership-management use case with authorization, participant resolution, membership changes and optional invitations.

Adding and removing members represent one lifecycle responsibility rather than separate pattern instances.

Application/framework boundary

DaPlanStan supplies authorization and membership rules. Active Record implements association persistence and enums. Action Mailer and Active Job provide mail infrastructure.

Runtime verification: NOT COMPLETED
Independent human sign-off: NOT ESTABLISHED

P05 — Bulk Itinerary Import Coordination

Pattern name: Controller (GRASP)
Stable instance name: Bulk Itinerary Import Coordination
Verification status: SOURCE-CONFIRMED

Source locations

- `app/controllers/trip_items_controller.rb:31–58, 128–179`
- `app/models/trip.rb:4–12`
- `app/models/trip_item.rb:1–28`
- `app/models/concerns/geocodable.rb:9–11`
- `app/jobs/geocode_trip_items_job.rb:4–21`
- `app/views/trip_items/import.html.erb:93–106`
- `config/routes.rb:30–34`

Participant-role mapping

`TripItemsController#import` — import system-event coordinator.
`#bulk_create_items` — batch matching and persistence coordination.
`#parse_import_time` — imported date/time parsing.
`Trip` — destination domain context.

`TripItem` — existing or newly created itinerary record.
`Geocodable#geocoded?` — coordinate-completeness operation.
`GeocodeTripItemsJob` — deferred geocoding collaborator.
Import view — preview and confirmation interface.

Strongest supporting evidence

The import action processes preview and confirmation stages. `bulk_create_items` checks input names, identifies existing records, updates selected missing fields, creates new itinerary ideas and schedules geocoding when needed.

Strongest false-positive explanation tested

Much of the import algorithm is implemented directly inside one Rails controller helper. It may be interpreted as procedural import code rather than meaningful GRASP responsibility assignment.

Reason retained

The controller coordinates one coherent system-level import operation involving preview, confirmation, duplicate handling, persistence and deferred processing.

The presence of procedural logic does not remove the identifiable system-event responsibility.

Application/framework boundary

DaPlanStan defines import decisions and sequencing. Active Record supplies persistence and callback APIs. Active Job performs scheduled work. The existing Geocodable behavior is reused without creating another pattern instance.

Runtime verification: NOT COMPLETED
Independent human sign-off: NOT ESTABLISHED

P06 — OAuth Sign-in and Identity Linking Coordination

Pattern name: Controller (GRASP)
Stable instance name: OAuth Sign-in and Identity Linking Coordination
Verification status: SOURCE-CONFIRMED

Source locations

- `app/controllers/sessions_controller.rb:4–41, 54–81`
- `app/models/user_identity.rb:1–8`
- `app/models/user.rb:11–22`
- `app/models/traveler.rb:26–28`
- `app/models/account.rb:1–11`
- `app/models/invite.rb:12–18`
- `app/controllers/application_controller.rb:14–16`
- `config/initializers/omniauth.rb:1–16`
- `config/routes.rb:12–14`

Participant-role mapping

`SessionsController#create` — post-authentication system-event coordinator.
`#provision_account_or_gate` — account-eligibility coordination.
`#create_account_for` — account and initial-traveler creation.
`UserIdentity` — stored provider identity.
`User` — local authenticated identity.
`Traveler#accept!` — pending traveler association.
`Account` — user-owned account.
`Invite` — eligibility predicates.
`ApplicationController#current_user` — session identity resolution.
OmniAuth configuration — provider registration.

Strongest supporting evidence

The callback first checks provider/UID identity, then uses email-based user resolution when necessary. It creates missing provider identity records, establishes a session, links pending travelers and applies account-provisioning rules.

Strongest false-positive explanation tested

OmniAuth already handles authentication, and the controller may simply perform procedural post-authentication account maintenance.

Reason retained

DaPlanStan independently determines how an authenticated provider identity becomes a local

user with appropriate traveler and account access. This is a cohesive application-level identity coordination responsibility.

Application/framework boundary

OmniAuth and its providers perform OAuth protocol processing. Rails supplies sessions and persistence. DaPlanStan supplies identity linking, traveler acceptance and account-access decisions.

Google and Apple remain one application-level coordination instance.

Runtime verification: NOT COMPLETED
Independent human sign-off: NOT ESTABLISHED

P07 — Account Traveler Enrollment Coordination

Pattern name: Controller (GRASP)
Stable instance name: Account Traveler Enrollment Coordination
Verification status: SOURCE-CONFIRMED

Source locations

- `app/controllers/travelers_controller.rb:5–35, 54–56`
- `app/models/account.rb:1–11`
- `app/models/traveler.rb:1–8, 18–28`
- `app/models/user.rb:11–22`
- `app/mailers/invitation_mailer.rb:2–6`
- `app/views/settings/show.html.erb:114–127`
- `config/routes.rb:48–50`

Participant-role mapping

`TravelersController#create` — account-traveler enrollment coordinator.
`TravelersController#set_account` — account-owner access check.
`Account` — enrollment scope.
`Traveler` — account participant record.
`User` — registered-user identity lookup.
`InvitationMailer#invite` — account invitation construction.

Settings view — enrollment request interface.

Strongest supporting evidence

The controller handles name-only enrollment or email-based enrollment. It checks for duplicate participants, identifies registered users, creates the corresponding traveler records and requests invitations when an email is supplied.

Strongest false-positive explanation tested

The implementation could be ordinary account-level CRUD and duplicates some traveler-resolution decisions found in trip membership management.

Reason retained

The event establishes an account-level traveler independently of membership in a particular trip.

Unlike P04, it does not create a role-bearing `TripMember`. The domain outcome and initiating use case are distinct.

Application/framework boundary

DaPlanStan supplies enrollment rules and invitation decisions. Rails supplies routing and Active Record operations. Action Mailer and Active Job provide notification infrastructure.

Runtime verification: NOT COMPLETED
Independent human sign-off: NOT ESTABLISHED

P08 — Outbound Mail Delivery Logging

Pattern name: Observer (GoF)
Stable instance name: Outbound Mail Delivery Logging
Verification status: SOURCE-CONFIRMED

Source locations

Application source:

- `config/initializers/mail_logger.rb:1–7`
- `app/mailers/application_mailer.rb:1–5`
- `app/mailers/invitation_mailer.rb:2–13`
- `app/mailers/trip_mailer.rb:2–10`

Framework contract:

- Action Mailer 8.1.3 — `register_observer`
- Mail 2.9.0 — observer registration and `delivered_email` notification contract

Participant-role mapping

`MailLogger` — application-defined concrete observer.
`MailLogger.delivered_email(message)` — observer callback.
`ActionMailer::Base.register_observer(MailLogger)` — application registration operation through the framework API.
`ActionMailer::Base` — framework registration interface.
Mail gem — framework publisher, observer registry and notification infrastructure.
`InvitationMailer`, `TripMailer` — outgoing mail producers.

Strongest supporting evidence

The application explicitly defines `MailLogger.delivered_email` and registers `MailLogger` through `ActionMailer::Base.register_observer`.

The previously checked Action Mailer and Mail framework contracts establish that registered observers receive the delivery-notification callback.

Strongest false-positive explanation tested

The notification registry and dispatch mechanism are entirely framework-provided. DaPlanStan does not implement a standalone GoF Observer framework.

Reason retained

DaPlanStan supplies a concrete observer and explicitly registers it with a real observer contract. The framework invokes that observer without individual mailers calling it directly.

This supports a framework-mediated Observer classification.

Application/framework boundary

DaPlanStan owns `MailLogger` and its registration statement. Action Mailer and the Mail gem supply notification infrastructure. Postmark is an external delivery provider, not an application-defined Observer participant.

All mailers and messages share one Observer instance.

Runtime verification: NOT COMPLETED
Independent human sign-off: NOT ESTABLISHED

P09 — Trip-Scoped Turbo Refresh Notifications

Pattern name: Observer (GoF)
Stable instance name: Trip-Scoped Turbo Refresh Notifications
Verification status: SOURCE-CONFIRMED

Source locations

Application source:

- `app/models/inbox_item.rb:1–5`
- `app/jobs/geocode_trip_items_job.rb:4–21`
- `app/views/trips/show.html.erb:2–9`
- `app/views/inbox_items/index.html.erb:1`

Framework contract:

- Turbo Rails 2.0.23 — `Turbo::Broadcastable`
- Turbo Rails 2.0.23 — `Turbo::StreamsChannel`
- Turbo Rails 2.0.23 — `Turbo::Streams::Broadcasts`
- Rails Action Cable — stream subscription and delivery
- Turbo browser runtime — subscriber and refresh processing

Participant-role mapping

`InboxItem` — application model publication source.
`broadcasts_refreshes_to :trip` — application publication declaration.

`GeocodeTripItemsJob` — explicit application broadcast source.
Trip show view — application subscription declaration.
Inbox index view — second application subscription declaration.
`Turbo::Broadcastable` — model-triggered broadcast infrastructure.
`Turbo::StreamsChannel` — stream subscription and broadcast channel.
Action Cable — notification delivery infrastructure.
Turbo browser runtime — concrete stream subscriber and refresh reaction.

Strongest supporting evidence

`InboxItem` declares `broadcasts_refreshes_to :trip`. `GeocodeTripItemsJob` explicitly broadcasts a refresh to the trip. Two application views subscribe using `turbo_stream_from @trip`.

The previously checked Turbo Rails 2.0.23 contract establishes the trip-scoped subscription and broadcast mechanism.

Strongest false-positive explanation tested

DaPlanStan does not define its own Observer interface, subscriber callback or broadcast registry. The infrastructure and client notification behavior are supplied by Turbo and Action Cable.

Reason retained

The application explicitly selects the publication sources and subscribes pages to the same trip-scoped stream. The documented framework mechanism establishes notification delivery without direct publisher-to-page calls.

The behavior is defensible as one framework-mediated Observer collaboration.

Application/framework boundary

DaPlanStan declares publication and subscription usage. Turbo Rails, Action Cable and the Turbo browser runtime supply registration, delivery and refresh behavior.

Inbox and geocoding publishers, multiple broadcasts and multiple subscribed pages remain one Observer instance.

Runtime verification: NOT COMPLETED
Independent human sign-off: NOT ESTABLISHED

B. Rejected-Candidate Evidence Records

R01 — Geocoding Template Method

Proposed pattern: Template Method (GoF Behavioral)
Source area: Shared geocoding concern and TripItem model

Why it initially appeared plausible: `TripItem` overrides `geocode_from_address`, checks transport-item conditions and calls `super` to reuse the concern implementation.

Decisive rejection reason: There is no application-defined template algorithm invoking overridable primitive steps. `TripItem` guards the complete operation instead of supplying a primitive operation within an algorithm skeleton.

Better source explanation: Ordinary Ruby module method reuse with a model-specific guard.

Source locations: `app/models/concerns/geocodable.rb:4–16, 27–65`; `app/models/trip_item.rb:32–37`

Final status: REJECTED

R02 — Geocoding Strategy

Proposed pattern: Strategy (GoF Behavioral)
Source area: Primary and fallback geocoding

Why it initially appeared plausible: The concern contains two different lookup methods, `geocode_with_bias` and `geocode_fallback`.

Decisive rejection reason: Both are fixed methods within the same concern. The code executes them in a predetermined fallback sequence and does not select interchangeable implementations through a common Strategy role.

Better source explanation: Sequential conditional fallback within one algorithm.

Source locations: `app/models/concerns/geocodable.rb:13–14, 27–65`

Final status: REJECTED

R03 — Geocoding Adapter

Proposed pattern: Adapter (GoF Structural)
Source area: Geocoder API integration

Why it initially appeared plausible: The `Geocodable` concern invokes the external Geocoder library and converts lookup results into coordinates stored on application records.

Decisive rejection reason: No incompatible application target interface and adaptee interface are established. The concern directly invokes `Geocoder.search` and performs domain-specific persistence.

Better source explanation: Direct third-party API integration with application-specific query and coordinate-handling logic.

Source locations: `app/models/concerns/geocodable.rb:13–22, 27–65`; `config/initializers/geocoder.rb:1–7`

Final status: REJECTED

R04 — Itinerary Timeline–Map Event Coordination

Proposed pattern: Observer (GoF Behavioral)
Source area: Frontend Stimulus controllers and browser events

Why it initially appeared plausible: `TimelineController` dispatches `map:locate` and `map:reset`. `TripNavController` dispatches `map:resize`. `MapController` registers handlers for these events.

Decisive rejection reason: The publishers target the known map DOM element and issue map-specific requests. The code does not establish a substantive independent publisher/subscriber responsibility beyond targeted DOM events.

Better source explanation: Application-defined inter-controller communication using browser `CustomEvent` and `EventTarget` APIs.

Source locations: `app/javascript/controllers/timeline_controller.js:11–33`; `app/javascript/controllers/trip_nav_controller.js:31–43`; `app/javascript/controllers/map_controller.js:51–109`

Final status: REJECTED

R05 — Beta Invite Eligibility Evaluation

Proposed pattern: Information Expert (GRASP)
Source area: Beta invitation validation and account provisioning

Why it initially appeared plausible: `Invite` owns `used_at`, `expires_at` and `email`, and defines `usable?` and `email_matches?` using those fields.

Decisive rejection reason: These are simple domain predicates. They do not establish a substantive independent Information Expert collaboration comparable to the record-linking responsibility assigned to `InboxItem`.

The complete account eligibility and provisioning decision remains in `SessionsController`.

Better source explanation: Ordinary domain-model encapsulation of invitation validation predicates.

Source locations: `app/models/invite.rb:12–18`; `app/controllers/invites_controller.rb:5–15`; `app/controllers/sessions_controller.rb:63–75`

Final status: REJECTED

Verification Limitations

1. Source snapshot

All application-source claims are based on the uploaded `daplanstan-main.zip` snapshot.

2. Git provenance

The ZIP-to-commit correspondence with `da54d9ab8bfc118e99375fdd6c506efb20b66e5b` remains unverified.

The `baseline` source version identifies the analyzed snapshot without asserting verified upstream Git provenance.

3. Runtime testing

No complete Rails runtime or end-to-end integration test was performed during this source-based verification.

In particular, successful OAuth callbacks, mail delivery notifications, Turbo refreshes, background jobs and related multi-record workflows have not been established through executed integration tests.

4. Framework contracts

The two framework-mediated Observer classifications depend on the previously checked versioned contracts:

- Action Mailer 8.1.3 and Mail 2.9.0 for Outbound Mail Delivery Logging.
- Turbo Rails 2.0.23 and the corresponding Action Cable subscription infrastructure for Trip-Scoped Turbo Refresh Notifications.

The records distinguish framework-supplied mechanisms from DaPlanStan's application-defined participants.

5. Independent human verification

Static source verification was completed through LLM-assisted inspection. This does **not** constitute independent human verification.

Independent human reviewer sign-off remains pending unless a team member performs and records it separately.

6. Inventory status

The classifications remain frozen as the source-supported baseline proposal. Runtime limitations and pending independent sign-off do not introduce additional pattern instances or change the established source-level decisions.

Final Verification Record Summary

Record category	Count	Status
GRASP Controller	6	SOURCE-CONFIRMED
GRASP Information Expert	1	SOURCE-CONFIRMED
Observer	2	SOURCE-CONFIRMED
Total confirmed instances	**9**	**SOURCE-CONFIRMED**
Rejected candidate records	5	REJECTED
Uncertain candidates in frozen baseline	0	None
Distinct confirmed pattern types	3	Frozen

Final statement

Part 2 contains nine source-confirmed pattern verification records and five rejected-candidate records. Each accepted collaboration retains its frozen stable name and classification.

Static source verification is complete. Runtime testing is not complete, and independent human sign-off has not been established.

No new patterns were added, no frozen classification was modified and no evolution analysis was performed.

Source audit: the uploaded archive supports the stated inbound-email control flow, Controller responsibilities, explicit mail observer registration, Turbo publication/subscription declarations and rejected-candidate mechanisms.

Final check: 9 source-confirmed records, 5 rejected records, 0 uncertain candidates. Counts unchanged: **6 Controller + 1 Information Expert + 2 Observer = 9 instances across 3 pattern types.** No `llm.csv` was generated.

The frozen baseline, CSV contents, UML diagrams and verification evidence records are complete.

Do not search for new patterns and do not modify any frozen classification.
Now prepare the LLM audit record for Part 2.
Use this exact `llm.csv` schema:
`part,model_and_date,prompt_file,input_file,output_file,evaluation,verification_file`
The purpose is to document how LLM assistance was used during Part 2 and how each important output was checked against source evidence.
Use one row for each major prompt/output stage rather than one row for every small conversational message.
Group the completed workflow into these stages:

1. Initial requirements and source-inspection planning.
2. Geocoding investigation.
3. Geocoding candidate verification.
4. Inbound email investigation.
5. Inbound email candidate verification.
6. Trip creation and membership investigation.
7. Trip creation and membership verification.
8. Itinerary and expense investigation.
9. Bulk itinerary import verification.
10. External integrations and identity investigation.
11. External integration pattern verification.
12. Frontend interaction investigation.
13. Frontend Observer verification.
14. Repository-wide missed-pattern sweep.
15. Final adversarial review.
16. Human-verification worksheet preparation.
17. Baseline inventory freeze.
18. `patterns.csv` and `pattern_counts.csv` preparation.
19. UML preparation.
20. Verification and rejected-candidate evidence preparation.

For each row:

1. Use `Part 2 - Pattern Discovery and Verification` in the `part` field.

2. In `model_and_date`, record the actual model/date if known from the conversation record. If the exact model name is

unavailable, do not invent one; write `ChatGPT - 2026-10-08`.

3. Give clear, simple filenames for the prompt, input and output artifacts.

4. `prompt_file` should identify the saved prompt text for that stage.

5. `input_file` should identify the source ZIP or previous analysis artifact used for that stage.

6. `output_file` should identify the raw LLM output that should be preserved unchanged.

7. In `evaluation`, use one of:
   Correct
   Partial
   Incorrect
   Cannot confirm

   Then add a short correction or qualification where needed.

8. In `verification_file`, point to the source-check or later verification artifact that supports the evaluation.

9. Do not mark an output Correct merely because the LLM said it was correct. Base the evaluation on later source verification.

10. Preserve the distinction between:
    raw LLM output
    later human/source verification
    final frozen artifacts.

11. Note that several early candidate classifications were later rejected or revised. Their original outputs should still be preserved in the audit trail rather than silently deleted.

12. For outputs involving the Turbo or mail Observer, mention that the final acceptance depended on checked framework contracts.

13. For the human-verification worksheet, do not claim that the worksheet itself constitutes independent human sign-off.

14. For the final frozen inventory, use `Correct - source-supported baseline frozen; independent human sign-off and runtime testing not established`.

Before producing the CSV, check that:

1. All major stages of the prompt history are represented.
2. The original prompt/output files can be preserved as raw evidence.
3. Rejected candidates remain visible in the audit trail.
4. The final nine-instance count is not retroactively applied to early outputs that had fewer confirmed instances.
5. Evaluation wording reflects what was known at that stage.

At the end output:

1. Final `llm.csv` in a CSV code block.
2. A suggested filename list for the 20 prompt files and 20 raw-output files.
3. A short note explaining which files should remain unchanged as raw LLM evidence.

Do not generate evolution analysis.
Do not change `patterns.csv`, `pattern_counts.csv`, UML or the frozen verification records.

Worked for 1m 9s

Part 2 — LLM Audit Record

The audit contains **20 stages**, covering initial planning, source investigations, candidate verification, the adversarial review, baseline freezing and final documentation.

Historical model identifiers cannot be reliably established for each stage, so the requested fallback `ChatGPT - 2026-10-08` is used. Filenames are proposed archival names for the original conversation records; they do not imply that those files have already been saved.

1. Final `llm.csv`

```csv
part,model_and_date,prompt_file,input_file,output_file,evaluation,verification_file
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,01_requirements_plan_prompt.txt,daplans
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,02_geocoding_investigation_prompt.txt,"
```

```
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,03_geocoding_verification_prompt.txt,"d
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,04_inbound_email_investigation_prompt.t
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,05_inbound_email_verification_prompt.tx
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,06_trip_membership_investigation_prompt
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,07_trip_membership_verification_prompt.
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,08_itinerary_expenses_investigation_pro
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,09_bulk_import_verification_prompt.txt,
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,10_external_identity_investigation_prom
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,11_external_identity_verification_promp
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,12_frontend_investigation_prompt.txt,"d
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,13_frontend_observer_verification_promp
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,14_repository_sweep_prompt.txt,"daplans
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,15_adversarial_review_prompt.txt,"dapla
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,16_human_worksheet_prompt.txt,"15_adver
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,17_baseline_freeze_prompt.txt,"daplanst
Part 2 - Pattern Discovery and Verification,ChatGPT - 2026-10-08,18_baseline_csv_prompt.txt,17_baseline
```

2. Suggested filenames for 20 prompts and 20 raw outputs

The filenames below match the references used in `llm.csv`.

Stage	Prompt file	Raw LLM output file
01 — Requirements and planning	`01_requirements_plan_prompt.txt`	`01_requirements_plan_raw.txt`
02 — Geocoding investigation	`02_geocoding_investigation_prompt.txt`	`02_geocoding_investigation_raw.txt`
03 — Geocoding verification	`03_geocoding_verification_prompt.txt`	`03_geocoding_verification_raw.txt`
04 — Inbound email investigation	`04_inbound_email_investigation_prompt.txt`	`04_inbound_email_investigation_raw.txt`
05 — Inbound email verification	`05_inbound_email_verification_prompt.txt`	`05_inbound_email_verification_raw.txt`
06 — Trip creation and membership	`06_trip_membership_investigation_prompt.txt`	`06_trip_membership_investigation_raw.txt`
07 — Trip/membership verification	`07_trip_membership_verification_prompt.txt`	`07_trip_membership_verification_raw.txt`
08 — Itinerary and expenses	`08_itinerary_expenses_investigation_prompt.txt`	`08_itinerary_expenses_investigation_raw.txt`
09 — Bulk import verification	`09_bulk_import_verification_prompt.txt`	`09_bulk_import_verification_raw.txt`

10 — External integrations	`10_external_identity_investigation_prompt.txt`	`10_external_identity_investigation_raw.txt`
11 — External pattern verification	`11_external_identity_verification_prompt.txt`	`11_external_identity_verification_raw.txt`
12 — Frontend investigation	`12_frontend_investigation_prompt.txt`	`12_frontend_investigation_raw.txt`
13 — Frontend Observer verification	`13_frontend_observer_verification_prompt.txt`	`13_frontend_observer_verification_raw.txt`
14 — Repository-wide sweep	`14_repository_sweep_prompt.txt`	`14_repository_sweep_raw.txt`
15 — Final adversarial review	`15_adversarial_review_prompt.txt`	`15_adversarial_review_raw.txt`
16 — Human-verification worksheet	`16_human_worksheet_prompt.txt`	`16_human_worksheet_raw.txt`
17 — Baseline freeze	`17_baseline_freeze_prompt.txt`	`17_baseline_freeze_raw.txt`
18 — Baseline CSV preparation	`18_baseline_csv_prompt.txt`	`18_baseline_csv_raw.txt`
19 — UML preparation	`19_uml_preparation_prompt.txt`	`19_uml_preparation_raw.txt`
20 — Verification evidence	`20_verification_evidence_prompt.txt`	`20_verification_evidence_raw.txt`

3. Raw evidence preservation rules

Keep the 20 original prompts and 20 raw LLM outputs unchanged. Save each from the relevant conversation stage without correcting wording, changing provisional classifications or replacing early conclusions with the final nine-instance inventory.

In particular, preserve the original geocoding pattern suggestions, the provisional timeline–map Observer, the earlier uncertain Turbo classification and the two candidates discovered during the repository sweep. Their subsequent rejection, acceptance or revision is part of the audit evidence.

The `verification_file` column identifies later source-based review outputs that support or qualify earlier conclusions. These later LLM reviews are **not substitutes for independent human sign-off**. Where a corresponding source-check document or reviewer record is prepared, preserve it separately, with its reviewer, date and evidence; update references only to files that have actually been saved.

Also keep the final frozen `patterns.csv`, `pattern_counts.csv`, Mermaid sources and verification records separate from the raw LLM outputs. Do not overwrite stage 18 or 19 raw responses with corrected or exported artifacts.

Audit completeness check

Check	Result
Major workflow stages	20 represented
Suggested prompt files	20
Suggested raw-output files	20
Earlier rejected/revised candidates preserved	Yes
Historical nine-instance count imposed on early stages	No
Framework-contract dependency recorded	Mail and Turbo
Independent human sign-off claimed	No
Frozen baseline modified	No
Evolution analysis performed	No

Status: The `llm.csv` content is prepared for archival. The named prompt and output files still need to be saved from the original conversation records and checked against their actual contents. The frozen baseline remains **6 Controller + 1 Information Expert + 2 Observer = 9 instances across 3 pattern types**.

The Part 2 pattern-discovery work is complete.
Use only the frozen baseline and completed Part 2 artifacts. Do not search for new patterns and do not modify the nine frozen classifications.
Prepare the final Part 2 documentation section for the team report and a concise handoff for the teammate handling software evolution/refactoring.
The frozen baseline is:

1. Information Expert (GRASP) — Parsed Email to Trip Record Linking
2. Controller (GRASP) — Inbound Travel Email Intake Coordination
3. Controller (GRASP) — Trip Creation Coordination
4. Controller (GRASP) — Trip Membership Management Coordination
5. Controller (GRASP) — Bulk Itinerary Import Coordination
6. Controller (GRASP) — OAuth Sign-in and Identity Linking Coordination

7. Controller (GRASP) — Account Traveler Enrollment Coordination
8. Observer (GoF) — Outbound Mail Delivery Logging
9. Observer (GoF) — Trip-Scoped Turbo Refresh Notifications

Final counts:
Controller (GRASP) = 6
Information Expert (GRASP) = 1
Observer (GoF) = 2
Total instances = 9
Distinct pattern types = 3
Five rejected classifications must remain documented:

1. Geocoding Template Method
2. Geocoding Strategy
3. Geocoding Adapter
4. Itinerary Timeline–Map Event Coordination as Observer
5. Beta Invite Eligibility Evaluation as Information Expert

Prepare the report section with these headings:

1. Pattern Discovery Method

Explain briefly that the repository was inspected area by area, candidates were traced from actual source collaborations, false positives were challenged, the repository received a final coverage sweep and an adversarial review was performed before freezing the inventory.
Mention that pattern names were not assigned merely because Rails conventions, callbacks, enums, direct API calls or controllers existed.

2. Final Pattern Inventory

Summarize the nine instances in a concise table with:
Pattern ID
Pattern
Category
Instance name
Main participant
Short purpose
Use P01–P09 from the frozen inventory.

3. Pattern Counts

State:
6 Controller instances
1 Information Expert instance
2 Observer instances
9 total instances
3 distinct pattern types
Explain briefly that instances were counted by collaboration/use-case responsibility rather than by number of classes, methods, requests, providers or runtime occurrences.

4. Important Pattern Evidence

Give a concise paragraph for each pattern type rather than repeating all nine full verification records.
For Controller, explain the use-case coordination rule and give representative examples.
For Information Expert, explain why InboxItem received the parsed-email linking responsibility.
For Observer, explain that both confirmed instances are framework-mediated and clearly separate application participation from framework infrastructure.

5. Rejected Candidates and False Positives

Summarize the five rejected classifications and why retaining these rejection records improves the credibility of the analysis.

6. Verification and Limitations

State accurately:
Static source verification completed.
Runtime/end-to-end testing was not completed.
The source analyzed was `daplanstan-main.zip`.
The ZIP-to-commit correspondence with `da54d9ab8bfc118e99375fdd6c506efb20b66e5b` remains unverified.
The two Observer classifications rely on checked versioned framework contracts.
Independent human sign-off has not been established unless separately performed by a team member.

Do not present LLM-assisted verification as independent human approval.

7. Part 2 Deliverables

List the completed artifacts:
`patterns.csv`
`pattern_counts.csv`
P01–P09 editable UML Mermaid sources
verification evidence records
rejected-candidate records
`llm.csv`
20 prompt records
20 raw LLM outputs
Mention that the prompt and raw-output files still need to be physically archived from the conversation if they have not yet been saved.

8. Handoff to Software Evolution / Refactoring

Prepare a concise handoff explaining which baseline patterns are especially relevant when the next teammate introduces:
personalized itinerary generation
multiple booking providers
different cancellation policies
Do not design the evolution solution for them.
Instead tell them:
use this frozen baseline as the "before" source version;
preserve P01–P09 IDs when comparing before and after;
record whether each existing pattern is preserved, modified, extended, replaced or removed;
record genuinely new patterns separately;
do not change baseline counts retroactively because of evolution;
use `patterns_evolution.csv` for evolution status.
Also identify baseline areas likely to be affected without prescribing a pattern:
Trip and TripItem / itinerary import for personalized itinerary work;
external integration boundaries for multiple providers;
membership and domain/controller responsibilities only if cancellation changes touch them;

new booking/cancellation code should be compared against the frozen baseline rather than being treated as if it existed originally.

9. Part 2 Completion Statement

End with a short statement that baseline pattern discovery is complete and frozen at 9 instances across 3 pattern types, subject to the documented provenance, runtime and human-review limitations.
Keep the writing professional and concise enough to fit into the team report.
Do not perform Part 3 evolution analysis.
Do not propose refactoring solutions.
Do not add new baseline patterns.

The following is a report-ready Part 2 section based exclusively on the frozen baseline and completed source analysis. It preserves all nine classifications and includes the software evolution handoff without proposing any refactoring solutions.

Writing

Part 2 — Design Pattern Discovery and Verification

1. Pattern Discovery Method

Design patterns in DaPlanStan were identified through systematic source-code inspection rather than assumptions based on framework conventions.

The repository was examined area by area, including geocoding, inbound email processing, trip creation, membership management, itinerary importing, expenses, external integrations, authentication and frontend interactions. Candidate patterns were traced through actual classes, methods and collaborating objects.

Each candidate was evaluated against the defining responsibilities or mechanisms of the proposed pattern. Potential false positives were challenged, followed by a repository-wide coverage sweep and a final adversarial review before the baseline was frozen.

Rails controllers, callbacks, associations, enums, background jobs, direct API integrations and JavaScript events were not classified as design patterns merely because they existed. Only source-supported collaborations meeting the established classification and counting rules were retained.

2. Final Pattern Inventory

The final source-supported baseline contains nine pattern instances across three distinct pattern types.

ID	Pattern	Category	Instance name	Main participant	Short purpose
P01	Information Expert (GRASP)	GRASP — Responsibility Assignment	Parsed Email to Trip Record Linking	`InboxItem`	Interprets extracted email data and links itinerary and expense records.
P02	Controller (GRASP)	GRASP — Responsibility Assignment	Inbound Travel Email Intake Coordination	`TripMailbox#process`	Coordinates incoming-email routing, sender checks and processing delegation.
P03	Controller (GRASP)	GRASP — Responsibility Assignment	Trip Creation Coordination	`TripsController#create`	Creates trips, establishes owner membership and schedules cover-image retrieval.
P04	Controller (GRASP)	GRASP — Responsibility Assignment	Trip Membership Management Coordination	`TripMembersController`	Coordinates trip-member addition, removal, authorization and invitations.
P05	Controller (GRASP)	GRASP — Responsibility Assignment	Bulk Itinerary Import Coordination	`TripItemsController#import`	Coordinates itinerary JSON preview, confirmed import and geocoding scheduling.
P06	Controller (GRASP)	GRASP — Responsibility Assignment	OAuth Sign-in and Identity Linking Coordination	`SessionsController#create`	Resolves OAuth identities, establishes sessions and coordinates account access.
P07	Controller (GRASP)	GRASP — Responsibility Assignment	Account Traveler Enrollment Coordination	`TravelersController#create`	Coordinates account-level traveler enrollment and invitations.
P08	Observer (GoF)	GoF — Behavioral	Outbound Mail Delivery Logging	`MailLogger`	Logs mail-delivery notifications through Action Mailer's observer mechanism.
P09	Observer (GoF)	GoF — Behavioral	Trip-Scoped Turbo Refresh Notifications	`InboxItem`, `GeocodeTripItemsJob` and subscribed views	Publishes trip-scoped refresh notifications through Turbo Streams.

P08 and P09 are framework-mediated Observer implementations. Their framework-supplied infrastructure is distinguished from the application-defined participants in the verification records and UML diagrams.

3. Pattern Counts

Pattern type	Confirmed instances
Controller (GRASP)	6
Information Expert (GRASP)	1
Observer (GoF)	2
Total pattern instances	**9**
Distinct pattern types	**3**

Pattern instances were counted by meaningful collaboration or cohesive use-case responsibility rather than by the number of classes, methods, HTTP requests, authentication providers or runtime occurrences.

For example, trip-member addition and removal form one membership-management instance. Itinerary import preview and confirmation form one import instance. Similarly, multiple mailers and multiple trip-stream publishers or subscribers do not automatically create additional Observer instances.

4. Important Pattern Evidence

GRASP Controller

Six Controller instances were identified through application-level responsibility assignment. Each coordinates a meaningful system event involving multiple collaborators.

For example, `TripsController#create` establishes a new trip and its initial owner membership, while `TripMembersController` coordinates authorization, traveler resolution and membership changes. `SessionsController#create` similarly coordinates local identity and account-access decisions after OmniAuth authentication.

These responsibilities were retained because they represent cohesive application use cases, not merely because the classes inherit from Rails controllers or receive requests.

GRASP Information Expert

`InboxItem` was identified as an Information Expert because it stores the extracted email information required for its record-linking responsibility.

Its `confirm_and_create_items!` and `fill_missing_links!` methods interpret the stored parsed data, locate existing itinerary records and create missing trip items or qualifying expenses. The

requesting controller delegates this substantive work to the object possessing the central information.

GoF Observer

Two framework-mediated Observer collaborations were confirmed.

Outbound Mail Delivery Logging uses the application-defined `MailLogger.delivered_email` callback, explicitly registered through `ActionMailer::Base.register_observer`. Action Mailer and the Mail gem provide the observer registration and notification infrastructure.

Trip-Scoped Turbo Refresh Notifications uses `InboxItem` model broadcasts, explicit geocoding-job broadcasts and views subscribing through `turbo_stream_from @trip`. Turbo Rails and Action Cable provide subscription, delivery and browser-refresh behavior.

Both classifications distinguish DaPlanStan's application participation from mechanisms supplied by external frameworks.

5. Rejected Candidates and False Positives

Five candidate classifications were excluded from the final inventory.

Rejected classification	Reason for rejection
Geocoding — Template Method	No application-defined algorithm skeleton delegates overridable primitive operations.
Geocoding — Strategy	Biased and fallback geocoding are sequential methods rather than interchangeable strategy implementations.
Geocoding — Adapter	Direct Geocoder API usage does not establish translation between incompatible collaborator interfaces.
Itinerary Timeline–Map Event Coordination — Observer	The controllers use targeted browser DOM events to communicate with a known map element rather than a substantive independent Observer collaboration.
Beta Invite Eligibility Evaluation — Information Expert	Simple invitation-validity predicates do not establish a sufficiently substantial independent Information Expert responsibility.

Preserving these rejected candidates documents the alternative interpretations considered during analysis.

It also demonstrates that the final inventory was not inflated by treating ordinary Rails behavior,

API calls, model predicates or frontend events as design patterns without sufficient supporting evidence.

6. Verification and Limitations

Static source verification was completed for the nine retained instances. The verification process included source-location checks, participant-role mapping, candidate challenges, duplicate detection and comparison with relevant framework contracts.

The analyzed source snapshot was the uploaded `daplanstan-main.zip`.

The correspondence between this ZIP and the stated Git commit `da54d9ab8bfc118e99375fdd6c506efb20b66e5b` remains unverified. Therefore, the baseline is identified using the source-version label `baseline`, without claiming verified Git provenance.

No complete Rails runtime or end-to-end integration testing was performed. Consequently, the static pattern classifications should not be interpreted as proof that every associated workflow executes successfully.

The two Observer classifications rely on previously checked versioned framework contracts:

- Action Mailer 8.1.3 and Mail 2.9.0 for outbound mail delivery logging.
- Turbo Rails 2.0.23 and Action Cable for trip-scoped refresh notifications.

The source analysis was LLM-assisted. **Independent human reviewer sign-off has not been established** unless separately performed and documented by a team member.

These limitations are retained in the verification evidence rather than being represented as completed testing or approval.

7. Part 2 Deliverables

The following artifacts have been prepared as part of the pattern-discovery documentation:

Deliverable	Contents
`patterns.csv`	Nine baseline instances with participants, source locations and evidence.
`pattern_counts.csv`	Counts for the three confirmed pattern types and overall totals.
P01–P09 editable UML Mermaid sources	Nine diagrams showing the confirmed collaborations and participant relationships.

Verification evidence records	Nine source-confirmed records documenting supporting evidence and challenged interpretations.
Rejected-candidate records	Five records explaining why proposed classifications were excluded.
`llm.csv`	Twenty-stage audit content recording LLM assistance, evaluations and verification references.
20 prompt records	Original prompts for major discovery and documentation stages.
20 raw LLM outputs	Original responses retained to preserve the progression of candidate decisions.

The CSV contents, Mermaid sources and verification documentation have been prepared for inclusion in the team's deliverables.

The 20 original prompt files and 20 raw-output files must still be physically archived from the conversation if they have not already been saved. Original LLM outputs should remain unchanged, including earlier provisional classifications and subsequently rejected candidates.

The final frozen artifacts must be retained separately from the raw LLM evidence.

8. Handoff to Software Evolution / Refactoring

The next teammate will examine how DaPlanStan evolves to support personalized itinerary generation, multiple booking providers and different cancellation policies.

The nine-instance inventory from this analysis must serve as the unchanged "before" baseline.

Relevant baseline areas

Evolution requirement	Baseline code or responsibilities to examine	Existing pattern references
Personalized itinerary generation	`Trip`, `TripItem`, itinerary import and associated coordination	P03, P05; P01 where imported or interpreted trip records are relevant
Multiple booking providers	Existing external integration boundaries, provider interactions and booking-related records	P01, P02 or P06 only where their actual responsibilities are affected
Different cancellation policies	New booking/cancellation behavior and any affected trip, membership or controller responsibilities	P03, P04 or P07 only if those existing responsibilities are actually changed

These are investigation entry points, not proposed design solutions. Existing P01–P09 classifications should not be extended to new functionality without examining the evolved implementation.

Evolution comparison requirements

The next teammate should:

1. Use `baseline` as the original source version and preserve the stable P01–P09 identifiers.
2. Compare each existing collaboration before and after evolution.
3. Record whether an instance is **preserved, modified, extended, replaced or removed**, using the project's agreed status definitions.
4. Document genuinely new patterns separately from the existing baseline.
5. Record evolution status in `patterns_evolution.csv`.
6. Keep the original baseline counts unchanged, even if the evolved implementation introduces additional patterns or participants.

New booking-provider and cancellation-policy functionality must be evaluated as part of the evolved source, not attributed retroactively to DaPlanStan's original implementation.

The software evolution teammate is responsible for examining the resulting changes and documenting their design impact. This Part 2 handoff does not prescribe refactoring techniques or new pattern implementations.

9. Part 2 Completion Statement

Part 2 pattern discovery and source-based verification are complete.

The DaPlanStan baseline is frozen at **9 confirmed pattern instances across 3 distinct pattern types**, consisting of 6 GRASP Controller instances, 1 GRASP Information Expert instance and 2 GoF Observer instances.

The inventory, pattern counts, UML sources, verification evidence, rejected-candidate records and LLM audit documentation establish the source-supported baseline for subsequent software evolution analysis.

The baseline remains subject to the documented source-provenance, runtime-testing and independent human-review limitations. No future evolution work should retroactively modify its frozen classifications or counts.

This section can be inserted into the team report as Part 2. The handoff in Section 8 gives the next teammate the relevant source areas, stable pattern IDs and comparison rules without introducing any Part 3 design decisions.

Pasted text(202610...
Document

Worked for 6m 37s

DaPlanStan Part 2 — Submission Package

Package created and validated

The complete Part 2 folder has been packaged into a ZIP containing **65 files**. The consistency checks passed for the frozen inventory, source ranges, counts, and diagram files.

Complete Part 2 submission
Includes CSVs, UML, evidence, report, handoff and LLM audit

Download DaPlanStan_Part2_Complete_Submission.zip ↗

Individual files

- patterns.csv ↗
- pattern_counts.csv ↗
- llm.csv ↗
- Final Part 2 Report ↗
- Software Evolution Handoff ↗
- Final Consistency Check ↗
- Verification Evidence Records ↗

Package validation

Check	Result
Frozen pattern instances	9
Distinct pattern types	3
Controller / Information Expert / Observer	6 / 1 / 2
Mermaid `.mmd` diagrams	9
Vector `.svg` diagrams	9
LLM audit stages	20

Prompt archive files	20
Raw-output archive files	20
Source-range exceptions	0

Historical archive limitation: Six prompt texts were transcribed from available conversation messages; their byte-exact provenance has not been independently authenticated. No exact historical raw LLM responses could be recovered. The remaining 14 prompt slots and all 20 raw-output slots contain explicit unrecoverable markers rather than fabricated content.

The SVGs are valid vector diagrams rendered from the Mermaid relationships using Graphviz, rather than native Mermaid CLI exports. The editable `.mmd` sources are included.

The package preserves the **9-instance frozen baseline**. No new pattern discovery or software evolution analysis was performed.

- **patterns.csv** Spreadsheet
- **pattern_counts.csv** Spreadsheet
- **llm.csv** Spreadsheet
- 4 more

so now we are done with part2 right from the team division

Yes! ✅ **Your assigned Part 2 — Design Pattern Discovery and Verification — is complete.**

You have finished the source investigations, adversarial review, frozen baseline, CSVs, nine UML diagrams, verification records, final report and software-evolution handoff.

Pattern discovery and documentation	Complete
Frozen baseline	**9 instances · 3 types**
Submission ZIP	Created
Historical LLM evidence archive	Incomplete

Only the final submission checks remain: The original historical prompts and raw LLM responses need to be recovered where possible because the package currently contains placeholders for

missing originals. Independent human sign-off also remains pending if your team or faculty requires it.

So **yes, your Part 2 analysis and documentation work is finished.** The next teammate can begin **Software Evolution / Refactoring** using your frozen baseline without modifying P01–P09.

Download your Part 2 submission package ↗